

Object-oriented TypeScript, test first

OOP in TypeScript for Java programmers - with Deno, Celbridge, and test-driven development

Contents

1. [Introduction: TypeScript, Deno and Celbridge](#)
2. [Events: clicks, keys and timers](#)
3. [Arrow functions](#)
4. [Test first, properly](#)
5. [Classes, objects and constructors](#)
6. [Encapsulation](#)
7. [Model and view](#)
8. [Constants, static and enums](#)
9. [References, composition and modules](#)
10. [Interfaces and structural typing](#)
11. [Inheritance](#)

Appendix: [Java to TypeScript cheat sheet](#)

Chapter 1 - Introduction: TypeScript, Deno and Celbridge

You already know how to write programs in Java. This book shows you how to write object-oriented programs in **TypeScript**, the language most web software is now written in, and how to write them **test first**. This chapter takes it slowly. You start with one file that says "Hello, world!", and add one small idea at a time: the time of day, functions, your first tests, a function written test first, and lists of data. By the end you will have built seven small projects, each only a little bigger than the one before.

Hello, world!

The Hello, World project

What you will learn

- what TypeScript is, and how it compares with Java's "compile, then run"
- how a project is opened in Celbridge, built by Deno, tested, and previewed - automatically, every time you save
- the TypeScript basics for Java programmers: `const` and `let`, types, template literals, `if`, functions, `export` and `import`, arrays and `for ... of`
- how TypeScript finds and changes elements on a web page
- why we test; checking a function with an example in its doc comment; test files; reading the results
- **test-driven development**: writing the test before the code
- how to keep data in a JSON file and use it in a page

The projects

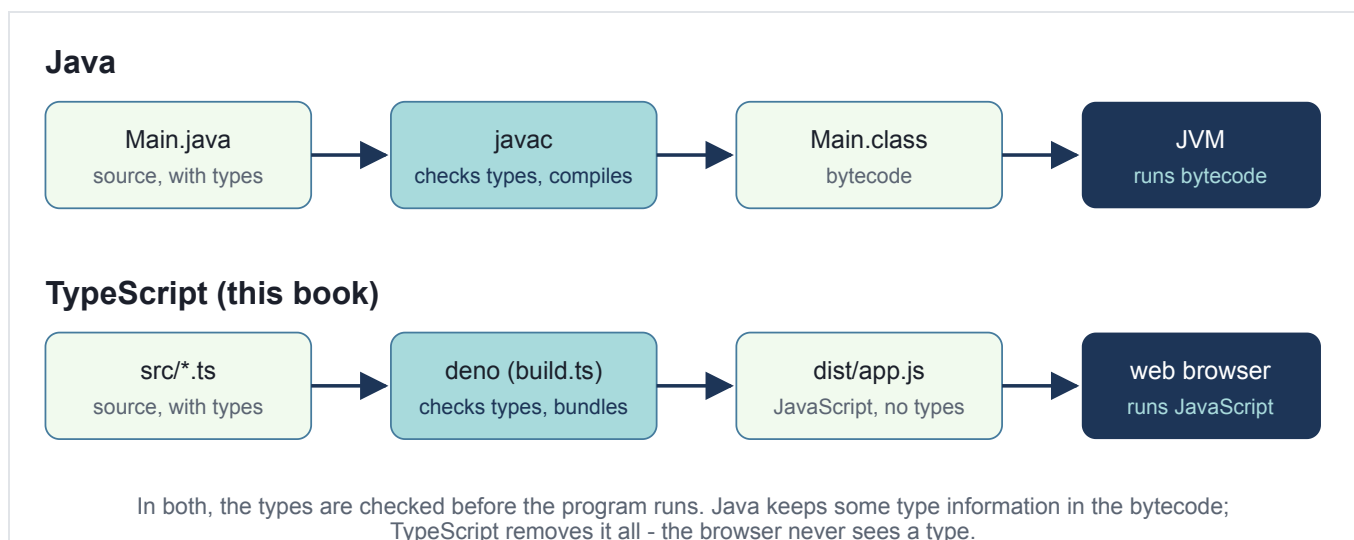
Project	What it shows
ch01_project01_hello_world	the smallest project: one file, <code>main.ts</code> , that changes the page
ch01_project02_whats_the_time	getting the time, and the <code>number</code> type
ch01_project03_first_tests	a function in its own file, an example in its doc comment that Deno checks, and the first test files
ch01_project04_age_groups	a function written test first
ch01_project05_bullet_list	an array, a loop, and a bulleted list on the page
ch01_project06_list_from_json	the list's data in a JSON file
ch01_project07_hello_page	everything together: functions, imports, a list, and tests

You do not have to type the projects in: each one is ready to open. But each step in the chapter starts from the project before and asks you to make a small change yourself (the **Exercises**). Try each exercise before reading on - the answer follows straight after it.

What is TypeScript?

Web browsers run one programming language: **JavaScript**. It has no types: a variable can hold a number, then a string, then a list, and nothing complains until the program goes wrong while it is running.

TypeScript is JavaScript with types added. You write `let count: number = 0;` and the TypeScript checker makes sure that `count` only ever holds numbers. Before the program runs, the types are checked and then **removed**, leaving ordinary JavaScript that any browser can run.



Java and TypeScript pipelines

This is close to what you know from Java. `javac` checks your types and turns `Main.java` into bytecode, which the Java Virtual Machine runs. In this book, Deno checks your types and turns your TypeScript into

one JavaScript file, `dist/app.js`, which the browser runs. The big difference is the last step: the browser never sees a single TypeScript type. Types are a safety net that works **before** the program runs.

TypeScript has classes, interfaces, `extends`, `implements`, `public`, `private` and `protected`, and they mean almost the same as in Java - you will meet them from Chapter 2 on. Some things are different: there is only one number type, functions do not have to live inside classes, and you compare strings with `===`. Each chapter points out where TypeScript differs from Java.

The tools

Everything in this book uses two tools.

- **Deno** runs TypeScript. It has a type checker, a bundler (to join many files into one for the browser), a test runner, a linter (to spot likely mistakes) and a code formatter built in, so there is nothing else to install. You need Deno 2.4 or later; check with `deno --version` in a terminal.
- **Celbridge** opens each project folder as a project, with an explorer, editors, a console at the bottom, and previews of web pages and Markdown files at the side.

The first build of the first project downloads one small package, Deno's library for tests, so it needs an internet connection. After that everything works offline.

Opening a project

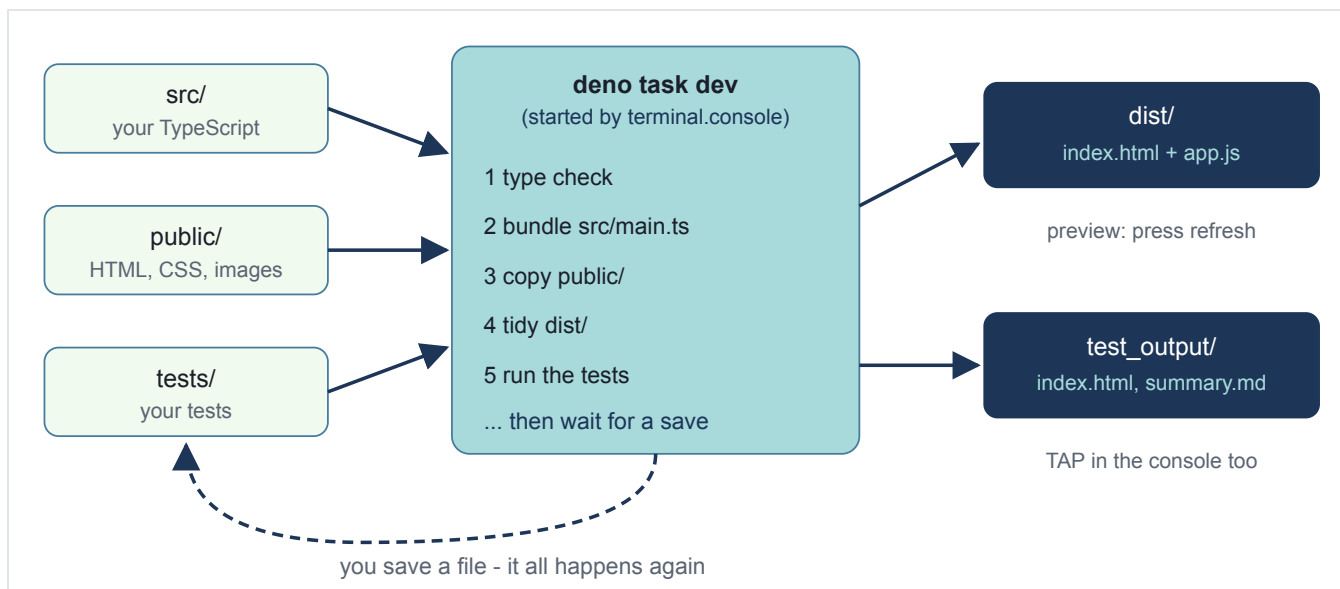
Open a project folder in Celbridge - start with `projects/ch01_project01_hello_world`. Three things open by themselves:

1. **the console**, at the bottom, which immediately runs `deno task dev` (see below)
2. `dist/index.html`, at the side, shown as a preview: this is your web page
3. `README.md`, the project's own notes

A fourth shortcut, with a clipboard icon, opens the **test report** (`test_output/index.html`).

What happens when you save

`deno task dev` does five things, then waits. Every time you save a file in `src/`, `public/` or `tests/`, it does them all again:



What deno task dev does

1. **type checks** every `.ts` file and lists any errors (the page is still built, so you can keep experimenting - but read the errors, they are nearly always a real bug)
2. **bundles** `src/main.ts`, and every file it imports, into one JavaScript file, `dist/app.js`
3. **copies** everything in `public/` (the HTML page, its CSS, any images) into `dist/`
4. **tidies** `dist/`, removing anything that is no longer in `public/`
5. **runs the tests** in `tests/`, prints the results, and writes a report to `test_output/`

When `dist/` changes, the **refresh** button on the `dist/index.html` preview lights up. Press it to see your changes.

The console has buttons down its side too: rebuild-and-watch, build once, test once, and lint. If `deno task dev` is running (it usually is), press **Ctrl+C** in the console before using them.

Note - There is no web server. The build makes one plain `<script>` file, which a browser is happy to load straight from your disk, so `dist/index.html` works in Celbridge's preview or in any browser. Without Celbridge, run `deno task dev` in a terminal in the project's folder, and open `dist/index.html` in a browser.

A project's files

Every project in the book has the same layout:

File or folder	What it is	Do you edit it?
src/	your TypeScript: <code>main.ts</code> , and other files as the project grows	yes - this is your program
tests/	your tests, in files ending <code>.test.ts</code>	yes
public/	<code>index.html</code> , <code>styles.css</code> , and any images	yes
dist/	the built page - made by the build	no - it is overwritten on every save
test_output/	the test report - made by the build	no
deno.json	the project's settings: its tasks (<code>dev</code> , <code>build</code> , <code>test</code> , ...)	rarely
build.ts , tools/	the build and the test report	no
terminal.console , *.celbridge	Celbridge's console and project settings	no

The rule to remember: **change** `src/` , `public/` and `tests/` ; **look at** `dist/` and `test_output/` .

Project 1: Hello, world!

The first project has one TypeScript file, `src/main.ts` , and it is four lines long:

`src/main.ts`

```
const output = document.querySelector("#output");
if (output !== null) {
  output.textContent = "Hello, world!";
}
```

And the web page it changes:

`public/index.html`

```
<h1 id="output">(if you can read this, dist/app.js has not run - build the project)</h1>
...
<script src="app.js"></script>
```

A web page is a tree of **elements** - headings, paragraphs, lists, buttons. The browser keeps a model of the page, with one object for every element, called the **DOM** (Document Object Model). Your TypeScript can find an element and change it.

- `document` is the whole page.
- `document.querySelector("#output")` finds the element whose `id` is `output` . It uses CSS selectors: `#output` means "the element with id output".
- `querySelector` gives back `null` if no element matches - if the id was mistyped, say. TypeScript knows this, and will not let you use `output` until you have checked that it is not `null` . That is what

the `if` is for. (Java would let you forget, and throw a `NullPointerException` when the program runs. TypeScript stops you before it runs.)

- `output.textContent = "Hello, world!";` replaces the text inside the element.

The `<script src="app.js">` line, at the end of the page, runs the JavaScript the build made from `main.ts`. It comes last so that the elements above it already exist when it runs. If you ever see the "(if you can read this ...)" text, the script did not run - look at the console for an error.

const

`const output = ...` declares a variable that can never be given a new value, like a `final` variable in Java. Notice there is no type: TypeScript **infers** the type from the value. It is still checked - you could not later put a number in `output`.

Try it - open `ch01_project01_hello_world`, change the message in `main.ts`, and save. Watch the console rebuild, then press refresh on the preview.

Exercise 1.1 - Hello, you

Change the program so that the name is in a constant of its own, and the page says "Hello, *your name*!". Try it before reading on.

Here is one way:

`src/main.ts`

```
const NAME = "Ada";

const output = document.querySelector("#output");
if (output !== null) {
  output.textContent = `Hello, ${NAME}!`;
}
```

The message is in **backticks** (```), not quotes. A string in backticks is a **template literal**: anything inside `${...}` is worked out and put into the string. It does the job of `"Hello, " + name + "!"` in Java, but is easier to read. (TypeScript strings can also use `"..."` or `'...'`; this book uses double quotes, and backticks when it needs `${...}`.)

`NAME` is in capitals because it is a fixed value that the program never changes - the same convention as Java constants.

Looking at the JavaScript

In Java you can look inside a `.class` file to see what `javac` made. Here, just open `dist/app.js`:

`dist/app.js`

```
(() => {
  // src/main.ts
  var output = document.querySelector("#output");
  if (output !== null) {
    output.textContent = "Hello, world!";
  }
})();
```

Almost the same as `main.ts`. The bundler wrapped it in `((() => { ... }()))`; to keep its variables private, and wrote `var` instead of `const` (an older keyword that every browser understands). As the projects grow, it is worth looking again: you will see the types disappear, and several files joined into one.

Project 2: What's the time?

The time is 2:27:16 PM

The hour is 14

The What's the Time? project

`src/main.ts`

```
// new Date() makes an object holding the date and time, right now – like Java's
// LocalDateTime.now().
const now = new Date();

// getHours() gives the hour, from 0 to 23. Its type is number – TypeScript's only type for
// numbers.
const hour: number = now.getHours();

const time = document.querySelector("#time");
if (time !== null) {
  // toLocaleTimeString() gives the time as text, written the way your computer is set up to show
  // it.
  time.textContent = `The time is ${now.toLocaleTimeString()}`;
}

const hourText = document.querySelector("#hour");
if (hourText !== null) {
  hourText.textContent = `The hour is ${hour}`;
}
```

`new Date()` makes a new object, just as `new` does in Java. `now.getHours()` calls one of its methods.

Types come after the name

`const hour: number = now.getHours();` gives the variable a type. In Java the type comes first (`int hour`); in TypeScript it comes after the name, with a colon. Here the type is not really needed – TypeScript can infer it – but writing it can make code clearer.

TypeScript's basic types:

Java	TypeScript	Notes
<code>int</code> , <code>long</code> , <code>double</code> , <code>float</code> ...	<code>number</code>	one type for all numbers; <code>7 / 2</code> is <code>3.5</code> (use <code>Math.floor(7 / 2)</code> for 3)
<code>boolean</code>	<code>boolean</code>	<code>true</code> and <code>false</code>
<code>String</code> , <code>char</code>	<code>string</code>	lower case <code>s</code> ; no separate character type
<code>String[]</code> , <code>ArrayList<String></code>	<code>string[]</code>	arrays grow and shrink, like an <code>ArrayList</code> - see project 5
<code>void</code>	<code>void</code>	for functions that return nothing

Exercise 1.2 - Good morning

Change the page so that it says "Good morning!" before midday, and "Good day!" from midday on. Try it before reading on.

Here is one way:

`src/main.ts`

```
const now = new Date();
const hour: number = now.getHours();

let message = "Good day!";
if (hour < 12) {
  message = "Good morning!";
}

const greeting = document.querySelector("#greeting");
if (greeting !== null) {
  greeting.textContent = message;
}
```

(with an element `<h1 id="greeting">` in `index.html`). `if` works exactly as in Java. Note `let`: it declares a variable that **can** be changed, which `message` needs to be. Use `const` unless you need to change the variable later, and then use `let`. (You may see `var` in older JavaScript code; it has confusing rules, and the linter will complain if you use it.)

Comparing values uses `===` and `!==` - three characters, not two. They work like Java's `==` for numbers, and for strings they compare the *contents*: `"cat" === "cat"` is true, with no need for `.equals()`. Never use `==` and `!=` in TypeScript: they convert between types in surprising ways (`0 == ""` is true!).

Exercise 1.3 - A function of its own

Move the decision - morning or day? - into a function called `greeting`, in a new file `src/greeting.ts`. The function takes the hour, and returns the greeting. `main.ts` should call it. Try it before reading on.

Here is one way - and it is how project 3 starts:

`src/greeting.ts`

```
// A named constant, rather than a "magic number" 12 in the middle of the code.
const MIDDAY = 12;

/** "Good morning" before midday (hours 0 to 11), and "Good day" from midday on (hours 12 to 23).
 */
export function greeting(hour: number): string {
  if (hour < MIDDAY) {
    return "Good morning";
  }
  return "Good day";
}
```

src/main.ts

```
import { greeting } from "../greeting.ts";

const now = new Date();
const hour: number = now.getHours();

const message = document.querySelector("#message");
if (message !== null) {
  message.textContent = `${greeting(hour)}!`;
}
```

Three new things:

- **Functions** are declared with the keyword `function`. The parameter's type comes after its name (`hour: number`), and the return type comes after the brackets (`): string`). In Java every method lives inside a class; in TypeScript a file can contain plain functions. They are like Java `static` methods without a class around them.
- `export` makes the function visible to other files. Without it, nothing in a file can be seen from outside.
- `import { greeting } from "../greeting.ts";` brings it into `main.ts`. The path is relative to the importing file, and in this book it always ends in `.ts`.

`/** ... */` is a doc comment, as in Java: editors show it when you hover over the function's name.

Testing

The greeting changes at midday. How do you check that `greeting` gets it right? You could look at the page now, and again this afternoon. And at exactly midday. And at midnight. That is slow, and easy to get wrong - and you would have to do it all again every time you changed the code.

A **test** is a small piece of code that runs your code and checks the answer. Because `greeting` is a function that takes the hour as a parameter, a test can try any hour it likes, instantly. Deno runs every test each time you save, so you find out straight away if a change broke something.

This is the main reason `greeting` lives in its own file, away from the page: code that does not touch the page can be tested by Deno on its own, without a browser.

A first check: an example in the doc comment

The simplest place to start checking `greeting` is right next to it, in its doc comment. Project 3's `greeting.ts` has an **example** there:

`src/greeting.ts`

```
/**
 * "Good morning" before midday (hours 0 to 11), and "Good day" from midday on (hours 12 to 23).
 *
 * @example
 * ```ts
 * import { assertEquals } from "@std/assert";
 *
 * assertEquals(greeting(9), "Good morning");
 * assertEquals(greeting(15), "Good day");
 * ```
 */
export function greeting(hour: number): string {
```

- `@example` marks the start of an example, as in a Javadoc comment. The example is a small piece of TypeScript, between two lines of three backticks with `ts` after the first.
- **Deno runs the example as a test**, every time you save. You do not need to import `greeting`: Deno does that for you, because the example is in `greeting`'s own file.
- `assertEquals(actual, expected)` checks that two values are equal; if they are not, the check **fails**. It comes from Deno's standard library, `@std/assert`, which the example imports. Note the order: **actual first, then expected** - the opposite way round to JUnit's `assertEquals`, if you have used it.
- The example is documentation too. Hover over `greeting` in `main.ts`, and your editor shows the comment and the example: how to use the function, and what it gives back.

Open `ch01_project03_first_tests` (project 3 also has a `tests/` folder - we come to that in a moment). Here is what the console showed when the example was the only check in the project. The results are in a format called **TAP** (the Test Anything Protocol), which is easy for people and programs to read:

```
TAP version 14
# src/greeting.ts
ok 1 - example in the doc comment (lines 11-17)
1..1

Tests: 0 failed, 1 passed, 0 skipped, 0 type errors, 0 lint warnings -> test_output/index.html
```

`ok` means it passed. The last line sums up, and says where the full report is. Open it with the clipboard shortcut:

All tests passing

ch01_project03_first_tests · 10/4/2026, 5:28:44 PM

1

passed

0

failed

0

skipped

0

type errors

0

lint warnings

All tests

src/greeting.ts

✓ example in the doc comment (lines 11-17)

► TAP output

Made by tools/test_report.ts · Markdown version: summary.md

The test report, with one example passing

The report is rewritten every time you save. It also counts **type errors** and **lint warnings** (more on those below), so it is the one place to look to see how your project stands. There is a Markdown copy of it, `test_output/summary.md`, too.

Note - A comment is not a check. An example written as `greeting(9); // "Good evening"` passes, even though the comment is wrong: Deno only runs the example, and nothing compares the answer with the comment. An example only checks what it puts in an `assertEquals`.

When a check fails

Try it: change `MIDDAY` to `8` in `greeting.ts`, and save. Now 9 o'clock is "Good day", and the example fails. The console shows exactly what went wrong:

```
not ok 1 - example in the doc comment (lines 11-17)
---
message: |-
  AssertionError: Values are not equal.

    [Diff] Actual / Expected

-   Good day
+   Good morning
    at src/greeting.ts (example at lines 11-17)
at: src/greeting.ts:11
...
```

In the difference ("diff"), the line starting `-` is what the code **actually** gave, and the line starting `+` is what the check **expected**. The last lines say where the failing example is. The report turns red and shows the same thing. Put `MIDDAY` back to 12, save, and it goes green again.

When examples are not enough: test files

An example shows how to use a function, and proves the example is true. But think about what else `greeting` should get right. The change happens at midday, and changes like that are where mistakes hide - so 11 o'clock, 12 o'clock and midnight all need checking too. Put them all in the doc comment and three problems appear:

- the comment is no longer a short, clear example - it is crowded with checks
- an example has **no name** saying what it checks - only "example in the doc comment (lines 11-17)"
- the first `assertEquals` that fails **stops** the rest of the example, so you do not find out about the others

So checks like these go in a **test file** instead. Test files end `.test.ts`, and live in `tests/`. Project 3's first test file is the simplest test there could be:

`tests/first.test.ts`

```
import { assertEquals } from "@std/assert";

Deno.test("one plus one is two", () => {
  assertEquals(1 + 1, 2);
});
```

- `Deno.test(...)` declares one test. It has a **name** - a sentence saying what should be true - and the **code** that checks it.
- The code is written `() => { ... }`. This is a function with no name (an *arrow function*, like a Java lambda). Chapter 3 explains arrow functions properly; for now, read it as "the test's code".
- Each test runs on its own: one failing test does not stop the others.

The second test file tests the greeting, including the edges:

`tests/greeting.test.ts`

```
import { assertEquals } from "@std/assert";
import { greeting } from "../src/greeting.ts";

Deno.test("9 o'clock is morning", () => {
  assertEquals(greeting(9), "Good morning");
});

Deno.test("3 in the afternoon (hour 15) is day", () => {
  assertEquals(greeting(15), "Good day");
});

// The edges – the hours either side of the change – are where mistakes hide.

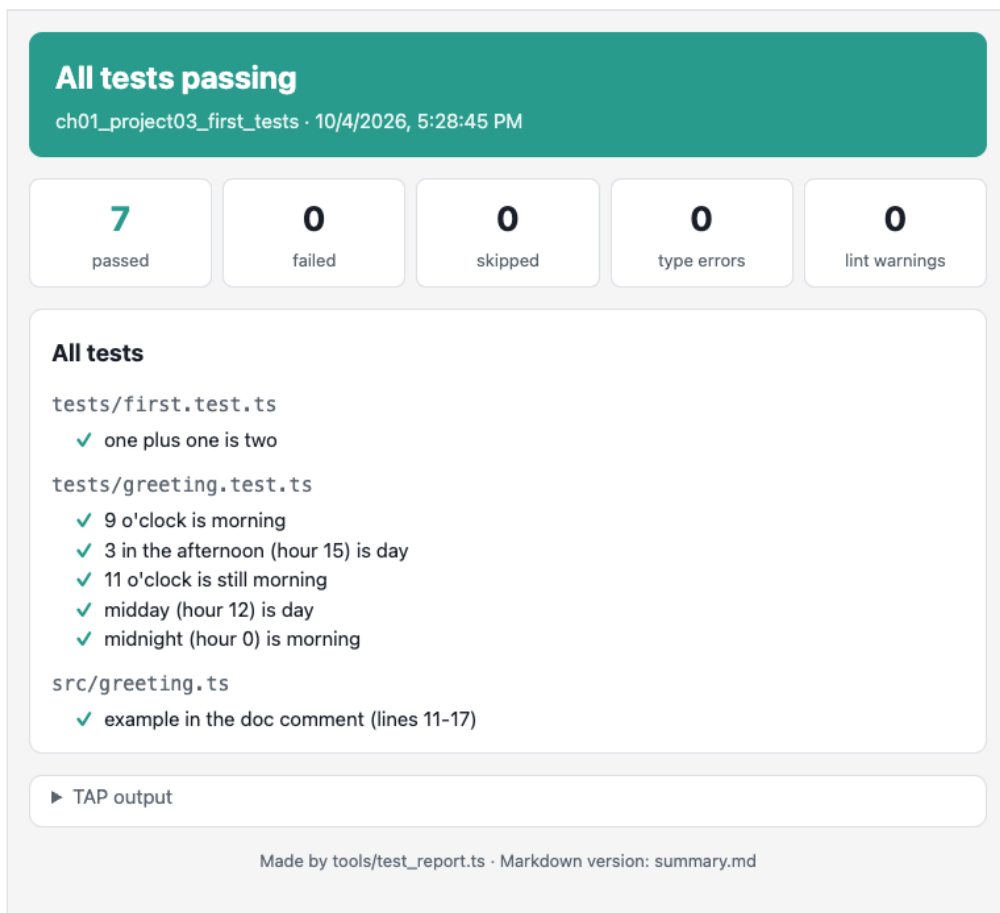
Deno.test("11 o'clock is still morning", () => {
  assertEquals(greeting(11), "Good morning");
});

Deno.test("midday (hour 12) is day", () => {
  assertEquals(greeting(12), "Good day");
});
```

A test file has to import the function it tests, just as `main.ts` does. With the example and the test files together, the console shows every check, each test by name:

```
TAP version 14
# tests/first.test.ts
ok 1 - one plus one is two
# tests/greeting.test.ts
ok 2 - 9 o'clock is morning
ok 3 - 3 in the afternoon (hour 15) is day
ok 4 - 11 o'clock is still morning
ok 5 - midday (hour 12) is day
ok 6 - midnight (hour 0) is morning
# src/greeting.ts
ok 7 - example in the doc comment (lines 11-17)
1..7

Tests: 0 failed, 7 passed, 0 skipped, 0 type errors, 0 lint warnings -> test_output/index.html
```



The test report, with the example and the tests

Examples or tests? This book uses both:

- a **doc comment example** shows how to use a function - one or two typical calls - and Deno checks that it stays true. It is documentation that cannot go out of date
- **test files** hold everything else: the edges, the unusual cases, many checks, each with a name

Why the edges matter

Look at which hours the test file checks. 9 and 15 are ordinary morning and afternoon hours, like the example's. 11 and 12 are the **edges**: the last hour of the morning and the first hour of the day.

Try it: change `MIDDAY` to `13`, and save. Now midday says "Good morning" - a bug. The example still passes (9 is still morning, and 15 is still day), but one test fails:

```
not ok 5 - midday (hour 12) is day
---
message: |-
  AssertionError: Values are not equal.

    [Diff] Actual / Expected

  - Good morning
  + Good day
    at tests/greeting.test.ts:21:3
at: tests/greeting.test.ts:20
...
```

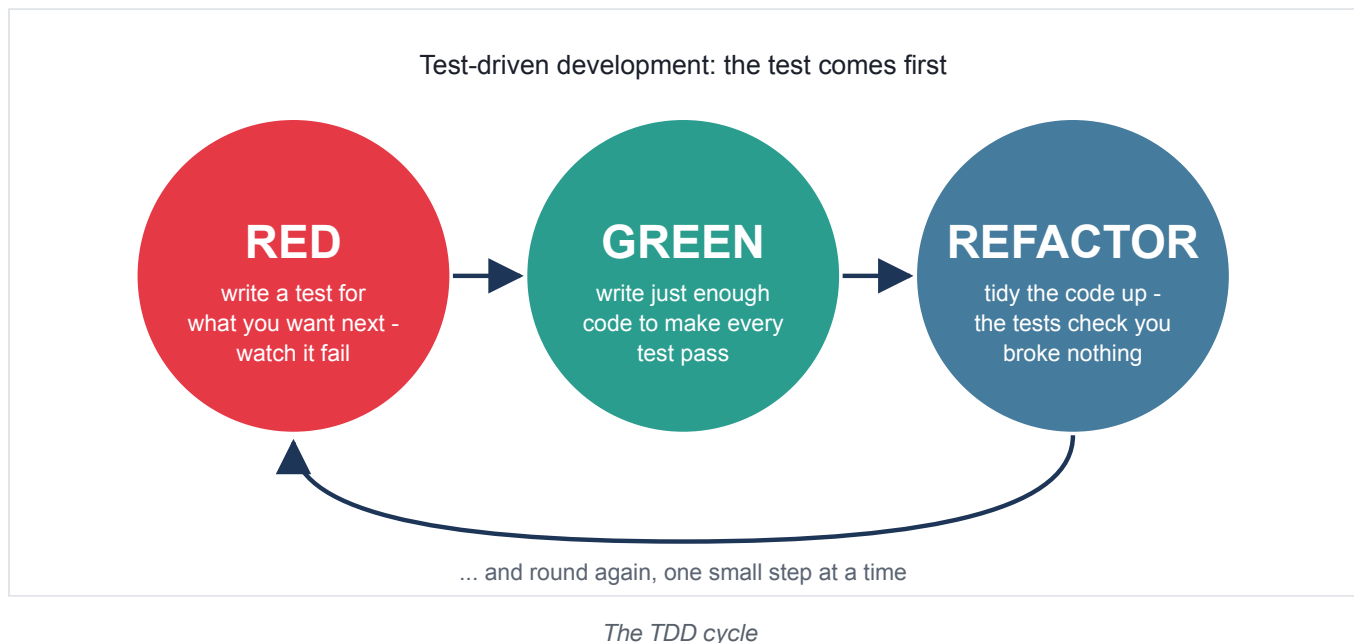
This time the failure has a name, saying exactly what should be true, and the line of the test that failed. Only the test at the edge caught the bug - which is why you should always test both sides of an edge. Put `MIDDAY` back to 12, and save.

Test-driven development

So far the tests were written *after* the code, to check it. **Test-driven development** (TDD) turns that round: you write a test for what the code *should* do **first**, watch it fail, and only then write the code. It sounds back to front, but it has big advantages:

- you have to decide exactly what the code should do before you write it
- every piece of code has a test from the moment it exists
- you only write code that a test needs - nothing extra
- you always know what to do next: make the failing test pass

TDD goes round a short cycle, many times:



1. **Red** - write one test for the next small thing the code should do. Run it, and watch it fail. (If it passes before you have written anything, the test is not testing what you think.)
2. **Green** - write just enough code to make every test pass. Not more.
3. **Refactor** - tidy the code up: better names, named constants, less repetition. The tests tell you straight away if tidying broke anything.

From Chapter 4 on, every project in this book is built this way. Here is a first, small example.

Project 4: Age groups, test first

We want a function, `ageGroup`, that takes an age in years and says which group it is in:

- under 12: **child**

- 13 to 19: **teenager**
- 20 and over: **adult**
- below 0 or above 150: **invalid** - nobody has an age like that

Before any code, look hard at that description. What group is a 12 year old? "Under 12" stops before 12, and "13 to 19" starts after it. Descriptions like this often have gaps, and writing tests makes you find them, because a test needs an exact answer. We will decide that 12 is still a child, so the groups are 0-12, 13-19 and 20-150.

Step 1 - red. Make a test file, `tests/age_group.test.ts`, with one test:

```
import { assertEquals } from "@std/assert";
import { ageGroup } from "../src/age_group.ts";

Deno.test("a 5 year old is a child", () => {
  assertEquals(ageGroup(5), "child");
});
```

Save. It fails - there is no `src/age_group.ts` yet, so the tests cannot even run. The console says `Module not found "src/age_group.ts"`, and the report's banner says "The tests could not run". That counts as red.

Step 2 - green. Write *just enough* code to pass. That is surprisingly little:

`src/age_group.ts`

```
export function ageGroup(age: number): string {
  return "child";
}
```

It looks like cheating - but every test passes, and that is all we can ask of the code so far. The linter warns that `age` is never used; that is fine for now, the next test will need it.

Step 3 - red again. Add a test for the next group:

```
Deno.test("a 15 year old is a teenager", () => {
  assertEquals(ageGroup(15), "teenager");
});
```

Save. The new test fails, because `ageGroup` always says "child":

1 test failing

ch01_project04_age_groups · 10/3/2026, 2:27:21 PM

1

passed

1

failed

0

skipped

0

type errors

0

lint warnings

Failing tests

✗ a 15 year old is a teenager

tests/age_group.test.ts:8

AssertionError: Values are not equal.

[Diff] Actual / Expected

- child

+ teenager

at tests/age_group.test.ts:9:3

All tests

tests/age_group.test.ts

✓ a 5 year old is a child

✗ a 15 year old is a teenager

▶ TAP output

Made by tools/test_report.ts · Markdown version: summary.md

The report after step 3: one test failing

Step 4 - green. Now the function has to look at the age:

```
export function ageGroup(age: number): string {
  if (age <= 12) {
    return "child";
  }
  return "teenager";
}
```

Both tests pass. **Then round again**, one group at a time: a test that 40 is an adult (red), an `if` for teenagers so that older ages fall through to "adult" (green); a test that -1 is invalid (red), a check at the top of the function (green); a test that 151 is invalid (red), and so on. Finally, add tests for the **edges** of every group - 0, 12, 13, 19, 20 and 150 - and check they all pass.

Refactor. With every test green, tidy up. The numbers 12, 19 and 150 are "magic numbers"; give them names. Here is the finished function, as it is in project 4:

```
src/age_group.ts
```

```

const OLDEST_CHILD = 12;
const OLDEST_TEENAGER = 19;
const OLDEST_POSSIBLE = 150;

export function ageGroup(age: number): string {
  if (age < 0 || age > OLDEST_POSSIBLE) {
    return "invalid";
  }
  if (age <= OLDEST_CHILD) {
    return "child";
  }
  if (age <= OLDEST_TEENAGER) {
    return "teenager";
  }
  return "adult";
}

```

(`||` means "or", and `&&` means "and", as in Java.) Save after the refactor: still all green, so the tidying broke nothing.

Last, give the finished function a doc comment with an example, showing how to use it:

`src/age_group.ts`

```

/**
 * The age group for an age in years: "child" (0 to 12), "teenager" (13 to 19), "adult" (20 to
 * 150),
 * or "invalid" for an age that cannot be right (below 0, or above 150).
 *
 * @example
 * ```ts
 * import { assertEquals } from "@std/assert";
 *
 * assertEquals(ageGroup(8), "child");
 * assertEquals(ageGroup(16), "teenager");
 * assertEquals(ageGroup(40), "adult");
 * ```
 */

```

The example shows one typical age from each group; the edges, and the invalid ages, stay in the test file, where each has a name. The project's `tests/age_group.test.ts` has all eleven tests, and its page shows a table of example ages:

Age Groups

Age	Group
5	child
12	child
13	teenager
19	teenager
20	adult
64	adult
150	adult
-1	invalid
200	invalid

The Age Groups project

Note - Small steps are the point. If you write a big piece of code and ten tests fail, it is hard to know where the mistake is. If you change two lines and one test fails, you know exactly where to look.

Lists of things

Project 5: A bulleted list

The Planets

8 planets

- Mercury
- Venus
- Earth
- Mars
- Jupiter
- Saturn
- Uranus
- Neptune

The Bullet List project

src/main.ts

```
// An array of strings. The type string[] means "array of string" – like Java's String[], but it
// can grow and shrink, like an ArrayList.
const PLANETS: string[] = ["Mercury", "Venus", "Earth", "Mars", "Jupiter", "Saturn", "Uranus",
"Neptune"];

const list = document.querySelector("#planets");
if (list !== null) {
  // for ... of goes through the array's values in order – Java's for (String planet : planets)
  for (const planet of PLANETS) {
    const item = document.createElement("li");
    item.textContent = planet;
    list.appendChild(item);
  }
}

const count = document.querySelector("#count");
if (count !== null) {
  count.textContent = `${PLANETS.length} planets`;
}
```

- An array is written in square brackets, and its type is `string[]`.
- `for (const planet of PLANETS)` takes each value in turn. Note the word `of`. There is also a `for ... in` loop, which does something different (it goes through the *positions*, as strings), so `for ... of` is the one you want.
- `document.createElement("li")` makes a new list item element; `list.appendChild(item)` puts it into the list on the page.
- An array's size is `PLANETS.length` – a property, not a method, so there are no brackets. (Java arrays use `.length` too; Java lists use `.size()`.)

Exercise 1.4 - The data in a file of its own

Move the array into its own file, `src/planets.ts`, and import it into `main.ts`. Try it before reading on.

Here is one way:

`src/planets.ts`

```
export const PLANETS: string[] = ["Mercury", "Venus", "Earth", "Mars", "Jupiter", "Saturn", "Uranus", "Neptune"];
```

`src/main.ts`

```
import { PLANETS } from "../planets.ts";
```

A constant can be exported and imported just like a function. Now the data and the code that shows it are separate.

Exercise 1.5 - The data as JSON

The planets are *data*, not code. Data is often kept in a **JSON** file (JavaScript Object Notation), the format almost every web service uses to send data. A JSON file holding an array of strings looks like the array in TypeScript, with double quotes:

`src/planets.json`

```
["Mercury", "Venus", "Earth", "Mars", "Jupiter", "Saturn", "Uranus", "Neptune"]
```

Move the array into `src/planets.json`, delete `planets.ts`, and change `main.ts` to use the JSON file. Here is how - and it is project 6:

`src/main.ts`

```
import planets from "../planets.json" with { type: "json" };

// TypeScript works out the type of the data from the file itself. Writing the type here makes it
// clear, and checks the file really does hold an array of strings.
const PLANETS: string[] = planets;
```

`with { type: "json" }` says "this file is JSON: read it, and **parse** it". To parse is to turn the text of the file into real values - here, an array of strings. When the project is built, the bundler reads `planets.json`, parses it, and puts the array into `dist/app.js`. So the page still needs no server, and you can edit `planets.json`, save, and refresh to see the new list.

Note - Web pages served from a web server usually *fetch* JSON while they run, and parse it with `JSON.parse(text)`, which turns a string of JSON into values. Project 6's tests show `JSON.parse` at work. A page opened straight from the disk is not allowed to fetch files, which is why this book reads JSON at build time instead.

tests/planets.test.ts

```
import { assertEquals } from "@std/assert";
import planets from "../src/planets.json" with { type: "json" };

Deno.test("JSON.parse turns JSON text into an array", () => {
  const text = '["Mercury", "Venus"]';
  const values: string[] = JSON.parse(text);
  assertEquals(values.length, 2);
  assertEquals(values[1], "Venus");
});

Deno.test("planets.json holds eight planets", () => {
  assertEquals(planets.length, 8);
});
```

Tests can check data as well as code: if someone deletes a planet from the file by mistake, the second test fails.

Project 7: Putting it together

The last project combines everything. You should find nothing in it new.

Good afternoon, student!

It is 2:27:19 PM.

5 things about TypeScript

- TypeScript is JavaScript with types added.
- The types are checked, then removed - browsers run plain JavaScript.
- There is one type for numbers, number - no int, long, float or double.
- let declares a variable that can change; const declares one that cannot.
- Strings are compared with ===, not with an equals() method.

The Hello, Page project

src/greeting.ts has a three-way version of the greeting, with a doc comment example for each function, and tests for the edges:

src/greeting.ts

```

const MORNING_STARTS = 5;
const AFTERNOON_STARTS = 12;
const EVENING_STARTS = 18;

/**
 * The part of the day for an hour from 0 to 23: "morning", "afternoon" or "evening".
 *
 * @example
 * ```ts
 * import { assertEquals } from "@std/assert";
 *
 * assertEquals(partOfDay(9), "morning");
 * assertEquals(partOfDay(20), "evening");
 * ```
 */
export function partOfDay(hour: number): string {
  if (hour >= MORNING_STARTS && hour < AFTERNOON_STARTS) {
    return "morning";
  }
  if (hour >= AFTERNOON_STARTS && hour < EVENING_STARTS) {
    return "afternoon";
  }
  return "evening";
}

/**
 * A greeting for `name` at `hour`.
 *
 * @example
 * ```ts
 * import { assertEquals } from "@std/assert";
 *
 * assertEquals(greeting("Ada", 9), "Good morning, Ada!");
 * ```
 */
export function greeting(name: string, hour: number): string {
  return `Good ${partOfDay(hour)}, ${name}!`;
}

```

`greeting` takes two parameters and calls `partOfDay` inside a template literal. `src/facts.ts` exports an array of strings, and `main.ts` imports both files, shows the greeting and the time, and builds a list of facts - the code from projects 3 and 5, side by side. Open `dist/app.js` too: all three files are now in one, and every type has gone.

When things go wrong

Three kinds of things go wrong, and each shows up in a different place:

What went wrong	Where you see it
a type error (e.g. a string where a <code>number</code> should be)	the console, straight after saving, and the report's "type errors"
a failing test (the code does the wrong thing)	the console's TAP output, and the report
a lint warning (a likely mistake, such as a variable that is never used)	the console's summary line, and the report's "lint warnings"
a problem while the page runs (e.g. an <code>id</code> that does not exist)	the browser: open <code>dist/index.html</code> in a web browser and its developer tools (F12, or Cmd+Option+I on a Mac), and look at the Console tab

The first three are found for you, every time you save. That is why we use TypeScript, tests and the linter: they catch problems before you ever look at the page.

Java and TypeScript, so far

Java	TypeScript
<code>final String NAME = "Ada";</code>	<code>const NAME = "Ada";</code>
<code>int hour = 9;</code>	<code>let hour = 9;</code> or <code>let hour: number = 9;</code>
<code>"Hi " + name + "!"</code>	<code>`Hi \${name}!`</code>
<code>a.equals(b)</code> (strings), <code>a == b</code> (numbers)	<code>a === b</code> for both
<code>static String greeting(int hour) { ... }</code> in a class	<code>function greeting(hour: number): string { ... }</code> in any file
<code>public</code> class or method, <code>import</code>	<code>export</code> , <code>import { name } from "../file.ts";</code>
<code>String[] planets = { ... };</code>	<code>const planets: string[] = [...];</code>
<code>for (String p : planets)</code>	<code>for (const p of planets)</code>
<code>planets.length</code> (array), <code>list.size()</code>	<code>planets.length</code>
a Javadoc comment, with an example nobody checks	a doc comment whose <code>@example</code> Deno runs and checks
JUnit <code>@Test</code> and <code>assertEquals(expected, actual)</code>	<code>Deno.test("name", () => { ... })</code> and <code>assertEquals(actual, expected)</code>
<code>javac</code> , then <code>java</code>	save; <code>deno task dev</code> does the rest; refresh the preview

Summary

- TypeScript is JavaScript plus types. Types are checked, then removed; the browser runs plain JavaScript

- Opening a project in Celbridge starts `deno task dev`, which type checks, builds `dist/`, runs the tests and writes `test_output/` - and does it again every time you save
- Change `src/`, `public/` and `tests/`; look at `dist/index.html` (press refresh) and the report
- `main.ts` finds elements with `querySelector`, checks they are not `null`, and changes their `textContent`
- `const` for values that never change, `let` for ones that do; types after names; one `number` type; template literals with `${...}`; `===` for comparing
- functions can live in any file; `export` them, and `import` them where they are needed
- an `@example` in a doc comment shows how to use a function, and Deno checks it; a comment on its own checks nothing
- a test file holds the edges and the unusual cases: each `Deno.test` names what should be true and checks it with `assertEquals(actual, expected)`; test both sides of every edge
- TDD: **red** (a failing test), **green** (just enough code), **refactor** (tidy up) - in small steps
- arrays (`string[]`), `for ... of`, and data kept in JSON files

Challenges

Each challenge says which project to start from. Work on a copy if you want to keep the original (copy the whole project folder). From challenge 2 on, write the test first.

1. Make it yours

Start from `ch01_project07_hello_page`. Greet yourself by name, add a sixth fact of your own, and change the page's colours in `public/styles.css` (the colours are at the top, in `:root`). Check that the heading now says "6 things about TypeScript" - without you changing it. Why did it change?

2. Good night

Start from `ch01_project07_hello_page`. At 2 o'clock in the morning the page says "Good evening". Make hours 22 to 4 say "Good night" instead. Test first: add a test that `partOfDay(2)` is `"night"`, see it fail, then change `partOfDay`. Then add tests for the edges - 21, 22, 4 and 5 - and make sure they all pass.

3. Break it on purpose

Start from `ch01_project07_hello_page`. Make each of these mistakes, one at a time. For each one, write down **where** it showed up (console, report, page) and **what the message said**, then undo it:

1. in `main.ts`, change `const hour: number = now.getHours();` to `const hour: number = "nine";`
2. add a line `let unused = 1;` to `main.ts`
3. in `greeting.ts`, change `AFTERNOON_STARTS` to `13`
4. in `main.ts`, change `"#facts"` to `"#fact"`

Which ones did nothing catch? Why not, and how could you find them?

4. Seniors

Start from `ch01_project04_age_groups`. Add a fifth group: people aged 65 and over are **"senior"**. Test first, one step at a time: a test that 70 is a senior (red), then the code (green). Then the edges: 64 and 65. Which existing test now fails, and why? Fix it so that it still tests what it was meant to.

5. Planets with facts

Start from `ch01_project06_list_from_json`. Change `planets.json` so that each planet has a name and a fact, like this:

```
[
  { "name": "Mercury", "fact": "the smallest planet" },
  { "name": "Venus", "fact": "the hottest planet" }
]
```

and show each one as **"Mercury - the smallest planet"**. Update the tests first: for example, that the first planet's name is `"Mercury"` (`planets[0].name`).

Hint: the type of the array is now `{ name: string; fact: string }[]`. To make part of the text bold, set the item's `innerHTML` instead of `textContent`: `item.innerHTML = `${planet.name} - ${planet.fact}`;`

6. Day and night colours

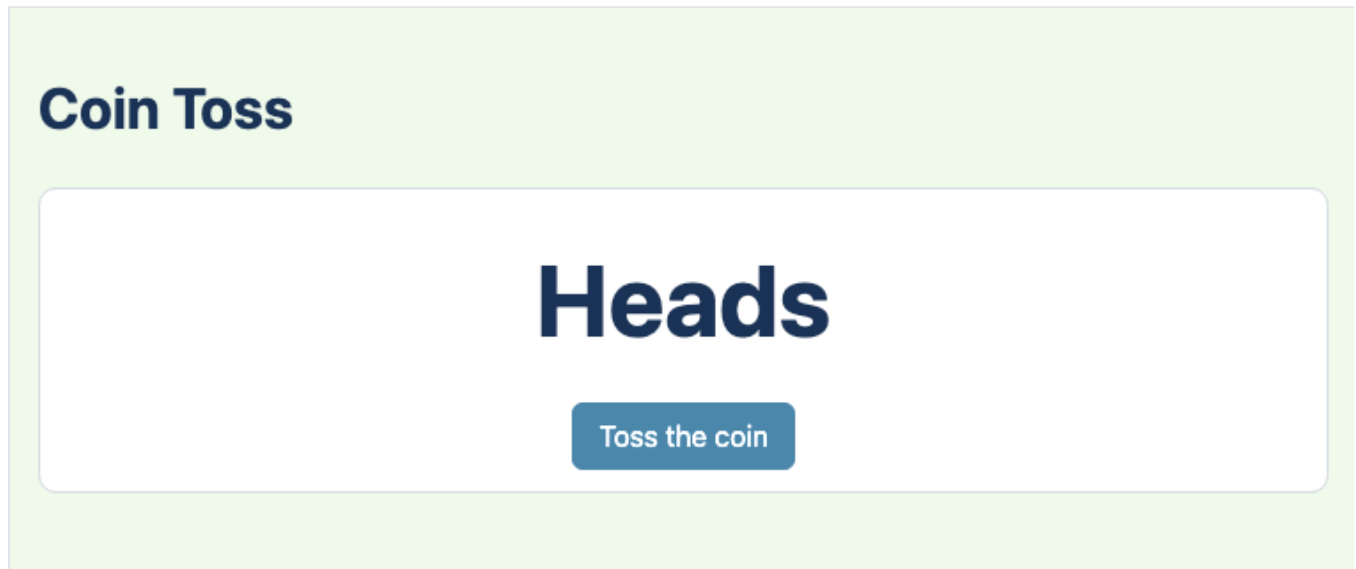
Start from `ch01_project07_hello_page`. Make the page's colours change with the time of day: a light theme in the day and a dark theme at night. Write a function `themeFor(hour: number): string` in a new file `src/theme.ts`, that returns `"day"` from 7 to 18 and `"night"` otherwise, test first: the edges in a test file, and an example in its doc comment showing one day hour and one night hour. In `main.ts`, set the page's class with `document.body.className = themeFor(hour);`, and in `styles.css` add rules for `body.day` and `body.night`.

Hint: the colours are CSS variables, so the night rule can just change them: `body.night { --bg: #16191f; --card: #1f232b; --text: #e8eae2; }`. To test the night theme without waiting for night, change `hour` in `main.ts` for a moment - or, better, trust your tests.

Next: [Chapter 2 - Events: clicks, keys and timers](#)

Chapter 2 - Events: clicks, keys and timers

The pages in Chapter 1 did their work once, when they loaded, and then sat still. Real pages respond: you click a button, type in a box, or wait for a clock to tick, and the page changes. This chapter explains how that works - the browser's **event model** - and how to write TypeScript that responds to events. Along the way you write your first TypeScript **class**.



The Coin Toss project

What you will learn

- how event-driven programs differ from programs that run from top to bottom
- the browser's event loop: events, the event queue, and listener functions
- responding to clicks with `addEventListener`, and passing a function without calling it
- a first TypeScript class, next to the same class in Java
- the pattern used all through this book: an object holds the state, events change it, and a `render` function shows it
- timer events: `setInterval`, `clearInterval` and `setTimeout`
- typing and key presses: the `input` and `keydown` events, and the event object
- keeping the parts you cannot predict (random numbers, the time) out of the code you test

The projects

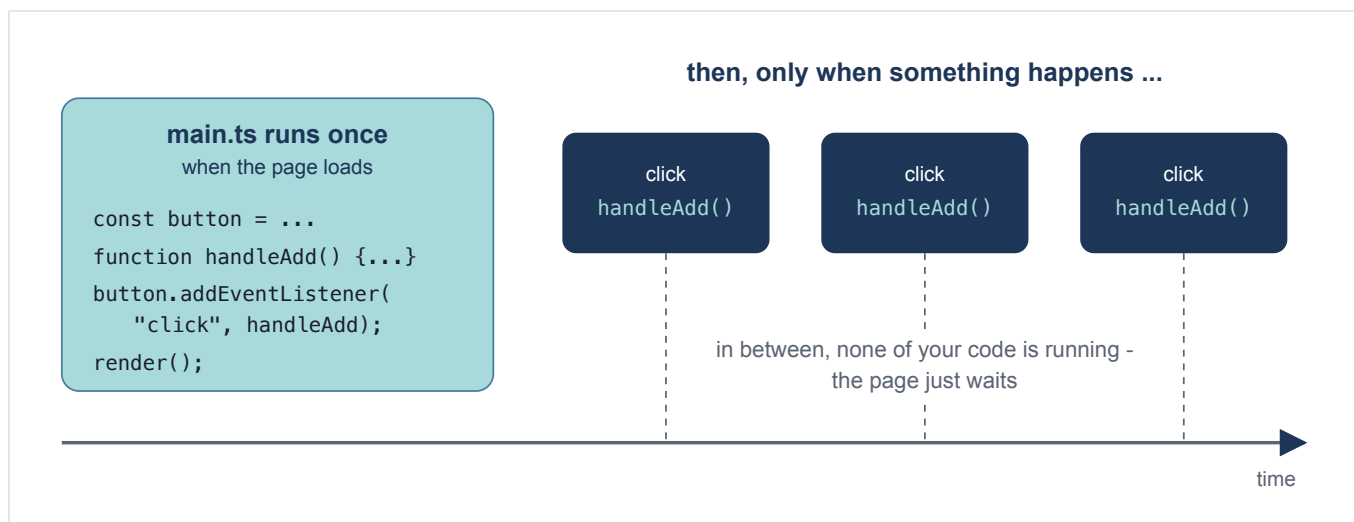
Project	What it shows
ch02_project01_coin_toss	a click listener; a random number passed into a function so it can be tested
ch02_project02_click_counter	a first class, <code>Counter</code> ; two buttons; a <code>render</code> function
ch02_project03_ticking_clock	a timer that ticks every second, and a button that pauses and resumes it
ch02_project04_character_count	the <code>input</code> and <code>keydown</code> events, and the event object

Programs that wait

The Java programs you started with ran from the top of `main` to the bottom, and then stopped. If they needed input, they stopped and waited for it (`scanner.nextLine()`), and nothing else could happen in the meantime.

A web page cannot work like that. The user might click any button, in any order, or type, or do nothing at all while a clock ticks. So a page is **event-driven**: its code runs when something happens. An **event** is anything that happens that a page might care about: a click, a key press, the text in a box changing, a timer going off, the page finishing loading.

This changes how `main.ts` works. It still runs from top to bottom - **once**, when the page loads. Its job is to set things up: find the elements, and tell the browser which function to call when each event happens. Then it finishes. From then on, your code only runs when an event calls it.

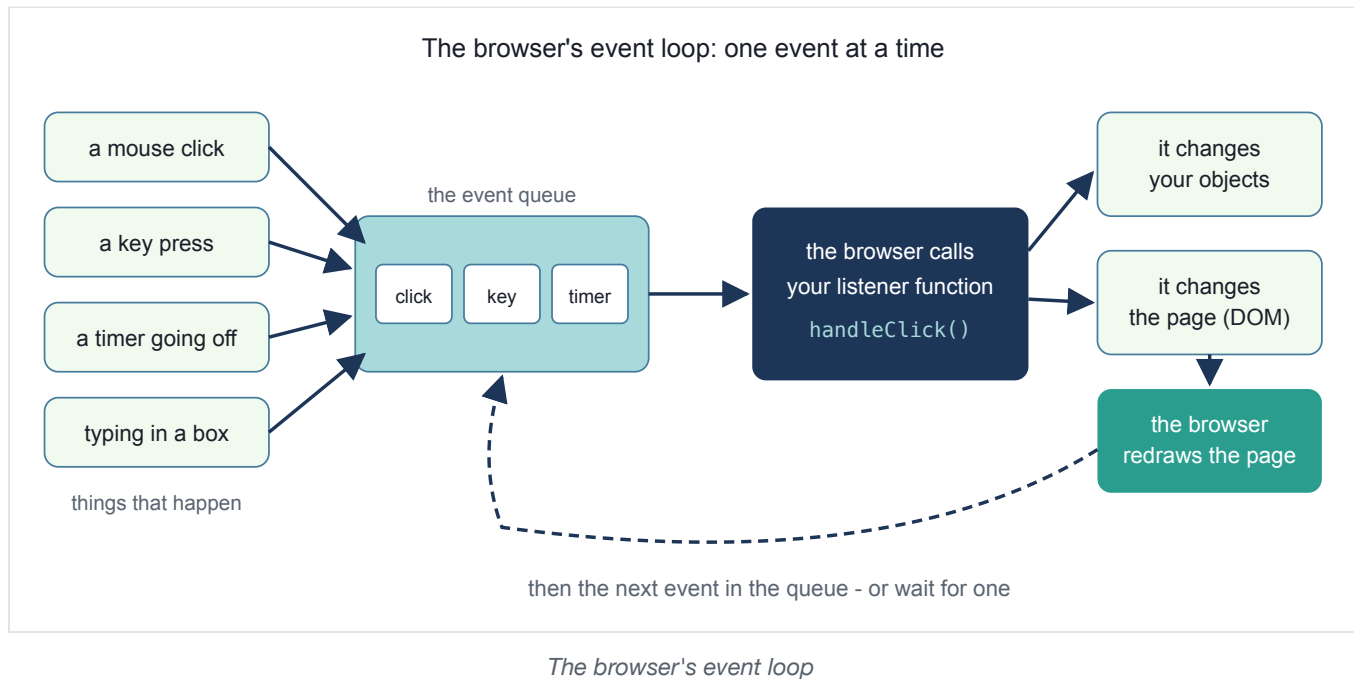


main.ts runs once; listeners run later

If you have written a Java Swing or JavaFX program with buttons, you have seen this before: you built the window, attached `ActionListeners` or `setOnAction` handlers, and the framework called them when the user clicked.

The event loop

Inside the browser, events are handled one at a time:



1. Things happen - the user clicks, a timer goes off - and each one is put at the back of a queue of events.
2. The browser takes the event at the front of the queue, and calls the function your code registered for it - the **listener** (also called a handler).
3. Your listener runs to the end. It changes your objects, and the page.
4. The browser redraws the page, so the user sees the change. Then it takes the next event, or waits for one.

Two things follow from this:

- **Only one listener runs at a time.** You never have to worry about two clicks being handled at once.
- **A slow listener freezes the page.** While a listener runs, nothing else can happen - the page cannot even redraw. So listeners should do their job quickly and finish. (In this book they always do; the risk comes with long calculations or loops that never end.)

Project 1: Responding to a click

Coin Toss

Heads

Toss the coin

The Coin Toss project

src/main.ts

```
import { coinFace } from "./coin.ts";

const tossButton = document.querySelector<HTMLButtonElement>("#toss");
const result = document.querySelector<HTMLInputElement>("#result");

/** Called by the browser each time the button is clicked. */
function handleToss(): void {
    // Math.random() gives a different number from 0 up to (but not including) 1 every time.
    const face = coinFace(Math.random());
    if (result !== null) {
        result.textContent = face;
    }
}

// "When this button is clicked, call handleToss." Note: handleToss, not handleToss() -
// we hand over the function itself, for the button to call later.
if (tossButton !== null) {
    tossButton.addEventListener("click", handleToss);
}
```

`querySelector<HTMLButtonElement>("#toss")` has something new in angle brackets: the kind of element we expect to find. It tells TypeScript that `tossButton` is a button, so it knows what a button can do (for example, be disabled). Each kind of element has a type: `HTMLButtonElement`, `HTMLInputElement`, `HTMLTextAreaElement`, and `HTMLInputElement` for any element at all.

`addEventListener("click", handleToss)` is the key line. It says: *when a "click" event happens on this button, call `handleToss`*. Look carefully: it says `handleToss`, **not** `handleToss()`.

- `handleToss()` - with brackets - **calls** the function, now, and gives its result (nothing).
- `handleToss` - without brackets - is **the function itself**, as a value, handed over to be called later.

In TypeScript a function is a value, like a number or a string: you can store it in a variable, or pass it to another function. That is how a listener is registered. If you write the brackets by mistake, TypeScript

refuses, with an error that ends `Argument of type 'void' is not assignable to parameter of type ...`.

`handleToss` has the return type `void`, as in Java: it returns nothing. It is called by the browser, which ignores any result anyway.

Try it - open `ch02_project01_coin_toss` and toss the coin a few times.

Keeping random numbers out of the tested code

How would you test a coin toss? The answer is random, so a test cannot know what to expect. The trick is to keep the random part out of the function that decides:

`src/coin.ts`

```
const HALF = 0.5;

/** "Heads" for a random number below 0.5, and "Tails" for 0.5 and above (Math.random() gives 0 to 1). */
export function coinFace(random: number): string {
  if (random < HALF) {
    return "Heads";
  }
  return "Tails";
}
```

`main.ts` makes the random number, with `Math.random()`, and passes it in. The tests pass in numbers they choose, so they know the right answer:

`tests/coin.test.ts`

```
Deno.test("a small random number gives heads", () => {
  assertEquals(coinFace(0.2), "Heads");
});

Deno.test("exactly a half gives tails", () => {
  assertEquals(coinFace(0.5), "Tails");
});
```

The same idea works for anything you cannot predict - the time of day was the same problem in Chapter 1, and `greeting(hour)` solved it the same way. Book 2 takes it further.

Project 2: A first class

Click Counter

3

3 clicks

Click me

Reset

The Click Counter project

The second project counts your clicks. The counting is done by a **class**. Here it is, next to the same class in Java:

src/Counter.ts

```
export class Counter {
  // A field, with its type. It starts at zero for every new Counter.
  private count: number = 0;

  /** Adds one to the count. */
  public increment(): void {
    this.count++;
  }

  /** Sets the count back to zero. */
  public reset(): void {
    this.count = 0;
  }

  /** The current count. */
  public getCount(): number {
    return this.count;
  }
}
```

Counter.java

```

public class Counter {
    private int count = 0;

    public void increment() {
        count++;
    }

    public void reset() {
        count = 0;
    }

    public int getCount() {
        return count;
    }
}

```

They are nearly the same. The differences:

- `export class` instead of `public class` - `export` is what lets other files import it
- types come after names: `count: number`, `getCount(): number`
- a method starts with its visibility and name - not with its return type
- **this. is compulsory.** In Java you can write `count++` inside a method, and Java works out you mean the field. In TypeScript you must write `this.count++`. Forgetting `this.` is the most common mistake in the first weeks; the error is `Cannot find name 'count'. Did you mean the instance member 'this.count'?`
- `public` is the default in TypeScript, so it could be left out. This book always writes it, so that visibility is clear

Objects are made with `new`, as in Java, and the class is imported from its file like anything else:

`src/main.ts`

```

import { Counter } from "./Counter.ts";
import { describeCount } from "./messages.ts";

const counter = new Counter();

```

The file is named after the class (`Counter.ts`), one class per file, as in Java. Chapter 5 covers classes properly: constructors, `toString`, and more.

Object, event, render

`src/main.ts`

```

const addButton = document.querySelector<HTMLButtonElement>("#add");
const resetButton = document.querySelector<HTMLButtonElement>("#reset");
const countDisplay = document.querySelector<HTMLDivElement>("#count");
const message = document.querySelector<HTMLParagraphElement>("#message");

/** Makes the page show the counter's current state. Called after every change. */
function render(): void {
  if (countDisplay !== null) {
    countDisplay.textContent = `${counter.getCount()}`;
  }
  if (message !== null) {
    message.textContent = describeCount(counter.getCount());
  }
}

function handleAdd(): void {
  counter.increment();
  render();
}

function handleReset(): void {
  counter.reset();
  render();
}

if (addButton !== null) {
  addButton.addEventListener("click", handleAdd);
}
if (resetButton !== null) {
  resetButton.addEventListener("click", handleReset);
}

render();

```

This is a pattern you will use in every project in the book:

1. an **object** (`counter`) holds the state
2. each **event** (a click) changes the object, and then calls `render()`
3. **`render()`** makes the page show the object's current state

The page never keeps its own copy of the count; it always shows what the `Counter` says. That keeps the two from getting out of step, and it means the logic - in `Counter` - can be tested without the page.

`render()` is also called once at the end of `main.ts`, so the page is right from the start.

`describeCount` is a plain function, in `src/messages.ts`, that turns the count into words ("No clicks yet", "1 click", "5 clicks"). Not everything has to be in a class.

`tests/Counter.test.ts` tests the class:


```

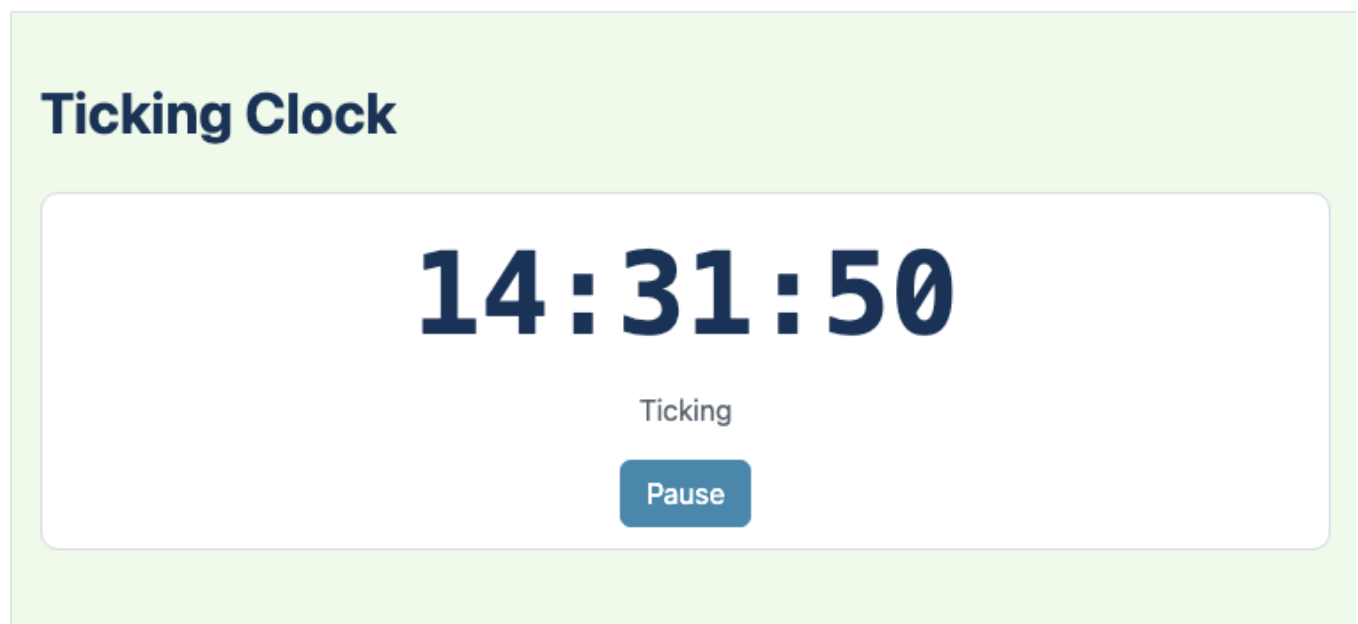
Deno.test("three increments make three", () => {
  const counter = new Counter();
  counter.increment();
  counter.increment();
  counter.increment();
  assertEquals(counter.getCount(), 3);
});

Deno.test("reset goes back to zero", () => {
  const counter = new Counter();
  counter.increment();
  counter.increment();
  counter.reset();
  assertEquals(counter.getCount(), 0);
});

```

Each test makes its own new `Counter`, so the tests cannot affect each other. The pattern - **arrange** (make the object), **act** (call methods), **assert** (check the result) - is the shape of most tests.

Project 3: Timer events



The Ticking Clock project

Not every event comes from the user. The browser can call a function **after a delay**, or **again and again**:

Function	What it does
<code>setTimeout(fn, ms)</code>	calls <code>fn</code> once, after <code>ms</code> milliseconds
<code>setInterval(fn, ms)</code>	calls <code>fn</code> every <code>ms</code> milliseconds, until stopped
<code>clearInterval(id)</code>	stops an interval; <code>id</code> is the number <code>setInterval</code> gave back

(1000 milliseconds is one second. As with `addEventListener`, you pass the function itself, without brackets.) A timer going off is just another event in the queue.

`src/main.ts`

```

import { formatTime } from "../time_format.ts";

const ONE_SECOND_MS = 1000;

// The id of the running interval, or null when the clock is paused. let, because it changes.
let timerId: number | null = null;

/** Shows the time now. Called once a second by the timer. */
function showTime(): void {
    const now = new Date();
    if (clock !== null) {
        clock.textContent = formatTime(now.getHours(), now.getMinutes(), now.getSeconds());
    }
}

function start(): void {
    showTime(); // straight away, rather than waiting a second
    timerId = setInterval(showTime, ONE_SECOND_MS);
    // ... the button now says "Pause"
}

function pause(): void {
    if (timerId !== null) {
        clearInterval(timerId);
        timerId = null;
    }
    // ... the button now says "Resume"
}

/** The button pauses a ticking clock, and resumes a paused one. */
function handlePause(): void {
    if (timerId === null) {
        start();
    } else {
        pause();
    }
}

```

`timerId: number | null` is a new kind of type: a **union**. It means "a number, or null". Java lets any object variable be `null`; TypeScript does not, unless the type says so. Here, `null` means "not running" - so `handlePause` can tell whether the clock is ticking by checking it. You will see a lot more of union types; they are one of TypeScript's most useful ideas.

`timerId` is declared **outside** the functions, with `let`, so that every function in the file can see it and change it, and it keeps its value between events.

Testing the formatting

The time itself cannot be tested (it keeps changing!), but turning hours, minutes and seconds into `"09:05:03"` can be:

```
src/time_format.ts
```

```
/** A number as two digits: 7 becomes "07", 42 stays "42". */
export function twoDigits(value: number): string {
  // padStart adds "0"s to the front of the string until it is 2 characters long.
  return `${value}`.padStart(2, "0");
}

/** For example, formatTime(9, 5, 3) is "09:05:03". */
export function formatTime(hours: number, minutes: number, seconds: number): string {
  return `${twoDigits(hours)}:${twoDigits(minutes)}:${twoDigits(seconds)}`;
}
```

``${value}`` turns the number into a string, and the string's `padStart` method does the rest. The tests check single digits, double digits, zero, and the last second of the day.

Project 4: Typing and keys

Character Count

Your message (press Escape to clear it)

Events are things that happen: a click, a key press, a timer going off.

69 characters left

The Character Count project

src/main.ts

```
import { charactersLeft, describeLeft, LIMIT } from "../characters.ts";

const box = document.querySelector<HTMLTextAreaElement>("#message");
const counter = document.querySelector<HTMLInputElement>("#counter");

/** Shows how many characters are left, in red when there are too many. */
function render(): void {
  if (box === null || counter === null) {
    return;
  }
  const left = charactersLeft(box.value, LIMIT);
  counter.textContent = describeLeft(left);
  counter.classList.toggle("bad", left < 0);
}

function handleInput(): void {
  render();
}

/** Escape clears the box. Every key press comes here, so check which key it was. */
function handleKey(event: KeyboardEvent): void {
  if (event.key === "Escape" && box !== null) {
    box.value = "";
    render();
  }
}

if (box !== null) {
  box.addEventListener("input", handleInput);
  box.addEventListener("keydown", handleKey);
}

render();
```

- The **input** event happens whenever the text in the box changes - typing, deleting, pasting. `box.value` is the text in it.
- The **keydown** event happens whenever a key is pressed. Its listener takes a parameter: the **event object**, a `KeyboardEvent`, which says which key it was (`event.key` is "Escape", "Enter", "a", ...). Every listener is given an event object; `handleInput` simply does not ask for it. A click listener could take a `MouseEvent`, with the mouse position.
- `counter.classList.toggle("bad", left < 0)` adds the CSS class `bad` (red text) when there are too many characters, and removes it otherwise.
- `render()` starts with a check: if either element is missing, `return` - there is nothing to do. After that check, TypeScript knows neither is `null`, so the rest of the function needs no more `if` s.

The counting is done by two small, tested functions in `src/characters.ts`: `charactersLeft(text, limit)` and `describeLeft(left)`, which gets "1 character left" and "3 too many" right.

Common events

Event	Happens when ...	Usually on
<code>click</code>	an element is clicked (or a button is pressed with the keyboard)	buttons, any element
<code>dblclick</code>	an element is double-clicked	any element
<code>input</code>	the value of a text box changes	<code><input></code> , <code><textarea></code>
<code>change</code>	a value is committed: a drop-down choice, a checkbox ticked	<code><select></code> , checkboxes
<code>keydown</code> , <code>keyup</code>	a key is pressed, released	text boxes, or <code>document</code> for the whole page
<code>mouseenter</code> , <code>mouseleave</code>	the mouse moves onto, off, an element	any element
<code>submit</code>	a form is submitted	<code><form></code>
<code>DOMContentLoaded</code>	the page has finished loading	<code>document</code>

Java and TypeScript

Java (Swing / JavaFX)	TypeScript in a web page
<code>button.addActionListener(listener) / button.setOnAction(...)</code>	<code>button.addEventListener("click", handleClick)</code>
an <code>ActionListener</code> object, or a lambda	a function, passed by name (without brackets)
<code>ActionEvent e</code> , <code>KeyEvent e</code>	<code>event: MouseEvent</code> , <code>event: KeyboardEvent</code>
<code>javax.swing.Timer</code> , <code>Timeline</code>	<code>setInterval(fn, ms)</code> , <code>setTimeout(fn, ms)</code>
any object variable can be <code>null</code>	only if its type says so: <code>number null</code>
<code>this.count</code> or just <code>count</code> in a method	<code>this.count</code> - always

Summary

- A web page is **event-driven**: `main.ts` runs once to set things up, and then your listener functions run when events happen
- The browser handles one event at a time, from a queue: it calls your listener, then redraws the page. Listeners should be quick
- `element.addEventListener("click", handleClick)` registers a listener. Pass the function - `handleClick` - do not call it
- TypeScript classes look like Java's, but `this.` is compulsory and types come after names
- **Object, event, render**: an object holds the state, each event changes it and calls `render()`, and `render()` shows the state on the page

- `setInterval/clearInterval` and `setTimeout` make timer events
- the `input` event fires as text changes; `keydown` gives a `KeyboardEvent` saying which key
- `number | null` is a union type: one or the other
- keep things you cannot predict (random numbers, the time) outside the functions you test, and pass them in

Challenges

Each challenge says which project to start from. Write the tests first.

1. Roll a die

Start from `ch02_project01_coin_toss`. Add a second button that rolls a six-sided die and shows 1 to 6. Write a function `dieFace(random: number): number` in a new file `src/die.ts`, test first: 0 gives 1, 0.999 gives 6, and the edges - just under 1/6 gives 1, exactly 1/6 gives 2.

Hint: `Math.floor(random * 6) + 1`. `Math.floor` rounds down, like casting to `int` in Java.

2. Take one away

Start from `ch02_project02_click_counter`. Add a **-1** button. Give `Counter` a `decrement()` method that takes one away - but never goes below zero. Tests first: "decrement takes one away" and "decrement never goes below zero".

3. Step size

Start from `ch02_project02_click_counter`. Add a drop-down list (`<select>`) so the user can count in 1s, 5s or 10s. Change `increment` to take the step: `increment(step: number = 1)`. The `= 1` is a **default parameter**: `increment()` with no argument still adds one. (Java would need two methods, `increment()` and `increment(int step)`; TypeScript does not allow two methods with the same name.) Tests first: "increment(5) adds five", and "increment() with no step still adds one".

Hint: a `<select>`'s chosen value is a string, in `select.value`; `Number(select.value)` turns it into a number. Read it inside the click listener, so you always get the latest choice.

4. Stopwatch

Start from `ch02_project03_ticking_clock`. Make a stopwatch: **Start**, **Stop** and **Reset** buttons, and a display that counts up in tenths of a second - "0:07.3", "1:02.0". Write and test a function `formatElapsed(ms: number): string` first: 0 is "0:00.0", 7300 is "0:07.3", 62000 is "1:02.0".

Hint: use `setInterval(tick, 100)`, and add 100 to the elapsed time on each tick. Inside `formatElapsed`: `Math.floor(ms / 60000)` is the minutes, `Math.floor(ms / 1000) % 60` the seconds, and `Math.floor(ms / 100) % 10` the tenths.

5. Getting close

Start from `ch02_project04_character_count`. Turn the counter orange when there are fewer than 20 characters left, and red when there are too many. Write a function `status(left: number): string` that returns `"ok"`, `"warning"` or `"over"`, test first (including the edges: 20, 19, 0 and -1), and use its result as the counter's CSS class.

Hint: `counter.className = status(left);` replaces all the element's classes with that one. Add `.ok`, `.warning` and `.over` rules to `styles.css`.

6. Ten-second challenge

Start from `ch02_project02_click_counter`. Turn the counter into a game: a **Start** button begins a 10-second round, clicks only count during the round, and when the time is up the click button is disabled and the page shows the score and the best so far, for example "Time's up! 23 clicks. Best: 31". Starting a new round resets the count but keeps the best.

Keep the timing in `main.ts`, but put the scoring logic in `Counter`, and test it: for example a `finishRound()` method that updates the best score, and a `getBest()` method. Tests such as "the best score starts at zero", "a higher score sets a new best", and "a lower score does not change the best".

Hint: `setTimeout(endRound, 10000);` calls `endRound` once, ten seconds later. `button.disabled = true;` greys a button out.

Next: [Chapter 3 - Arrow functions](#)

Chapter 3 - Arrow functions

You have written `() => { ... }` in every test since Chapter 1, and read it as "the test's code". It is an **arrow function**: TypeScript's version of a Java lambda. This chapter explains arrow functions properly, and shows what they make easy: writing listeners right where they are used, working through arrays with `map`, `filter` and `reduce`, and choosing which function to run while the program is running.

Marks Table

Sort by name

Sort by mark

☒ Passes only

8 students · average 60.9 · top: Emma (81)

Name	Mark	Grade
Emma	81	A
Aoife	72	A
Isla	68	B
Chloe	65	B
Grace	59	C
Darragh	54	C
Finn	47	D
Jack	41	D

The Marks Table project

What you will learn

- how to write an arrow function, and how it compares with a Java lambda and a `function`
- testing numbers with decimal places, with `assertAlmostEquals`
- arrow functions as event listeners, and how each one remembers the variables around it
- `?.`, a shorter way to write "if it is not null"
- the `this` trap: why a method must not be passed as a listener on its own
- the array methods `map`, `filter`, `reduce`, `find` and `toSorted`
- giving a type a name with `type`, and function types such as `(value: number) => number`
- storing functions in objects, and choosing one while the program runs

The projects

Project	What it shows
ch03_project01_temperatures	small functions written as arrow functions, and <code>assertAlmostEquals</code>
ch03_project02_scoreboard	buttons made from an array, each with an arrow-function listener that remembers its team and points
ch03_project03_marks_table	<code>map</code> , <code>filter</code> , <code>reduce</code> , <code>find</code> and <code>toSorted</code> on data from a JSON file
ch03_project04_unit_converter	functions stored in objects, function types, and choosing a function from a drop-down list

Functions are values

Chapter 2 showed that a function is a value. `addEventListener("click", handleToss)` hands the function `handleToss` to the button, to call later; `handleToss()`, with brackets, would call it straight away. A value can be stored in a variable, put in an array or an object, passed to another function, and returned from one - and so can a function.

Arrow functions make this easy, because they let you write a function anywhere a value can go, without giving it a name first.

Project 1: Arrow functions

Temperatures		
Celsius	Fahrenheit	Feels
-10 °C	14 °F	freezing
0 °C	32 °F	cold
15 °C	59 °F	mild
22 °C	71.6 °F	warm
37 °C	98.6 °F	hot
100 °C	212 °F	hot

The Temperatures project

Here is the same function written three ways:

The same function, three ways

```
function toFahrenheit(celsius: number): number { return celsius * 9 / 5 + 32; }
```

a function declaration (Chapter 1)

```
const toFahrenheit = (celsius: number): number => { return celsius * 9 / 5 + 32; };
```

an arrow function with a { block } body - it needs return

```
const toFahrenheit = (celsius: number): number => celsius * 9 / 5 + 32;
```

an arrow function with an expression body - the value after => is returned

```
Java lambda:    celsius -> celsius * 9 / 5 + 32
TypeScript arrow: (celsius) => celsius * 9 / 5 + 32    => (equals), not -> (minus)
```

The same function, three ways

src/temperature.ts

```
/** Celsius to Fahrenheit. A one-line arrow function: the value after => is returned. */
export const toFahrenheit = (celsius: number): number => celsius * 9 / 5 + 32;

/** Fahrenheit to Celsius. */
export const toCelsius = (fahrenheit: number): number => (fahrenheit - 32) * 5 / 9;
```

Read `toFahrenheit` from left to right:

- `const toFahrenheit =` - a constant, as in Chapter 1. Its value is a function
- `(celsius: number)` - the parameters, in brackets, with their types
- `: number` - the return type (optional - TypeScript can work it out - but this book writes it)
- `=>` - the arrow: "gives". Read the whole thing as "celsius gives celsius times 9 over 5, plus 32"
- `celsius * 9 / 5 + 32` - the **body**. When the body is a single expression, its value is returned: no return, no braces

When a function needs more than one line, give it a body in braces - and then it needs `return`, just like a function:

```
/** How a temperature in Celsius feels. An arrow function with a { block } body needs a return. */
export const describe = (celsius: number): string => {
  if (celsius < 0) {
    return "freezing";
  }
  if (celsius < 10) {
    return "cold";
  }
  // ...
  return "hot";
};
```

(Note the `;` after the closing brace: the whole line is a `const` declaration.)

An arrow function stored in a `const` is called, exported and imported exactly like a function declared with `function`; `main.ts` cannot tell the difference:

src/main.ts

```
import { describe, roundToTenth, toFahrenheit } from "../temperature.ts";
```

Java lambdas and TypeScript arrows

If you have written a Java lambda, arrow functions will look familiar:

Java	TypeScript
<code>c -> c * 9 / 5 + 32</code>	<code>(c) => c * 9 / 5 + 32</code>
<code>(a, b) -> a + b</code>	<code>(a, b) => a + b</code>
<code>() -> System.out.println("hi")</code>	<code>() => console.log("hi")</code>
<code>x -> { ... return y; }</code>	<code>(x) => { ... return y; }</code>
the type is an interface: <code>Function<Double, Double></code>	the type is written as a function type: <code>(c: number) => number</code>
a lambda can only use variables that are effectively final	an arrow can use - and change - any variable it can see

The arrow is `=>` (an equals sign), not Java's `->` (a minus sign). Brackets round a single parameter are optional in TypeScript, but the formatter, and this book, always write them.

Which to use?

Both kinds of function work. This book uses `function` for the listeners and helpers it has met so far, and from now on uses arrow functions for small functions and anything passed to another function - which is most of the time. One difference is worth knowing: a `function` declaration can be called from code *above* it in the file, but an arrow function stored in a `const` cannot be used until the line that creates it has run - the same as any other `const`.

Exercise 3.1 - Rewrite a function

Rewrite `twoDigits` from Chapter 2 (`ch02_project03_ticking_clock/src/time_format.ts`) as an arrow function. Try it before reading on.

```
// before
export function twoDigits(value: number): string {
  return `${value}`.padStart(2, "0");
}
```

Here is one way:

```
export const twoDigits = (value: number): string => `${value}`.padStart(2, "0");
```

The tests do not change, and they still pass: they call `twoDigits(7)` either way.

Tests are arrow functions too

Look at a test again:

```
tests/temperature.test.ts
```

```
Deno.test("water boils at 212 °F", () => {
  assertEquals(toFahrenheit(100), 212);
});
```

`Deno.test` is a function that takes two arguments: the test's name, and an arrow function with no parameters - the test's code. Deno stores the function, and calls it when it runs the tests. It is the same idea as `addEventListener`: you hand over a function, to be called later.

Testing numbers with decimal places

A normal body temperature, 36.6 °C, should be 97.88 °F. But the test

```
assertEquals(toFahrenheit(36.6), 97.88)
```

 fails:

```
[Diff] Actual / Expected
- 97.880000000000001
+ 97.88
```

Computers store numbers with decimal places in binary, and most decimals (like 0.1, or 36.6) cannot be stored exactly, so a calculation can come out a tiny amount wrong - Java's `double` has exactly the same problem (in either language, `0.1 + 0.2` is `0.30000000000000004`). So numbers that come out of calculations are compared with `assertAlmostEquals`, which passes if they are very close:

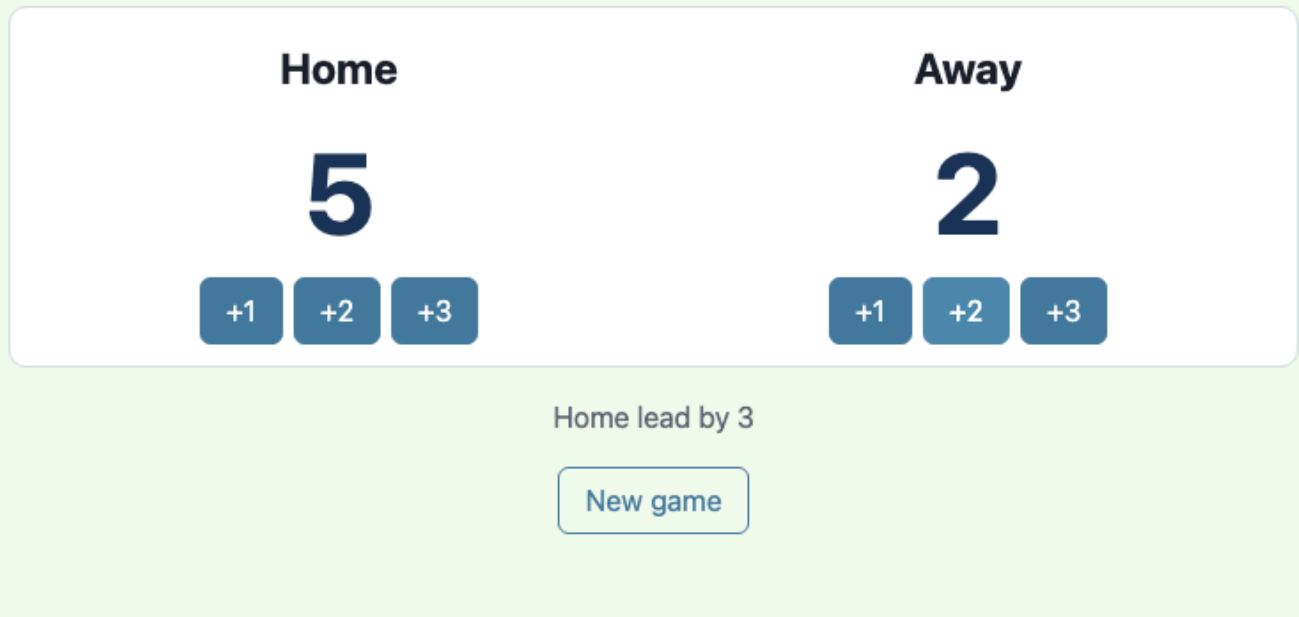
```
import { assertAlmostEquals, assertEquals } from "@std/assert";

Deno.test("36.6 °C is about 97.88 °F", () => {
  assertAlmostEquals(toFahrenheit(36.6), 97.88);
});
```

For showing on the page, `roundToTenth` rounds to one decimal place: `Math.round(value * 10) / 10`.

Project 2: Arrow functions as listeners

Basketball Scoreboard



The Basketball Scoreboard project

A scoreboard needs six buttons: +1, +2 and +3 for each team. In Chapter 2 each button's listener was a named function - that would be six functions here, all nearly the same. With arrow functions, the buttons and their listeners can be made in a loop:

src/main.ts

```
const POINTS: number[] = [1, 2, 3];

/** Puts a +1, +2 and +3 button for `team` into the element with id `containerId`. */
const addButtons = (team: Team, containerId: string): void => {
  const container = document.querySelector<HTMLElement>(containerId);
  if (container === null) {
    return;
  }
  for (const points of POINTS) {
    const button = document.createElement("button");
    button.textContent = `+${points}`;
    // The listener is written right here, as an arrow function. It can use `team` and `points`,
    // and it remembers them: each button gets its own team and its own number of points.
    button.addEventListener("click", () => {
      team.addPoints(points);
      render();
    });
    container.appendChild(button);
  }
};

addButtons(home, "#home-buttons");
addButtons(away, "#away-buttons");
```

The listener `() => { team.addPoints(points); render(); }` is written right where it is needed. It uses two variables from outside itself: `team` (a parameter of `addButtons`) and `points` (the loop variable). The

listener runs much later - when the button is clicked, long after `addButtons` has finished - and yet it still knows which team and how many points. An arrow function **remembers the variables it could see when it was made**. (The function together with the variables it remembers is called a **closure**.) Each time round the loop a new arrow function is made, so each button remembers its own `points`; and each call of `addButtons` has its own `team`.

`Team` is a small class like Chapter 2's `Counter`, with a constructor (constructors work as in Java; Chapter 5 says more):

`src/Team.ts`

```
export class Team {
  private name: string;
  private score: number = 0;

  // A constructor, as in Java - but always called "constructor". Chapter 5 has more.
  constructor(name: string) {
    this.name = name;
  }

  /** Adds a basket: 1, 2 or 3 points. */
  public addPoints(points: number): void {
    this.score += points;
  }
  // ... getName(), getScore(), reset()
}
```

and `leader(home, away)`, in `src/leader.ts`, works out "Home lead by 3" or "All square". Both are tested.

Exercise 3.2 - An arrow listener

In Chapter 2's coin toss (`ch02_project01_coin_toss`), replace the named function `handleToss` with an arrow function written inside the `addEventListener` call. Try it before reading on.

Here is one way:

```
if (tossButton !== null) {
  tossButton.addEventListener("click", () => {
    if (result !== null) {
      result.textContent = coinFace(Math.random());
    }
  });
}
```

Which is better? For a listener of a few lines that is used once, the arrow function keeps the code together. For a long listener, or one used by several buttons, a named function is clearer.

? - a shorter null check

You have written a lot of `if (button !== null) { ... }`. When all you want to do is call one method on something that might be `null`, there is a shorter way:

```
resetButton?.addEventListener("click", () => {
  home.reset();
  away.reset();
  render();
});
```

`resetButton?.addEventListener(...)` means: if `resetButton` is `null` (or `undefined`), do nothing; otherwise, call `addEventListener`. The `?.` is called **optional chaining**. The marks table and the unit converter use it for their listeners.

The `this` trap

`Team` has a `reset()` method. It is tempting to pass it straight to the reset button:

```
resetButton?.addEventListener("click", home.reset); // wrong!
```

TypeScript accepts it, the linter accepts it, the tests pass - and the button does nothing. No error appears anywhere; the home score just stays as it was.

The reason is `this`. Inside a method, `this` means "the object the method was called on": in `home.reset()`, it is `home`. But `home.reset` without brackets is just the function, cut off from `home`. When the browser later calls a listener, it sets `this` to the *element* that was clicked - so `reset` runs `this.score = 0` on the button, quietly giving the button a property called `score`, and leaves `home` alone. (Java has no equivalent: a Java method reference like `home::reset` always remembers its object.)

The fix is to wrap the call in an arrow function, which calls the method *on* the object:

```
resetButton?.addEventListener("click", () => home.reset()); // right
```

The rule: **never pass a method on its own as a listener (or to any other function) - wrap it in an arrow function**. Arrow functions have no `this` of their own, so inside one, `this` means whatever it meant outside - which is one reason they suit listeners.

Project 3: Working through arrays

Marks Table

Sort by name

Sort by mark

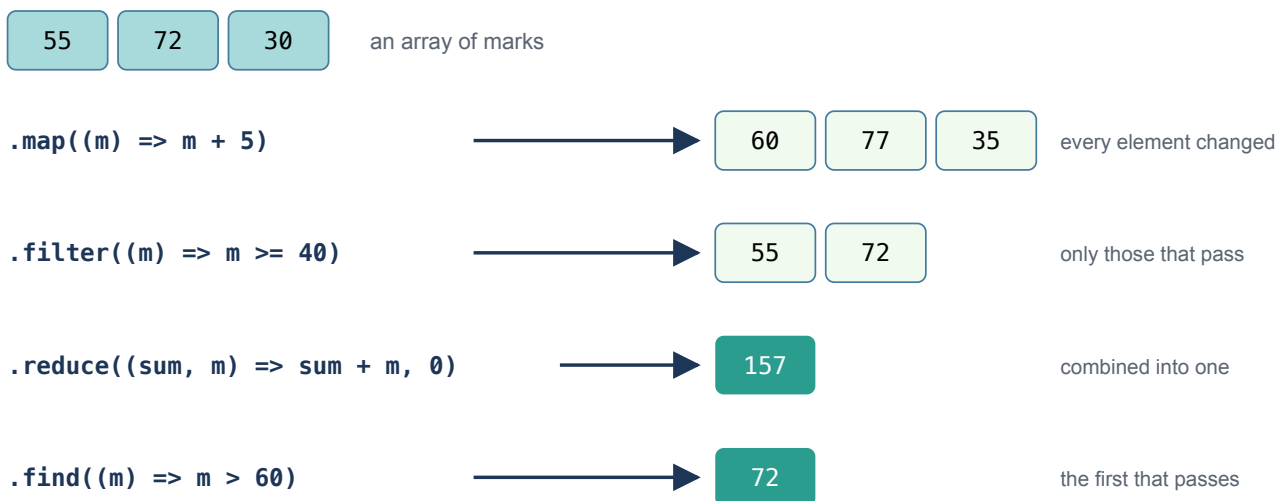
☒ Passes only

8 students · average 60.9 · top: Emma (81)

Name	Mark	Grade
Emma	81	A
Aoife	72	A
Isla	68	B
Chloe	65	B
Grace	59	C
Darragh	54	C
Finn	47	D
Jack	41	D

The Marks Table project

Arrays have methods that take a function and apply it to the elements. They replace most of the `for` loops you would write in Java (and do the job of Java's streams):



The original array is never changed - each method gives back something new.

map, filter, reduce and find

Method	Gives back	The function you pass ...
<code>map(f)</code>	a new array, every element changed by <code>f</code>	turns one element into its new value
<code>filter(f)</code>	a new array, only the elements for which <code>f</code> is true	answers true or false for one element
<code>reduce(f, start)</code>	one value, built up element by element	combines the value so far with one element
<code>find(f)</code>	the first element for which <code>f</code> is true - or <code>undefined</code>	answers true or false for one element
<code>toSorted(f)</code>	a sorted copy	compares two elements: negative, zero or positive

None of them changes the original array.

The marks come from a JSON file, as in Chapter 1, and each student has a name and a mark:

`src/marks.ts`

```
/** One student and their mark. `type` gives a name to a type, so it can be used again. */
export type Student = { name: string; mark: number };
```

`type Student = ...` gives the object type a **name**, so the code can say `Student[]` instead of `{ name: string; mark: number }[]` everywhere. (Chapter 10 compares `type` with `interface`.)

map, filter, reduce, find

`src/marks.ts`

```
/** Just the marks: [72, 38, 65, ...]. */
export const marksOf = (students: Student[]): number[] => students.map((student) => student.mark);

/** Only the students who passed. */
export const passed = (students: Student[]): Student[] => students.filter((student) => student.mark
>= PASS_MARK);

/** The student with this name, or undefined if there is none. */
export const findStudent = (students: Student[], name: string): Student | undefined =>
students.find((student) => student.name === name);
```

Each is one line, with an arrow function saying what to do with *one* student. Compare `passed` with the Java you would write:

```
List<Student> passed = new ArrayList<>();
for (Student student : students) {
    if (student.getMark() >= PASS_MARK) {
        passed.add(student);
    }
}
```

`find` gives back `Student | undefined` - a union, like `number | null` in Chapter 2. If no student matches, the answer is `undefined` (JavaScript's other "nothing", meaning "no value here"), and TypeScript

makes sure the code checks for it.

`reduce` takes two things: a function, and a starting value. The function is given the total so far and the next element, and returns the new total:

```
const total = marks.reduce((sum, mark) => sum + mark, 0);
```

With the marks `[50, 60, 70]`, it works out `0 + 50`, then `50 + 60`, then `110 + 70`: 180.

Average, test first

`average` divides the total by the number of marks. Before writing it, think about the edges. What is the average of no marks at all? A test makes you decide - we will say 0:

tests/marks.test.ts

```
Deno.test("the average of no marks is 0, not NaN", () => {
  assertEquals(average([]), 0);
});
```

Without any special handling, `average([])` would work out `0 / 0`, which in JavaScript is `NaN` ("not a number"), and this test is red:

```
- NaN
+ 0
```

So the function checks first:

src/marks.ts

```
/** The average of some marks - 0 if there are none. */
export const average = (marks: number[]): number => {
  if (marks.length === 0) {
    return 0;
  }
  const total = marks.reduce((sum, mark) => sum + mark, 0);
  return total / marks.length;
};
```

Green. Without the test, the page would one day have shown "average NaN" - the first time the "passes only" box was ticked on a class where everybody failed.

Sorting

src/marks.ts

```

/** A copy, highest mark first. (toSorted leaves the original array as it was.) */
export const byMark = (students: Student[]): Student[] => students.toSorted((a, b) => b.mark -
a.mark);

/** A copy, in alphabetical order of name. */
export const byName = (students: Student[]): Student[] => students.toSorted((a, b) =>
a.name.localeCompare(b.name));

```

The function passed to `toSorted` is a **comparator**, exactly like Java's `Comparator`: given two elements `a` and `b`, it returns a negative number if `a` should come first, a positive number if `b` should come first, or zero if it does not matter. `b.mark - a.mark` is positive when `b` has the higher mark, so higher marks come first. Strings have a `localeCompare` method that does the same job for alphabetical order (like Java's `compareTo`).

Note - Arrays also have a `sort` method, but it sorts the array **in place** - it changes the original. `toSorted` makes a sorted copy and leaves the original alone, which avoids surprises: the test "sorting leaves the original array as it was" checks it.

Chaining, and building the page

src/main.ts

```

const render = (): void => {
  const chosen = passesOnly ? passed(STUDENTS) : STUDENTS;
  const shown = sortByMark ? byMark(chosen) : byName(chosen);

  if (rows !== null) {
    // map turns each student into a row of HTML; join puts the rows together into one string.
    rows.innerHTML = shown
      .map((student) => `<tr><td>${student.name}</td><td>${student.mark}</td>
<td>${gradeFor(student.mark)}</td></tr>`)
      .join("");
  }
  // ... the summary line
};

```

`map` can turn the students into strings of HTML just as easily as into numbers, and `join("")` glues an array of strings into one string. One statement builds the whole table - no `createElement` loop. Because each method returns a new array, they can be **chained**: one after another, each working on the result of the last.

`render` decides what to show from two booleans, `sortByMark` and `passesOnly`, and the buttons and checkbox only change those and call `render()` - the object, event, render pattern from Chapter 2, with plain variables as the state.

Exercise 3.3 - Count the fails

Write a function `failedCount(students: Student[]): number` that says how many students failed. Write the test first. Try it before reading on.

Here is one way:

```

Deno.test("one student in the made-up class failed", () => {
  assertEquals(failedCount(CLASS), 1);
});

export const failedCount = (students: Student[]): number =>
  students.filter((student) => student.mark < PASS_MARK).length;

```

`filter` gives a new array; `.length` counts it.

Project 4: Functions in data

Unit Converter

Kilometres to miles ▾

42.195

42.195 km = 26.22 miles

From

To

1 km

0.62 miles

5 km

3.11 miles

10 km

6.21 miles

100 km

62.14 miles

The Unit Converter project

The unit converter can convert temperatures, distances, weights and lengths. Instead of an `if` for each kind of conversion, each conversion is an object that **holds its own function**:

`src/converters.ts`

```

/** The type of a conversion function: it takes a number and gives back a number. */
export type ConvertFunction = (value: number) => number;

/** One conversion: its name, the two units, and the function that does it. */
export type Converter = {
  name: string;
  from: string;
  to: string;
  convert: ConvertFunction;
};

export const CONVERTERS: Converter[] = [
  { name: "Celsius to Fahrenheit", from: "°C", to: "°F", convert: (celsius) => celsius * 9 / 5 + 32 },
  { name: "Kilometres to miles", from: "km", to: "miles", convert: (km) => km * 0.621371 },
  { name: "Kilograms to pounds", from: "kg", to: "lb", convert: (kg) => kg * 2.20462 },
  { name: "Metres to feet", from: "m", to: "ft", convert: (metres) => metres * 3.28084 },
];

```

`(value: number) => number` is a **function type**: "a function that takes a number and returns a number". It looks like an arrow function, but it is a type - it describes functions, the way `string[]` describes arrays. `ConvertFunction` gives it a name.

Notice that the arrow functions in the array do not say what type their parameters are - `(celsius) => ...`, not `(celsius: number) => ...`. They do not need to: TypeScript knows that `convert` must be a `ConvertFunction`, so it knows `celsius` is a number. (Try `(celsius) => celsius.length` - it is an error.)

The page puts the converters' names in a drop-down list, and when the choice changes it finds the chosen converter and calls *its* function:

src/main.ts

```

const converter = findConverter(choice.value);
if (converter === undefined) {
  return;
}
result.textContent = describeConversion(Number(input.value), converter);

const converted = convertAll(EXAMPLES, converter.convert);

```

`converter.convert` - without brackets - is the chosen function itself, handed to `convertAll`:

src/converters.ts

```

/** Applies any conversion function to every value. The function is passed in, like a listener. */
export const convertAll = (values: number[], convert: ConvertFunction): number[] =>
  values.map(convert);

```

`values.map(convert)` passes the function straight to `map`. Because `convertAll` takes *any* function, the test can hand it a simple one of its own, and know exactly what to expect:

tests/converters.test.ts

```

Deno.test("convertAll applies the function to every value", () => {
  assertEquals(convertAll([1, 2, 3], (x) => x * 10), [10, 20, 30]);
});

```

To add a new conversion, you add one object to `CONVERTERS` - nothing else in the program changes. The code that *uses* conversions does not need to know what conversions exist. Choosing which function (or object) to use while the program runs is the idea behind the **Strategy** pattern, which the design patterns book (Book 3) begins with.

Java and TypeScript

Java	TypeScript
<code>c -> c * 9 / 5 + 32</code>	<code>(c) => c * 9 / 5 + 32</code>
<code>Function<Double, Double> , Predicate<Student> , ...</code>	<code>(c: number) => number , (s: Student) => boolean</code>
<code>list.stream().map(f).collect(toList())</code>	<code>array.map(f)</code>
<code>list.stream().filter(p)</code>	<code>array.filter(p)</code>
<code>list.stream().reduce(0, Integer::sum)</code>	<code>array.reduce((sum, x) => sum + x, 0)</code>
<code>list.stream().filter(p).findFirst()</code> (an <code>Optional</code>)	<code>array.find(p)</code> (the element, or <code>undefined</code>)
<code>list.sort(comparator)</code> (changes the list)	<code>array.toSorted(comparator)</code> (a copy); <code>array.sort(...)</code> changes it
<code>home::reset</code> (remembers <code>home</code>)	<code>() => home.reset()</code> - never <code>home.reset</code> on its own
<code>if (button != null) button.addListener(...)</code>	<code>button?.addEventListener(...)</code>
a class or record for a simple data shape	<code>type Student = { name: string; mark: number };</code>

Summary

- an arrow function is `(parameters) => body` ; with an expression body its value is returned, with a `{ block }` body it needs `return`
- arrow functions can be written anywhere a value can go - as listeners, as arguments to `map` , in objects
- an arrow function remembers the variables it could see when it was made (a closure), so a listener made in a loop remembers its own values
- `assertAlmostEquals` compares numbers that may not be exact
- `x?.method()` calls the method only if `x` is not `null` or `undefined`
- never pass a method on its own as a listener: wrap it, `() => object.method()`
- `map` , `filter` , `reduce` , `find` and `toSorted` replace most loops, and leave the original array alone; `find` may give `undefined`

- `type Name = ...` names a type; `(value: number) => number` is a function type
- functions can be stored in objects and chosen while the program runs

Challenges

Each challenge says which project to start from. Write the tests first, and use arrow functions.

1. Kelvin

Start from `ch03_project01_temperatures`. Scientists use the Kelvin scale: 0 °C is 273.15 K. Add an arrow function `toKelvin(celsius: number): number`, test first (0 °C, 100 °C, and absolute zero, -273.15 °C, which is 0 K), and add a Kelvin column to the table.

2. Free throw and undo

Start from `ch03_project02_scoreboard`. Add an **Undo** button for each team that takes away the last basket scored. Give `Team` a method `undo()` and test it first: after +2 and +3, undo leaves 2; undo with no baskets does nothing.

Hint: `Team` can keep an array of the baskets (`private baskets: number[] = []`;). `push` adds one to the end and `pop` removes the last one. The score is then the sum of the baskets - a job for `reduce`.

3. Highest and lowest

Start from `ch03_project03_marks_table`. Add a `bottomStudent` function, and show "Top: ... Bottom: ..." in the summary. Test first, including an empty class (`undefined`).

4. Grade counts

Start from `ch03_project03_marks_table`. Under the table, show how many students got each grade: "A: 2 · B: 2 · C: 2 · D: 2 · F: 2". Write a function `countGrade(students: Student[], grade: string): number`, test first.

Hint: `filter` then `.length`. To build the line, `map` over `["A", "B", "C", "D", "F"]` and `join(" · ")`.

5. New conversions, no new code

Start from `ch03_project04_unit_converter`. Add three conversions: miles to kilometres, pounds to kilograms, and litres to pints (1 litre is 1.75975 pints). Tests first. You should only need to add to `CONVERTERS` and to the tests - check that nothing else had to change. Then fix the test that checked there were four converters: what is a better way to test that list?

6. Search as you type

Start from `ch03_project03_marks_table`. Add a search box: as you type, the table shows only the students whose names contain what you have typed, ignoring upper and lower case ("em" finds Emma). It should work together with sorting and "passes only". Write `searchByName(students: Student[], text: string): Student[]`, test first - including an empty search, which matches everybody.

Hint: `name.toLowerCase().includes(text.toLowerCase())` . Listen for the `input` event (Chapter 2), keep the search text in a variable like `passesOnly` , and apply the search in `render` .

Next: [Chapter 4 - Test first, properly](#)

Chapter 4 - Test first, properly

Chapter 1 showed you test-driven development once, on a small function. From this chapter on, it is how every feature in the book is built: a failing test first, then just enough code, then a tidy-up. This chapter makes that a habit. You will work through three classic exercises one small step at a time, and pick up the tools that make tests easier to write and easier to read: `assertThrows`, `assertStrictEquals`, `steps`, and helper functions.

FizzBuzz

Multiples of 3 are Fizz, multiples of 5 are Buzz, multiples of both are FizzBuzz.

Count up to

1	2	Fizz	4	Buzz	Fizz	7
8	Fizz	Buzz	11	Fizz	13	14
FizzBuzz	16	17	Fizz	19	Buzz	Fizz
22	23	Fizz	Buzz	26	Fizz	28
29	FizzBuzz					

The FizzBuzz project

What you will learn

- red-green-refactor in really small steps, and why small is the point
- naming tests as sentences, and laying each one out as arrange / act / assert
- refactoring - changing how code is written without changing what it does - while the tests stay green
- throwing an error with `throw new Error(...)`, and testing it with `assertThrows`
- the difference between `assertEquals` (the same contents) and `assertStrictEquals` (the very same object)
- grouping related cases into steps with `t.step`
- helper functions that give every test a fresh start
- katas, and "start red" exercises where the tests are written for you
- what goes in a doc comment example, and what goes in a test file

The projects

Project	What it shows
ch04_project01_fizzbuzz	the FizzBuzz kata built one red-green-refactor cycle at a time; test names as sentences; arrange / act / assert; examples and tests
ch04_project02_roman_numerals	refactoring a pile of <code>if</code> s into a table and a loop, with the tests green throughout; steps with <code>t.step</code>
ch04_project03_password_rules	throwing errors and <code>assertThrows</code> ; <code>assertStrictEquals</code> ; a helper that makes a fresh object for each test
ch04_project04_leap_years_start_red	a "start red" kata: the tests are written, and four of them fail until you write the code

The cycle, again

Here is the cycle from Chapter 1:

1. **Red** - write one test for the next small thing the code should do. Run it and watch it fail.
2. **Green** - write the least code that makes every test pass.
3. **Refactor** - tidy up the code (and the tests) while everything is green.

Two words in there matter more than they look: **one** test, and the **least** code. It feels slow if you have programmed before. It is not: a small step that goes wrong is fixed in seconds, because you know exactly which two lines you just changed. A big step that goes wrong can cost an afternoon.

Watching the test fail matters too. A test you have never seen fail might not be testing anything: it might call the wrong function, or check the wrong thing, and pass whatever the code does. Seeing it go red, for the reason you expected, proves the test can catch a mistake.

Project 1: FizzBuzz, cycle by cycle

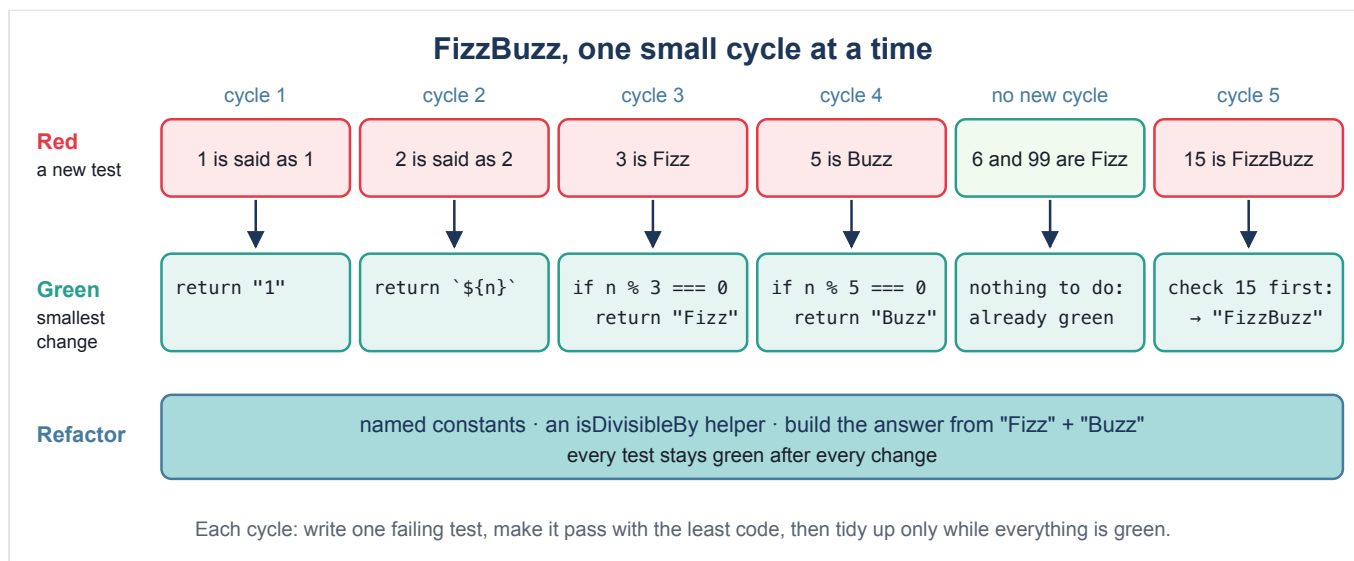
FizzBuzz is a counting game. Count from 1, but for a multiple of 3 say "Fizz", for a multiple of 5 say "Buzz", and for a multiple of both say "FizzBuzz". So: 1, 2, Fizz, 4, Buzz, Fizz, 7, 8, Fizz, Buzz, 11, Fizz, 13, 14, FizzBuzz ...

We want a function `fizzBuzz(n: number): string` that gives the answer for one number. Before any code, write a **test list**: the examples you expect to need, simplest first. It is a plan on a scrap of paper, not code:

- 1 is "1"
- 2 is "2"
- 3 is "Fizz"
- 5 is "Buzz"
- 6 is "Fizz" (and other multiples of 3)

- 15 is "FizzBuzz"

Each line will become one cycle. Here they all are, before we go through them one by one:



FizzBuzz, one small cycle at a time

Cycle 1: 1 is "1"

Red. Make `tests/fizz_buzz.test.ts` with one test:

`tests/fizz_buzz.test.ts`

```
import { assertEquals } from "@std/assert";
import { fizzBuzz } from "../src/fizz_buzz.ts";

Deno.test("1 is said as 1", () => {
  assertEquals(fizzBuzz(1), "1");
});
```

Save. There is no `src/fizz_buzz.ts` yet, so the tests cannot even load:

```
error: Module not found "src/fizz_buzz.ts".
  at tests/fizz_buzz.test.ts:2:26

Tests: the tests could not run (see above) -> test_output/index.html
```

That is red.

Green. The least code that passes is this:

`src/fizz_buzz.ts`

```
export const fizzBuzz = (n: number): string => "1";
```

Yes, really. It passes the only test there is, and that is all we can ask. The linter warns that `n` is never used - fine for a minute; the next test will need it.

Cycle 2: 2 is "2"

Red. Add the next test from the list:

```
Denotest("2 is said as 2", () => {
  assertEquals(fizzBuzz(2), "2");
});
```

```
not ok 2 - 2 is said as 2
---
message: |-
  AssertionError: Values are not equal.

      [Diff] Actual / Expected

-   1
+   2
at tests/fizz_buzz.test.ts:9:3
```

Remember how to read the diff: `-` is what the code **actually** gave, `+` is what the test **expected**.

Green. Now the function has to use `n`:

```
export const fizzBuzz = (n: number): string => `${n}`;
```

Why not jump straight to this in cycle 1? Because with only one example, `"1"` was just as correct, and less code. The second example *forced* the general version. This is called **triangulation**: you only generalise when a second test makes you.

Cycles 3 and 4: Fizz and Buzz

Red. `"3 is Fizz"` fails with `- 3` and `+ Fizz`. **Green:**

```
export const fizzBuzz = (n: number): string => {
  if (n % 3 === 0) {
    return "Fizz";
  }
  return `${n}`;
};
```

(`%` is the remainder, as in Java: `n % 3 === 0` means "n divides by 3 exactly".) The expression body had to become a block body with a `return` - Chapter 3's two kinds of arrow function.

Red. `"5 is Buzz"` fails with `- 5` and `+ Buzz`. **Green:** a second `if`, the same shape.

A test that passes straight away

The next line of the test list is "other multiples of 3 are Fizz":

```
Denotest("other multiples of 3 are Fizz", () => {
  assertEquals(fizzBuzz(6), "Fizz");
  assertEquals(fizzBuzz(99), "Fizz");
});
```

Save - and it passes first time. That is not a cycle; there is nothing to make green. Is the test useless, then? No: it records something we believe about the code, and it will catch anyone who later "simplifies" the function to `if (n === 3)`. Keep tests like this when they say something new. But be suspicious when a test you *expected* to fail passes: check it is calling what you think.

Cycle 5: FizzBuzz

Red.

```
Deno.test("15 is FizzBuzz", () => {
  assertEquals(fizzBuzz(15), "FizzBuzz");
});
```

15 divides by 3, so the first `if` answers "Fizz" and never looks any further:

The screenshot shows a test report interface. At the top, a red banner indicates "1 test failing" with the timestamp "ch04_project01_fizzbuzz · 10/3/2026, 3:25:33 PM". Below this, a summary row shows: 5 passed, 1 failed, 0 skipped, 0 type errors, and 0 lint warnings. The "Failing tests" section lists "15 is FizzBuzz" with a diff showing the actual result "Fizz" and the expected result "FizzBuzz". The "All tests" section lists several passing tests and one failing test, "15 is FizzBuzz". A "TAP output" section is also visible.

Test Name	Status
15 is FizzBuzz	Failed

AssertionError: Values are not equal.

```
[Diff] Actual / Expected
- Fizz
+ FizzBuzz
at tests/fizz_buzz.test.ts:26:3
```

1 is said as 1
2 is said as 2
3 is Fizz
5 is Buzz
other multiples of 3 are Fizz
15 is FizzBuzz

The report with "15 is FizzBuzz" failing

Green. The least code is one more check, put **first** so that it wins:

```
export const fizzBuzz = (n: number): string => {
  if (n % 15 === 0) {
    return "FizzBuzz";
  }
  if (n % 3 === 0) {
    return "Fizz";
  }
  if (n % 5 === 0) {
    return "Buzz";
  }
  return `${n}`;
};
```

Every test passes. The function works. Now - and only now - we tidy it.

Refactor

Refactoring means changing how code is written without changing what it does. The tests are what make it safe: run them after every small change, and if one goes red, undo the last change.

What is untidy here? The numbers 3, 5 and 15 are magic numbers. "Divides exactly" is written out three times. And 15 is only there because it is 3 times 5: if the rules changed to Fizz for 4, someone would have to remember to change 15 to 20. So, one change at a time, saving and checking after each:

1. name the numbers `FIZZ_DIVISOR` and `BUZZ_DIVISOR`
2. pull out a small helper, `isDivisibleBy`
3. build the answer up - "Fizz", then "Buzz" on the end - so a multiple of 15 needs no check of its own

`src/fizz_buzz.ts`

```
const FIZZ_DIVISOR = 3;
const BUZZ_DIVISOR = 5;

/** True if `n` divides exactly by `divisor` (nothing left over). */
const isDivisibleBy = (n: number, divisor: number): boolean => n % divisor === 0;

/** What to say for one number: "Fizz", "Buzz", "FizzBuzz", or the number itself. */
export const fizzBuzz = (n: number): string => {
  // Build the answer up: a multiple of 15 collects both words, so it needs no check of its own.
  let answer = "";
  if (isDivisibleBy(n, FIZZ_DIVISOR)) {
    answer += "Fizz";
  }
  if (isDivisibleBy(n, BUZZ_DIVISOR)) {
    answer += "Buzz";
  }
  return answer === "" ? `${n}` : answer;
};
```

`isDivisibleBy` is not exported: it is a detail of this file. It is still tested - through `fizzBuzz`, whose tests would fail if it were wrong.

The third step is the risky one, and the tests earn their keep. Put the two `if`s the other way round by mistake, and the moment you save:

```
not ok 6 - 15 is FizzBuzz
...
-   BuzzFizz
+   FizzBuzz
```

Without that test, "BuzzFizz" might have sat on the page for weeks.

Note - Refactor on green only. If a test is red, you do not know whether a change broke something or the code was broken already. Get to green first - even by undoing - then tidy.

Test names are sentences

Look at the names of the tests so far:

```
ok 1 - 1 is said as 1
ok 2 - 2 is said as 2
ok 3 - 3 is Fizz
ok 4 - 5 is Buzz
ok 5 - other multiples of 3 are Fizz
```

Read down the list and you have the rules of the game. That is deliberate. A test name is the first thing you see when a test fails - often all you see - so it should say *what the code should do*, as a sentence, in the words of the problem. Compare:

Not like this	Like this
test1	1 is said as 1
testFizz	3 is Fizz
fizzBuzz works	15 is FizzBuzz
edge case	no answers up to 0

"fizzBuzz works" failing tells you nothing; "15 is FizzBuzz" failing tells you exactly what is broken before you have opened a file.

Arrange, act, assert

Most tests have three parts, always in this order:

1. **Arrange** - set up what the test needs: values, objects
2. **Act** - do the one thing being tested
3. **Assert** - check the result

The FizzBuzz tests are so short that all three fit on one line. The test for `fizzBuzzUpTo`, which gives the answers from 1 up to a count, spells them out:

```
tests/fizz_buzz.test.ts
```

```

Deno.test("the first five answers, in order", () => {
  // Arrange: how many answers we want
  const count = 5;

  // Act: get them
  const answers = fizzBuzzUpTo(count);

  // Assert: check them
  assertEquals(answers, ["1", "2", "Fizz", "4", "Buzz"]);
});

```

You will not usually write the comments - a blank line between the parts is enough - but keep the shape. A test that acts, asserts, acts again and asserts again is really two tests; split it, so that each failure points at one thing.

`fizzBuzzUpTo` uses the counting `for` loop, which is the same as Java's:

`src/fizz_buzz.ts`

```

export const fizzBuzzUpTo = (count: number): string[] => {
  const answers: string[] = [];
  for (let n = 1; n <= count; n++) {
    answers.push(fizzBuzz(n));
  }
  return answers;
};

```

(`let`, not `const`, because `n` changes.) Its edge is a count of 0, tested by "no answers up to 0". The page in `main.ts` just calls `fizzBuzzUpTo` with the number in the box and gives each answer a CSS class for its colour.

Examples and tests

In Chapter 1, `greeting` had an example in its doc comment, and Deno ran it as a check. Now that `fizzBuzz` is finished, it gets one too. This is the last step, after the refactor:

`src/fizz_buzz.ts`

```

/**
 * What to say for one number: "Fizz", "Buzz", "FizzBuzz", or the number itself.
 *
 * @example
 * ```ts
 * import { assertEquals } from "@std/assert";
 *
 * assertEquals(fizzBuzz(3), "Fizz");
 * assertEquals(fizzBuzz(5), "Buzz");
 * assertEquals(fizzBuzz(15), "FizzBuzz");
 * assertEquals(fizzBuzz(7), "7");
 * ```
 */
export const fizzBuzz = (n: number): string => {

```

`fizzBuzzUpTo` has one too: `assertEquals(fizzBuzzUpTo(5), ["1", "2", "Fizz", "4", "Buzz"]);`.

Why add the example last, and not first? Because the tests and the example do different jobs. The tests **drove** the code: each one was the next small step, and together they pin down every rule and every edge. Most of them are not much use to someone who just wants to *call* `fizzBuzz`. The example is for that person: four calls that show what the function is for. It is written once the function has settled, and it stays true, because Deno checks it on every save.

You can see the difference when something breaks. Make the refactor mistake from above - put the two `if`s the other way round - and save:

```
not ok 7 - 15 is FizzBuzz
...
not ok 8 - other multiples of 15 are FizzBuzz
...
# src/fizz_buzz.ts
not ok 11 - example in the doc comment (lines 14-22)
---
message: |-
  AssertionError: Values are not equal.

      [Diff] Actual / Expected

-   BuzzFizz
+   FizzBuzz
    at src/fizz_buzz.ts (example at lines 14-22)
```

The two tests say *what* is broken, by name. The example only says *where* it is: line 14 to 22 of `fizz_buzz.ts`. And the example stopped at its first failure, so `fizzBuzz(7)` was never checked at all. That is fine for an example - its job is to show, not to diagnose - but it is why the checks that matter live in the test file.

	A doc comment example	A test file
Written for	someone using the function	someone changing it
What goes in it	one to four typical calls	every rule, every edge, every refused input
Its name	none - "example in the doc comment (lines 14-22)"	a sentence saying what should be true
When one check fails	the rest of the example is not run	every other test still runs
When it is written	once the function has settled	before the code, one test at a time

From here on, the book's projects use both: tests for everything the code must do, and an example on each exported function or class whose use is not obvious from its name.

Exercise 4.1 - What about 0?

`fizzBuzzUpTo` starts at 1, but nothing stops someone calling `fizzBuzz(0)`. What does it give? Work it out from the code, then write a test that records the answer. Try it before reading on.

Here is one way. 0 divided by anything leaves nothing over, so 0 divides by 3 *and* by 5:

```
Deno.test("0 is FizzBuzz, because 0 divides by both 3 and 5", () => {
  assertEquals(fizzBuzz(0), "FizzBuzz");
});
```

It passes straight away - but now the surprising answer is written down, and the test name explains it. If you decided 0 should be "0" instead, you would write that test, watch it go red, and change the code.

Project 2: Roman numerals - refactoring as you go

Roman Numerals

Number

MCMXCIV

Some years

Year	Roman
1066	MLXVI
1776	MDCCLXXVI
1916	MCMXVI
1999	MCMXCIX
2000	MM
2026	MMXXVI

The Roman Numerals project

The Romans wrote numbers with seven letters: I (1), V (5), X (10), L (50), C (100), D (500) and M (1000). Symbols are added up, biggest first: VIII is $5 + 1 + 1 + 1 = 8$. But four of the same symbol in a row is not allowed, so a smaller symbol written *in front of* a bigger one is taken away: IV is 4, IX is 9, XL is 40, XC is 90, CD is 400, CM is 900. So 1994 is M CM XC IV: MCMXCIV.

The goal is `toRoman(n: number): string`. This kata is good practice for the **refactor** step, because the code goes through some ugly stages on the way to a neat one.

The first numbers

The first cycles go as FizzBuzz did: "1 is I" (green with `return "I"`), "2 is II" (green with `"I".repeat(n)`), "4 is IV", "5 is V", "9 is IX". After nine numbers, the code looks like this - and passes every test:

```
export const toRoman = (n: number): string => {
  if (n === 4) {
    return "IV";
  }
  if (n === 9) {
    return "IX";
  }
  if (n >= 5) {
    return "V" + "I".repeat(n - 5);
  }
  return "I".repeat(n);
};
```

Red. Then "10 is X":

```
-  VIIII
+   X
```

The pile of ifs

Getting the tens green, then 14, 19, 27, 39, gives this:

```
export const toRoman = (n: number): string => {
  let result = "";
  let rest = n;
  while (rest >= 10) {
    result += "X";
    rest -= 10;
  }
  if (rest >= 9) {
    result += "IX";
    rest -= 9;
  }
  if (rest >= 5) {
    result += "V";
    rest -= 5;
  }
  if (rest >= 4) {
    result += "IV";
    rest -= 4;
  }
  while (rest >= 1) {
    result += "I";
    rest -= 1;
  }
  return result;
};
```

(`while` is the same as in Java.) Every test is green, so it is time to refactor - and this time there is real repetition to remove. Five blocks all do the same thing: *if what is left is at least this big, write this symbol and take its value away*. Only two things change from block to block: the value and the symbol. When blocks differ only in their data, the data can go in a table, and one loop can replace them all. (Two blocks use `if` rather than `while`, but a `while` would do the same there: 9 can never fit twice into what is left.)

The table

src/roman.ts

```
/** One numeral and its value. */
type Numeral = { value: number; symbol: string };

// Biggest first. The pairs like CM (900) and IV (4) are the "take away" numerals: listing them as
// numerals of their own means the loop below never needs a special case for them.
const NUMERALS: Numeral[] = [
  { value: 1000, symbol: "M" },
  { value: 900, symbol: "CM" },
  { value: 500, symbol: "D" },
  // ... 400 CD, 100 C, 90 XC, 50 L, 40 XL, 10 X, 9 IX, 5 V
  { value: 4, symbol: "IV" },
  { value: 1, symbol: "I" },
];

export const toRoman = (n: number): string => {
  let result = "";
  let rest = n;
  // Take the biggest numeral that fits, as many times as it fits, then move on to the next one.
  for (const numeral of NUMERALS) {
    while (rest >= numeral.value) {
      result += numeral.symbol;
      rest -= numeral.value;
    }
  }
  return result;
};
```

Do it in small steps: first a table with just X, IX, V, IV and I, replacing the blocks you have, and check the tests; *then* add L, XL and the rest - each new row with a test that needed it ("50 is L" red, add the row, green). The finished loop handles 1994 and 3999 without a single new line of logic.

This is the same idea as Chapter 3's unit converter: when the code differs only by data, put the data in an array and let one piece of code work through it.

When a refactor goes wrong

Suppose that while typing the table you put IV above V. Save, and:

```
# Subtest: each symbol on its own
ok 1 - 1 is I
not ok 2 - 5 is V
...
-   IVI
+   V
ok 3 - 10 is X
...
# Subtest: symbols written together are added up
ok 1 - 2 is II
ok 2 - 3 is III
not ok 3 - 6 is VI
not ok 4 - 8 is VIII
not ok 5 - 27 is XXVII
```

Every failing number has a 5 in it, and 5 came out as IVI: the loop took 4 (IV) before it ever looked at 5. The test output pointed straight at the problem. Notice how the output is laid out: groups, with numbered cases inside them. Those are **steps**.

Grouping with `t.step`

The Roman numeral tests fall into natural groups: single symbols, symbols added together, symbols taken away, and bigger numbers. Each group is one `Deno.test`, and each case in it is a **step**:

```
Deno.test("a smaller symbol in front of a bigger one is taken away", async (t) => {
  await t.step("4 is IV", () => {
    assertEquals(toRoman(4), "IV");
  });
  await t.step("9 is IX", () => {
    assertEquals(toRoman(9), "IX");
  });
  // ...
});
```

The test function is now given a parameter, `t` (its type is `Deno.TestContext`), and `t.step(name, fn)` runs one named case inside the test. A failing step does not stop the others, and the report shows every step by name, so you see *all* the numbers that went wrong, not just the first:

The screenshot shows a Deno test report with a green header indicating "All tests passing". Below the header, statistics show 32 passed, 0 failed, 0 skipped, 0 type errors, and 0 lint warnings. The "All tests" section lists various test cases, all marked with green checkmarks, including individual symbols, combined symbols, and subtraction cases. A "TAP output" section is visible at the bottom.

All tests passing
ch04_project02_roman_numerals - 10/4/2026, 5:37:24 PM

32	0	0	0	0
passed	failed	skipped	type errors	lint warnings

All tests
tests/roman.test.ts

- ✓ each symbol on its own
 - ✓ 1 is I
 - ✓ 5 is V
 - ✓ 10 is X
 - ✓ 50 is L
 - ✓ 100 is C
 - ✓ 500 is D
 - ✓ 1000 is M
- ✓ symbols written together are added up
 - ✓ 2 is II
 - ✓ 3 is III
 - ✓ 6 is VI
 - ✓ 8 is VIII
 - ✓ 27 is XXVII
 - ✓ 3000 is MMM
- ✓ a smaller symbol in front of a bigger one is taken away
 - ✓ 4 is IV
 - ✓ 9 is IX
 - ✓ 40 is XL
 - ✓ 90 is XC
 - ✓ 400 is CD
 - ✓ 900 is CM
- ✓ bigger numbers use all the rules together
 - ✓ 14 is XIV
 - ✓ 49 is XLIX
 - ✓ 1994 is MCMXCIV
 - ✓ 2026 is MMXXVI
 - ✓ 3999 is MMMCMXCIX
- ✓ whole numbers from 1 to 3999 can be Roman
- ✓ 0, negative numbers, numbers over 3999 and fractions cannot be Roman

src/roman.ts

- ✓ example in the doc comment (lines 33-39)
- ✓ example in the doc comment (lines 46-52)

► TAP output

Made by tools/test_report.ts - Markdown version: summary.md

Steps in the test report

Two new keywords come with steps: `async` and `await`. A step does not finish instantly the way a function call does - Deno starts it and finishes it a little later. `await` means "wait here until this step is finished", and

a function that uses `await` must be marked `async`. Book 2 says more about waiting; for now, the rule is simple: **write `async (t)`, and put `await` in front of every `t.step`**. Forget an `await`, and Deno tells you plainly:

```
not ok 2 - 1 is I
----
message: |-
  Didn't complete before parent. Await step with `await t.step(...)`.
```

Writing `await t.step(...)` thirty times would be dull, so the project uses a **helper function** that turns a list of examples into steps:

`tests/roman.test.ts`

```
/** One example: a number and the numeral it should become. */
type Example = { n: number; numeral: string };

/** Runs one step per example, each named like "4 is IV". A helper keeps every group short. */
const checkExamples = async (t: Deno.TestContext, examples: Example[]): Promise<void> => {
  for (const example of examples) {
    await t.step(`${example.n} is ${example.numeral}`, () => {
      assertEquals(toRoman(example.n), example.numeral);
    });
  }
};

Deno.test("a smaller symbol in front of a bigger one is taken away", async (t) => {
  await checkExamples(t, [
    { n: 4, numeral: "IV" },
    { n: 9, numeral: "IX" },
    { n: 40, numeral: "XL" },
    // ...
  ]);
});
```

The helper is `async` too, so it is awaited as well. Its return type, `Promise<void>`, is what an `async` function that returns nothing gives back - read it as "something you can `await`, with no answer at the end". The names of the steps are made from the data with a template literal, so they are still sentences: "4 is IV".

Note - Steps suit many cases of the same kind, like a table of examples. Cases about different behaviour are clearer as separate `Deno.test` s.

Numbers that cannot be Roman

What is `toRoman(0)`? The loop never runs, so it is `""` - which is wrong, but quietly wrong. The Romans had no zero, and nothing above 3999 can be written with these seven letters. So the project has a second function, tested on its own:

`src/roman.ts`

```

/**
 * True if `n` can be written in Roman numerals: a whole number from 1 to 3999.
 *
 * @example
 * ```ts
 * import { assertEquals } from "@std/assert";
 *
 * assertEquals(canBeRoman(1994), true);
 * assertEquals(canBeRoman(0), false);
 * ```
 */
export const canBeRoman = (n: number): boolean => Number.isInteger(n) && n >= SMALLEST && n <=
LARGEST;

```

(`Number.isInteger(2.5)` is `false`.) The tests check its edges: 1 and 3999 are fine; 0, -5, 4000 and 2.5 are not. The page calls `canBeRoman` before `toRoman`, and shows a message instead of a numeral when the answer is no. Another choice would be for `toRoman` itself to refuse, by throwing an error - which is exactly what the next project does.

Both functions have an example in their doc comment. `toRoman`'s comment used to say, in words, that 4 is "IV" and 1994 is "MCMXCIV". Now it says so in two `assertEquals` lines, so if a change to the table ever broke 1994, the documentation would go red along with the tests.

Exercise 4.2 - A group of your own

Add a group of steps called "famous years" to `tests/roman.test.ts`, with 1066 (MLXVI), 1776 (MDCCLXXVI) and 1916 (MCMXVI). Try it before reading on.

Here is one way:

```

Deno.test("famous years", async (t) => {
  await checkExamples(t, [
    { n: 1066, numeral: "MLXVI" },
    { n: 1776, numeral: "MDCCLXXVI" },
    { n: 1916, numeral: "MCMXVI" },
  ]);
});

```

All green, and three new lines in the report. Adding a case is now adding one line of data.

Project 3: Password rules

Password Rules

Password ☒ show

Not yet

✓ at least 10 characters

✓ a capital letter

✓ a small letter

✓ a digit (0-9)

✗ a symbol such as ! or #

Nothing you type leaves this page.

The Password Rules project

The third project checks passwords against a list of rules: at least 10 characters, a capital letter, a small letter, a digit and a symbol. As you type, each rule is ticked or crossed.

Each rule is an object holding a description and a function - functions in data, as in Chapter 3's converters:

src/rules.ts

```
/** One rule: a description for people, and a test for the computer. */
export type Rule = {
  description: string;
  isMetBy: (password: string) => boolean;
};

export const HAS_DIGIT: Rule = {
  description: "a digit (0-9)",
  isMetBy: (password) => containsAny(password, DIGITS),
};

// If making the password lower case changes it, it must have had a capital letter in it.
export const HAS_UPPER_CASE: Rule = {
  description: "a capital letter",
  isMetBy: (password) => password.toLowerCase() !== password,
};
```

Each rule was written test first, with a password that meets it and one that does not, and the edges: "a minimum length rule accepts a password of exactly that length" and "... rejects a password one character short". The capital and small letter rules use a trick - a password has a capital letter if making it lower case changes it - and a trick deserves a test that tries to fool it: "digits and symbols are neither capital nor small letters" checks that "123!" has neither.

Throwing an error

The length rule is made by a function, because the length can vary:

src/rules.ts

```
export const minLength = (length: number): Rule => {
  if (!Number.isInteger(length) || length < 1) {
    throw new Error(`a minimum length must be a whole number of at least 1, not ${length}`);
  }
  return {
    description: `at least ${length} characters`,
    isMetBy: (password) => password.length >= length,
  };
};
```

What should `minLength(0)` do? A rule that every password meets is never what anyone meant. It is not a problem with a password - it is a mistake in the *program*, and the best thing the code can do is stop, loudly, at once, so the programmer finds it. That is what `throw` is for.

`throw new Error("...")` works as it does in Java: the function stops at once, nothing is returned, and the error travels back up through every function call until something catches it - or, if nothing does, the program (or the test) stops with the message. Two differences from Java:

- TypeScript has no checked exceptions and no `throws` clause. Nothing in `minLength`'s type says it can throw - so say so in its comment
- you will nearly always throw a plain `Error`, with a message saying what was wrong and the value that was wrong. (Book 2 shows how to make your own kinds of error, and how to catch them)

Testing that something throws

The test comes first, as always:

tests/rules.test.ts

```
Deno.test("a minimum length of 0 is a mistake, and throws", () => {
  assertThrows(() => minLength(0));
});
```

`assertThrows` takes a **function**, calls it, and passes only if the function throws. Before the `if` and the `throw` were written, the test was red:

```
not ok 1 - a minimum length of 0 is a mistake, and throws
---
message: |-
  AssertionError: Expected function to throw.
```

Notice the `() =>`. It is tempting to write `assertThrows(minLength(0))` - but that calls `minLength(0)` *before* `assertThrows` starts, the error flies out of the test itself, and the test fails with the very error it was supposed to expect:

```
not ok 1 - no arrow
---
message: |-
  Error: a minimum length must be a whole number of at least 1, not 0
```

The type checker spots it as well: `Argument of type 'Rule' is not assignable to parameter of type '() => unknown'`. - `assertThrows` wanted a function, and was given the rule that `minLength` would have returned. Wrapping the call in an arrow function hands `assertThrows` something it can call when it is ready - the same idea as `() => home.reset()` in Chapter 3.

`assertThrows` can also check what kind of error was thrown and what its message says. The second argument is the kind (`Error`), and the third is some text the message must **include**:

```
Deno.test("a minimum length that is not a whole number throws, and says why", () => {
  assertThrows(() => minLength(2.5), Error, "must be a whole number");
});
```

Checking part of the message, not all of it, keeps the test from breaking every time someone rewords the message - but still catches the wrong error being thrown. If the text does not match, the failure says what the message really was:

```
AssertionError: Expected error message to include "must be a positive number", but got "a
minimum length must be a whole number of at least 1, not 2.5".
```

The checker class

`PasswordChecker` is given its list of rules when it is made, so the same class can check a strict password or a relaxed one. Its constructor throws too: a checker with no rules would accept anything.

```
src/PasswordChecker.ts
```

```

export class PasswordChecker {
  private rules: Rule[];

  // A checker with no rules would accept anything, which is never what was meant: fail loudly.
  constructor(rules: Rule[]) {
    if (rules.length === 0) {
      throw new Error("a password checker needs at least one rule");
    }
    this.rules = rules;
  }

  /** The rules this password does not meet – the very same rule objects the checker was given. */
  public failedRules(password: string): Rule[] {
    return this.rules.filter((rule) => !rule.isMetBy(password));
  }

  /** What is missing, in words, ready to show on the page. */
  public problems(password: string): string[] {
    return this.failedRules(password).map((rule) => `needs ${rule.description}`);
  }

  /** True if the password meets every rule. */
  public isAcceptable(password: string): boolean {
    return this.failedRules(password).length === 0;
  }
}

```

A constructor that throws is tested the same way: `assertThrows(() => new PasswordChecker([]), Error, "at least one rule")`.

A class can have an example too, in the doc comment just above `export class`. It shows how to make an object and what to call on it:

`src/PasswordChecker.ts`

```

/**
 * Checks passwords against the rules it was made with.
 *
 * @example
 * ```ts
 * import { assertEquals } from "@std/assert";
 * import { HAS_DIGIT, minLength } from "../rules.ts";
 *
 * const checker = new PasswordChecker([minLength(8), HAS_DIGIT]);
 * assertEquals(checker.isAcceptable("open sesame 2"), true);
 * assertEquals(checker.problems("sesame"), [
 *   "needs at least 8 characters",
 *   "needs a digit (0-9)",
 * ]);
 * ```
 */
export class PasswordChecker {

```

Deno imports `PasswordChecker` for you, because the example is in its file - but only that file's own exports. The rules come from `rules.ts`, so the example imports them, with a path relative to `PasswordChecker.ts` (`../rules.ts`), just as the file's own imports are.

A fresh object for every test

Nearly every checker test needs a checker. In JUnit you would make it in a `@BeforeEach` method and keep it in a field. `Deno.test` has no `beforeEach`; instead, write a small **helper function** and call it at the start of each test:

`tests/PasswordChecker.test.ts`

```
/** A fresh checker with three simple rules - the "arrange" step most tests share. */
const makeChecker = (): PasswordChecker => new PasswordChecker([minLength(8), HAS_DIGIT,
HAS_UPPER_CASE]);

Deno.test("a password that meets every rule is acceptable", () => {
  // Arrange
  const checker = makeChecker();

  // Act
  const acceptable = checker.isAcceptable("Elephant42");

  // Assert
  assertEquals(acceptable, true);
});

Deno.test("the problems list every rule that is broken, in order", () => {
  const checker = makeChecker();
  assertEquals(checker.problems("cat"), ["needs at least 8 characters", "needs a digit (0-9)",
"needs a capital letter"]);
});
```

This is better than a shared object in two ways. Every test gets a **new** checker, so nothing one test does can leak into the next - tests that pass alone but fail together are miserable to track down. And the helper's call is right there in the test, so you can see the whole arrange step without scrolling up to find a hidden setup method. (Deno does have a `describe / it / beforeEach` style, in `@std/testing/bdd`; this book does not need it.)

Equal, or the same object?

`failedRules` promises to give back "the very same rule objects the checker was given". How do you test *the very same*? `assertEquals` is no good: it checks that two values have the same **contents** - it compares arrays element by element and objects property by property, which is why

`assertEquals(checker.problems("Elephant42"), [])` works even though the two empty arrays are different arrays. Java programmers know this as the difference between `equals` and `==`.

`assertStrictEquals` checks that the two values are **the same object** (it uses `===`):

Equal, or the same object?

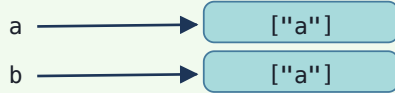
assertEquals

same contents?

assertStrictEquals

the very same object? (===)

Two objects that look the same



✓ passes

✗ fails

One object with two names



✓ passes

✓ passes

Equal, or the same object?

tests/PasswordChecker.test.ts

```
Deno.test("the failed rules are the very same objects the checker was given", () => {
  const checker = makeChecker();
  const failed = checker.failedRules("Elephants");
  assertEquals(failed.length, 1);
  // Not just a rule that looks like HAS_DIGIT: HAS_DIGIT itself.
  assertStrictEquals(failed[0], HAS_DIGIT);
});
```

Give it two lookalike objects and it fails, with a message that tells you exactly that:

AssertionError: Values have the same structure but are not reference-equal.

```
{
  description: "a digit (0-9)",
  isMetBy: [Function: isMetBy],
}
```

(That was `assertStrictEquals({ ...HAS_DIGIT }, HAS_DIGIT)`; `{ ...HAS_DIGIT }` makes a copy - Chapter 9 explains it.)

For numbers, strings and booleans the two assertions behave the same, because `===` compares their values. The difference only shows with objects and arrays. Use `assertEquals` almost always - you usually care what is in a result, not where it lives - and `assertStrictEquals` when *identity* is the point.

One surprise: `assertEquals(minLength(8), minLength(8))` **fails**. The two rules have the same description, but each call makes a new `isMetBy` arrow function, and two functions are only equal if they are the same function. Compare what you can see - `minLength(8).description` - or test the behaviour, by calling `isMetBy`.

Exercise 4.3 - Negative lengths

Is `minLength(-3)` tested? Write the test. Should it be red or green when you first run it? Try it before reading on.

Here is one way:

```
Deno.test("a negative minimum length throws", () => {
  assertThrows(() => minLength(-3), Error, "at least 1");
});
```

It is green at once: `length < 1` already covers it. That is fine - the test records that negative lengths were thought about.

Project 4: Start red

The last project is different: the tests are already written, and most of them fail.



The leap years project, starting red

A leap year has 366 days. The rules, from the comment at the top of `src/leap_year.ts`:

- a year that divides by 4 is a leap year,
- except a year that divides by 100, which is not,
- except a year that divides by 400, which is.

The rules began in 1583, so asking about an earlier year should throw an error. `isLeapYear` is written - it always says `false` - and the eight tests in `tests/leap_year.test.ts` are waiting. Four of them pass already (every year that is *not* a leap year), and four fail. Your job is to make them all pass, which is Challenge 2.

A "start red" exercise is how you might join a team: someone has written down what the code must do, as tests, and you make it true. Work it like any TDD exercise:

- **pick one failing test** - the simplest - and make it pass with the least code. Ignore the others
- run the tests after every small change: one more green, none newly red?
- refactor on green
- never change a test to make it pass. The tests are the specification. (If you are sure a test is wrong, that is a conversation with whoever wrote it.)

(The project has a file `.book-check.json` saying "this project is meant to have 4 failing tests"; it is for the book's own checking tool, and you can ignore it.)

Katas

FizzBuzz, Roman numerals and leap years are **katas**. In karate, a kata is a set of moves practised over and over until they come without thinking. A coding kata is a small, well-known problem you solve again and again - not to get the answer, which you already know, but to practise the way you get there: small steps, red first, refactor on green. Do FizzBuzz again next week from an empty folder, and see if your steps get smaller and your tests read better.

Other well-known katas: the **string calculator** (Challenge 5), **word wrap**, the **bowling game**, and **Gilded Rose** (messy working code to refactor without breaking it).

The habit, from here on

Every later chapter builds its features this way. In short:

1. write a test list before writing code
2. one test at a time: see it fail, for the right reason
3. make it pass with the least code
4. refactor on green - the code *and* the tests
5. name tests as sentences; arrange, act, assert
6. test the edges: 0, empty, one, the boundary and one either side of it, and the input that should be refused
7. once a function has settled, give it a doc comment example: a few typical calls, for the next person who uses it

Java and TypeScript

Java (JUnit 5)	TypeScript (Deno)
<code>@Test void fifteenIsFizzBuzz() { ... }</code>	<code>Deno.test("15 is FizzBuzz", () => { ... });</code>
<code>@DisplayName("15 is FizzBuzz")</code>	the name <i>is</i> the sentence
<code>assertEquals(expected, actual)</code>	<code>assertEquals(actual, expected)</code> - the other way round!
<code>assertSame(expected, actual)</code>	<code>assertStrictEquals(actual, expected)</code>
<code>assertThrows(IllegalArgumentException.class, () -> make(0))</code>	<code>assertThrows(() => make(0), Error, "part of the message")</code>
<code>throw new IllegalArgumentException("...")</code>	<code>throw new Error("...")</code>
checked exceptions, <code>throws</code> in the method signature	no checked exceptions; say "throws" in a comment
a Javadoc <code>@code</code> sample, never run	a doc comment <code>@example</code> , run and checked with the tests
<code>@BeforeEach</code> and a field	a helper function called at the start of each test
<code>@Nested</code> classes, <code>@ParameterizedTest</code>	steps with <code>t.step</code> , and a helper looping over a table of examples
<code>for (int i = 1; i <= n; i++)</code>	<code>for (let i = 1; i <= n; i++)</code>

Summary

- test first, in small steps: one failing test, the least code, then refactor on green
- watch every new test fail before making it pass; a test that passes straight away still has value if it records something new
- generalise only when a second example forces you (triangulation)
- refactoring changes how code is written, not what it does; when blocks differ only by data, put the data in a table and loop over it
- test names are sentences that state the rule; each test is arrange, act, assert
- `throw new Error("message")` for mistakes that should stop the program; test it with `assertThrows(() => ..., Error, "part of the message")` - always with the `() =>`
- `assertEquals` checks contents; `assertStrictEquals` checks it is the very same object
- `t.step` groups cases inside a test: `async (t)` and `await t.step(...)`
- helper functions replace `beforeEach`: a fresh object for every test, made where you can see it
- a doc comment example shows a few typical calls and is checked on every save; it has no name and stops at its first failure, so the rules and edges belong in test files
- katas are practice problems, done again and again; in a "start red" exercise, the tests are the specification

Challenges

Each challenge says which project to start from. All of them are test first: write a failing test, see it fail, then write the code.

1. Fizz, Buzz, Whizz

Start from `ch04_project01_fizzbuzz`. Add a new word: multiples of 7 are "Whizz". 21 is "FizzWhizz", 35 is "BuzzWhizz", and 105 is "FizzBuzzWhizz". Write the test list first, then go one cycle at a time. How much did the refactored `fizzBuzz` make this easier than the version with `n % 15` would have?

2. Make it green

Start from `ch04_project04_leap_years_start_red`. Make all eight tests pass, one at a time, without changing any test. Keep a note of the order you made them pass in, and what the code looked like after each. Then refactor: named constants for 4, 100, 400 and 1583, and the shortest clear version of the rule you can find. The page should then list the leap years from 2020 to 2060.

3. More password rules

Start from `ch04_project03_password_rules`. Add two rules, each test first: `NO_SPACES` (a password must not contain a space) and `notContaining(word: string)`, which makes a rule that the password must not contain `word`, ignoring upper and lower case (so `notContaining("password")` rejects "MyPassword1!"). `notContaining("")` makes no sense, so it should throw. Add `NO_SPACES` and `notContaining("password")` to the page's rules.

4. Back from Roman

Start from `ch04_project02_roman_numerals`. Write `fromRoman(numeral: string): number`, test first, with its own groups of steps: single symbols, symbols added up, symbols taken away, bigger numbers. Add a second box to the page that turns a numeral back into a number.

Hint: the `NUMERALS` table works backwards too. While the numeral starts with a symbol from the table (`numeral.startsWith(symbol, position)` checks from a position), add its value and move past it. Then add a group that checks every number from 1 to 3999 survives the round trip: `fromRoman(toRoman(n))` is `n` - a loop of 3999 `assertEquals` calls in one test (not 3999 steps!).

5. The string calculator kata

Start from a copy of `ch04_project04_leap_years_start_red` (delete its source and tests, and its `.book-check.json`). Write `add(numbers: string): number`, one rule at a time, a test for each:

1. an empty string gives 0
2. one number gives that number: `"4"` gives 4
3. two numbers, separated by a comma: `"1,2"` gives 3
4. any amount of numbers: `"1,2,3,4"` gives 10
5. new lines work as separators too: `"1\n2,3"` gives 6

6. a negative number throws an error whose message lists **all** the negatives: `"1,-2,-3"` throws `"negatives not allowed: -2, -3"`

Give it a page with a text box and the answer.

Hint: `numbers.split(",")` gives an array of strings, and `Number("4")` gives 4. For new lines, one way is to replace them with commas first: `numbers.replaceAll("\n", ",")`. For rule 6, `filter` out the negatives and `join(", ")` them into the message. Refactor after every green - by rule 6 the code should still be short.

6. Strict Roman

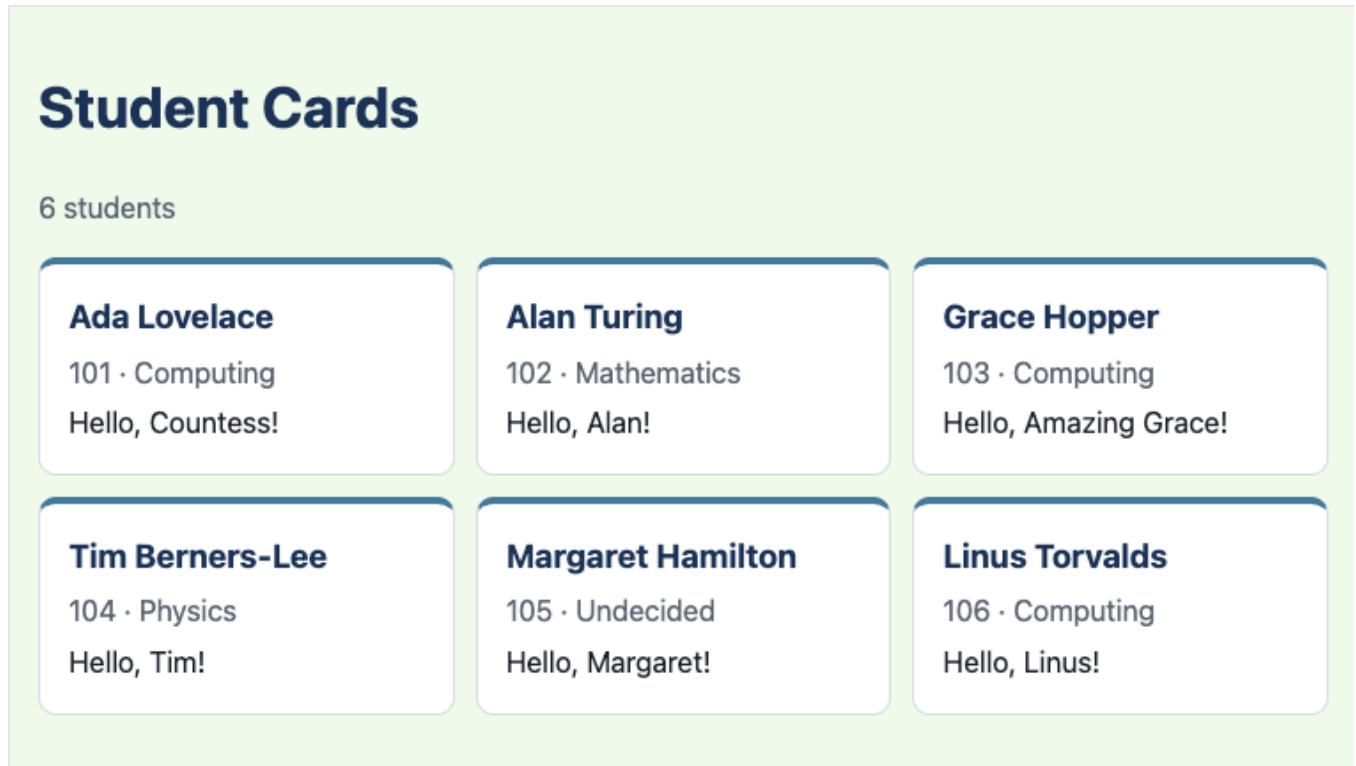
Start from your solution to Challenge 4. Made the way the hint suggests, `fromRoman` happily reads `"IIII"` (4), `"VX"` (5) and `"IIX"` (2) - none of which a Roman would write - and turns `"HELLO"` into 0. Make `fromRoman` throw an error, with a message naming the numeral, for anything that is not a proper Roman numeral. Test first: a group of steps for numerals that must be refused (empty string, lower case, other letters, `"IIII"`, `"VV"`, `"IC"`, `"MMMM"`) and make sure all the valid ones still work. Show the error's message on the page instead of a number.

Hint: you do not need a rule for every kind of mistake. There is exactly one proper way to write each number - and `toRoman` already knows it. Work the number out, turn it back into a numeral with `toRoman`, and compare. On the page, `try { ... } catch (error) { ... }` works as in Java; inside the `catch`, `error instanceof Error ? error.message : String(error)` gets the message.

Next: [Chapter 5 - Classes, objects and constructors](#)

Chapter 5 - Classes, objects and constructors

You have already written small classes: `Counter` in Chapter 2 and `Team` in Chapter 3. This chapter looks at classes properly, as you knew them in Java - objects, constructors and `toString()` - and shows where TypeScript does things its own way: constructors that declare their fields for you, default and optional parameters instead of several constructors, and turning plain data from a JSON file into real objects. Every class is built test first.



The Student Cards project

What you will learn

- classes and objects in TypeScript, compared with Java
- writing a constructor, and the shorter way: **parameter properties**
- default parameters (`course: string = "Undecided"`) and optional parameters (`nickname?: string`)
- why a TypeScript class has only one constructor, and what replaces Java's overloading
- writing `toString()`, and where TypeScript calls it for you
- making objects from JSON data with `map((d) => new Student(...))`
- what to test about a constructor, and about `toString()`
- keeping randomness out of a class by passing the random number in

The projects

Project	What it shows
ch05_project01_student_cards	a <code>Student</code> class with parameter properties, a default and an optional parameter, and <code>toString()</code> ; students made from JSON data
ch05_project02_die	a <code>Die</code> with any number of sides, built in small red-green steps; the random number is passed in
ch05_project03_module_list	a course's <code>Module</code> objects, filtered by semester and added up; <code>toString()</code> inside template literals and <code>join</code>

Classes and objects

A quick reminder of the words, which mean the same as in Java:

- a **class** is a blueprint: it says what data each object holds (its **fields**, also called properties) and what it can do (its **methods**)
- an **object** (an **instance** of the class) is one thing made from that blueprint, with its own values in the fields
- a **constructor** is the code that runs when an object is made with `new`, to set up its fields

From Chapter 2 you also know the differences you will see on every line: `export class` instead of `public class`, types written after names (`count: number`), `this.` always written in front of a field, and methods with `public` and a return type written out.

The constructor is where TypeScript differs most from Java, so that is where this chapter begins.

Project 1: Student cards

Student Cards

6 students

Ada Lovelace

101 · Computing
Hello, Countess!

Alan Turing

102 · Mathematics
Hello, Alan!

Grace Hopper

103 · Computing
Hello, Amazing Grace!

Tim Berners-Lee

104 · Physics
Hello, Tim!

Margaret Hamilton

105 · Undecided
Hello, Margaret!

Linus Torvalds

106 · Computing
Hello, Linus!

The Student Cards project

The first project shows a card for each student in a JSON file. Each card is made from a `Student` object. We build the class test first, as in Chapter 4.

The first test

The first thing a student must do is remember their name. The test says how a `Student` is made, and what it should give back:

`tests/Student.test.ts`

```
import { assertEquals } from "@std/assert";
import { Student } from "../src/Student.ts";

Deno.test("a new student has the full name it was given", () => {
  const student = new Student(101, "Ada", "Lovelace", "Computing");

  assertEquals(student.getFullName(), "Ada Lovelace");
});
```

There is no `src/Student.ts` yet, so this is as red as it gets - the tests cannot even start:

```
TS2307 [ERROR]: Cannot find module 'src/Student.ts'.
    at tests/Student.test.ts:4:25

error: Module not found "src/Student.ts".
    at tests/Student.test.ts:4:25

Tests: the tests could not run (see above) -> test_output/index.html
```

The long way

Here is the class written the way you would write it in Java: fields declared at the top, a constructor that takes the values as parameters, and a line for each field to copy the parameter in:

src/Student.ts (first version)

```
export class Student {
  private id: number;
  private firstName: string;
  private surname: string;
  private course: string;

  constructor(id: number, firstName: string, surname: string, course: string) {
    this.id = id;
    this.firstName = firstName;
    this.surname = surname;
    this.course = course;
  }

  public getFullName(): string {
    return `${this.firstName} ${this.surname}`;
  }
}
```

Compared with Java:

- the constructor is always called `constructor`, not the name of the class
- like a Java constructor, it has no return type
- `this.id = id;` works exactly as in Java: `this.id` is the field, `id` is the parameter. In TypeScript `this.` is never optional, so the field and the parameter can share a name with no confusion

Save, and the test is green:

```
ok 1 - a new student has the full name it was given
```

```
Tests: 0 failed, 1 passed, 0 skipped, 0 type errors, 0 lint warnings -> test_output/index.html
```

Note - Forget the `this.` and TypeScript tells you what you meant: Cannot find name 'firstName'. Did you mean the instance member 'this.firstName'?

Refactor: parameter properties

That class says everything three times. `firstName` is declared as a field, then again as a parameter, and then one is copied into the other. With four fields that is twelve lines of paperwork, and it is easy to add a field and forget one of the three.

TypeScript has a shorter way. Put `private` in front of a constructor parameter, and that parameter **is** the field: TypeScript declares the field and copies the argument into it for you. This is called a **parameter property**:

The same class, written two ways

The long way (as in Java)

```
export class Student {  
  private id: number;  
  private firstName: string;  
  constructor(id: number,  
              firstName: string) {  
    this.id = id;  
    this.firstName = firstName;  
  }  
}
```

fields + parameters + assignments: 3 places

Parameter properties

```
export class Student {  
  constructor(  
    private id: number,  
    private firstName: string,  
  ) {}  
}
```

the word **private** in front of a parameter declares the field **and** sets it from the argument: 1 place, nothing to forget



the field



the parameter



copying one into the other

Long form and parameter properties

src/Student.ts

```
export class Student {  
  constructor(  
    private id: number,  
    private firstName: string,  
    private surname: string,  
    private course: string,  
  ) {}  
  
  public getFullName(): string {  
    return `${this.firstName} ${this.surname}`;  
  }  
}
```

The constructor's body is empty - `{}` - because there is nothing left to do. The class has exactly the same four private fields as before, and the methods use them in the same way, with `this.`. The parameters are written one per line, with a comma after the last, so that adding one later is a one-line change.

This is a **refactor**: the code changed, the behaviour did not. The test proves it - it is still green, without being touched. From now on, this book writes constructors this way whenever the constructor's job is just to store its arguments.

Note - The word in front matters. A parameter with no `private` (or `public`, `protected`, or `readonly`) in front is just a parameter, gone when the constructor ends. Write `constructor(name: string) {}` and then `this.name` in a method, and TypeScript says `Property 'name' does not exist on type 'Student'.`

Exercise 5.1 - Shorten Team

Chapter 3's `Team` (in `ch03_project02_scoreboard/src/Team.ts`) declares a field and copies it in the constructor:

```
export class Team {
  private name: string;
  private score: number = 0;

  constructor(name: string) {
    this.name = name;
  }
  // ...
}
```

Rewrite it with a parameter property. Try it before reading on.

Here is one way:

```
export class Team {
  private score: number = 0;

  constructor(private name: string) {}
  // ...
}
```

`score` stays a normal field: it does not come from the constructor's arguments, it always starts at 0. A class can mix both kinds. Its tests pass unchanged.

A default parameter, test first

Not every student has chosen a course when they are added. Java would give you a second constructor for that. In TypeScript, we start with a test that says what should happen:

`tests/Student.test.ts`

```
Deno.test("a student with no course given is Undecided", () => {
  const student = new Student(102, "Alan", "Turing");

  assertEquals(student.getCourse(), "Undecided");
});
```

(`getCourse()` is a one-line method that returns `this.course`.) Two things go wrong, and both are useful. The type checker complains that the constructor wants four arguments:

```
TS2554 [ERROR]: Expected 4 arguments, but got 3.
  const student = new Student(102, "Alan", "Turing");
                        ~~~~~
  at tests/Student.test.ts:13:19

  An argument for 'course' was not provided.
```

and the test itself runs (tests run even with type errors, so you can see them go red) and fails, because a missing argument in JavaScript is simply `undefined`:


```
not ok 2 - a student with no course given is Undecided
---
message: |-
  AssertionError: Values are not equal.

      [Diff] Actual / Expected

-   undefined
+   "Undecided"
```

The fix is a **default parameter** - a value written after the type with `=`, used when no argument is given:

src/Student.ts

```
/** The course a student is on until they choose one. */
const DEFAULT_COURSE = "Undecided";
```

```
private course: string = DEFAULT_COURSE, // a default: used when no course (or undefined) is
given
```

Both problems go away: the type checker now knows that `course` may be left out, and when it is, the field gets `"Undecided"`. Green.

Default parameters work in any function or method, not only constructors - `roll(random: number, times: number = 1)` would be fine too.

Optional parameters

Some students have a nickname they would rather be called by. Most do not. For a value that may simply be missing, with no sensible default, write `?` after the parameter's name:

src/Student.ts

```
constructor(
  private id: number,
  private firstName: string,
  private surname: string,
  private course: string = DEFAULT_COURSE, // a default: used when no course (or undefined) is
given
  private nickname?: string, // optional: may be left out, and is then undefined
) {}
```

`nickname?: string` makes the field's type `string | undefined` - the same kind of union as `Student | undefined` from `find` in Chapter 3 - and TypeScript makes sure you check before using it:

```
/** The name to greet the student by: their nickname if they have one, otherwise their first
name. */
public getGreetingName(): string {
  if (this.nickname === undefined) {
    return this.firstName;
  }
  return this.nickname;
}
```

Both cases get a test:

tests/Student.test.ts

```
Deno.test("a student with a nickname is greeted by it", () => {
  const student = new Student(101, "Ada", "Lovelace", "Computing", "Countess");

  assertEquals(student.getGreetingName(), "Countess");
});

Deno.test("a student with no nickname is greeted by their first name", () => {
  const student = new Student(102, "Alan", "Turing", "Mathematics");

  assertEquals(student.getGreetingName(), "Alan");
});
```

Arguments are matched to parameters by position, so to give a nickname but no course, you pass `undefined` for the course - and a default parameter treats `undefined` exactly like a missing argument:

```
Deno.test("passing undefined for the course also gives the default", () => {
  const student = new Student(105, "Margaret", "Hamilton", undefined, "Maggie");

  assertEquals(student.getCourse(), "Undecided");
});
```

That `undefined` in the middle is a little ugly. It is fine for one or two optional values; when a constructor collects many of them, there is a better design (Challenge 6 asks you to find it).

One constructor, not several

In Java you might have written three constructors for `Student`: one with every value, one without the course, one without anything. That is **overloading** - several methods with the same name and different parameter lists - and Java picks one by the arguments you pass.

A TypeScript class has **one** constructor. Try to write two and you get:

```
TS2392 [ERROR]: Multiple constructor implementations are not allowed.
```

The reason is that types vanish when TypeScript is turned into JavaScript, so at run time there is nothing to choose between constructors with. Default and optional parameters do the same job, in one place:

Java	TypeScript
<code>Student(int id, String first, String surname, String course)</code>	<code>new Student(101, "Ada", "Lovelace", "Computing")</code>
<code>Student(int id, String first, String surname)</code> calling <code>this(id, first, surname, "Undecided")</code>	<code>new Student(102, "Alan", "Turing")</code> - the default fills in the course
a fifth constructor for the nickname	<code>new Student(101, "Ada", "Lovelace", "Computing", "Countess")</code>

The rule for the parameter list: **required parameters first, then the ones with defaults or ?**. An optional parameter followed by a required one is an error - A required parameter cannot follow an optional parameter. - because there would be no way to leave the optional one out.

Note - TypeScript does have a way to declare overloads (several signatures above one implementation), but the one implementation still has to sort out the arguments itself. Defaults and optional parameters are simpler, and are what this book uses.

toString, test first

When you print an object in Java, `toString()` decides what you see. TypeScript (really JavaScript) has the same idea. Here is the test for the summary we want:

tests/Student.test.ts

```
Deno.test("toString gives the id, full name and course", () => {
  const student = new Student(101, "Ada", "Lovelace", "Computing");

  assertEquals(student.toString(), "(Student) 101 Ada Lovelace, Computing");
});
```

Before writing the method, run it. You might expect a type error - `Student` has no `toString` - but there is none. Every object already has a `toString()`, just as every Java object inherits one from `Object`. The test fails with the built-in version's answer, which is even less helpful than Java's `Student@6d06d69c`:

```
not ok 3 - toString gives the id, full name and course
---
message: |-
  AssertionError: Values are not equal.

      [Diff] Actual / Expected

-   [object Object]
+   (Student) 101 Ada Lovelace, Computing

Tests: 1 failed, 2 passed, 0 skipped, 0 type errors, 0 lint warnings -> test_output/index.html
```

Now write our own:

src/Student.ts

```
/** A short summary of the student, for showing or logging - like Java's toString(). */
public toString(): string {
  return `(Student) ${this.id} ${this.getFullName()}, ${this.course}`;
}
```

Green. As in Java, `toString()` **returns** a string; it does not print anything. What happens to the string is up to the code that asked for it.

The point of `toString()` is that you rarely call it yourself. JavaScript calls it whenever an object has to become a string - in a template literal, for example:

```
Deno.test("a template literal uses toString", () => {
  const student = new Student(102, "Alan", "Turing");

  assertEquals(`${student}`, "(Student) 102 Alan Turing, Undecided");
});
```

"" + student, String(student) and join on an array of students all call it too. One thing does not: console.log(student) shows the object's fields instead - in Deno, Student { id: 102, firstName: "Alan", ... }. To log your summary, log `\${student}`, as main.ts does.

Note - In Java you would write @Override above toString(). TypeScript's override keyword is only for classes that extend another class, which Student does not; Chapter 11 uses it.

From JSON data to objects

The students come from a JSON file, as the marks did in Chapter 3. Some have no course, and most have no nickname:

src/students.json

```
[
  { "id": 101, "firstName": "Ada", "surname": "Lovelace", "course": "Computing", "nickname":
    "Countess" },
  { "id": 102, "firstName": "Alan", "surname": "Turing", "course": "Mathematics" },
  ...
  { "id": 105, "firstName": "Margaret", "surname": "Hamilton" },
  ...
]
```

Importing the file gives an array of **plain objects** - values, with no methods. In Chapter 3 that was all we needed. Now we want Student objects, with getGreetingName(), toString() and the default course. map turns each piece of data into an object by calling the constructor:

From plain data to objects

students.json (data)

```
{ id: 101,
  firstName: "Ada", ... }
```

```
{ id: 105,
  firstName: "Margaret" }
```

just values - no methods

data.map((d) =>

new Student(d.id, ...))
the constructor runs once for each

Student objects

```
Ada Lovelace · Computing
getFullName() toString() ...
```

```
Margaret Hamilton · Undecided
getFullName() toString() ...
```

values + behaviour; defaults filled in

One class (the blueprint), as many objects as there are rows of data

From plain data to objects

src/students.ts

```

/** The shape of one student in students.json. course and nickname may be missing. */
export type StudentData = {
  id: number;
  firstName: string;
  surname: string;
  course?: string;
  nickname?: string;
};

/** One Student object for each piece of data. A missing course becomes the default. */
export const studentsFrom = (data: StudentData[]): Student[] =>
  data.map((d) => new Student(d.id, d.firstName, d.surname, d.course, d.nickname));

```

The `?` works in a type too: `course?: string` means the property may be missing. A missing property reads as `undefined`, so `d.course` passes `undefined` to the constructor - and the default parameter turns that into `"Undecided"`. The data and the class fit together with no `if`.

The tests use a small array of their own, and check one thing the diagram promises - that the data has no methods of its own, and the objects do:

tests/students.test.ts

```

Deno.test("the objects made have the class's methods - plain data does not", () => {
  const students = studentsFrom(DATA);

  assertEquals(students[0].toString(), "(Student) 1 Ada Lovelace, Computing");
  assertEquals(`${DATA[0]}`, "[object Object]");
});

```

main.ts makes the objects once, then builds a card from each object's methods:

src/main.ts

```

import data from "./students.json" with { type: "json" };
import { studentsFrom } from "./students.ts";

const STUDENTS = studentsFrom(data);

for (const student of STUDENTS) {
  const card = document.createElement("article");
  card.className = "card student";

  const name = document.createElement("h2");
  name.textContent = student.getFullName();
  // ... the id and course, and "Hello, Countess!"

  // toString is for logging: open the browser's console to see one line per student.
  console.log(`${student}`);
}

```

Try it - Open `dist/index.html` in a browser and open the developer tools' console: six lines, one per student, each from `toString()`. Then change the log to `console.log(student)` and compare.

Testing constructors

What should you test about a constructor? Not that TypeScript can copy a value into a field - that would be testing TypeScript. Test what **your** code decides:

- that the values given come back out (through the methods that use them, such as `getFullName()`)
- every default - "a student with no course given is Undecided"
- every optional parameter, both given and left out
- `toString()`, exactly, because other code (and people) rely on what it says

Many mistakes never reach a test, because the type checker catches them first:

Mistake	What TypeScript says
<code>new Student(101)</code>	Expected 3-5 arguments, but got 1.
<code>new Student("101", "Ada", "Lovelace")</code>	Argument of type 'string' is not assignable to parameter of type 'number'.
<code>Student(101, "Ada", "Lovelace")</code> - no <code>new</code>	Value of type 'typeof Student' is not callable. Did you mean to include 'new'?

Notice the "3-5": TypeScript counts the required parameters and the ones that may be left out.

Two objects with the same values. Chapter 4 showed that `assertEquals` compares what things contain, while `assertStrictEquals` asks whether they are the very same thing. With objects made by a constructor:

```
assertEquals(new Student(1, "Ada", "Lovelace"), new Student(1, "Ada", "Lovelace")); // passes
assertStrictEquals(new Student(1, "Ada", "Lovelace"), new Student(1, "Ada", "Lovelace")); // fails
```

The second fails with `Values have the same structure but are not reference-equal.` - two `new`s make two objects, just as in Java, where `==` would be `false`. `assertEquals` also checks the class: a `Student` is not equal to a plain object with the same fields (the diff shows `- Student {` and `+ {`). Chapter 9 says much more about objects and references.

Project 2: A die

Dice Roller

Sides

6



Roll

6

d6 showing 6

Last rolls: 5, 2, 6

The Dice Roller project

Board games use dice with 4, 6, 8, 10, 12 or 20 sides. A `Die` class needs to know how many sides it has - six, unless you say otherwise - and to roll.

Rolling is random, and random code is hard to test: what would the test expect? Chapter 2's coin toss solved this by passing the random number `in`. The die does the same: `roll(random)` is given a number from 0 up to (but not including) 1 - the kind `Math.random()` gives - and the page passes `Math.random()`, while the tests pass numbers they choose.

The constructor

src/Die.ts

```
/** The ordinary die has six sides. */
const DEFAULT_SIDES = 6;

export class Die {
  // What the die shows: null until it has been rolled for the first time.
  private value: number | null = null;

  constructor(private sides: number = DEFAULT_SIDES) {}

  public getSides(): number {
    return this.sides;
  }
}
```

A parameter property with a default: `new Die()` has six sides, `new Die(20)` has twenty. The class also has an ordinary field, `value`, which does not come from the constructor: every new die starts "not rolled yet", which `number | null` says honestly (a die showing 0, or 1, before anyone rolled it would be a small lie). Its tests:

tests/Die.test.ts

```
Deno.test("a die has six sides unless you say otherwise", () => {
  assertEquals(new Die().getSides(), 6);
});

Deno.test("a die can have any number of sides", () => {
  assertEquals(new Die(20).getSides(), 20);
});

Deno.test("a new die has not been rolled", () => {
  assertEquals(new Die().getValue(), null);
});
```

Rolling, in small steps

Chapter 4's habit: one small test, the least code that passes it, then the next test. The smallest random number is 0, and it should roll a 1:

tests/Die.test.ts

```
Deno.test("the lowest random number rolls a 1", () => {
  const die = new Die();

  assertEquals(die.roll(0), 1);
});
```

Red - with a type error, and a test that fails because there is no such method yet:

```
TS2339 [ERROR]: Property 'roll' does not exist on type 'Die'.
  assertEquals(die.roll(0), 1);
                  ~~~~
    at tests/Die.test.ts:15:20

not ok 3 - the lowest random number rolls a 1
---
message: |-
  TypeError: die.roll is not a function
```

The least code that passes:

```
public roll(random: number): number {
  return 1;
}
```

Green - but the console also says 1 lint warnings. The linter has noticed what we know: `random` is never used. That is a good sign that the next test is needed. The highest random numbers, just below 1, should roll a 6:


```

/** Just below 1: the largest number Math.random() can give is a tiny bit less than 1. */
const ALMOST_ONE = 0.999999;

Deno.test("a random number just below 1 rolls a 6", () => {
  const die = new Die();

  assertEquals(die.roll(ALMOST_ONE), 6);
});

```

not ok 4 - a random number just below 1 rolls a 6

```

---
message: |-
  AssertionError: Values are not equal.

```

```

      [Diff] Actual / Expected

```

```

-   1
+   6

```

Now "always 1" will not do. Multiplying the random number by the number of sides gives a number from 0 up to (not including) 6; `Math.floor` cuts off the decimals, giving 0 to 5; adding 1 gives 1 to 6. And the die should remember what it rolled:

src/Die.ts

```

public roll(random: number): number {
  this.value = Math.floor(random * this.sides) + 1;
  return this.value;
}

```

Green, with no lint warning. More tests pin down the rest: 0.5 rolls a 4, a twenty-sided die can roll a 20, and after a roll `getValue()` gives what was rolled. Each one passes straight away - which is fine: they record what the die promises, so that a later change cannot quietly break it.

Two kinds of toString

`toString` has two cases, so it has two tests:

src/Die.ts

```

/** For example "d6 showing 4", or "d20 (not rolled yet)". */
public toString(): string {
  if (this.value === null) {
    return `d${this.sides} (not rolled yet)`;
  }
  return `d${this.sides} showing ${this.value}`;
}

```

("d6" and "d20" are how gamers write dice.)

Making objects while the page runs

The page starts with a six-sided die. Choosing a different number of sides does not change the die - it makes a **new** one:

src/main.ts

```
let die = new Die();
let history: number[] = [];
```

```
sidesChoice?.addEventListener("change", () => {
  die = new Die(Number(sidesChoice.value));
  history = [];
  render();
});

rollButton?.addEventListener("click", () => {
  // The only place Math.random() is called: the Die is given the number, so it can be tested.
  const value = die.roll(Math.random());
  history.push(value);
  render();
});
```

`die` is a `let`, because it is replaced. The old die is simply forgotten (JavaScript, like Java, cleans up objects nothing refers to any more). `render()` shows `${die}` - its `toString()` - under the big number.

Exercise 5.2 - A coin is a die

A coin is a die with two sides. Write two tests: a two-sided die rolls a 1 for 0.4, and a 2 for 0.6. Do you need to change `Die`? Try it before reading on.

Here is one way:

```
Deno.test("a two-sided die rolls a 1 for 0.4 and a 2 for 0.6", () => {
  const coin = new Die(2);

  assertEquals(coin.roll(0.4), 1);
  assertEquals(coin.roll(0.6), 2);
});
```

No change is needed - both pass straight away. The constructor made the number of sides a value, not a fixed 6, so one class covers every die.

Project 3: A module list

Module List

Whole year

Semester 1

Semester 2

Semester 1: 5 modules, 35 credits

COMP1001 Programming 1 (10 credits)

Semester 1

COMP1002 Computer Architecture (5 credits)

Semester 1

MATH1001 Discrete Mathematics (5 credits)

Semester 1

COMP1003 Web Development (5 credits)

Semester 1

COMP1010 Team Project (10 credits)

All year

The Module List project

The last project lists the modules of a course year, with buttons to show one semester at a time and the total credits. It brings the chapter together: a class with a default and an optional parameter, objects made from JSON, and `toString()` doing quiet work.

The Module class

Most modules are worth 5 credits. Most run in one semester, but a few - such as a team project - run all year:

`src/Module.ts`

```
/** Most modules are worth this many credits. */
const DEFAULT_CREDITS = 5;

export class Module {
  constructor(
    private code: string,
    private title: string,
    private credits: number = DEFAULT_CREDITS,
    private semester?: number, // optional: undefined means the module runs all year
  ) {}
}
```

Here `undefined` is not just "not given" - it has a meaning, and the class gives that meaning a name, so no other code needs to know how it is stored:

```

/** True when the module has no semester, because it runs all year. */
public isYearLong(): boolean {
    return this.semester === undefined;
}

/** Does this module run in the given semester? A year-long module runs in both. */
public runsIn(semester: number): boolean {
    return this.isYearLong() || this.semester === semester;
}

/** For example "COMP1001 Programming 1 (10 credits)". */
public toString(): string {
    return `${this.code} ${this.title} (${this.credits} credits)`;
}

```

The tests cover the default ("a module is worth 5 credits unless you say otherwise"), the optional semester both ways, and `runsIn` for both kinds of module:

tests/Module.test.ts

```

Deno.test("a year-long module runs in both semesters", () => {
    const module = new Module("COMP1010", "Team Project", 10);

    assertEquals(module.runsIn(1), true);
    assertEquals(module.runsIn(2), true);
});

```

Functions that work on objects

The list of modules is made from JSON exactly as the students were, and two small functions answer the page's questions. They are the array methods from Chapter 3, now working on objects and calling their methods:

src/modules.ts

```

/** One Module object for each piece of data. Missing credits become the default. */
export const modulesFrom = (data: ModuleData[]): Module[] =>
    data.map((d) => new Module(d.code, d.title, d.credits, d.semester));

/** The modules that run in a semester - including the year-long ones. */
export const modulesIn = (modules: Module[], semester: number): Module[] =>
    modules.filter((module) => module.runsIn(semester));

/** The credits of all the modules added together. */
export const totalCredits = (modules: Module[]): number =>
    modules.reduce((total, module) => total + module.getCredits(), 0);

```

Not everything has to be a method. `totalCredits` is about a *list* of modules, not one module, so it is a plain function. (A `Course` class that holds its modules is a good idea too - that is composition, in Chapter 9.)

toString doing quiet work

Each line of the list is just the module, in a template literal:

src/main.ts

```
const item = document.createElement("li");
item.textContent = `${module}`; // toString: "COMP1001 Programming 1 (10 credits)"
```

and `join`, which glues an array into one string, calls `toString()` on each element:

tests/modules.test.ts

```
Deno.test("toString makes joining a list of modules easy", () => {
  const modules = modulesFrom(DATA).slice(0, 2);

  assertEquals(modules.join("; "), "A1 Alpha (10 credits); B2 Beta (5 credits)");
});
```

(`slice(0, 2)` is a copy of the first two elements.) The page's state is one variable, `semester`, which is a number or `null` for "the whole year"; the three buttons set it and call `render()` - the pattern from Chapters 2 and 3.

Exercise 5.3 - Year-long credits

Write a function `yearLongCredits(modules: Module[]): number` that adds up the credits of the year-long modules only. Write the test first, using the made-up `DATA` in `tests/modules.test.ts`. Try it before reading on.

Here is one way:

```
Deno.test("only the year-long module's credits are counted", () => {
  assertEquals(yearLongCredits(modulesFrom(DATA)), 5);
});

export const yearLongCredits = (modules: Module[]): number =>
  totalCredits(modules.filter((module) => module.isYearLong()));
```

It reuses `totalCredits` instead of writing another `reduce`, and asks each module `isYearLong()` instead of checking `getSemester() === undefined` itself.

A class or a type?

Chapter 3 described a student with a `type`: `type Student = { name: string; mark: number }`. This chapter uses classes. When is each right?

- a **type** describes the **shape** of plain data - what comes out of a JSON file, for example. It has no constructor and no methods, and it vanishes when the code is compiled
- a **class** is for things with **behaviour**: defaults decided in one place (the constructor), methods such as `getGreetingName()` and `runsIn()`, and private fields that other code cannot reach

This chapter uses both, side by side: `StudentData` is the shape of the data in the file; `Student` is the object the program works with. `studentsFrom` is the bridge between them.

Java and TypeScript

Java	TypeScript
<pre>public Student(int id) { this.id = id; }</pre>	<pre>constructor(private id: number) {}</pre> - a parameter property
the constructor is named after the class	the constructor is always called <code>constructor</code>
several overloaded constructors	one constructor, with default (<code>= "Undecided"</code>) and optional (<code>?</code>) parameters
<pre>this(id, first, surname, "Undecided")</pre> to call another constructor	a default parameter
a missing argument is a compile error	a missing argument is <code>undefined</code> at run time - and a type error, so you see it
<pre>@Override public String toString()</pre>	<pre>public toString(): string</pre> (override only when the class extends another)
<pre>"" + student, println(student)</pre> call <code>toString()</code>	<pre>`\${student}`, String(student), join</pre> call <code>toString()</code> ; <code>console.log(student)</code> does not
default <code>toString(): Student@6d06d69c</code>	default <code>toString(): [object Object]</code>
<code>a == b</code> on objects: the same object?	<code>assertStrictEquals</code> (and <code>===</code>): the same object; <code>assertEquals</code> : the same contents

Summary

- a class is a blueprint; `new` makes an object from it and runs the constructor
- the constructor is called `constructor`; `this.` is always needed for fields
- a **parameter property** - `private` in front of a constructor parameter - declares the field and sets it in one go; use it when the constructor just stores its arguments
- a class has only one constructor; **default parameters** (`= value`) and **optional parameters** (`name?: type`, which may be `undefined`) replace Java's overloading. Required parameters come first
- passing `undefined` to a parameter with a default gives the default
- write `toString()` to return a useful summary; template literals, `String()` and `join` call it, `console.log` does not
- plain data from JSON has no methods; `data.map((d) => new Student(...))` turns it into objects
- test what your constructor decides: values kept, defaults, optional parameters, and `toString()` exactly
- pass anything random into a method, so that tests can choose it

Challenges

Each challenge says which project to start from. Write the tests first, and watch each one fail before you make it pass.

1. Initials

Start from `ch05_project01_student_cards`. Give `Student` a method `getInitials()` that returns the first letters of the first name and surname, such as "AL" for Ada Lovelace. Write the tests first (including a surname with a hyphen, Tim Berners-Lee, which should still give "TB"), then show the initials in a circle at the top of each card.

2. Critical roll

Start from `ch05_project02_die`. Give `Die` a method `isMaximum(): boolean`, true when the die shows its highest number. Test first: a d20 that rolled 20; a d20 that rolled 19; a die that has not been rolled yet (false). On the page, show "Maximum!" and give the number a different colour when `isMaximum()` is true.

3. Who teaches it?

Start from `ch05_project03_module_list`. Add an optional `lecturer` to `Module`, and to some (not all) of the modules in `modules.json`. `toString()` should add " - taught by Dr Byrne" only when there is a lecturer. Write a test for each case first. Where must the new parameter go in the constructor, and why?

4. Enough credits?

Start from `ch05_project03_module_list`. A full year is 60 credits. Write a function `creditCheck(modules: Module[], target: number = 60): string` that gives "60 credits - complete", "55 credits - 5 short" or "65 credits - 5 over". Write the tests first, for all three cases and for a different target (a part-time year of 30). Show the result under the list when the whole year is shown.

Hint: use `totalCredits`. A default parameter works in a plain function exactly as in a constructor.

5. A cup of dice

Start from `ch05_project02_die`. Many games roll two or three dice and add them up. Write a class `DiceCup` whose constructor takes how many dice and how many sides each has (six unless you say otherwise), and makes that many `Die` objects. `roll(random: () => number): number` rolls every die and returns the total, and `toString()` gives, for example, "2d6: 3 + 5 = 8". Write the tests first. Add a choice of 1, 2 or 3 dice to the page.

Hint: the constructor can fill a private `Die[]` with a `for` loop and `push`. `roll` takes a *function* (a function type, Chapter 3) because each die needs its own random number: the page passes `Math.random` itself (no brackets); a test passes a fake that hands out numbers it chose, one per call - for example an arrow function that returns the next number from an array each time.

6. Too many arguments

Start from `ch05_project01_student_cards`. Students also need a year of study (1 unless you say otherwise) and an optional email address. Adding two more parameters to the end of the constructor would mean calls like `new Student(105, "Margaret", "Hamilton", undefined, undefined, 2)`. Change the design so that the constructor takes **one** argument, a `StudentData` object, and `studentsFrom` becomes

`data.map((d) => new Student(d))` . Rewrite the tests first, so that they show how much easier the new calls are to read. Add the year to `toString()` and to the cards.

Hint: the constructor can no longer use parameter properties for each value. Keep private fields, and set them in the constructor's body from the object, using the default when a value is `undefined` . Think about what is better and what is worse about the new design - is it clearer what a `Student` needs?

Next: [Chapter 6 - Encapsulation](#)

Chapter 6 - Encapsulation

An object should look after its own data. If any code anywhere can set a pet's age to -3 or a bank balance to a million, then every bug anywhere can do it too. **Encapsulation** means hiding an object's data behind methods that check every change, so the object is always in a sensible state. You know the idea from Java; this chapter shows how TypeScript does it - which is mostly the same, with two twists: a second kind of `private`, and `get / set` accessors.

Bank Account

Aoife Byrne · account IE-0001

Balance: €20.00

€

Deposit

Withdraw

Not enough money: the balance is €20.00

The Bank Account project

What you will learn

- why data is hidden, and what "the outside" and "the inside" of a class are
- `private` fields and methods, and why TypeScript's `private` is only checked by the compiler
- `#private` fields and methods, which stay private while the program runs
- `readonly` fields, which are set once and never changed
- `getX()`, `isX()` and `setX()` methods, the Java way, and `get / set` accessors, the TypeScript way
- making setters and constructors refuse bad values by throwing an `Error`, and catching it on the page
- invariants: rules an object keeps true all the time
- testing refused input with `assertThrows`, and checking that a refusal changes nothing

The projects

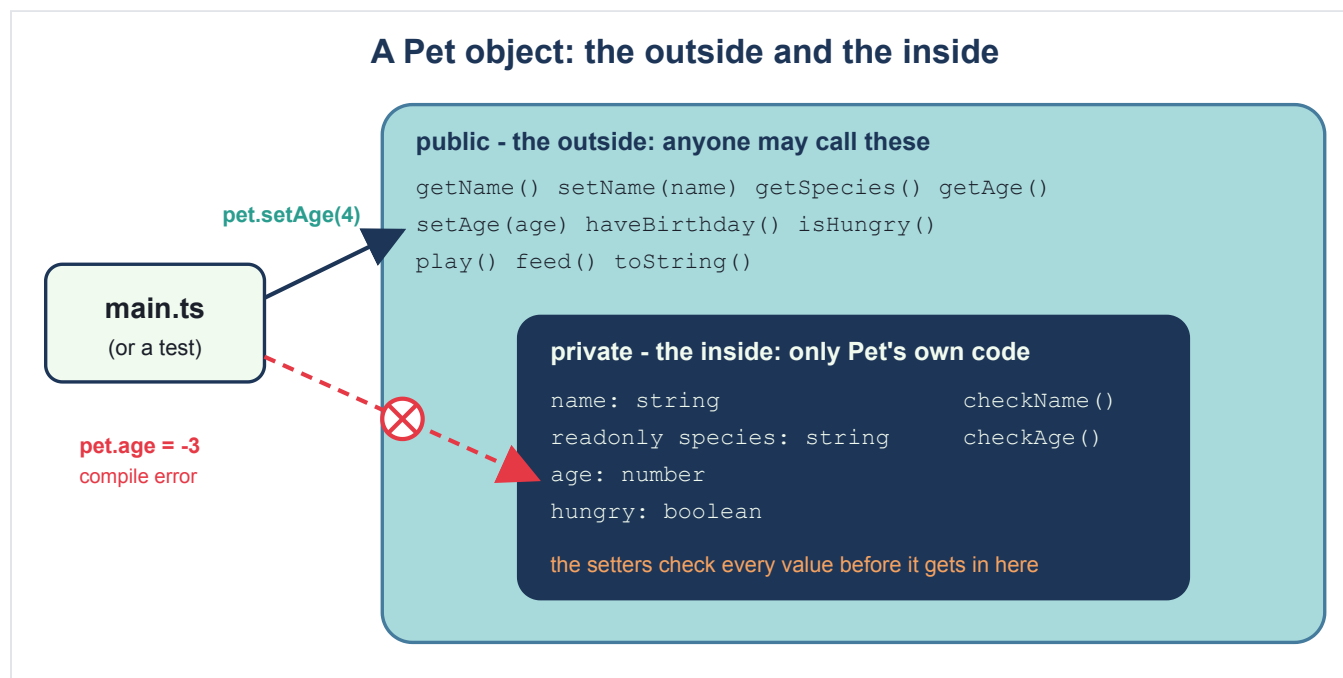
Project	What it shows
ch06_project01_pet	the Java Pet exercise: <code>private</code> and <code>readonly</code> fields, getters, setters that check their values, <code>isHungry()</code> , <code>try / catch</code> on the page
ch06_project02_bank_account	a <code>#private</code> balance, a <code>get</code> accessor with no setter, deposit and withdraw rules, the invariant "never below zero"
ch06_project03_thermostat	a <code>get / set</code> pair whose setter checks the value, a computed property, and when a plain public field is fine

The outside and the inside

Chapter 2's `Counter` was already encapsulated, though nobody said so. Its `count` field was `private`, and the only ways to change it were `increment()` and `reset()`. So there was no way for `main.ts` to set the count to -7 or 2.5. The class promised that the count was always a whole number, 0 or more - and it could keep that promise because nobody else could touch the field.

Every class has two parts:

- **the outside** - its `public` methods (and any public fields). This is what other code can use. It is a promise: change it, and other code breaks
- **the inside** - its `private` fields and helper methods. Only the class's own code can use them. You can change them whenever you like, and as long as the outside behaves the same, nothing else notices



A Pet object: the outside and the inside

Hiding the inside does two jobs. It **protects** the data: if every change goes through a method, the method can check it, and bad values never get in. And it **frees** you to change the inside: in Chapter 3's challenge 2, the scoreboard's `Team` changed from storing a score to storing a list of baskets, and not one test had to change, because the tests only used the outside.

That second point matters for testing too. A test that only calls public methods tests what the class *does*, not how it does it - so you can refactor the inside with the tests staying green. This chapter's tests never reach inside an object.

Project 1: The pet

Pet

(PET) Rex is a dog, and is 4 years old.

Rex is hungry!

PlayFeedBirthday

Rename

A pet needs a name

The Pet project

The first project is the Pet exercise from the Java course: a pet with a name, a species and an age (5 if none is given), private fields, getters and setters, and a `toString()`. This version adds one thing the Java exercise left out: the setters **check** what they are given.

Private fields, getters and setters

`src/Pet.ts`

```

const DEFAULT_AGE = 5;

export class Pet {
  private name: string;
  // readonly: set once, in the constructor, and never again. A dog stays a dog.
  private readonly species: string;
  private age: number;
  private hungry: boolean = false;

  constructor(name: string, species: string, age: number = DEFAULT_AGE) {
    // The constructor uses the same checks as the setters, so a Pet cannot start life invalid
    either.
    this.name = this.checkName(name);
    this.species = species;
    this.age = this.checkAge(age);
  }

  public getName(): string {
    return this.name;
  }

  /** Renames the pet. Throws an Error (and keeps the old name) if the new name is blank. */
  public setName(name: string): void {
    this.name = this.checkName(name);
  }
  // ...
}

```

This is the Java pattern, written in TypeScript: every field `private`, a `getX()` to read it, a `setX()` to change it. Try to reach a private field from `main.ts`:

```

const pet = new Pet("Rex", "dog", 3);
pet.age = -3;

```

and the compiler stops you:

```

TS2341 [ERROR]: Property 'age' is private and only accessible within class 'Pet'.
pet.age = -3;
  ~~~

```

Private methods work the same way. `checkName` and `checkAge` are helpers for the class's own use; `pet.checkAge(3)` in `main.ts` gives `Property 'checkAge' is private and only accessible within class 'Pet'.` Keeping helpers private keeps the outside small: fewer methods for other code to know about, and fewer to keep working forever.

Note - In TypeScript, a member with no visibility written is `public` (in Java it would be "package-private"). This book always writes `public` anyway, so that you can see the choice was made on purpose.

isX() for booleans

`hungry` is a boolean, so its getter follows the same convention as in Java: `isHungry()`, not `getHungry()`. It reads as a question, which is how it is used:

src/main.ts

```
mood.textContent = pet.isHungry() ? `${pet.getName()} is hungry!` : `${pet.getName()} is happy.`;
```

There is no `setHungry(true)`. Instead there are `play()` and `feed()`, which say what happens to the pet rather than which field changes. A setter for every field is a habit worth questioning: only add the ways of changing an object that make sense for it.

readonly

A pet's species never changes, so `species` is `readonly`. A `readonly` field must be given its value where it is declared or in the constructor, and can never be changed after that - not even by the class's own methods. Add a method that tries:

```
public adopt(): void {  
    this.species = "cat";  
}
```

and the compiler says:

```
TS2540 [ERROR]: Cannot assign to 'species' because it is a read-only property.  
    this.species = "cat";  
    ~~~~~
```

`readonly` is TypeScript's version of a Java `final` field. As there is nothing a setter could do, `Pet` has a `getSpecies()` and no `setSpecies()`. (Like Java's `final`, `readonly` only stops the field being pointed at something else. If a `readonly` field holds an array, the array itself can still be changed - Chapter 9 explains why.)

Setters that say no, test first

A setter that just copies its value into the field protects nothing: `setAge(-3)` would do exactly what `pet.age = -3` would have done. The point of a setter is that it can **refuse**. Here is `setAge` being made to refuse, red-green-refactor, starting from the plain version:

```
public setAge(age: number): void {  
    this.age = age;  
}
```

Red. First the test. It says what should happen to a negative age: an `Error` is thrown, and the old age is still there.

tests/Pet.test.ts

```
Deno.test("setAge refuses a negative age", () => {  
    const pet = new Pet("Rex", "dog", 3);  
  
    assertThrows(() => pet.setAge(-1), Error, "Age must be a whole number");  
    assertEquals(pet.getAge(), 3);  
});
```

`assertThrows` (Chapter 4) calls the arrow function and passes only if it throws. The second and third arguments say what kind of error and what its message must include. The plain setter throws nothing, so the test is red:

```
setAge refuses a negative age => ./tests/Pet.test.ts:58:6
error: AssertionError: Expected function to throw.
```

Green. The smallest change that passes:

```
public setAge(age: number): void {
  if (age < 0) {
    throw new Error(`Age must be a whole number, 0 or more (not ${age})`);
  }
  this.age = age;
}
```

The check comes **before** the change. If the age is refused, `throw` leaves the method at once, and `this.age = age` never runs - so the old age is kept, and the second line of the test passes too.

Red again. An age of 2.5 makes no sense either. The next test,

```
Deno.test("setAge refuses an age that is not a whole number", () => {
  const pet = new Pet("Rex", "dog", 3);

  assertThrows(() => pet.setAge(2.5), Error, "Age must be a whole number");
});
```

fails with the same `Expected function to throw.`, and is made green by widening the check to `!Number.isInteger(age) || age < 0`. (`Number.isInteger` is true for whole numbers only; it is false for 2.5, and for `NaN`, which is what `Number("abc")` gives.)

Red a third time. The setter is safe, but the constructor is not: `new Pet("Rex", "dog", -1)` still makes a pet aged -1. The test `assertThrows(() => new Pet("Rex", "dog", -1), Error, "Age must be")` fails with `Expected function to throw.`

Refactor. Copying the `if` into the constructor would make it pass, but then the rule would be written twice, and one day the copies would disagree. Instead, the check moves into a private helper that both use:

src/Pet.ts

```
/** Changes the age. Throws an Error (and keeps the old age) unless it is a whole number, 0 or more. */
public setAge(age: number): void {
  this.age = this.checkAge(age);
}

/** Gives back the age if it is allowed; throws an Error if not. */
private checkAge(age: number): number {
  if (!Number.isInteger(age) || age < 0) {
    throw new Error(`Age must be a whole number, 0 or more (not ${age})`);
  }
  return age;
}
```

and the constructor says `this.age = this.checkAge(age);`. All the tests are green, and the age rule lives in one place.

Why does the constructor not just call `this.setAge(age)` ? Try it, with the name:

```
constructor(name: string, species: string, age: number = DEFAULT_AGE) {  
  this.setName(name);  
  // ...  
}
```

```
TS2564 [ERROR]: Property 'name' has no initializer and is not definitely assigned in the  
constructor.  
  private name: string;  
    ~~~~~
```

TypeScript checks that every field gets a value in the constructor, but it does not look inside the methods the constructor calls. Assigning the field directly - `this.name = this.checkName(name)` - keeps the check and lets the compiler see the assignment.

Note - This is also why `Pet` does not use Chapter 5's parameter properties (`constructor(private name: string)`). A parameter property copies the argument into the field before the constructor's body runs, unchecked. They are perfect for values with no rules - Project 2's account owner - but a field with a rule needs a check on the way in.

Wrong message? The test says so

`assertThrows` with a message checks the message as well. If `setAge` threw `new Error("No negative ages")`, the test would still be red:

```
error: AssertionError: Expected error message to include "Age must be a whole number", but got "No  
negative ages".
```

Is that too fussy? The message is part of the outside: the page shows it to the user. Check a distinctive part of it, not every word, so that the wording can be improved without breaking tests.

Catching the error on the page

When the user types a blank name and presses Rename, `setName` throws. If nothing catches the error, the listener stops halfway, and the user sees nothing at all. `main.ts` catches it with `try / catch`, which works as in Java:

```
src/main.ts
```

```
document.querySelector("#rename")?.addEventListener("click", () => {
  if (nameBox === null) {
    return;
  }
  // setName throws an Error if the name is blank. try/catch (as in Java) catches it, and the
  // page shows the Error's message instead of stopping.
  try {
    pet.setName(nameBox.value);
    showMessage("");
  } catch (error) {
    showMessage(error instanceof Error ? error.message : String(error));
  }
  render();
});
```

Two differences from Java. There is one `catch`, with no type in brackets: TypeScript cannot know what was thrown (JavaScript lets you `throw` anything, even a string), so `error` has the type `unknown`. And because it is `unknown`, the code must check what it is before using it: `error instanceof Error` works as in Java, and after that check TypeScript knows `error.message` exists. Book 2's chapter on errors goes much further; for now, this is the pattern to copy.

Notice who does what. `Pet` decides what is allowed and says why not. `main.ts` decides how to show it. The rule is not written in `main.ts` at all - so it cannot be forgotten by the next page that uses `Pet`.

Exercise 6.1 - A species is needed too

`new Pet("Rex", "")` makes a pet with no species. Make the constructor refuse a blank species, with the message "A pet needs a species". Write the test first. Try it before reading on.

Here is one way. The test:

```
Deno.test("a pet cannot be made with a blank species", () => {
  assertThrows(() => new Pet("Rex", " "), Error, "A pet needs a species");
});
```

is red (Expected function to throw.) until the constructor checks:

```
if (species.trim() === "") {
  throw new Error("A pet needs a species");
}
this.species = species;
```

As there is no setter for the species, the constructor is the only place this check is needed.

How private is `private`?

There is a hole in `private`. The compiler rejects `pet.age = -3` - but it accepts this:

```
const pet = new Pet("Rex", "dog", 3);
pet["age"] = -3;
console.log(pet.toString());
```


`deno check` reports no error, the linter is happy, and it prints:

```
(PET) Rex is a dog, and is -3 years old.
```

The reason is that TypeScript's types, including `private`, exist only while the code is being checked. The build removes them. Here is the start of `Pet` in `dist/app.js`:

`dist/app.js`

```
var Pet = class {  
  name;  
  // readonly: set once, in the constructor, and never again. A dog stays a dog.  
  species;  
  age;  
  hungry = false;  
};
```

No `private`, no `readonly`, no types: just a JavaScript class with ordinary properties that any code can change. TypeScript's `private` is a check the compiler makes for you, and the square bracket form `pet["age"]` is a deliberate way round it (meant for tests and special cases). Java's `private` is enforced by the JVM while the program runs, too; TypeScript's is not.

In practice, `private` is enough for nearly everything. Nobody writes `pet["age"]` by accident, and in your own program, the compiler's check is what you need. But JavaScript has its own, real privacy, and TypeScript supports it.

Project 2: The bank account

Bank Account

Aoife Byrne · account IE-0001

Balance: €20.00

€

Deposit

Withdraw

Not enough money: the balance is €20.00

The Bank Account project

#private fields

A field whose name starts with `#` is private in JavaScript itself:

src/BankAccount.ts

```
export class BankAccount {  
  // # makes the field private at run time too. No `private` keyword is needed (or allowed) with #.  
  #balance: number = 0;  
  
  // Parameter properties (Chapter 5): public, so anyone can read them; readonly, so nobody can  
  // change them.  
  constructor(public readonly owner: string, public readonly accountNumber: string) {}  
}
```

The `#` is part of the name: the field is `#balance`, and the class writes `this.#balance` everywhere. Outside the class, the compiler rejects it:

```
TS18013 [ERROR]: Property '#balance' is not accessible outside class 'BankAccount' because it has a  
private identifier.  
account.#balance = 1000000;
```

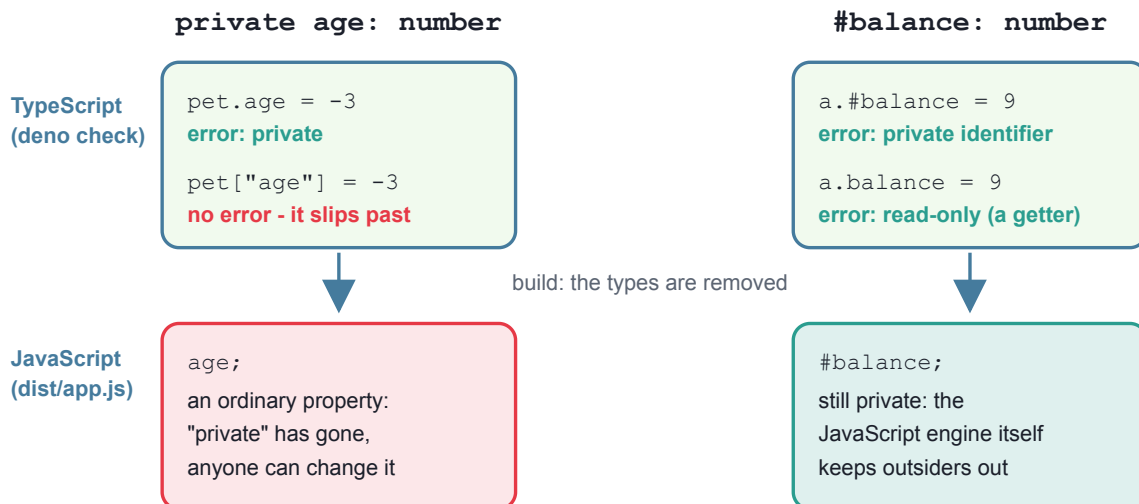
And this time the privacy survives the build: the bundle still says `#balance`, and the JavaScript engine itself refuses access from outside the class. It does not even show up when the object is listed. A short Deno script makes an account, deposits 2000 and prints it:

```
console.log(Object.keys(account));  
console.log(account);
```

```
[ "owner", "accountNumber" ]  
BankAccount { owner: "Aoife", accountNumber: "IE-0001" }
```

The balance is in there, but nobody outside can see it. (Writing `private #balance` is an error - An accessibility modifier cannot be used with a private identifier. - because `#` already says it.)

private age vs #balance



private age vs #balance

Which should you use? Both are fine, and you will see both in other people's code. `private` looks like Java and is what most TypeScript code still uses. `#` is real privacy, and is the JavaScript standard. This book uses `private` for classes written the Java way, like `Pet`, and `#` when a class uses `get / set` accessors, as in the next two sections - there it also solves a naming problem. Pick one style per class; do not mix them for no reason.

A getter with no setter

The balance needs to be read - the page shows it - but must never be set from outside. In Java you would write `getBalance()`. TypeScript can do that too, but it also has **get accessors**:

src/BankAccount.ts

```
/** The balance in cents. A get accessor: read it like a field, `account.balance` - but it cannot be set. */
public get balance(): number {
  return this.#balance;
}
```

`get balance()` is a method, but it is used **like a field**, without brackets:

src/main.ts

```
balance.textContent = formatEuro(account.balance);
```

Reading `account.balance` runs the method and gives its result. As there is no matching `set`, the property is read-only from outside:

```
TS2540 [ERROR]: Cannot assign to 'balance' because it is a read-only property.
account.balance = 1000000;
```

Here you can see the naming problem that `#` solves. The accessor is called `balance`, so the field behind it cannot be called `balance` too. With `private`, you would have to invent a name (often `_balance`). With `#`, the field is `#balance` and the accessor is `balance` - two different names that obviously belong together.

The owner and account number have no rules and never change, so they are `public readonly` parameter properties: readable as `account.owner`, and `account.owner = "Me"` is the same. Cannot assign to `'owner'` because it is a read-only property. A public readonly field needs no getter at all.

Money in cents

`BankAccount` counts in whole **cents**. Chapter 3 showed that decimals like 0.1 cannot be stored exactly; adding up euros as decimals would, sooner or later, give a balance like `0.30000000000000004` (deposit €0.10, then €0.20). Whole numbers are exact. `src/money.ts` converts at the edges - `toCents(12.5)` is 1250, `formatEuro(1250)` is `"€12.50"` - and everything in between is whole cents. Banks (and Java programs, with `BigDecimal` or `long` cents) do the same.

The invariant

The account has a rule that must be true for every account, all the time:

the balance is a whole number of cents, and never below zero.

A rule like this is called an **invariant**. Encapsulation is what makes an invariant possible: the balance starts at 0, which obeys the rule, and the only two methods that change it both check before they change it. So it can never be broken - not by the page, not by a bug, not by anybody.

`src/BankAccount.ts`

```
/** Adds money. Throws an Error if the amount is not a whole number of cents above zero. */
public deposit(cents: number): void {
    this.#checkAmount(cents);
    this.#balance += cents;
}

/** Takes money out. Throws an Error, and changes nothing, if the amount is bad or too big. */
public withdraw(cents: number): void {
    this.#checkAmount(cents);
    // Check first, change after: if the check fails, the balance has not been touched.
    if (cents > this.#balance) {
        throw new Error(`Not enough money: the balance is ${formatEuro(this.#balance)}`);
    }
    this.#balance -= cents;
}

// A #private method: a helper that only this class can call.
#checkAmount(cents: number): void {
    if (!Number.isInteger(cents) || cents <= 0) {
        throw new Error(`An amount must be a whole number of cents, more than 0 (not ${cents})`);
    }
}
```

Methods can be `#private` too: `#checkAmount` is called as `this.#checkAmount(...)`.

Check first, change after

It is tempting to write `withdraw` the other way round - take the money out, then complain if the balance went negative:

```
public withdraw(cents: number): void {
  this.#checkAmount(cents);
  this.#balance -= cents;
  if (this.#balance < 0) {
    throw new Error(`Not enough money: the balance is ${formatEuro(this.#balance)}`);
  }
}
```

It does throw. But this test catches the problem:

tests/BankAccount.test.ts

```
Deno.test("a refused withdrawal leaves the balance as it was", () => {
  const account = accountWith(2000);

  assertThrows(() => account.withdraw(3000));

  assertEquals(account.balance, 2000);
});
```

a refused withdrawal leaves the balance as it was => ./tests/BankAccount.test.ts:55:6
error: AssertionError: Values are not equal.

[Diff] Actual / Expected

```
-  -1000
+  2000
```

The error was thrown *after* the damage was done: the account is now at -1000, breaking the invariant, and the page would go on to show it. (The message test fails too - the message now says the balance is €-0.01.) A method that refuses should leave the object **exactly as it was**. So: check everything first, change things last. Every refusal test in this chapter checks both halves - that the error is thrown, and that nothing changed.

Testing many bad values with `t.step`

Zero, negative amounts, fractions of a cent and `NaN` must all be refused, by both methods. Rather than eight nearly identical tests, one test loops over the bad values, with a step (Chapter 4) for each, so a failure says exactly which value got through:

tests/BankAccount.test.ts

```

Deno.test("bad amounts are refused, for deposits and withdrawals alike", async (t) => {
  const account = accountWith(2000);
  const badAmounts = [0, -500, 12.5, NaN];

  for (const amount of badAmounts) {
    await t.step(`deposit(${amount}) is refused`, () => {
      assertThrows(() => account.deposit(amount), Error, "An amount must be");
    });
    await t.step(`withdraw(${amount}) is refused`, () => {
      assertThrows(() => account.withdraw(amount), Error, "An amount must be");
    });
  }
  assertEquals(account.balance, 2000);
});

```

`accountWith(cents)` is a small helper at the top of the test file that makes a fresh account with some money in it - Chapter 4's alternative to a `beforeEach`.

Exercise 6.2 - Why refuse a negative withdrawal?

What would `account.withdraw(-500)` do if `withdraw` did not call `#checkAmount`? Write a test that would catch it. Try it before reading on.

Here is one way. Without the check, `-500 > balance` is false, so the withdrawal goes ahead, and `this.#balance -= 500` adds 500: a way to make money out of nothing. The `t.step` test above already covers it (`withdraw(-500)` is refused). To see it go red, comment out the `this.#checkAmount(cents);` line in `withdraw` and run the tests - then put it back.

try / catch, once

The page has two buttons that can be refused, so `main.ts` has one helper that runs any change to the account and shows either a success message or the refusal:

`src/main.ts`

```

/** Runs one change to the account; if the account refuses it, shows the refusal's message. */
const attempt = (change: () => void, done: string): void => {
  if (message === null) {
    return;
  }
  try {
    change();
    message.textContent = done;
    message.className = "good";
  } catch (error) {
    message.textContent = error instanceof Error ? error.message : String(error);
    message.className = "bad";
  }
  render();
};

document.querySelector("#withdraw")?.addEventListener("click", () => {
  const cents = amountInCents();
  attempt(() => account.withdraw(cents), `Withdrew ${formatEuro(cents)}`);
});

```

`attempt` takes the change as a function `( () => void`, Chapter 3), so the `try / catch` is written once. The page does not check the amount itself: an empty box gives 0, letters give `NaN`, and the account refuses both with its own message.

Project 3: The thermostat

Thermostat

-

Target
21.0 °C

+

Heating on

Room temperature (the sensor): 17.0 °C

35

Set target

The target must be from 5 to 30 °C (not 35)

The Thermostat project

Setters as properties

The bank account had a getter with no setter. The thermostat's target temperature needs both: the page reads it, and the page sets it. In Java this would be `getTarget()` and `setTarget(...)`. In TypeScript it can be a **get/set pair**:

```
src/Thermostat.ts
```

```

const MIN_TARGET = 5;
const MAX_TARGET = 30;
const STEP = 0.5;
const START_TARGET = 20;

export class Thermostat {
  #target: number = START_TARGET;

  // ...

  /** The temperature the heating aims for. */
  public get target(): number {
    return this.#target;
  }

  /** Runs on every `thermostat.target = ...`. Throws an Error, and changes nothing, if the value
  is not allowed. */
  public set target(celsius: number) {
    if (celsius < MIN_TARGET || celsius > MAX_TARGET) {
      throw new Error(`The target must be from ${MIN_TARGET} to ${MAX_TARGET} °C (not
${celsius})`);
    }
    if (!Number.isInteger(celsius / STEP)) {
      throw new Error(`The target must be a whole or half degree (not ${celsius})`);
    }
    this.#target = celsius;
  }
}

```

To other code, `target` looks exactly like a field:

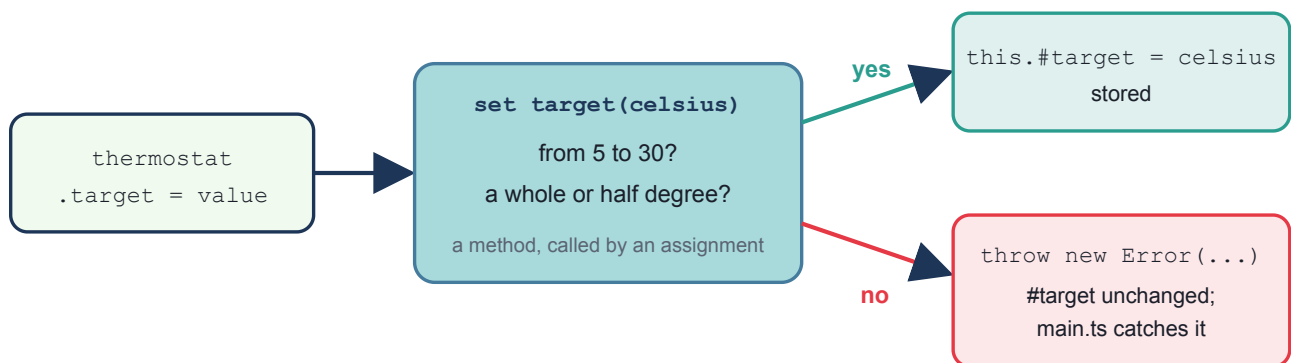
```

thermostat.target = 21.5; // runs set target(21.5)
console.log(thermostat.target); // runs get target()

```

But every assignment runs the setter, and the setter can refuse:

What an assignment to `thermostat.target` really does



Reading `thermostat.target` calls `get target()` the same way - the caller never sees `#target`.

What an assignment to `thermostat.target` really does

This invariant is "the target is always from 5 to 30 °C, in half degrees". `Number.isInteger(celsius / STEP)` checks the half degrees: `21.5 / 0.5` is 43, a whole number; `21.3 / 0.5` is 42.6, not one.

Testing a setter

A setter is called by an assignment, so the test puts an assignment inside `assertThrows` :

tests/Thermostat.test.ts

```
Deno.test("a target outside 5-30 °C is refused, and the old target is kept", async (t) => {
  const thermostat = new Thermostat(18);

  for (const celsius of [4.5, 30.5, -10, 100]) {
    await t.step(`${celsius} °C is refused`, () => {
      // Braces make it clear that the body is an assignment, run for what it does, not for a
      // value.
      assertThrows(() => {
        thermostat.target = celsius;
      }, Error, "The target must be from 5 to 30 °C");
    });
  }
  assertEquals(thermostat.target, 20);
});
```

Look at the values: 4.5 and 30.5 are the first refused values on each side of the allowed range. Another test checks that 5 and 30 themselves are accepted. Edges, as always (Chapter 1's `ageGroup`): an off-by-one mistake, `<=` where `<` was meant, is caught by exactly these values.

The class uses its own setter

The + and - buttons call `up()` and `down()` :

src/Thermostat.ts

```
/** Half a degree warmer - but never above the maximum, so pressing + too often is harmless. */
public up(): void {
  // Goes through the setter, so the rules are checked in one place only.
  this.target = Math.min(this.#target + STEP, MAX_TARGET);
}
```

Two decisions are in there. First, pressing + at 30 °C is not an error - it is what people do with thermostats - so `up` stops at the maximum (`Math.min` picks the smaller number) instead of throwing. Typing 35 into the box is refused, because the user asked for something impossible. The tests say so: "up stops at 30 °C instead of going past it".

Second, `up` writes `this.target = ...`, not `this.#target = ...`. Inside the class, assigning to `this.target` runs the setter too. That way, even the class's own methods go through the one place that checks the rules.

A computed property

src/Thermostat.ts

```

/** A get accessor with no field behind it: worked out fresh every time it is read. */
public get heating(): boolean {
    return this.room < this.#target;
}

```

There is no `heating` field. Every time `thermostat.heating` is read, it is worked out from the room temperature and the target, so it can never be out of date. If `heating` were a field, every method that changed `room` or `target` would have to remember to update it as well - and one day one would forget. It is read-only, of course: `thermostat.heating = true` gives `Cannot assign to 'heating' because it is a read-only property.`

(A boolean *property* is usually named without "is": `if (thermostat.heating)` reads well. A boolean *method* gets the `is`: `isHungry()`.)

Not everything needs hiding

The room temperature has no getter and no setter:

`src/Thermostat.ts`

```

// `room` is the temperature the sensor reads. Any number is possible, so there is nothing to
// check and nothing to hide: a public field is fine. If a rule turns up later, it can become a
// get/set pair, and no code that uses it has to change.
constructor(public room: number) {}

```

In Java, a public field is a decision you are stuck with. If you later need a check, you must replace `t.room = 22` with `t.setRoom(22)` in every file that uses it - so Java programmers write getters and setters for every field, just in case. In TypeScript, `room` can become a `get / set` pair at any time, and `thermostat.room = 22` keeps working, now running the setter. So a field with no rules can simply be public; you add the accessors when a rule appears, not before.

Exercise 6.3 - A rule for the sensor

A sensor reading below -50 °C must be a fault. Make `room` refuse it, without changing `main.ts` or any existing test. Write the test first. Try it before reading on.

Here is one way:

```

Deno.test("a room reading below -50 °C is refused", () => {
    const thermostat = new Thermostat(18);

    assertThrows(() => {
        thermostat.room = -51;
    }, Error, "sensor fault");
});

```

```

const LOWEST_READING = -50;

export class Thermostat {
  #target: number = START_TARGET;
  #room: number;

  constructor(room: number) {
    this.#room = room;
    this.room = room; // through the setter, so the first reading is checked too
  }

  public get room(): number {
    return this.#room;
  }

  public set room(celsius: number) {
    if (celsius < LOWEST_READING) {
      throw new Error(`Below ${LOWEST_READING} °C: a sensor fault (not ${celsius})`);
    }
    this.#room = celsius;
  }
}
// ... the rest as before, still using this.room

```

The parameter property had to go (it would skip the check), and the constructor gives `#room` a value directly first, so the compiler can see it is assigned. Everything that used `thermostat.room`

- `main.ts`, the old tests, `heating` - is unchanged.

Accessors or methods?

You now have two ways to give the outside a value: `getBalance()` or `get balance()`. Both work, and TypeScript code uses both. Some guidelines:

- use a **get accessor** for something that feels like a property of the object: cheap to work out, no parameters, no side effects, and the same answer if you read it twice in a row (`balance`, `target`, `heating`)
- use a **set accessor** when setting a value is all that happens, with a check (`target`)
- use a **method** for anything that *does* something or needs more than one value: `deposit(500)`, `withdraw(500)`, `haveBirthday()`, `feed()`. `account.balance = account.balance + 500` would hide the idea of a deposit; `account.deposit(500)` says it
- a getter should never be slow or surprising. `account.balance` looks like reading a field; if it secretly did a lot of work or changed something, the next person to read the code would be misled
- `getX()` / `isX()` / `setX()` methods, as in `Pet`, are never wrong - they are what a Java programmer expects - but in TypeScript accessors are usually neater

Whichever you choose, the setter or method that changes a value checks it first, and refuses by throwing an `Error` before anything has changed.

Java and TypeScript

Java	TypeScript
<code>private int age;</code> - enforced by the compiler and the JVM	<code>private age: number;</code> - enforced by the compiler only
(no equivalent)	<code>#age: number;</code> - private in the running JavaScript too
no visibility written: package-private	no visibility written: <code>public</code> (this book writes it anyway)
<code>private final String species;</code>	<code>private readonly species: string;</code>
<code>public int getAge() / public void setAge(int age)</code>	either the same methods, or <code>public get age(): number / public set age(value: number)</code>
<code>public boolean isHungry()</code>	<code>public isHungry(): boolean</code> , or a <code>get hungry()</code> property
<code>pet.setAge(4)</code>	<code>pet.setAge(4)</code> , or <code>pet.age = 4</code> running a setter
change a public field to get/set: every caller must change	change a public field to <code>get / set</code> : no caller changes
<code>throw new IllegalArgumentException("...")</code>	<code>throw new Error("...")</code>
<code>catch (IllegalArgumentException e)</code> - typed, one per kind	<code>catch (error)</code> - one catch, <code>error</code> is <code>unknown</code> ; check <code>error instanceof Error</code>

Summary

- encapsulation: keep an object's data on the inside, and let the outside change it only through methods that check every change
- `private` members are only checked by the compiler; `pet["age"]` gets round it, and the build removes it
- `#private` fields and methods (`#balance`, `this.#balance`) stay private while the program runs
- `readonly` fields are set once, in their declaration or the constructor, and never again
- getters are `getX()` or, for booleans, `isX()`; or a `get x()` accessor, read like a field
- a `get` with no `set` makes a read-only property; a `get` with no field behind it is computed fresh each time
- a `set x(value)` accessor runs on every assignment, so it can check and refuse
- refuse bad values with `throw new Error("...")`, checking **before** changing anything; the constructor uses the same checks as the setters
- an invariant is a rule that is true for every object at every moment; encapsulation is what makes it possible
- test refusals with `assertThrows(fn, Error, "part of the message")`, and check that the object did not change
- catch errors on the page with `try / catch` and `error instanceof Error`
- a field with no rules can be public: in TypeScript it can become a `get / set` pair later without changing its callers

Challenges

Each challenge says which project to start from. Write the tests first - including tests for what must be refused, and that a refusal changes nothing.

1. Tidy names

Start from `ch06_project01_pet`. Make the pet's names tidier: `setName(" Rex ")` stores `"Rex"` (the spaces at either end removed), and a name longer than 20 characters is refused with the message "A name can have at most 20 characters". The same rules apply in the constructor. Test the edges: exactly 20 characters is allowed, 21 is not.

2. Fahrenheit too

Start from `ch06_project03_thermostat`. Add a read-only property `targetFahrenheit` (20 °C is 68 °F) and show it on the page under the target, as "68.0 °F". Test it first, including after `up()`. Should it have a field? Should it have a setter? Write down why or why not.

3. An overdraft

Start from `ch06_project02_bank_account`. Some accounts may go overdrawn, down to an agreed limit. Give the constructor an optional third parameter, `overdraftLimit`, in cents (0 if not given). `withdraw` may then take the balance down to minus the limit, but no further. Write the tests first: an account with no overdraft behaves exactly as before; an account with a €100 overdraft can withdraw down to -10000 cents but not -10001; a negative limit is refused by the constructor. What is the account's invariant now?

4. Transfers

Start from `ch06_project02_bank_account`. Add `transferTo(other: BankAccount, cents: number)`, which moves money from one account to another. It must be all or nothing: if the transfer is refused, **neither** balance changes. Tests first, including a refused transfer (not enough money) and a transfer to the same account (refuse it). Add a second account to the page with a Transfer button.

Hint: `transferTo` can call `this.withdraw(cents)` and then `other.deposit(cents)` - think about which order makes "all or nothing" easy. Like Java's `private`, `#private` belongs to the class, not to one object: inside `BankAccount`'s code, `other.#balance` is allowed. Do you need it?

5. Pet, the TypeScript way

Start from `ch06_project01_pet`. Rewrite `Pet` with accessors instead of Java-style methods: `name` and `age` as `get/set` pairs that still check their values, `species` as a `public readonly` field, and `hungry` as a read-only `get` property. Change the tests first (`pet.getAge()` becomes `pet.age`, `pet.setAge(-1)` becomes an assignment inside braces), watch them fail to compile, then change the class and `main.ts`. Which version do you find easier to read, and why?

Hint: the fields behind the accessors need different names - `#name`, `#age` and `#hungry` solve that. `haveBirthday()`, `play()` and `feed()` stay as methods: they do things.

6. Holiday mode

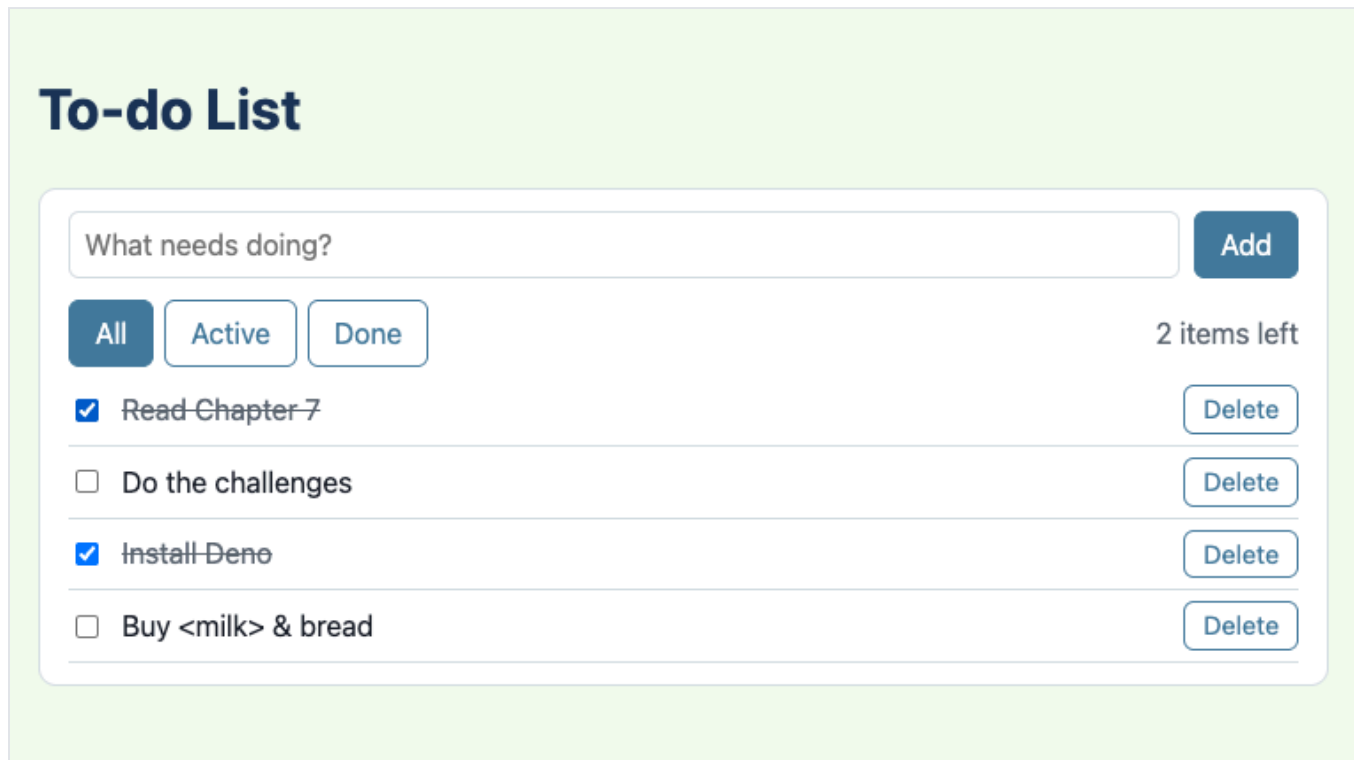
Start from `ch06_project03_thermostat`. Add a holiday mode for when nobody is home. While it is on, the thermostat keeps the house at a frost-protection target of 7 °C: `target` reads 7, `heating` uses 7, and setting the target, `up()` and `down()` are refused with an Error saying the thermostat is on holiday. When the holiday ends, the target goes back to what it was before. Add methods `startHoliday()` and `endHoliday()` and an `isOnHoliday()` method (or `onHoliday` property), and a Holiday button on the page that switches between the two. Tests first: the target during and after a holiday, every refused change, and starting a holiday twice.

Hint: keep the user's target in `#target` all the time, and a boolean `#holiday`. The `target` getter then decides what to give back. Where should the "on holiday" check go so that it is written only once? What should the page do with the - and + buttons during a holiday?

Next: [Chapter 7 - Model and view](#)

Chapter 7 - Model and view

Since Chapter 2 your pages have followed one pattern: an object, an event, a render. As pages grow, that pattern needs a firmer shape. This chapter splits every program into a **model** (classes that hold the data and the rules, and know nothing about the page), a **view** (code that shows the model, and does nothing else) and a thin `main.ts` that joins them. The model gets all the tests; the view becomes so simple it hardly needs any.



The To-do List project

What you will learn

- what a model and a view are, and why the model must never touch the DOM
- writing a small view class whose `render` method copies the model onto the page
- `requireElement(selector)`: a helper that fails loudly, naming the element, when the page is missing one
- why `innerHTML` must never be given text a user typed, and how `textContent` keeps it safe
- passing functions (handlers) into a view, so the view can report clicks without changing the model
- event, model, render at a larger scale: many kinds of event, one path
- testing the model only, and why that is enough

The projects

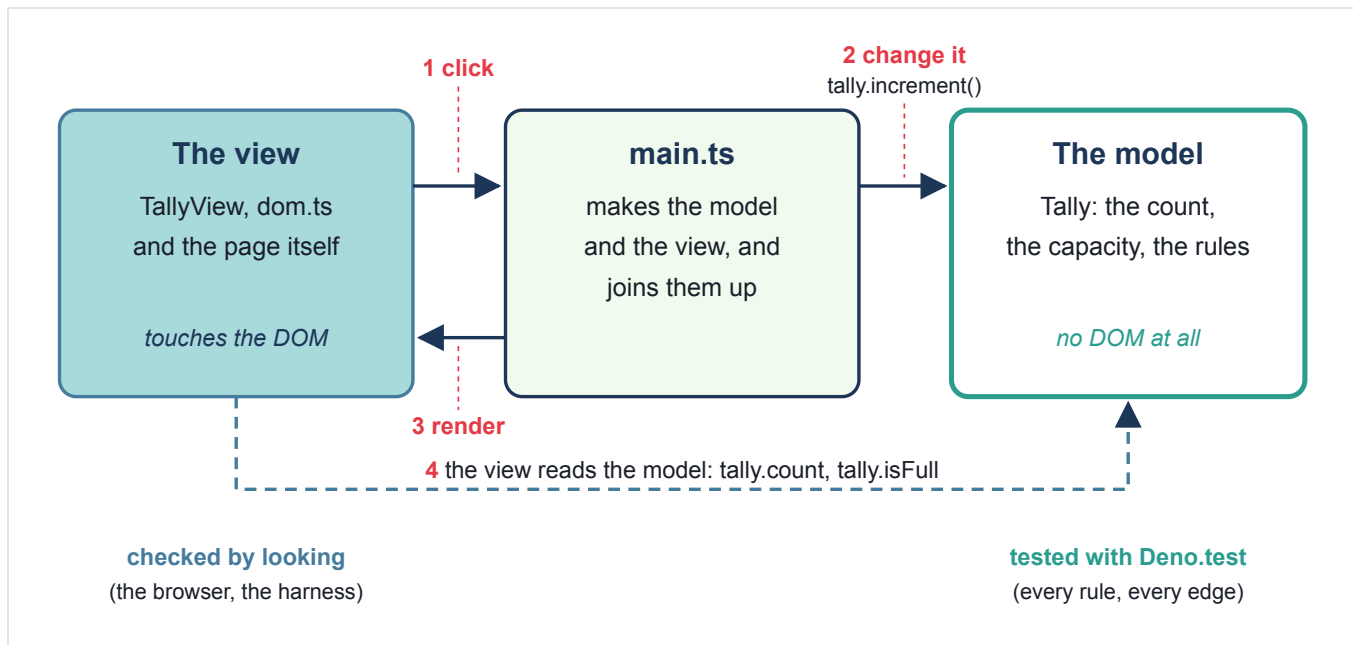
Project	What it shows
ch07_project01_tally_counter	a <code>Tally</code> model, a small <code>TallyView</code> class, <code>requireElement</code> , and the model developed test first
ch07_project02_shopping_list	a model with rules about user input, a view that builds elements with <code>textContent</code> , and the <code>innerHTML</code> danger
ch07_project03_todo_filters	two model classes, filters, a view given an object of handlers, and many events taking one path

Why split model and view?

Look back at Chapter 3's marks table. Its `main.ts` held the state (two booleans), the decisions (which students to show, in which order) and the DOM code (building table rows), all in one file. That worked because the page was small. Now imagine adding a search box, a second table and a chart. Every new feature would touch the same file, and none of it could be tested, because tests run in Deno, and Deno has no page.

The answer, used by almost every user interface ever written, is to split the program into parts with different jobs:

- the **model** holds the data and the rules: "the count never goes below zero", "the same item cannot be added twice". It is made of ordinary classes and functions. It never touches the DOM - it does not know there *is* a page
- the **view** shows the model. It reads the model and copies what it finds onto the page. It makes no decisions about the data
- `main.ts` makes the model and the view and joins them, with listeners that change the model and then ask the view to render



Model, view and main.ts

Follow the numbers: (1) the user clicks; (2) the listener in `main.ts` changes the model; (3) it calls the view's `render` ; (4) the view reads the model and updates the page. Every event in this chapter takes that path.

The payoff is on the right of the picture. Everything that can go wrong with the *data* lives in the model, and the model can be tested with `Deno.test` , rule by rule and edge by edge. The view is left with nothing to get wrong except "is it showing the right thing?" - which you can check by looking.

Note - Up to now the rule has been " `main.ts` is the only file that touches the DOM". From this chapter on, the rule is: **model files never touch the DOM**. DOM code lives in `main.ts` and in view files - here `dom.ts` and the classes whose names end in `View` . That makes it easy to see which files are tested and which are not.

If you have met MVC (model-view-controller) in Java - Swing, or a web framework - this is the same idea. `main.ts` plays the controller.

Project 1: A tally counter

Room Counter

People in the lab

4

8 spaces left

-

+

Reset

The Room Counter project

A lab has room for 12 people. A tally counter at the door counts them in and out: + when someone comes in, - when someone leaves. The page shows how many are inside and how many spaces are left, and says "Full" when nobody else may come in.

The model, test first

Start with the model, and start with a test. The class is `Tally`; the first rules are that a new tally starts at zero and that `increment` adds one. Those are quick to make pass. The interesting rule is at the edge: if the room is empty and the - button is pressed, the count must not become -1. Write that test before the code that makes it true:

`tests/Tally.test.ts`

```
Deno.test("decrement never goes below zero", () => {
  const tally = new Tally();
  tally.decrement();
  assertEquals(tally.count, 0);
});
```

With the simplest possible `decrement - this.current--;` - the test is red:

```
decrement never goes below zero => ./tests/Tally.test.ts:18:6
error: AssertionError: Values are not equal.
```

[Diff] Actual / Expected

```
-  -1
+  0
```

Make it green with the smallest change - do nothing when the room is empty. Then the other edge, the capacity: a room for 2 must stop at 2.

```
Deno.test("increment stops at the room's capacity", () => {
  const tally = new Tally(2);
  tally.increment();
  tally.increment();
  tally.increment();
  assertEquals(tally.count, 2);
});
```

Red again (- 3 / + 2), and green after one more guard. Then refactor: both guards ask a question about the tally ("is it full?", "is it empty?"), and the view will want to ask the same questions to switch its buttons off. So the questions become getters, and the methods use them:

src/Tally.ts

```
const DEFAULT_CAPACITY = 20;

export class Tally {
  private current: number = 0;

  // A parameter property (Chapter 5): `capacity` is a public, readonly field, set by the
  constructor(public readonly capacity: number = DEFAULT_CAPACITY) {
    if (capacity < 1) {
      throw new Error(`A room must hold at least 1 person, not ${capacity}`);
    }
  }

  /** How many people are in the room now. */
  public get count(): number {
    return this.current;
  }

  /** True when nobody else may come in. */
  public get isFull(): boolean {
    return this.current === this.capacity;
  }

  // ... isEmpty and spacesLeft are getters like these

  /** One more person comes in - unless the room is full. */
  public increment(): void {
    if (this.isFull) {
      return;
    }
    this.current++;
  }

  // ... decrement() and reset()
}
```

Everything here comes from the last two chapters: a parameter property with a default, `readonly`, a field that is `private` and read through a `get` accessor, and an error thrown for a room that could never be used. Notice what is *not* here: no `document`, no `HTMLElement`, no `textContent`. The class would work just as well behind a command-line program or a phone app.

Exercise 7.1 - A room for nobody

`new Tally(0)` makes no sense. Write a test that says so, using `assertThrows` (Chapter 4), and see it fail before the check in the constructor exists. Try it before reading on.

Here is one way:

tests/Tally.test.ts

```
Deno.test("a room must hold at least one person", () => {
  assertThrows(() => new Tally(0), Error, "at least 1 person");
});
```

Without the `if` in the constructor, the red output is short:

```
a room must hold at least one person => ./tests/Tally.test.ts:16:6
error: AssertionError: Expected function to throw.
```

The third argument checks that the error's message *contains* "at least 1 person", so the test fails if the wrong error is thrown.

requireElement: failing loudly

Before writing the view, deal with a problem that has been quietly waiting since Chapter 1. Every `querySelector` so far has been followed by `if (x !== null)`, or by `x?.`. That keeps TypeScript happy - but think about what it does when you mistype an id. `querySelector("#cuont")` gives `null`, the `if` skips the code, and the page silently does nothing. No error, no message, nowhere to look.

For an element that the page *must* have, a missing element is a bug, and a bug should be loud. Here is a helper that finds the element or throws an error that names it:

src/dom.ts

```
/**
 * The element that matches `selector` - or an error naming the selector, if there is none.
 * querySelector quietly gives null for a typo like "#cuont"; this makes the mistake loud,
 * in the browser's console, the moment the page starts.
 */
export const requireElement = <T extends HTMLElement>(selector: string): T => {
  const element = document.querySelector<T>(selector);
  if (element === null) {
    throw new Error(`No element matches "${selector}" - check index.html`);
  }
  return element;
};
```

The `<T extends HTMLElement>` is new. You have been *using* it since Chapter 1: in `querySelector<HTMLButtonElement>("#add")`, the part in angle brackets tells the function which kind of element to expect. `requireElement` simply passes that choice on. Read the first line as "requireElement works for any type `T` that is some kind of `HTMLElement`; it takes a selector and returns a `T`". So `requireElement<HTMLButtonElement>("#add")` returns an `HTMLButtonElement` - never `null`, so no `if`

is needed afterwards. (A function with a type in angle brackets is called **generic**, like Java's `<T> T first(List<T> list)`. Book 2 covers generics properly; this is all you need for now.)

If you leave out the `<HTMLElement>`, `T` becomes plain `HTMLElement`, and the compiler stops you using button-only properties:

```
TS2339 [ERROR]: Property 'disabled' does not exist on type 'HTMLElement'.
```

To see the helper earn its keep, change `id="count"` to `id="cuont"` in `public/index.html` and rebuild. The page stops at once, and the browser's console (in the developer tools) shows, in red:

```
Error: No element matches "#count" - check index.html
    at requireElement (app.js:6:13)
    at <instance_members_initializer> (app.js:62:17)
    at new TallyView (app.js:61:19)
```

The message says what is missing and the stack says who wanted it. Compare that with a blank page and no clue.

Note - `requireElement` is not tested. It uses `document`, and Deno has no `document`: a test that makes a `TallyView` fails with `ReferenceError: document is not defined`. That is fine - it is four lines, and you have just checked it by hand. It is also exactly why the model must stay free of the DOM: one `document` in `Tally.ts` and none of its tests could run.

The view class

The view's job is to make the page match the model. It needs the elements, and a method that copies the model's state onto them:

`src/TallyView.ts`

```
import { requireElement } from "../dom.ts";
import type { Tally } from "../Tally.ts";

export class TallyView {
  private readonly countText = requireElement<HTMLElement>("#count");
  private readonly statusText = requireElement<HTMLElement>("#status");
  private readonly addButton = requireElement<HTMLButtonElement>("#add");
  private readonly removeButton = requireElement<HTMLButtonElement>("#remove");

  /** Makes the page match the tally. Safe to call as often as you like. */
  public render(tally: Tally): void {
    this.countText.textContent = `${tally.count}`;
    this.statusText.textContent = tally.isFull ? "Full - nobody else may come in" :
    `${tally.spacesLeft} spaces left`;
    this.statusText.classList.toggle("bad", tally.isFull);
    // The buttons that would do nothing are switched off, so the page shows the model's rules.
    this.addButton.disabled = tally.isFull;
    this.removeButton.disabled = tally.isEmpty;
  }
}
```

A few things to notice:

- The elements are found **once**, when `new TallyView()` runs: field initialisers run as part of the constructor, as in Java. If one is missing, the error comes immediately, not on the first click.
- The fields are `private readonly`: nothing outside can reach into the view's elements, and the view never swaps one for another.
- `import type` imports `Tally` for its *type* only - the view never makes a `Tally`, it is given one. (Chapter 9 says more about `import type`.)
- `render` asks the model questions (`count`, `isFull`, `spacesLeft`, `isEmpty`) and makes no decisions of its own about the data. "Is the room full?" is answered by `Tally`, which is tested. The view only decides how "full" *looks*: red text and a disabled button.
- `render` can be called any number of times, in any state, and always gives the right page. It does not "add one to the number on the screen"; it shows whatever the model says now.

`classList.toggle("bad", tally.isFull)` adds the class when the second argument is true and removes it when it is false - one line instead of an `if` and an `else`. `button.disabled = true` greys a button out and stops it being clicked.

Joining them up

`main.ts` is now short, and every listener has the same shape:

`src/main.ts`

```
const ROOM_CAPACITY = 12;

const tally = new Tally(ROOM_CAPACITY);
const view = new TallyView();

const render = (): void => view.render(tally);

requireElement<HTMLButtonElement>("#add").addEventListener("click", () => {
  tally.increment();
  render();
});
requireElement<HTMLButtonElement>("#remove").addEventListener("click", () => {
  tally.decrement();
  render();
});
// ... #reset the same way

render();
```

Change the model, then render. The listener never touches the count on the screen, and the view never changes the count in the model. And the final `render()` matters: without it the page would show the placeholder text until the first click.

Try it - In `main.ts`, comment out the `render()` inside the `+` listener and rebuild. Click `+` three times: the page still says 0, and `-` is still greyed out. The model has three people in it; the page just was not told. Forgetting to render is the commonest model-view bug, and no test of the model will catch it.

Project 2: A shopping list

Shopping List

Add

milk is already on the list

4 to buy

Milk

Remove

bread

Remove

cheese

Remove

eggs

Remove

The Shopping List project

A tally only has buttons. A shopping list has an input box, which brings two new problems: the model must decide what input it accepts, and the view must show text that a *user* typed.

Rules about input

The model, `ShoppingList`, keeps the items in an array. Its rules: no blank items, no item twice (ignoring case - "Milk" and "milk" are the same thing), and spaces round an item are removed.

When the user types something the list refuses, the page should say why. So the model has a method that answers "what is wrong with this?" - a message, or `null` if nothing is:

```
src/ShoppingList.ts
```

```

export class ShoppingList {
  private readonly items: string[] = [];

  public problemWith(text: string): string | null {
    const name = text.trim();
    if (name === "") {
      return "Type an item first";
    }
    const lower = name.toLowerCase();
    if (this.items.some((item) => item.toLowerCase() === lower)) {
      return `${name} is already on the list`;
    }
    return null;
  }

  /** Adds an item, without the spaces round it. Throws if problemWith would complain. */
  public add(text: string): void {
    const problem = this.problemWith(text);
    if (problem !== null) {
      throw new Error(problem);
    }
    this.items.push(text.trim());
  }
  // ... remove(index), and the getters all and size
}

```

`some` is another array method, like `find` (Chapter 3): it is true if the function is true for at least one element.

The duplicate rule was written test first, and the first attempt was too simple:

`this.items.includes(name)`. This test caught it:

`tests/ShoppingList.test.ts`

```

Deno.test("an item already on the list is a problem, whatever its case", () => {
  assertEquals(listOf(["Milk"]).problemWith("milk"), "milk is already on the list");
});

```

an item already on the list is a problem, whatever its case => ./tests/ShoppingList.test.ts:32:6
error: AssertionError: Values are not equal.

[Diff] Actual / Expected

```

- null
+ "milk is already on the list"

```

`includes` compares strings exactly, and `"Milk" === "milk"` is false. Comparing lower-case copies turns it green.

`listOf` is a helper in the test file that makes a list with some items already on it, so each test can set itself up in one line (Chapter 4's "helper functions instead of `beforeEach`"):


```
const listOf = (names: string[]): ShoppingList => {
  const list = new ShoppingList();
  for (const name of names) {
    list.add(name);
  }
  return list;
};
```

Why both `problemWith` and a throwing `add`? They do different jobs. `problemWith` is for the view: a polite answer to show the user. The `throw` in `add` guards the model's **invariant** (Chapter 6): whatever code calls it, a blank or duplicate item can never get onto the list. A test checks both:

```
Deno.test("adding a problem item throws, and leaves the list alone", () => {
  const list = listOf(["milk"]);
  assertThrows(() => list.add("MILK"), Error, "already on the list");
  assertEquals(list.all, ["milk"]);
});
```

Handing out a copy

The view needs the items, but it must not be able to change them. If `all` returned the model's own array, any code could `push` onto it and skip every rule. So it returns a copy:

```
/** A copy of the items, so code outside cannot change the list behind the model's back. */
public get all(): string[] {
  return [...this.items];
}
```

`[...this.items]` makes a new array with the same elements (the `...` spreads them out; Chapter 9 looks at copying in detail). A test proves the copy is a copy:

```
Deno.test("changing the copy from all does not change the list", () => {
  const list = listOf(["milk"]);
  list.all.push("cake");
  assertEquals(list.size, 1);
});
```

innerHTML and textContent

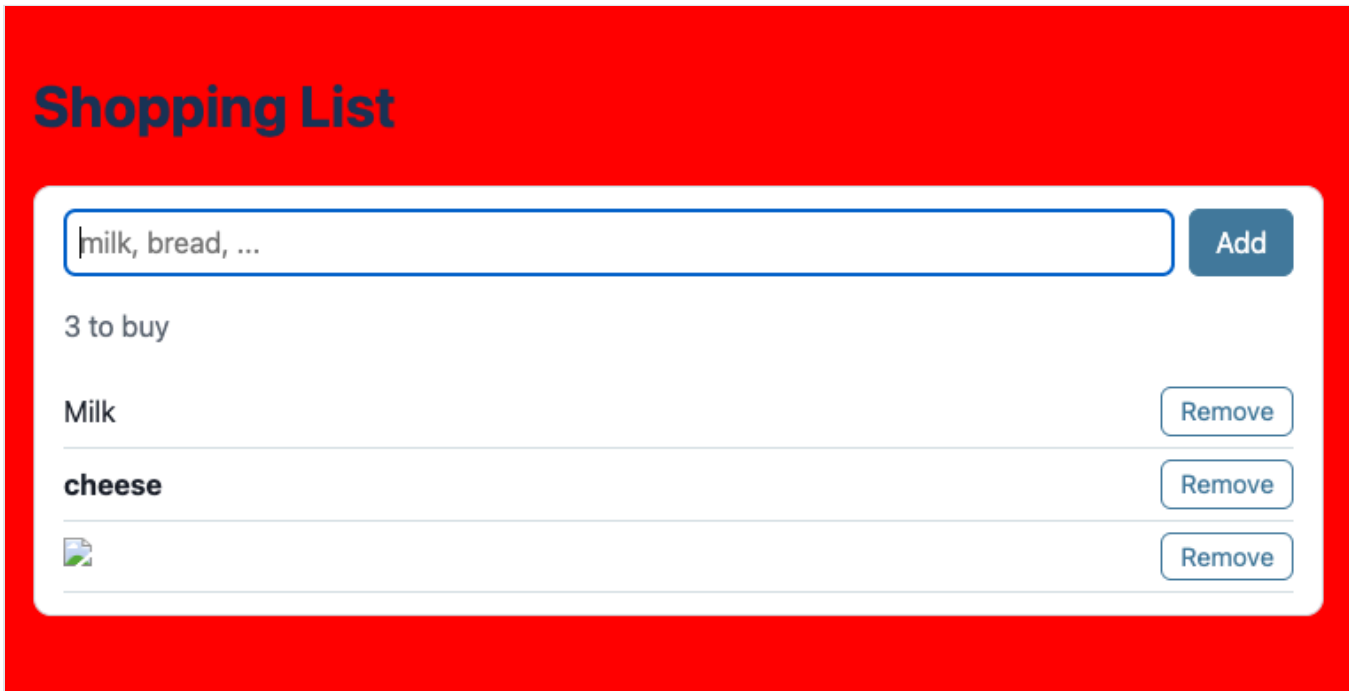
Chapter 3's marks table built its rows in one statement, with `innerHTML` and a template literal. It is tempting to do the same here:

```
// WRONG: the names were typed by the user, and innerHTML treats them as HTML.
this.listElement.innerHTML = items.map((name) => `<li><span>${name}</span><button
class="secondary">Remove</button></li>`).join("");
```

It works for "milk". Now type `<b>cheese</b>` into the box. With this line, "cheese" appears in bold - the browser did not show your text, it *obeyed* it. Then try this:

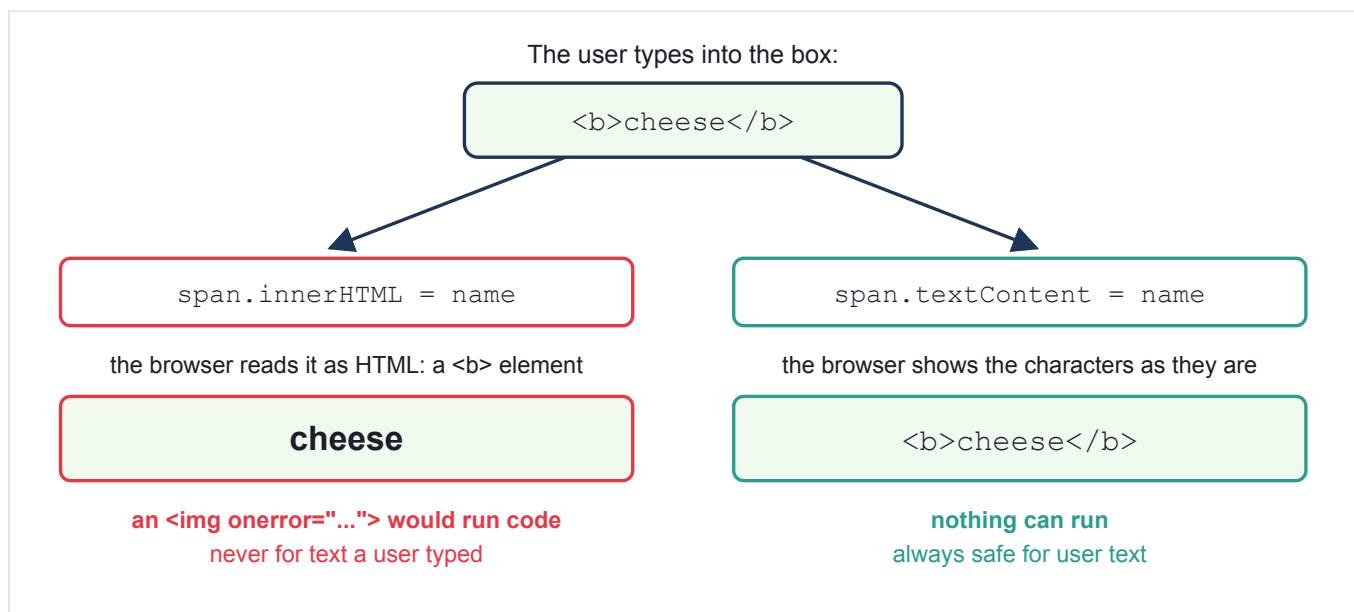
```
<img src=x onerror="document.body.style.background=`red`">
```

The browser makes an image, fails to load `x`, runs the `onerror` code - and the whole page turns red:



The shopping list, built with innerHTML

Turning a page red is harmless. But that box would run *any* JavaScript, and on a real site the "user" might be an attacker whose text is saved and later shown to other people: their code would then run in every visitor's browser, able to read the page and send what it finds anywhere. This is called **cross-site scripting** (XSS), and it is one of the most common security holes on the web.



innerHTML and textContent

The rule is simple:

- **textContent** puts characters on the page, exactly as they are. `<b>` is shown as the three characters `<`, `b`, `>`. Nothing is ever run. Use it for anything that came from a user, a file or another computer

- `innerHTML` is read as HTML. Use it only for HTML *you* wrote, with nothing typed by a user inside it - or not at all

Chapter 3's marks table was safe because its names came from your own JSON file. The shopping list is not, so its view builds each item with `createElement` and `textContent`:

`src/ShoppingListView.ts`

```
/** One <li>: the item's name, as text, and a Remove button. */
private itemElement(name: string, index: number): HTMLLIElement {
  const item = document.createElement("li");
  const label = document.createElement("span");
  label.textContent = name; // text, never HTML: "<b>milk</b>" is shown just as it was typed
  const removeButton = document.createElement("button");
  removeButton.textContent = "Remove";
  removeButton.className = "secondary";
  // An arrow function, so `this` is still the view (Chapter 3's this trap), and `index` is
  remembered.
  removeButton.addEventListener("click", () => this.onRemove(index));
  item.appendChild(label, removeButton);
  return item;
}
```

`append` is like `appendChild`, but takes any number of children. The screenshot at the top of this section shows the result: `<b>cheese</b>` on the list, exactly as typed.

A view that reports clicks

The Remove buttons belong to the view - it makes them - but removing an item is a change to the model, which is `main.ts`'s business. So the view is *given* a function to call when a Remove button is clicked, in its constructor:

`src/ShoppingListView.ts`

```
/** What the view calls when a Remove button is clicked: main.ts decides what that means. */
export type RemoveHandler = (index: number) => void;

export class ShoppingListView {
  private readonly listElement = requireElement<HTMLUListElement>("#items");
  private readonly messageText = requireElement<HTMLElement>("#message");
  private readonly summaryText = requireElement<HTMLElement>("#summary");

  constructor(private readonly onRemove: RemoveHandler) {}

  /** Makes the page show these items. */
  public render(items: string[]): void {
    // Empty the list first. replaceChildren() with nothing in the brackets removes every child.
    this.listElement.replaceChildren();
    items.forEach((name, index) => this.listElement.appendChild(this.itemElement(name, index)));
    this.summaryText.textContent = items.length === 0 ? "Nothing on the list yet" :
    `${items.length} to buy`;
  }
  // ... showMessage(text), itemElement(name, index)
}
```

`RemoveHandler` is a function type (Chapter 3), and the constructor uses a parameter property to keep it. `forEach` calls a function for every element, passing the element *and its index* - handy here, because each Remove button needs to know which item it removes.

`render` starts by emptying the list and then builds every item again. That sounds wasteful, but for a list of tens or hundreds of items it is instantly fast, and it means the page can never get out of step with the model.

In `main.ts`, the handler is an arrow function: change the model, then render.

`src/main.ts`

```
const view = new ShoppingListView((index) => {
  list.remove(index);
  view.showMessage("");
  render();
});

const addItem = (): void => {
  // Ask the model first: if there is a problem, show it and change nothing.
  const problem = list.problemWith(input.value);
  if (problem !== null) {
    view.showMessage(problem);
    return;
  }
  list.add(input.value);
  view.showMessage("");
  input.value = "";
  render();
};
```

`addItem` is called by the Add button and by Enter in the input box (a `keydown` listener, as in Chapter 2). It does not decide what counts as a problem - the model does - it only decides what to do with the answer.

Exercise 7.2 - An empty list

When the list is empty the page should say "Nothing on the list yet" rather than "0 to buy". Which file changes, and does the model need a new test? Try it before reading on.

Here is one way: it is one line in the view's `render`, shown above - `items.length === 0 ? "Nothing on the list yet" : ...`. The model does not change at all: it already says how many items there are. Wording is the view's job. (If the wording ever grew real rules - "1 thing to buy", "2 things to buy" - those rules would move out of the view into a function that can be tested, as the next project shows.)

Project 3: A to-do list with filters

To-do List

Add

All

Active

Done

2 items left

☒ Read Chapter 7

Delete

☐ Do the challenges

Delete

☒ Install Deno

Delete

☐ Buy <milk> & bread

Delete

The To-do List project

The last project puts everything together at a larger scale. A to-do list has more kinds of event - adding, ticking off, deleting, choosing a filter - and its state is more than one object. But every event still takes the same path.

Two model classes

One to-do is a small class:

src/ToDo.ts

```
export class ToDo {
  private isDone: boolean = false;

  constructor(public readonly id: number, public readonly text: string) {}

  /** True once the to-do has been ticked off. */
  public get done(): boolean {
    return this.isDone;
  }

  /** Done becomes not done, and not done becomes done. */
  public toggle(): void {
    this.isDone = !this.isDone;
  }
}
```

And the list hands out ids and finds to-dos by them:

src/ToDoList.ts

```

export class TodoList {
  private todos: Todo[] = [];
  private nextId: number = 1;

  /** Adds a to-do (without the spaces round it) and gives it back. Throws for blank text. */
  public add(text: string): Todo {
    const trimmed = text.trim();
    if (trimmed === "") {
      throw new Error("A to-do needs some text");
    }
    const todo = new Todo(this.nextId, trimmed);
    this.nextId++;
    this.todos.push(todo);
    return todo;
  }

  /** Ticks off (or un-ticks) the to-do with this id. */
  public toggle(id: number): void {
    this.get(id).toggle();
  }

  /** The to-dos that `filter` shows, oldest first. */
  public visible(filter: Filter): Todo[] {
    return this.todos.filter((todo) => matchesFilter(todo, filter));
  }

  /** How many to-dos are not done yet. */
  public get remaining(): number {
    return this.visible("active").length;
  }
  // ... remove(id), and a private get(id) that throws if there is no such to-do
}

```

Why ids, when the shopping list used positions? Because the to-do list is filtered. When "Active" is chosen, the first to-do *on the screen* might be the third in the list, so "delete item 0" would delete the wrong one. An id belongs to the to-do itself and never changes, so the view can say "delete to-do 3" whatever is showing. A test makes sure ids are never reused:

tests/TodoList.test.ts

```

Deno.test("ids are never reused after a remove", () => {
  const list = sampleList();
  list.remove(3);
  assertEquals(list.add("Water plants").id, 4);
});

```

Filters

A filter is one of three words. A plain `string` would also allow "all" or "Done", and the mistake would show up only as an empty list. So the type lists the words allowed:

src/filter.ts

```
export type Filter = "all" | "active" | "done";

/** Every filter, in the order the buttons appear. */
export const FILTERS: Filter[] = ["all", "active", "done"];

/** True if `todo` should be shown when `filter` is chosen. */
export const matchesFilter = (todo: Todo, filter: Filter): boolean => {
  if (filter === "active") {
    return !todo.done;
  }
  if (filter === "done") {
    return todo.done;
  }
  return true;
};
```

"all" | "active" | "done" is a union, like `number | null` - but of three particular strings. `visible("Done")` is now a compile error, not a bug. Chapter 8 compares this with Java-style enums.

Keeping words out of the view

The view shows "2 items left". Suppose `render` simply wrote:

```
this.remainingLabel.textContent = `${remaining} items left`;
```

Then, with one to-do left, the page would say "1 items left". The fix is a rule - one is singular, zero gets its own words - and rules belong where they can be tested. So the words moved into a plain function in its own file:

`src/messages.ts`

```
/** "1 item left", "3 items left", "Nothing left to do". */
export const remainingText = (count: number): string => {
  if (count === 0) {
    return "Nothing left to do";
  }
  return count === 1 ? "1 item left" : `${count} items left`;
};
```

`messages.ts` is not the model (it knows nothing about to-dos) and not the view (it touches no DOM). It is a small piece of logic that the view *uses*, and because it touches no DOM, it is tested:

`tests/messages.test.ts`

```
Deno.test("remaining: none, one, and more than one", () => {
  assertEquals(remainingText(0), "Nothing left to do");
  assertEquals(remainingText(1), "1 item left");
  assertEquals(remainingText(2), "2 items left");
});
```

`emptyText(filter)`, the message shown instead of an empty list ("Nothing done yet"), lives there too. The general rule: **if a piece of view code has an `if` in it that could be wrong, move the `if` somewhere it can be tested.**

Exercise 7.3 - Write the test first

Suppose `remainingText` did not exist yet. Write its test first and run it (it fails to compile, which counts as red). Then write the function. Which edge would you most expect to get wrong?

Here is one way: the test above, written first. The edge most people miss is 1 - "1 items left" is the bug that started this section. The 0 case is a choice: "0 items left" would be correct, but the test makes you decide.

A view with several handlers

The to-do view needs to report three kinds of click: a checkbox, a Delete button, a filter button. Rather than three constructor parameters, it takes one object with three functions in it:

`src/ToDoView.ts`

```
/** What main.ts wants to happen for each kind of click. One function type per event. */
export type TodoHandlers = {
  onToggle: (id: number) => void;
  onRemove: (id: number) => void;
  onFilter: (filter: Filter) => void;
};

export class ToDoView {
  private readonly listElement = requireElement<HTMLUListElement>("#todos");
  private readonly emptyMessage = requireElement<HTMLElement>("#empty");
  private readonly remainingLabel = requireElement<HTMLElement>("#remaining");

  constructor(private readonly handlers: TodoHandlers) {
    // The filter buttons never change, so their listeners are added once, here.
    for (const filter of FILTERS) {
      this.filterButton(filter).addEventListener("click", () => this.handlers.onFilter(filter));
    }
  }

  /** Makes the page show these to-dos, this count, and which filter is chosen. */
  public render(todos: Todo[], remaining: number, filter: Filter): void {
    this.listElement.replaceChildren();
    for (const todo of todos) {
      this.listElement.appendChild(this.todoElement(todo));
    }
    this.emptyMessage.textContent = todos.length === 0 ? emptyText(filter) : "";
    this.remainingLabel.textContent = remainingText(remaining);
    for (const name of FILTERS) {
      this.filterButton(name).classList.toggle("chosen", name === filter);
    }
  }
  // ... filterButton(filter) and todoElement(todo)
}
```

`TodoHandlers` is an object type (Chapter 3) whose properties are function types. Each to-do's checkbox calls `this.handlers.onToggle(todo.id)` and each Delete button calls `this.handlers.onRemove(todo.id)`, from arrow-function listeners made in `todoElement`. The text goes in with `textContent`, as in the shopping list.

Notice what the view is given: the to-dos to show, the count, and the filter - already worked out. It does not filter or count anything itself.

Event, model, render - at scale

Here is the whole of the wiring in `main.ts`:

`src/main.ts`

```
const list = new TodoList();
let filter: Filter = "all";

const view = new TodoView({
  onToggle: (id) => {
    list.toggle(id);
    render();
  },
  onRemove: (id) => {
    list.remove(id);
    render();
  },
  onFilter: (chosen) => {
    filter = chosen;
    render();
  },
});

const render = (): void => view.render(list.visible(filter), list.remaining, filter);
```

Four kinds of event (with adding a to-do, below these lines), and every one is the same two steps: change the state, then `render()`. The state is the `TodoList` and the chosen filter: the filter is part of what the page shows, so it is state too, even though it is a plain variable.

`render` refers to `view`, and the handlers refer to `render`, which is defined after them. That is fine: the arrow functions do not *run* until a click happens, and by then both constants exist.

This is the payoff of the pattern. Nowhere does a listener reach into the page and tweak one element ("strike through this line", "change that number"). Each change is made in exactly one place, the model, and `render` rebuilds the page from it. Add a fifth kind of event and you write two lines; the page cannot end up showing a count that disagrees with the list.

Testing the model only

The project has 17 tests, and every one is of a model class or a plain function: `Todo`, `TodoList`, `matchesFilter`, `remainingText` and `emptyText`. Here are the filters, using a helper that makes a list of three with the middle one done:

`tests/TodoList.test.ts`

```
Denotest("the active filter hides done to-dos", () => {
  assertEquals(textsOf(sampleList().visible("active")), ["Buy milk", "Ring Gran"]);
});

Denotest("the done filter shows only done to-dos", () => {
  assertEquals(textsOf(sampleList().visible("done")), ["Post letter"]);
});
```

What is *not* tested is `TodoView` and `main.ts`. Is that a gap? Look at what is in them: finding elements, `createElement`, `textContent`, adding listeners, and calls to the model. None of it makes a decision about the data, so there is very little that a test could catch that a glance at the page would not. (Testing views is possible - with a fake `document` from a library, or a headless browser like the one this book uses for its screenshots - but it is slower and more fragile, and it pays only once the model tests are in place.)

That is the deal model-view offers: **keep the view thin, and put everything that can be wrong where it can be tested.**

Java and TypeScript

Java	TypeScript
Swing: a model class, a <code>JPanel</code> view, listeners as the controller	a model class, a view class, <code>main.ts</code> as the controller
<code>label.setText(name)</code> - always plain text	<code>element.textContent = name</code> - plain text; <code>innerHTML</code> is not
JSP/Thymeleaf escape <code><</code> and <code>></code> for you	the DOM does not: choose <code>textContent</code> yourself
<code>Objects.requireNonNull(x, "message")</code>	<code>requireElement(selector)</code> - throws, naming what is missing
<code><T extends Component> T find(String id)</code>	<code><T extends HTMLElement>(selector: string): T</code>
an <code>ActionListener</code> passed to a view	a function (or an object of functions) passed to a view's constructor
<code>Collections.unmodifiableList(items)</code> or <code>List.copyOf(items)</code>	<code>[...this.items]</code> - a copy
an enum <code>Filter { ALL, ACTIVE, DONE }</code>	<code>type Filter = "all" "active" "done"</code>

Summary

- the **model** holds the data and the rules and never touches the DOM; the **view** shows the model and makes no decisions about the data; `main.ts` makes both and joins them
- every event takes the same path: change the model, then render; `render` rebuilds the page from the model, so it can be called at any time
- a view class finds its elements once, in its constructor, and keeps them in `private readonly` fields
- `requireElement<T>(selector)` returns the element or throws an error naming the selector, so a typo in an id is loud instead of silent
- `textContent` shows text exactly as it is; `innerHTML` runs it as HTML - never give `innerHTML` anything a user typed (cross-site scripting)
- a view reports clicks by calling functions it was given (`RemoveHandler`, `TodoHandlers`) - it does not change the model itself

- give out copies (`[...items]`), find things by id rather than position when the view filters, and use a union of strings for a fixed set of choices
- test the model, and move any rule that creeps into the view (like "1 item" vs "2 items") into a function that can be tested

Challenges

Each challenge says which project to start from. Write the tests first, keep the model free of the DOM, and keep every user's text out of `innerHTML`.

1. Nearly full

Start from `ch07_project01_tally_counter`. When the room is at least 80% full (but not full), the status should turn orange and say "Nearly full - 2 spaces left". Add a getter `isNearlyFull` to `Tally`, test first: test a room of 10 with 7, 8 and 10 people in it. Then use it in the view.

2. Clear the list

Start from `ch07_project02_shopping_list`. Add a **Clear all** button that empties the list. Give `ShoppingList` a `clear()` method, test first. The button should be disabled when the list is already empty - which file decides that, and what does it ask the model?

3. Two rooms

Start from `ch07_project01_tally_counter`. Count two rooms on one page: the lab (12 people) and the library (30). Use two `Tally` objects and **two** `TallyView` objects. Write a test first that shows two tallies are independent (incrementing one leaves the other at zero). Then change `TallyView` so it can be told which part of the page is its own.

4. Clear done

Start from `ch07_project03_todo_filters`. Add a **Clear done** button that removes every to-do that is done. Give `TodoList` a method `clearDone(): number` that returns how many it removed, test first (including a list where nothing is done). Show "Removed 2" for a moment, and disable the button when there is nothing to clear.

Hint: `filter` keeps the to-dos that are not done; compare the lengths before and after. A getter that says whether anything is done lets the view disable the button without counting.

5. Move up and down

Start from `ch07_project03_todo_filters`. Give each to-do ▲ and ▼ buttons that move it up or down the list. Write `moveUp(id)` and `moveDown(id)` test first: an ordinary move, and both edges (the first to-do cannot move up, the last cannot move down - nothing happens). Then think about filters: what should "up" mean when "Active" is chosen and the to-do above is done and hidden?

Hint: find the to-do's position in the private array, then swap two elements: `[a[i], a[i - 1]] = [a[i - 1], a[i]]`. Add the new handlers to `TodoHandlers`; the compiler will show you every place that needs them.

6. Remember the list

Start from `ch07_project03_todo_filters`. Make the to-dos survive a page reload, using the browser's `localStorage`. The model must still know nothing about the browser: give `TodoList` a method that turns the list into plain data (`{ id, text, done }[]`) and a way to build a list from such data, and test that a list survives the round trip - ids, text, done, and the next id handed out. `main.ts` does the saving and loading.

Hint: `localStorage.setItem("todos", JSON.stringify(data))` saves a string; `localStorage.getItem("todos")` gives it back, or `null` the first time; `JSON.parse` turns it back into data. Save after every change - there is already one place every change goes through.

Next: [Chapter 8 - Constants, static and enums](#)

Chapter 8 - Constants, static and enums

Some values never change: the colours a traffic light can show, the diets on a menu, the letter printed on a ticket. Some values belong to a whole class rather than to any one object: the number the *next* ticket will get. This chapter is about both. You will see TypeScript's three kinds of "never changes" (`const`, `readonly` and `static readonly`), static fields and methods, and the three ways TypeScript can say "one of these values and nothing else" - including why this book uses string-literal unions rather than Java-style enums.

Traffic Light



Main Street

Red and amber

This colour stays on for 2 seconds.

Next

☐ Change by itself

The Traffic Light project

What you will learn

- when to use `const`, `readonly` and `static readonly`, and how they compare with Java's `final` and `static final`
- string-literal union types such as `"red" | "amber" | "green"`, and `switch` statements that the compiler checks cover every value
- testing every value of a union, with a loop instead of copy and paste
- static fields and static methods, and calling them on the class: `Ticket.issuedCount()`
- what static state does to tests, and how to keep tests independent of each other
- TypeScript's `enum`, unions and `as const`, side by side - and which to choose
- checking strings from a JSON file against a union, and throwing an error for a bad one

The projects

Project	What it shows
ch08_project01_traffic_light	a union type for the colours, a <code>switch</code> over every value, <code>static readonly</code> and <code>readonly</code> in one class, <code>Record</code>
ch08_project02_menu_order	<code>enum</code> , union and <code>as const</code> compared; diets and courses checked as they come in from JSON; a static factory method
ch08_project03_ticket_numbers	a static counter shared by every ticket, static methods, and keeping tests independent when there is static state

Three kinds of "never changes"

In Java you write `final` for a variable that is never reassigned, and `public static final` for a class constant such as `Math.PI`. TypeScript splits the job three ways, depending on *where* the value lives:

You want	TypeScript	Java
a variable (or a module-level value) that is never reassigned	<code>const LIMIT = 3;</code>	<code>final int limit = 3;</code>
a field that each object sets once, in the constructor	<code>readonly name: string</code>	<code>private final String name;</code>
one value for the whole class, used as <code>ClassName.VALUE</code>	<code>static readonly PREFIX = "A";</code>	<code>public static final String PREFIX = "A";</code>

You have used the first since Chapter 1, and Chapter 6 introduced the second. All three are checked by the compiler, and all three give the same kind of message when you try to break them:

```
TS2588 [ERROR]: Cannot assign to 'LIMIT' because it is a constant.
TS2540 [ERROR]: Cannot assign to 'name' because it is a read-only property.
TS2540 [ERROR]: Cannot assign to 'STARTING_COLOUR' because it is a read-only property.
```

The naming rule is the same as in Java: class constants and module constants that are fixed settings are written in `UPPER_CASE` (`MILLISECONDS_PER_SECOND`, `Ticket.PREFIX`); ordinary `const` variables inside functions are written in `camelCase`.

Project 1: A traffic light

Traffic Light



Main Street

Red and amber

This colour stays on for 2 seconds.

Next

☐ Change by itself

The Traffic Light project

A UK traffic light shows four things, always in the same order: red, then red and amber together (get ready), then green, then amber, then red again. Press **Next** to step through them, or tick **Change by itself** to let a timer do it.

A type with exactly four values

How should the program store "the colour the light is showing"? A `string` would work, but it would also accept `"blue"`, `"Red"` and `"gren"`. Java's answer is an `enum`. TypeScript's simplest answer is a **string-literal union type**:

`src/light_colour.ts`

```
/** One of the four things a UK traffic light can show. */
export type LightColour = "red" | "red-amber" | "green" | "amber";
```

You have met unions before: `number | null` means "a number, or null". Here each part of the union is not a whole type but a single string value, so `LightColour` means "one of these four strings, and nothing else". At run time a `LightColour` is just a string - you can print it, compare it with `===`, put it in a template literal - but the compiler checks every place one is made:

```
TS2345 [ERROR]: Argument of type '"purple"' is not assignable to parameter of type 'LightColour'.
export const a = nextColour("purple");
```

Your editor knows the four values too: type `nextColour("` and it offers them.

nextColour, test first

The light needs one rule: which colour comes after which. That is a function from `LightColour` to `LightColour`, and a good one to write test first.

Red. The first test, in `tests/light_colour.test.ts`:

```
Deno.test("after red comes red and amber", () => {
  assertEquals(nextColour("red"), "red-amber");
});
```

Green. The simplest code that passes:

```
export const nextColour = (colour: LightColour): LightColour => "red-amber";
```

(The linter points out that `colour` is never used. The next test will need it.)

Red again. A second test:

```
Deno.test("after red and amber comes green", () => {
  assertEquals(nextColour("red-amber"), "green");
});
```

It fails, because `nextColour` always says `"red-amber"`:

```
not ok 2 - after red and amber comes green
---
message: |-
  AssertionError: Values are not equal.

      [Diff] Actual / Expected

-   red-amber
+   green
```

Green - and a surprise. Now the function has to look at the colour. A `switch` suits a union well: one `case` per value. Write just the two cases the tests need:

```
export const nextColour = (colour: LightColour): LightColour => {
  switch (colour) {
    case "red":
      return "red-amber";
    case "red-amber":
      return "green";
  }
};
```

Both tests pass - but the build reports a type error:

```
TS2366 [ERROR]: Function lacks ending return statement and return type does not include
'undefined'.
export const nextColour = (colour: LightColour): LightColour => {
  ~~~~~
```


The compiler knows that `colour` could also be `"green"` or `"amber"`, and that for those the function would fall off the end and return nothing - which is not a `LightColour`. It is acting like a test you did not have to write: *every value must be handled*. So the "simplest code that passes" here is all four cases:

`src/light_colour.ts`

```
/** The colour that comes after this one. A UK light goes red, red and amber, green, amber, red... */
export const nextColour = (colour: LightColour): LightColour => {
  // A switch over a union: the compiler knows the four possible values. Because the return type is
  // LightColour, leaving a case out is a compile error ("Function lacks ending return statement").
  switch (colour) {
    case "red":
      return "red-amber";
    case "red-amber":
      return "green";
    case "green":
      return "amber";
    case "amber":
      return "red";
  }
};
```

Then add the tests for green and amber, one behaviour each, so that every case is pinned down by a test as well as by the compiler.

Two things to notice about this `switch`. There is no `break`: each case ends with `return`. And there is no `default`: A `default` would make the compiler happy even with a case missing - which is exactly what you do not want. Leave it out, and when someone adds a fifth colour to the union, the compiler shows them every `switch` that needs a new case. (A typo is caught too: `case "gren":` gives `Type '"gren"' is not comparable to type 'LightColour'.`)

Note - This only works because the return type is written. Without `: LightColour`, the two-case version compiles: TypeScript quietly works out the return type as `"red-amber" | "green" | undefined`, and the problem only shows up later, if at all, wherever the result is used. One more reason this book always writes return types.

Testing every value

A union has a small, fixed number of values, so you can test *all* of them. To do that the tests need the values in an array:

`src/light_colour.ts`

```
/** Every colour, in the order the light shows them - so the tests can check every one. */
export const LIGHT_COLOURS: LightColour[] = ["red", "red-amber", "green", "amber"];
```

and then a loop does the work of four copied tests:

`tests/light_colour.test.ts`

```

Deno.test("every colour comes back to itself after four steps", () => {
  for (const colour of LIGHT_COLOURS) {
    const afterFour = nextColour(nextColour(nextColour(nextColour(colour))));
    assertEquals(afterFour, colour);
  }
});

```

This test says something the four single tests do not: the sequence is a *cycle*. Another test checks that four steps visit every colour exactly once, and another that every colour has a name to show on the page (colourName, a second switch).

There is a weakness here. The colours are written twice - once in the type, once in the array - and nothing stops someone adding a colour to the type and forgetting the array. The tests would then quietly skip the new colour. Project 2 shows how to write the values once and get both.

static readonly and readonly together

The TrafficLight class holds the current colour. It uses both kinds of read-only field:

src/TrafficLight.ts

```

export class TrafficLight {
  // static readonly: one value for the whole class, shared by every light, never reassigned.
  // Used from outside as TrafficLight.STARTING_COLOUR - no object needed.
  public static readonly STARTING_COLOUR: LightColour = "red";

  /** How many seconds each colour stays on. Record<LightColour, number> needs a key for every
  colour. */
  public static readonly SECONDS: Record<LightColour, number> = {
    "red": 5,
    "red-amber": 2,
    "green": 5,
    "amber": 3,
  };

  private colour: LightColour = TrafficLight.STARTING_COLOUR;

  // readonly (a parameter property): each light has its own name, set once in the constructor.
  constructor(public readonly name: string) {}
}

```

STARTING_COLOUR and SECONDS are the same for every traffic light, so they belong to the **class**: there is one copy, however many lights exist, and you reach them through the class name -

TrafficLight.SECONDS, never this.SECONDS - even inside the class. name is different for each light ("Main Street", "Station Road"), so it belongs to each **object**; readonly means it is set in the constructor and never changed. The test "each light keeps its own name" makes two lights and checks each one.

Record<LightColour, number> is new. It is a built-in type meaning "an object with one property for every LightColour, each holding a number". Because the keys come from the union, the compiler checks that none is missing. Leave out "red-amber" and you get:

```

TS2741 [ERROR]: Property '"red-amber"' is missing in type '{ red: number; green: number; amber: number; }' but required in type 'Record<LightColour, number>'.

```

("red-amber" needs quotes as a key because of the -; the others are quoted to match.)

`main.ts` uses the seconds to set a timer for each colour, and a module-level `const` for the conversion - a plain value, with no class needed:

`src/main.ts`

```
// A module-level const: a plain value that is never reassigned. No class needed.  
const MILLISECONDS_PER_SECOND = 1000;
```

Note - `readonly` is shallow. `TrafficLight.SECONDS = {...}` is an error, but `TrafficLight.SECONDS.red = 99` compiles: the property `SECONDS` cannot be pointed at a different object, but the object it points to can still be changed. Java's `final` behaves the same way with a `final` array or list. Chapter 9 looks at this properly.

Exercise 8.1 - One full cycle

Write a test that checks one full cycle of the light - all four colours - takes 15 seconds. Use `LIGHT_COLOURS` and `TrafficLight.SECONDS`, so the test still makes sense if a colour is added. Try it before reading on.

Here is one way:

```
Deno.test("one full cycle takes 15 seconds", () => {  
  const total = LIGHT_COLOURS.reduce((sum, colour) => sum + TrafficLight.SECONDS[colour], 0);  
  assertEquals(total, 15);  
});
```

`TrafficLight.SECONDS[colour]` looks up a property using a variable, as you would with a Java `Map`. TypeScript allows it because `colour` is a `LightColour`, and `Record<LightColour, number>` promises a number for every one.

Project 2: Menu order - three ways to write an enum

Menu Order

Show:

Everything

Vegetarian

Vegan

Starter

Tomato soup	Vegan	€6.50	Add
Garlic mushrooms	Vegetarian	€7.50	Add

Main

Vegetable curry	Vegan	€14.50	Add
Cheese and onion pie	Vegetarian	€15.50	Add

Dessert

Fruit salad	Vegan	€5.95	Add
Apple crumble	Vegetarian	€6.95	Add
Chocolate brownie	Vegetarian	€6.50	Add

Your order

- Garlic mushrooms - €7.50
- Vegetable curry - €14.50

Total: €22.00

Clear the order

The Menu Order project

The menu has dishes in three courses, and each dish is vegan, vegetarian or contains meat. The buttons at the top show only the dishes that suit a diet, and **Add** puts a dish on the order.

Java's enum

The Java version of a diet is an `enum` :

```
public enum DietType {
    VEGAN,
    VEGETARIAN,
    CARNIVORE
}
```

It solves the problem that integer codes (`public static final int VEGAN = 0;`) cannot: a `DietType` variable can only hold one of the three values, and the code reads as words. TypeScript can do the same thing three ways. `src/three_ways.ts` has all three side by side, and `tests/three_ways.test.ts` shows how each behaves. Nothing on the page uses that file; it is there for comparison.

Three ways to write a fixed set of values

What you write (TypeScript)

1. enum

```
enum DietEnum {  
  Vegan = "vegan", ... }  
→
```

2. string-literal union (this book's choice)

```
type Diet =  
  "vegan" | "vegetarian" | "meat";  
→
```

3. as const array (or object) + union from it

```
const DIETS = ["vegan", ...] as const;  
type Diet = (typeof DIETS)[number];  
use when the program needs to loop over the values  
→
```

What runs (JavaScript)

a real object is made:

DietEnum.Vegan is "vegan"
but a plain "vegan" is not accepted as one

nothing at all - the type is erased;
values are plain strings: "vegan"

a plain array of strings:

["vegan", "vegetarian", "meat"]
the type is erased, the array stays

In every case the compiler rejects "vegen" - the difference is what is left when the program runs.

Three ways to write a fixed set of values

Way 1: enum

src/three_ways.ts

```
export enum DietEnum {  
  Vegan = "vegan",  
  Vegetarian = "vegetarian",  
  Meat = "meat",  
}
```

This looks like Java, and it is used like Java: `DietEnum.Vegan`. Each member is given a string value, so at run time `DietEnum.Vegan` is the string `"vegan"`. But there are surprises.

An enum is not just a type. Almost all of TypeScript disappears when it is turned into JavaScript: types, `private`, `readonly`, return types. An enum does not - the compiler writes a real object into the JavaScript to hold the members. That is why the test can list them with `Object.values(DietEnum)`. Some tools refuse enums for this reason. Node.js can now run TypeScript files by simply deleting the types, and an enum stops it:

```
SyntaxError [ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX]: TypeScript enum is not supported in strip-only mode
```

A plain string is not accepted. Even though `DietEnum.Vegan` is `"vegan"`, you cannot write the string itself:

```
TS2322 [ERROR]: Type '"vegan"' is not assignable to type 'DietEnum'.  
export const d1: DietEnum = "vegan";
```

That matters as soon as data arrives from outside - a JSON file, a form, a `<select>` - because data is always plain strings.

Members without values are numbers. Leave the values out and the members are numbered from 0, as in Java's `ordinal()`:

src/three_ways.ts

```
/** An enum without values: its members are numbered from 0. */
export enum NumberedCourse {
  Starter,
  Main,
  Dessert,
}
```

and the object the compiler makes maps both ways, names to numbers and numbers back to names. The test shows the result, which surprises most people:

tests/three_ways.test.ts

```
Deno.test("a numbered enum also maps the numbers back to the names", () => {
  assertEquals(NumberedCourse[1], "Main");
  // A surprise: listing its keys gives the numbers as well as the names.
  assertEquals(Object.keys(NumberedCourse), ["0", "1", "2", "Starter", "Main", "Dessert"]);
});
```

None of this makes enums wrong - plenty of TypeScript code uses them. But they are the one part of the language that behaves differently from everything around it.

Way 2: a string-literal union

You have already met this way in Project 1:

src/three_ways.ts

```
export type DietUnion = "vegan" | "vegetarian" | "meat";
```

It is only a type, so it vanishes completely from the JavaScript, and the values are ordinary strings - "vegan" from a JSON file is a perfectly good `DietUnion`, once you have checked it. It works with `switch`, the compiler checks for missing cases, and the editor offers the values. Its one weakness is the one Project 1 ran into: because it is only a type, the program cannot loop over its values at run time.

Way 3: `as const`

Write the values once, as an array, and add `as const`:

src/diet.ts

```
/** Every diet a dish can have, from strictest to least strict. */
export const DIETS = ["vegan", "vegetarian", "meat"] as const;

/** "vegan" | "vegetarian" | "meat" - the type of one element of DIETS. */
export type Diet = (typeof DIETS)[number];
```

Without `as const`, TypeScript would say `DIETS` is a `string[]` - any strings, and the array could be changed. `as const` tells it: these exact values, in this order, never changed. Then the second line *works out the union from the array*. Read it from the inside out: `typeof DIETS` is "the type of `DIETS`", and `[number]` asks "what type do you get if you index it with a number?" - the type of one element: `"vegan" | "vegetarian" | "meat"`. (You do not need to be able to write this line from memory; copy the pattern.)

Now the values exist at run time *and* the union exists for the compiler, and since both come from the same line, they can never get out of step. This is what fixes Project 1's weakness. The page loops over the courses the same way, to build one section per course in the right order:

`src/main.ts`

```
// COURSES exists at run time (it is an array, not just a type), so the page can loop over it.  
for (const course of COURSES) {
```

There is also an `as const` *object*, which gives enum-style names to plain strings, so you can write `Diet.Vegan`:

`src/three_ways.ts`

```
export const Diet = { Vegan: "vegan", Vegetarian: "vegetarian", Meat: "meat" } as const;  
/** "vegan" | "vegetarian" | "meat": the type of any value in the Diet object. */  
export type Diet = (typeof Diet)[keyof typeof Diet];
```

You will see this pattern in other people's code, often as a replacement for enums. It works, but the type line is hard to read, and `"vegan"` is already as clear as `Diet.Vegan`.

Which to use?

This book's rule:

- use a **string-literal union** for a fixed set of values - it is the simplest, and it is just strings at run time
- when the program also needs to **list** the values (to loop over them, to make buttons, to check data), write them as an `as const` **array** and work the union out from it
- you will meet `enum` in other people's code, so recognise it - but you do not need it

Checking data from JSON

The dishes come from `src/menu.json`, where every course and diet is just a string:

```
{ "name": "Tomato soup", "course": "starter", "diet": "vegan", "price": 650 },
```

TypeScript reads the JSON file and works out its type: `diet: string`, not `diet: Diet`. If you try to use the data directly, the compiler stops you, and it is right to - nothing has checked that the file does not say `"vegen"`:

```
TS2345 [ERROR]: Argument of type 'string' is not assignable to parameter of type '"starter" | "main" | "dessert"'.  
    "main" | "dessert".
```

So each string is checked on the way in. Here is the test first:

tests/diet.test.ts

```
Deno.test("a string that is not a diet is rejected", () => {
  assertThrows(() => toDiet("pescatarian"), Error, `"pescatarian" is not a diet`);
});
```

and the function, which uses `find` on the `as const` array:

src/diet.ts

```
/** Turns a string (from a JSON file, say) into a Diet - or throws if it is not one. */
export const toDiet = (text: string): Diet => {
  // find gives back an element of DIETS, so its type is Diet | undefined - no cast needed.
  const diet = DIETS.find((value) => value === text);
  if (diet === undefined) {
    throw new Error(`"${text}" is not a diet - expected one of ${DIETS.join(", ")}`);
  }
  return diet;
};
```

This is where the run-time array earns its place: with only a union type, there would be nothing to search. The "every value" test for it loops over `DIETS` and checks that each one comes back unchanged. A further test runs the real `menu.json` through the checks, so a typo in the data turns the tests red before any customer sees it.

Try it - Change one dish's diet in `menu.json` to `"vegen"` and save. The test "every dish in menu.json has a real course and diet" fails, naming the bad value. Put it back.

A static factory method

Making a `Dish` from the JSON data means checking two strings and calling the constructor. That job does not belong to any particular dish - there is no dish yet - so it is a **static method**:

src/Dish.ts

```
export class Dish {
  constructor(
    public readonly name: string,
    public readonly course: Course,
    public readonly diet: Diet,
    public readonly priceInCents: number,
  ) {}

  /**
   * A static "factory" method: it belongs to the class, not to a dish, and makes a Dish from the
   * JSON data - checking the course and diet on the way, so a typo in menu.json is caught.
   */
  public static fromData(data: DishData): Dish {
    return new Dish(data.name, toCourse(data.course), toDiet(data.diet), data.price);
  }
}
```


It is called on the class, before any dish exists: `menu.map((data) => Dish.fromData(data))`. Java programmers know the idea from `List.of(...)` and `Integer.valueOf(...)`: a static method that makes objects is often called a **factory method**, and Book 3 has a whole pattern built on it.

`Order` has a static method too. `formatPrice` turns cents into `"€6.95"`. It uses no fields - it needs a number, not an order - so it is static, and the page calls `Order.formatPrice(...)` for dishes as well as totals. (Prices are whole cents because, as Chapter 3 showed, adding decimals can give `12.499999999`; adding whole numbers never does.)

Exercise 8.2 - Who can eat what

Write a test, using `DIETS` and `filter`, which checks that a vegetarian can eat vegan and vegetarian dishes and nothing else. Try it before reading on.

Here is one way:

```
Deno.test("a vegetarian can eat vegan and vegetarian dishes", () => {
  assertEquals(DIETS.filter((dish) => suits(dish, "vegetarian")), ["vegan", "vegetarian"]);
});
```

The project's own tests do the same with `map`, checking the answer for every diet at once:

```
assertEquals(DIETS.map((dish) => suits(dish, "vegetarian")), [true, true, false]);
```

Project 3: Ticket numbers - static fields and methods

Ticket Numbers

Customers

Take a ticket

Your ticket is A004.

Waiting

A002 A003 A004

Counter

Now serving

A001

Serve next

Tickets issued today: 4

New day

The Ticket Numbers project

At a deli counter you take a numbered ticket and wait to be called. Each new ticket must get the next number: A001, A002, A003. Where should "the next number" be stored?

Not in a ticket - each ticket only knows its own number, and a new ticket does not know about the ones before it. The next number belongs to the **class** `Ticket` as a whole. In Java, and in TypeScript, that is a **static field**:

src/Ticket.ts

```
export class Ticket {
  /** Printed before every number. static readonly: one value for the class, never changed. */
  public static readonly PREFIX = "A";

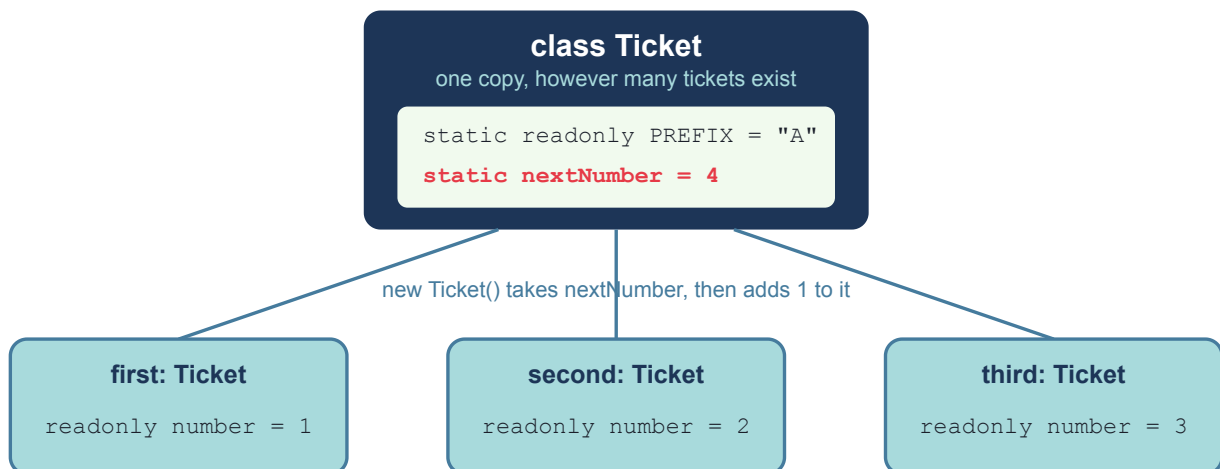
  /** How many digits the number is padded to: 7 is shown as 007. */
  public static readonly DIGITS = 3;

  // static, but not readonly: one counter, shared by every ticket, that goes up each time a ticket
  // is made. There is exactly one nextNumber, however many tickets exist - even none.
  private static nextNumber = 1;

  // readonly, not static: every ticket has its own number, fixed when the ticket is made.
  public readonly number: number;

  constructor() {
    // Inside the class, static members are still reached through the class name, never `this`.
    this.number = Ticket.nextNumber;
    Ticket.nextNumber++;
  }
}
```

static belongs to the class; everything else belongs to each object



Use `Ticket.PREFIX` and `Ticket.issuedCount()` - on the class. Use `first.number` - on an object.

static belongs to the class; everything else belongs to each object

`nextNumber` is **static** but **not** **readonly**: it changes every time a ticket is made. It is **private**, so nothing outside the class can set it to 500. Each ticket's `number` is the opposite: not static (every ticket has its own) but **readonly** (it never changes once printed).

Static methods

A static method is called on the class, and has no object to work on:

src/Ticket.ts

```

/** How any number looks when printed. Static: it needs a number, not a ticket. */
public static format(number: number): string {
    return `${Ticket.PREFIX}${String(number).padStart(Ticket.DIGITS, "0")}`;
}

/** How many tickets have been made since numbering last started at 1. */
public static issuedCount(): number {
    return Ticket.nextNumber - 1;
}

```

The page shows "Tickets issued today: 4" by calling `Ticket.issuedCount()` - no ticket needed. (`padStart` is Chapter 2's: it pads "7" to "007".) The ordinary method `toString()` uses `Ticket.format(this.number)`; an instance method can use static members freely, but not the other way round. A static method has no `this` object, so it cannot reach an object's fields:

```

TS2339 [ERROR]: Property 'count' does not exist on type 'typeof Broken'.
    return this.count;

```

(That was a test class, `Broken`, with an ordinary field `count` and a static method trying to read it. Inside a static method, `this` means the class itself - `typeof Broken` - which has no `count`.)

TypeScript is stricter than Java in one way here. Java lets you reach a static member through an object (`ticket.PREFIX`), with only a warning. TypeScript does not:

```

TS2576 [ERROR]: Property 'PREFIX' does not exist on type 'Ticket'. Did you mean to access the
static member 'Ticket.PREFIX' instead?

```

Static state and tests

Here is how the first two tests of `Ticket` looked when they were written:

```

Deno.test("the first ticket is number 1", () => {
    const ticket = new Ticket();
    assertEquals(ticket.number, 1);
});

Deno.test("each new ticket gets the next number", () => {
    const first = new Ticket();
    const second = new Ticket();
    assertEquals(first.number, 1);
    assertEquals(second.number, 2);
});

```

Each test is right on its own. Run together, the second fails:

```

ok 1 - the first ticket is number 1
not ok 2 - each new ticket gets the next number
---
message: |-
  AssertionError: Values are not equal.

      [Diff] Actual / Expected

-   2
+   1
    at tests/Ticket.test.ts:12:3

```

The first ticket of test 2 is number **2**, because test 1 already made ticket 1, and the static counter is shared by every ticket in the program - including tickets made in other tests. Run test 2 on its own (`deno test --filter "next number"`) and it passes. A test whose result depends on which other tests ran before it is a nasty thing: it can pass on your machine and fail on someone else's, or start failing when somebody adds a test above it.

Every new object starts fresh; a static field does not. So the class needs a way to start again, which it needs anyway for the "New day" button:

`src/Ticket.ts`

```

/** Starts the numbering again from 1: for a new day - and for tests, which each need a fresh
start. */
public static resetNumbering(): void {
  Ticket.nextNumber = 1;
}

```

and **every** test that makes tickets calls it first:

`tests/Ticket.test.ts`

```

Deno.test("each new ticket gets the next number", () => {
  Ticket.resetNumbering();
  const first = new Ticket();
  const second = new Ticket();
  assertEquals(first.number, 1);
  assertEquals(second.number, 2);
});

```

In `tests/TicketQueue.test.ts` a small helper does it, as Chapter 4 suggested instead of a `beforeEach`:

`tests/TicketQueue.test.ts`

```

/** A fresh queue, with numbering starting at 1 - every test's starting point. */
const freshQueue = (): TicketQueue => {
  Ticket.resetNumbering();
  return new TicketQueue();
};

```

When not to use static

A static field that changes is a **global variable** wearing a class's clothes. It is shared by everything, so anything can be affected by anything else - you have just seen that in the tests. The last queue test shows the other side of it:

tests/TicketQueue.test.ts

```
Deno.test("two queues share one counter: the numbers never repeat", () => {
  Ticket.resetNumbering();
  const deli = new TicketQueue();
  const bakery = new TicketQueue();
  assertEquals(deli.take().toString(), "A001");
  assertEquals(bakery.take().toString(), "A002");
  assertEquals(deli.take().toString(), "A003");
});
```

For one shop with one counter, that is what you want. For a post office with a parcels desk and a payments desk, each starting at 1, it is wrong - and no amount of `resetNumbering` will fix it, because there is only one counter. The fix is to move the counter out of the class and into an object - a `TicketMachine` with an ordinary field - so that each desk has its own. Challenge 5 asks you to do exactly that.

A good rule:

- **static readonly** constants are always fine: they never change, so sharing them is harmless
- **static methods** that use only their parameters (like `format`) are fine, and easy to test
- **static fields that change** need a reason. Ask: "could there ever be two of these?" If the answer is yes, use an object instead

Java and TypeScript

Java	TypeScript
<code>final int limit = 3;</code>	<code>const limit = 3;</code>
<code>private final String name;</code>	<code>private readonly name: string;</code> (or a <code>readonly</code> parameter property)
<code>public static final double RATE = 0.1;</code>	<code>public static readonly RATE = 0.1;</code>
<code>private static int nextNumber = 1;</code>	<code>private static nextNumber = 1;</code>
<code>public static String format(int n)</code>	<code>public static format(n: number): string</code>
<code>ticket.PREFIX</code> works (with a warning)	<code>ticket.PREFIX</code> is an error: use <code>Ticket.PREFIX</code>
<code>enum Diet { VEGAN, VEGETARIAN, MEAT }</code>	<code>type Diet = "vegan" "vegetarian" "meat";</code> (recommended), or <code>enum</code>
<code>Diet.values()</code>	<code>DIETS</code> , an array <code>as const</code> , with <code>type Diet = (typeof DIETS)[number];</code>
<code>Diet.valueOf("VEGAN")</code> (throws if unknown)	your own <code>toDiet("vegan")</code> , using <code>DIETS.find</code>
a <code>switch</code> over an enum, often with <code>default</code>	a <code>switch</code> over a union with no <code>default</code> , so the compiler checks every case
<code>EnumMap<Diet, Integer></code>	<code>Record<Diet, number></code>

Summary

- `const` for variables that are never reassigned, `readonly` for fields each object sets once, `static readonly` for one value shared by the whole class
- a string-literal union such as `"red" | "amber" | "green"` allows only those strings; at run time the values are ordinary strings
- a `switch` over a union, with a written return type and no `default`, will not compile until every value has a case
- keep the values in an array so tests can loop over every one; write it `as const` and work the union out from it, `(typeof VALUES)[number]`, so the two can never disagree
- `Record<Union, T>` is an object with a property for every value of the union
- TypeScript's `enum` works but is unlike the rest of the language: it makes a real object, rejects plain strings, and numbers members by default. This book uses unions
- strings from outside (JSON, forms) must be checked before they become a union value: `find`, then throw if `undefined`
- static fields and methods belong to the class: `Ticket.issuedCount()`. A static method has no object, so cannot use an object's fields
- a static field that changes is shared by every test: reset it at the start of each test, and think twice before using one

Challenges

Each challenge says which project to start from. Write the tests first.

1. What should drivers do?

Start from `ch08_project01_traffic_light`. Add a function `instruction(colour: LightColour): string` that says what a driver should do: "Stop", "Stop - get ready to go", "Go if the way is clear", "Stop unless it is unsafe to do so". Write a test for each colour first, then use a `switch` with no `default`, and show the instruction under the colour name on the page.

2. A new diet

Start from `ch08_project02_menu_order`. Add a fourth diet, "pescatarian" (eats fish but no other meat), between "vegetarian" and "meat", and mark Fish and chips as pescatarian in `menu.json`. Write the tests for `suits` first: what can a pescatarian eat, and can a vegetarian eat fish? Then let the compiler show you which `switch` statements need a new case - and look carefully for any code the compiler *cannot* help with. Add a "Pescatarian" filter button.

3. Back to the start

Start from `ch08_project03_ticket_numbers`. The ticket printer only has room for three digits, so after A999 the next ticket should be A001 again. Test first: reset, make 999 tickets in a loop, and check that the next one is number 1. Make sure the existing tests still pass.

4. Courses in order

Start from `ch08_project02_menu_order`. Customers add dishes in any order, but the kitchen wants the order listed starter first, then mains, then desserts. Add a method `Order.byCourse(): Dish[]` that returns the dishes sorted by course (keeping dishes of the same course in the order they were added), test first, and use it to show the order.

Hint: `COURSES.indexOf(dish.course)` gives 0, 1 or 2. Use it in a `toSorted` comparator. The order of the courses then comes from one place - the `COURSES` array - and nowhere else.

5. Two desks

Start from `ch08_project03_ticket_numbers`. A post office has two desks: Parcels, whose tickets start with "P", and Payments, whose tickets start with "M". Each desk numbers its tickets from 1. Write the test first: take two parcels tickets and one payments ticket, and check they are P001, P002 and M001. You will find that a static counter cannot pass it.

Hint: make a class `TicketMachine` whose constructor takes the prefix (`constructor(private readonly prefix: string)`) and which has an ordinary field `private nextNumber = 1` and a method `take(): Ticket`. A `Ticket` is then given its number and prefix by the machine, in its constructor. Each `TicketQueue` gets its own machine. Do the tests still need `resetNumbering`? Write a sentence explaining why not.

6. A junction

Start from `ch08_project01_traffic_light`. At a crossroads, the north-south light and the east-west light must never both let traffic go. Make a class `Junction` with two `TrafficLight`s and a `next()` method that moves the junction through a fixed sequence of phases: north-south goes through red-amber, green and amber while east-west stays red, then the other way round. Show both lights on the page. The important test: **for every phase** in the whole cycle, at least one of the two lights is red.

Hint: describe the phases with a union type, e.g. `type Phase = "ns-ready" | "ns-go" | ...`, with an `as const` array of them, and a `switch` (or a `Record<Phase, ...>`) that says what colour each light shows in each phase. `TrafficLight` will need a way to be set to a colour, not only moved on. The test loops over every phase.

Next: [Chapter 9 - References, composition and modules](#)

Chapter 9 - References, composition and modules

Objects are rarely alone. A playlist holds songs, a course holds modules, and a module holds students. This chapter is about objects that hold other objects: what it means when two variables refer to the *same* object, how that causes some of the most confusing bugs there are, and how tests catch them. Then it organises a program with more than a handful of classes into folders of ES modules, and compares them with Java's packages.

Aliasing Lab

1. Make basket B from basket A:

`const b = a;``const b = [...a];``const b = structuredClone(a);`

2. Change basket B:

`Add eggs to B``One more of B's first item``Start again`

```
const b = [...a];
```

Basket A

- bread × 2
- milk × 2

Basket B

- bread × 2
- milk × 2
- eggs × 6

A and B are different arrays, but share 2 item objects

The Aliasing Lab project

What you will learn

- the difference between values that are **copied** (numbers, strings, booleans) and values that are **shared** (objects and arrays) - and that Java works the same way
- why `const` does not stop an array or object changing
- three ways to "copy" an array - none at all, a spread copy, and `structuredClone` - and what each shares
- how to find **aliasing bugs** with tests, and how a class protects its own arrays
- **composition** and **aggregation**: objects that *have* other objects, and who owns what
- ES modules in depth: named exports, `import type`, folders and an `index.ts`, and how they compare with Java packages
- why test data should come from a function that builds it fresh, not from a shared constant

The projects

Project	What it shows
ch09_project01_aliasing	two baskets, three ways to copy one into the other, and tests that pin down what is shared
ch09_project02_playlist	a <code>Playlist</code> has <code>Songs</code> ; copying arrays in and out of a class; sharing objects that cannot change
ch09_project03_course_modules	a <code>Course</code> has <code>Modules</code> has <code>Students</code> , in a <code>model/</code> folder with an <code>index.ts</code> ; a fresh fixture for every test

Values and references

Start with something you already know from Java. Copy a number, change the copy, and the original does not change:

tests/values.test.ts

```
Deno.test("numbers are copied: changing b leaves a alone", () => {
  const a = 10;
  let b = a;
  b += 1;
  assertEquals(a, 10);
  assertEquals(b, 11);
});
```

Now do the same with an array:

```
Deno.test("arrays are shared: b is a second name for the same array", () => {
  const a = ["bread"];
  const b = a;
  b.push("milk");
  assertEquals(a, ["bread", "milk"]);
  assertEquals(b, a);
});
```

`b.push("milk")` changed `a`. That is because `const b = a` did not copy the array. An array is an object, and a variable never holds an object - it holds a **reference** to it: where to find it. `const b = a` copies the reference, so now there are two references to one array, and whatever you do through `b` you see through `a`. Two names for the same object are called **aliases**.

The rule in TypeScript (really, in JavaScript):

- **primitives** - `number`, `string`, `boolean`, `null`, `undefined` - are copied
- **objects** - including arrays, and objects made from your own classes - are shared

If that sounds familiar, it should: it is exactly Java's rule. In Java an `int` is copied and an `ArrayList` or a `Student` is shared, because Java variables of class type hold references too. The only surprise for a Java

programmer is strings: a TypeScript `string` is a primitive, not an object - but since strings can never be changed in either language, it makes no difference in practice.

`assertStrictEquals(b, a)` (from Chapter 4) asks "are these the **same object**?" - it uses `===`. `assertEquals` asks "do these have the **same contents**?". Two different objects can have the same contents:

```
Deno.test("two objects with the same contents are equal, but not the same object", () => {
  const a = { name: "milk", quantity: 2 };
  const b = { name: "milk", quantity: 2 };
  assertEquals(a, b); // deep equality: same contents
  assertEquals(a === b, false); // identity: two different objects
});
```

In Java terms, `===` on objects is `==` (the same object), and `assertEquals` is a deep `equals()` that compares every field for you. TypeScript has no `equals` method to override - Book 2's collections chapter comes back to that.

`const` is not "constant"

`const` stops the **variable** changing. It says nothing about the object the variable refers to:

```
Deno.test("const stops you changing the variable, not the array it refers to", () => {
  const shopping = ["bread"];
  shopping.push("milk"); // allowed: the variable still refers to the same array
  assertEquals(shopping.length, 2);
});
```

What `const` forbids is pointing the variable at something else. `shopping = ["cake"];` is a compile error:

```
TS2588 [ERROR]: Cannot assign to 'shopping' because it is a constant.
```

This is Java's `final` again: a `final List<String>` can still have things added to it. To say "this array must not be changed", TypeScript has a different tool - the type `readonly string[]` - which you will use in Project 2.

Parameters are references too

When you pass an array to a function, the function's parameter is one more alias:

```
Deno.test("a function can change the caller's array through its parameter", () => {
  const addMilk = (list: string[]): void => {
    list.push("milk");
  };
  const shopping = ["bread"];
  addMilk(shopping);
  assertEquals(shopping, ["bread", "milk"]);
});
```

But pointing the parameter at a *new* array changes only the parameter - the caller's variable still refers to the old one. The next test in `values.test.ts` shows it. Java's "pass by value" chapter said exactly this:

Java (and TypeScript) pass a **copy of the reference**. The function can change the object through it, but cannot make the caller's variable refer to something else.

Try it - `tests/values.test.ts` tests the language, not the project. If one of its tests surprises you, change it - make `a` an array in the first test, say - and watch it go red.

Project 1: The Aliasing Lab

The Aliasing Lab has two shopping baskets. Basket A always starts as bread and milk. Three buttons make basket B from basket A, three different ways; two more change B. After every click the page shows both baskets, so you can see whether changing B changed A.

A basket is an array of items, and each item is an object:

`src/basket.ts`

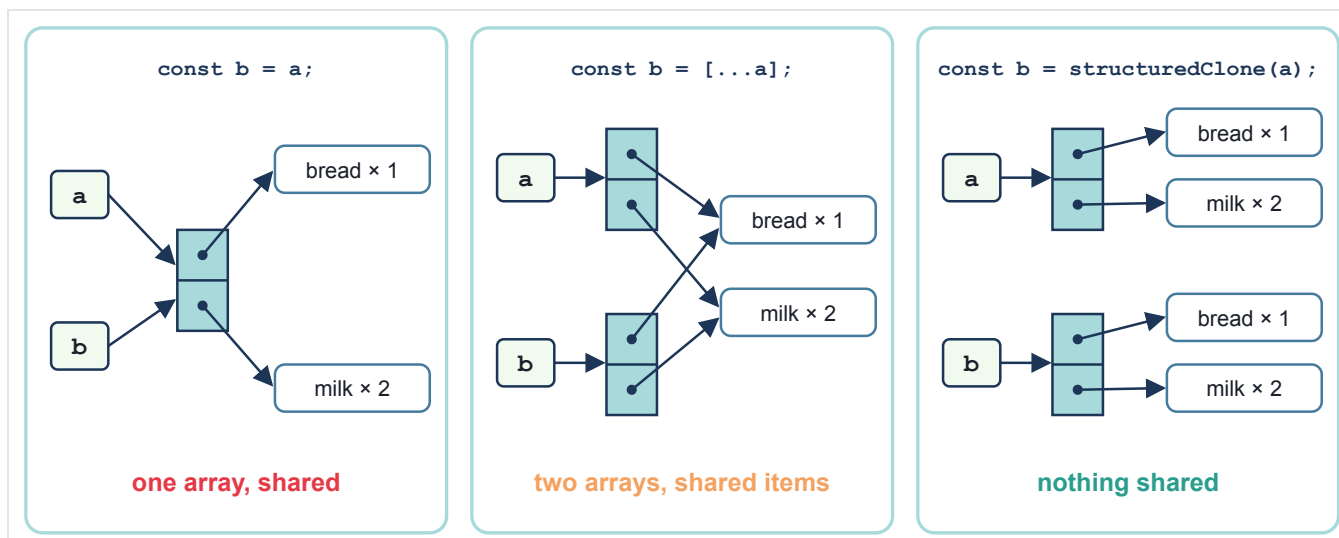
```
/** One line in a basket. An object, so it is shared, not copied, when you assign it. */
export type Item = { name: string; quantity: number };

/** The three ways the page can make basket B from basket A. */
export type CopyMode = "same array" | "spread copy" | "structuredClone";
```

`CopyMode` is a union of string literals, from Chapter 8, and `copyBasket` switches over it:

```
/** Makes B from A. Only one of the three ways gives B nothing in common with A. */
export const copyBasket = (basket: Item[], mode: CopyMode): Item[] => {
  switch (mode) {
    case "same array":
      // Not a copy at all: a second name for the same array.
      return basket;
    case "spread copy":
      // A new array - but its elements are the same Item objects as before.
      return [...basket];
    case "structuredClone":
      // A new array of new Item objects: nothing is shared.
      return structuredClone(basket);
  }
};
```

Here is what each one gives you:



Three ways to make b from a

- `const b = a` copies nothing. One array, two names. Add eggs to B and they appear in A.
- `[...a]` - the **spread** operator - makes a new array and puts A's elements into it. Add eggs to B and A is untouched. But the elements are references to the *same* `Item` objects, so "one more of B's first item" changes A's bread too. This is a **shallow copy**: one level deep.
- `structuredClone(a)` makes a **deep copy**: a new array, new item objects, new everything inside them. Nothing you do to B can reach A.

The screenshot at the top of the chapter is the spread copy: eggs went into B only, but the bread went up to 2 in both.

Spread works on objects too: `{ ...item }` is a new object with the same properties - also shallow. And Java? Java has no spread, but `new ArrayList<>(list)` is the same shallow copy, and Java has no built-in deep copy at all.

Exercise 9.1 - Predict, then check

Make B with `const b = [...a];`, then click **One more of B's first item** twice and **Add eggs to B** once. What does basket A show? Work it out before you try it in the page.

Here is the answer: bread × 3, milk × 2. The two clicks on "one more" changed the bread object, which both arrays share; the eggs went into B's array only.

Testing what is shared

Each behaviour in the page has a test. The tests use both kinds of equality:

`tests/basket.test.ts`

```

Deno.test("spread copy: the items are still shared, so one more milk in B is one more in A", () => {
  const a = makeBasket();
  const b = copyBasket(a, "spread copy");
  oneMore(b, 1);
  assertStrictEquals(b[1], a[1]);
  assertEquals(a[1].quantity, 3);
});

Deno.test("structuredClone: B is equal to A, but not the same objects", () => {
  const a = makeBasket();
  const b = copyBasket(a, "structuredClone");
  assertEquals(b, a);
  assertNotStrictEquals(b, a);
  assertNotStrictEquals(b[0], a[0]);
});

```

`assertNotStrictEquals` is the opposite of `assertStrictEquals`: it passes when the two are *not* the same object. The second test says precisely what a deep copy is: equal contents, different objects, all the way down.

Notice `makeBasket()`. Every test calls it and gets its own brand new basket:

```

/** A new basket every time it is called - new array, new item objects. */
const makeBasket = (): Item[] => [
  { name: "bread", quantity: 1 },
  { name: "milk", quantity: 2 },
];

```

If the tests shared one `const BASKET`, then the test that adds eggs would leave the eggs there for every test after it. Project 3 shows that bug happening.

Note - `structuredClone` copies **data**, not classes. Cloning an object made with `new Box(3)` gives a plain object: `instanceof Box` is `false` and its methods are gone - and TypeScript still believes it is a `Box`. Use it for plain data like `Item`; for your own classes, write a method that makes the copy.

An aliasing bug, found test first

The page needs one more function: `withItem(basket, item)`, which gives back a basket with the item added. Unlike `addItem`, it must leave the basket it is given alone - code that calls it expects to still have the old basket.

Test first. One test for what it gives back, one for what it must not do:

```
tests/basket.test.ts
```

```

Deno.test("withItem gives a basket with the new item at the end", () => {
  const basket = withItem(makeBasket(), EGGS);
  assertEquals(describeBasket(basket), "bread × 1, milk × 2, eggs × 6");
});

Deno.test("withItem leaves the original basket as it was", () => {
  const original = makeBasket();
  withItem(original, EGGS);
  assertEquals(original.length, 2);
});

```

A first attempt that looks perfectly reasonable:

```

export const withItem = (basket: Item[], item: Item): Item[] => {
  basket.push(item);
  return basket;
};

```

The first test passes. The second is red:

```

not ok 8 - withItem leaves the original basket as it was
---
message: |-
  AssertionError: Values are not equal.

      [Diff] Actual / Expected

-   3
+   2

```

`basket` is an alias of the caller's array, so `push` changed the caller's basket - and returned the same array, so the caller now has two names for one changed basket. Nothing on the screen would have told you; the first test would never have caught it either, because the result *looks* right.

The fix is to build a new array, with spread:

`src/basket.ts`

```

/** A new basket with the item added. The basket it is given is left as it was. */
export const withItem = (basket: Item[], item: Item): Item[] => [...basket, item];

```

`[...basket, item]` means "a new array: everything in `basket`, then `item`". Green. This is the same idea as Chapter 3's `toSorted`, which makes a sorted copy where `sort` changes the array: functions that change their arguments are a common source of surprises, so prefer ones that return something new, and make the ones that do change things obvious (`addItem` says so in its comment and returns `void`).

Project 2: A playlist has songs

Playlist

Library

1. Harbour Lights - The Tidewaters (3:34) +
2. Paper Planes at Noon - Mira Quinn (3:07) +
3. Slow Train North - Old Atlas (4:23) +
4. Neon Rain - Kilowatt Kids (3:18) +
5. Lantern - Mira Quinn (4:01) +
6. Copper Sky - The Tidewaters (5:05) +
7. Half a Melody - June & the Jetsam (2:52) +
8. Static Hearts - Kilowatt Kids (3:49) +

My playlist

4 songs · 15:05

1. Harbour Lights - The Tidewaters (3:34) ↑ ×
2. Slow Train North - Old Atlas (4:23) ↑ ×
3. Paper Planes at Noon - Mira Quinn (3:07) ↑ ×
4. Lantern - Mira Quinn (4:01) ↑ ×

The Playlist project

So far the "objects inside objects" were plain data. Now they are objects of your own classes. A `Playlist` holds `Song` objects: a playlist **has** songs. In Java you have met this as **composition**: a class with a field that refers to objects of another class. (Chapter 11's inheritance is the other relationship: a `Cat` **is an** `Animal`. "Has a" or "is a" is usually the first design question about two classes.)

There are two flavours of "has a":

- **composition** - the whole **owns** its parts. The parts are made by the whole and live only inside it: a `Car` and its `Engine`, a course and its modules (Project 3)
- **aggregation** - the whole **refers to** parts that exist on their own and can be shared: a playlist and its songs. The song is in the library whether or not any playlist uses it, and the same song can be in many playlists

In TypeScript both look the same - a private field holding objects. The difference is who makes the objects, and whether anyone else has a reference to them. And that is where aliasing matters.

Sharing is safe when nothing can change

When you click + in the library, the playlist gets the library's own `Song` object, not a copy. The test says so:

`tests/playlist.test.ts`

```
Deno.test("the playlist holds the very same Song objects, not copies of them", () => {
  const playlist = makePlaylist();
  assertStrictEquals(playlist.getSongs()[0], HARBOUR);
  assertEquals(playlist.contains(PLANES), true);
});
```


Sharing `Song` objects is fine, because a `Song` can never change:

`src/Song.ts`

```
export class Song {
  constructor(
    public readonly title: string,
    public readonly artist: string,
    public readonly seconds: number,
  ) {
    if (!Number.isInteger(seconds) || seconds <= 0) {
      throw new Error(`A song must last a whole number of seconds above 0, not ${seconds}`);
    }
  }
}
```

Parameter properties with `readonly` (Chapters 5 and 6): once made, a song stays as it is. No alias can surprise you if nobody can change the object. That is a rule worth remembering: **share objects that cannot change; be careful with objects that can.**

`contains` uses `includes`, which compares with `===`, so it asks about *this very object*:

```
Deno.test("an equal but different Song object is not 'contained'", () => {
  const lookalike = new Song("Harbour Lights", "The Tidewaters", 214);
  assertEquals(makePlaylist().contains(lookalike), false);
});
```

Here, that is what we want - the page only ever adds the library's own objects. Project 3 needs the other answer, and compares ids instead.

The playlist's own array

The songs are shared, but the *array* of songs belongs to the playlist. The playlist keeps it private, and only its methods change it - `add` refuses a song when the playlist is full, `removeAt` and `moveUp` check the position. Encapsulation, from Chapter 6.

But a getter can quietly undo all that. The page needs to show the songs, so `Playlist` has a `getSongs()` method, and the obvious first version is:

```
public getSongs(): Song[] {
  return this.songs;
}
```

It returns the private array itself - an alias. Any code that calls it can `push` a song, skipping the "full" check, or empty the playlist. A test that tries it:

`tests/playlist.test.ts`

```
Deno.test("changing the array from getSongs does not change the playlist", () => {
  const playlist = makePlaylist();
  const songs = playlist.getSongs();
  songs.push(HARBOUR);
  assertEquals(playlist.count(), 3);
});
```

Red:

```
not ok 11 - changing the array from getSongs does not change the playlist
---
message: |-
  AssertionError: Values are not equal.

      [Diff] Actual / Expected

    - 4
    + 3
```

`private` protected the *field*, but not the array the field refers to: once a reference has escaped, anyone holding it can change the array. Java has exactly the same trap (returning a private `ArrayList` from a getter), and the same fix - return a copy:

```
public getSongs(): Song[] {
  return [...this.songs];
}
```

Green. Now refactor, with the tests still passing, to say it in the type as well:

`src/Playlist.ts`

```
/** A copy of the songs, in order. Changing the copy cannot change the playlist. */
public getSongs(): readonly Song[] {
  return [...this.songs];
}
```

`readonly Song[]` is an array type with no methods that change it - no `push`, `pop`, `splice` or `sort`.

Now TypeScript stops careless code before it runs. In fact it stops the test:

```
TS2339 [ERROR]: Property 'push' does not exist on type 'readonly Song[]'.
  songs.push(HARBOUR);
      ~~~~~
```

That is the type doing its job. But `readonly` exists only at compile time - it vanishes from the JavaScript - so the copy is still the real protection, and the test should still check it. The test plays the part of careless code with `as`, which tells TypeScript to treat a value as another type:

```
// getSongs returns readonly Song[], so TypeScript stops honest code changing it. `as Song[]`
plays
// careless code that ignores that - and the copy must protect the playlist even then.
const songs = playlist.getSongs() as Song[];
songs.push(HARBOUR);
```

(`as` is a way of overruling the compiler, so it is rare in this book - a test that deliberately misbehaves is one of the few good reasons.)

The same leak can happen on the way *in*. The constructor accepts a starting array of songs. If it kept that array (`this.songs = songs`), the caller could change the playlist later through their own variable. So it copies that too:

```

constructor(public readonly name: string, songs: Song[] = []) {
  if (songs.length > MAX_SONGS) {
    throw new Error(`A playlist holds at most ${MAX_SONGS} songs, not ${songs.length}`);
  }
  // A copy: if we kept the caller's array, the caller could change our songs later.
  this.songs = [...songs];
}

```

Try it - change the constructor to `this.songs = songs;` and save. The test "changing the array given to the constructor does not change the playlist" goes red - and so does the one you are about to write in Exercise 9.2. Put the copy back.

Copying arrays in and out of a class is called **defensive copying**. It is cheap for small arrays, and it means the class's rules (at most ten songs) cannot be broken from outside.

One more `import` detail in `Playlist.ts`: it never calls `new Song(...)`, it only mentions the type, so it says so:

```
import type { Song } from "./Song.ts";
```

Project 3 explains `import type`.

Exercise 9.2 - Two playlists, one array

Write a test: make an array of two songs, make two playlists from that same array, move a song up in one, and check that the other has its songs in the original order. Will it pass? Try it before reading on.

Here is one way:

```

Deno.test("two playlists made from one array do not affect each other", () => {
  const songs = [HARBOUR, PLANES];
  const first = new Playlist("First", songs);
  const second = new Playlist("Second", songs);
  first.moveUp(1);
  assertEquals(titles(second), ["Harbour Lights", "Paper Planes at Noon"]);
});

```

It passes straight away, because the constructor copies. Without that copy, both playlists would share one array, and moving a song in one would move it in the other. The test is now the last one in `tests/playlist.test.ts`.

Project 3: Course modules

Course Modules

BSc in Computing, Year 2 · 4 modules · 25 credits · 7 students

Module ▼

WEB2 Web Development 2 (5 credits) · 4 students

- Aoife Byrne (C001)
- Darius Nowak (C004)
- Finn Walsh (C006)
- Ben Okafor (C002)

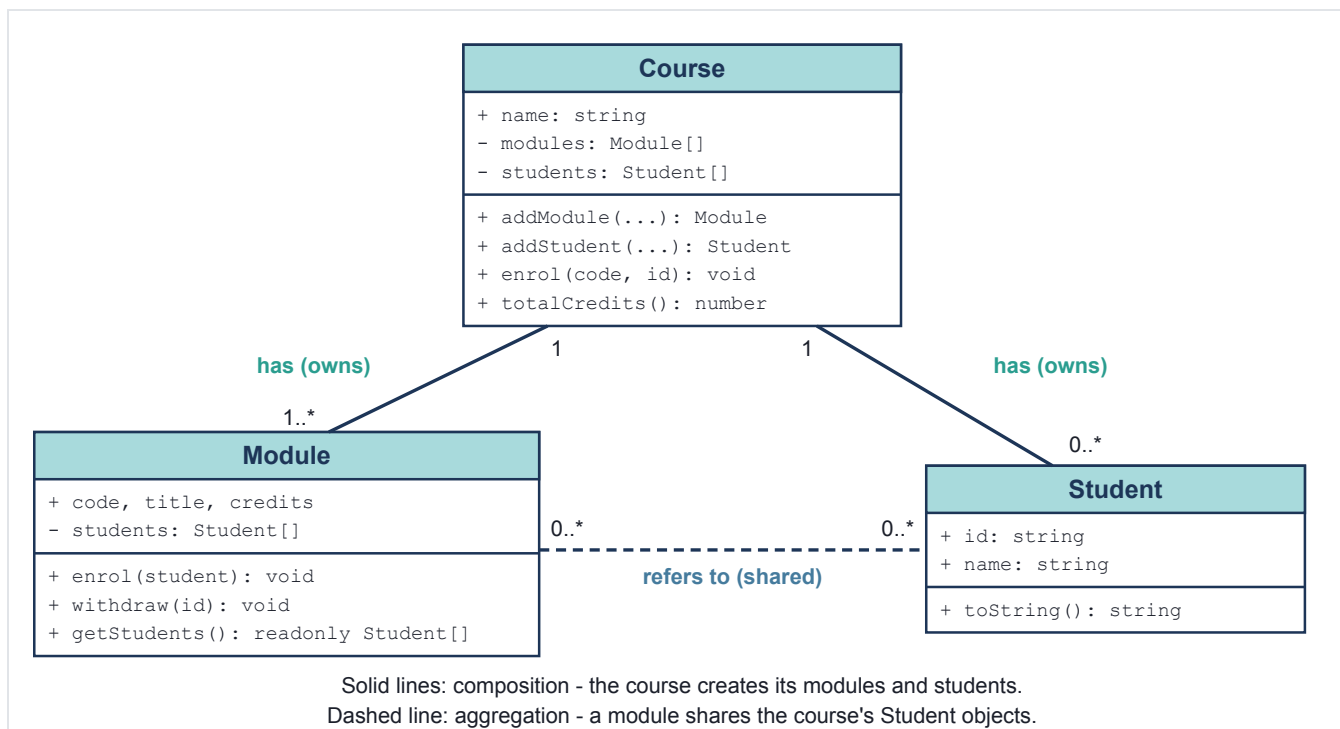
▼

The Course Modules project

A course has modules, a module has students. Pick a module and the page lists who is on it; you can enrol and withdraw students. Three classes, a data file, a function that builds the course from the data, and some text for the page - enough files that it is worth organising them.

Note - this project uses "module" in two senses: a **course module** (the `Module` class, like OOP2 or Databases 1) and an **ES module** (a TypeScript file that exports things). The text says `Module` (in code font) for the class.

Who owns what



Course, Module and Student

The class diagram uses plain lines, as in the Java book, with how many objects are at each end. Read it as: one course has one or more modules and any number of students; a module and a student can be linked any number of times.

The **course owns** its modules and students. Nothing else creates them - the course does, in `addModule` and `addStudent`. That is composition:

src/model/Course.ts

```

/** Makes a new module on this course, and gives it back so it can be used straight away. */
public addModule(code: string, title: string, credits: number): Module {
  if (this.getModule(code) !== undefined) {
    throw new Error(`${this.name} already has a module ${code}`);
  }
  const module = new Module(code, title, credits);
  this.modules.push(module);
  return module;
}

```

A **module refers to** students that the course already has. Aoife takes OOP2 and Web Development 2, and both modules hold the *same* `Student` object - aggregation. The test checks it with the real data:

tests/build_course.test.ts

```

Deno.test("a student on two modules is one shared object", () => {
  const course = buildCourse(data);
  const onOop = moduleOf(course, "OOP2").getStudents()[0];
  const onWeb = moduleOf(course, "WEB2").getStudents()[0];
  assertEquals(onOop.name, "Aoife Byrne");
  assertStrictEquals(onOop, onWeb);
});

```

As with songs, the shared `Student` is safe because its fields are `readonly`.

`getModule(code)` hands out the course's **real** `Module` object, not a copy - the page needs it, to enrol and withdraw. Is that an aliasing risk? Not here, because `Module` protects itself: its student array is private, `getStudents()` returns a `readonly` copy, and the only ways to change it are `enrol` (which refuses a student who is already enrolled) and `withdraw`. Sharing an object whose methods guard its own rules is fine. Sharing a bare array is not.

`Module` compares students by `id` rather than with `===`:

`src/model/Module.ts`

```
public isEnrolled(student: Student): boolean {
  // some(...) is true if the arrow function is true for at least one element.
  // Comparing ids, not objects (===), means a second Student object for the same person - made
  // from the same data, say - still counts as the same student.
  return this.students.some((enrolled) => enrolled.id === student.id);
}
```

Identity (`===`) asks "the same object?"; an id asks "the same person?". For a playlist the first was right; for students it is the second. Choosing which question to ask is part of designing the class.

ES modules in depth

Since Chapter 1, every file you have written has been an **ES module** (ECMAScript module - the standard JavaScript module system). A module is simply a file with `import` or `export` in it. It has its own scope: nothing in it can be seen from outside unless it is exported, and nothing from another file can be used unless it is imported. This section fills in the details.

Named exports

This book only uses **named exports** - `export` in front of a class, function, constant or type:

```
export class Course { ... }
export const MAX_SONGS = 10;
export type Item = { name: string; quantity: number };
```

and imports them by name, in braces:

```
import { courseSummary, moduleSummary } from "../report.ts";
```

The names must match - import a name that a file does not export and the compiler says so (here, asking `index.ts` for a `Teacher` class that does not exist):

```
TS2305 [ERROR]: Module '".../src/model/index.ts"' has no exported member 'Teacher'.
```

JavaScript also has **default exports** (`export default class Course`, imported as `import Course from "../Course.ts"`, with no braces and any name you like). This book avoids them: a named export has one name everywhere, so searching for it finds every use, and editors can add the import for you.

Relative imports always end in `.ts`. Leave it off and Deno will not guess:

```
TS2307 [ERROR]: Cannot find module '../src/model/Student'. Maybe add a '.ts' extension or run with
--sloppy-imports
```

import type

`report.ts` uses `Course` and `Module` only as **types** - in parameter lists - and never makes one with `new`. So it imports them as types only:

`src/report.ts`

```
import type { Course, Module } from "../model/index.ts";
```

A type-only import disappears completely when the TypeScript is turned into JavaScript, because types do not exist at run time. Writing `import type` says so to the reader ("this file only *talks about* courses, it never makes one"), and makes sure the bundler does not load a file just for its types. Try to use the import as a value and you are told:

```
TS1361 [ERROR]: 'Course' cannot be used as a value because it was imported using 'import type'.
```

When one import brings in both values and types, mark just the types with `type` inside the braces. You saw that back in Chapter 3's `main.ts` (`import { average, ..., type Student, ... }`), and Project 1's `main.ts` does the same:

```
import { addItem, COPY_MODES, copyBasket, copyCode, type CopyMode, describeBasket, type Item,
oneMore, sharing } from "../basket.ts";
```

Note - Deno's linter has an optional rule, `verbatim-module-syntax`, that insists on `import type` whenever everything imported is a type. It is not switched on in these projects, but `deno lint --rules-include=verbatim-module-syntax src/` will show you where it would apply.

Folders and `index.ts`

Project 3 puts its classes in a folder:

```
src/
  main.ts
  report.ts
  data/
    course.json
    course_data.ts      types for the JSON: StudentData, ModuleData, CourseData
    build_course.ts     JSON -> objects
  model/
    index.ts
    Course.ts
    Module.ts
    Student.ts
```

The `model/` folder has an `index.ts` that does nothing but **re-export** the three classes:

src/model/index.ts

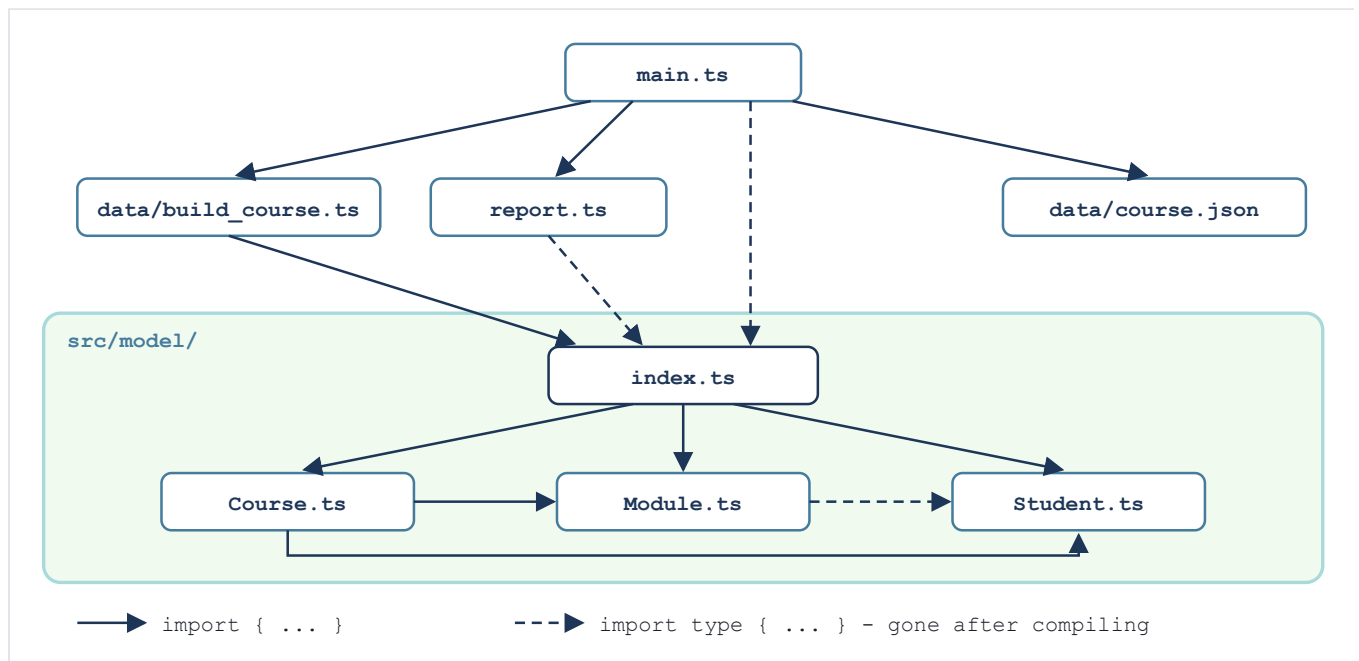
```
export { Course } from "./Course.ts";
export { Module } from "./Module.ts";
export { Student } from "./Student.ts";
```

Code outside the folder imports from `./model/index.ts` and never needs to know which file each class is in:

src/data/build_course.ts

```
import { Course } from "../model/index.ts";
import type { CourseData } from "../course_data.ts";
```

The `index.ts` is the folder's front door. It lists what the folder offers the rest of the program; anything it does not re-export is, by agreement, for the folder's own use. If you later split `Course.ts` in two, only `index.ts` changes. Here is who imports whom:



Imports in Project 3

Notice that the arrows only go one way. The model knows nothing about the page or the report; the report and the page depend on the model. Keeping dependencies flowing in one direction - and never in a circle - is what keeps a larger program understandable. (Book 2's code quality chapter measures this with a tool.) Files inside the folder import each other directly (`Course.ts` imports `./Module.ts`), not through `index.ts`, so the front door cannot cause a circle.

`course_data.ts` contains only types. It still has to be a module of its own, so others can import them - with `import type`, since there is nothing else in it.

Java packages and ES modules

Java	TypeScript
a package is a folder of classes, declared with <code>package model;</code> at the top of each file	a module is one file; folders are just folders - no declaration
<code>public class Course</code> is visible outside the package	<code>export class Course</code> is visible to any file that imports it
no modifier: visible only inside the package	not exported: visible only inside the file
<code>import model.Course;</code>	<code>import { Course } from "../model/index.ts";</code> (a path, relative to this file)
<code>import model.*;</code>	<code>import * as model from "../model/index.ts";</code> then <code>new model.Course(...)</code> - rarely needed
the class name must match the file name	not enforced; this book does it anyway (<code>Course.ts</code>)
one public class per file	any number of exports per file; this book keeps one class per file
there is no "package front door"	an <code>index.ts</code> re-exporting the folder's public names

The biggest difference: in Java the **package** is the unit of privacy; in TypeScript it is the **file**. Something not exported cannot be used anywhere else at all - even in the same folder.

Fresh fixtures, every time

The tests for Project 3 need a course to work on. It is tempting to build one at the top of the file and share it:

```
const COURSE = makeCourse();

Deno.test("enrol adds Bob to Maths", () => {
  COURSE.enrol("MATH", "S2");
  assertEquals(moduleOf(COURSE, "MATH").count(), 1);
});

Deno.test("nobody is on Maths yet", () => {
  assertEquals(moduleOf(COURSE, "MATH").count(), 0);
});
```

Each test is correct on its own. Together, the second fails:

```
ok 21 - enrol adds Bob to Maths
not ok 22 - nobody is on Maths yet
---
message: |-
  AssertionError: Values are not equal.

      [Diff] Actual / Expected

-   1
+   0
```

The first test enrolled Bob on the shared course, and the second test saw him there. It is an aliasing bug in the `tests`: both tests hold a reference to the same course. Run the second test on its own (`deno test --filter "nobody" tests/shared.test.ts`) and it passes - the worst kind of failure, one that depends on which tests run and in what order.

The fix is the one Project 1 used: a **function** that builds fresh data every time it is called. It lives in its own file so all the test files can import it:

`tests/fixtures.ts`

```
/** Two modules, three students; Ann is on Programming. A new course every time. */
export const makeCourse = (): Course => {
  const course = new Course("Test course");
  course.addStudent("S1", "Ann");
  course.addStudent("S2", "Bob");
  course.addStudent("S3", "Cat");
  course.addModule("MATH", "Maths", 5);
  course.addModule("PROG", "Programming", 10);
  course.enrol("PROG", "S1");
  return course;
};
```

Every test starts with `const course = makeCourse();` and gets a course no other test has touched. (Data that can never change - the `Song` constants in Project 2's tests - is safe to share.) Java's JUnit does this with `@BeforeEach`; Deno has no `beforeEach`, and as Chapter 4 said, a helper function is clearer anyway: you can see in the test exactly where its data comes from.

`fixtures.ts` has a second helper. `course.getModule("MATH")` gives `Module | undefined`, and checking for `undefined` in every test would bury what the test is about. `moduleOf` throws instead, so the test fails loudly if the module is missing:

```
/** The course's module with this code. Throws if there is none, so tests need no undefined checks. */
export const moduleOf = (course: Course, code: string): Module => {
  const module = course.getModule(code);
  if (module === undefined) {
    throw new Error(`The test course has no module ${code}`);
  }
  return module;
};
```

Java and TypeScript

Java	TypeScript
<code>int</code> , <code>double</code> , <code>boolean</code> copied; objects and arrays shared	<code>number</code> , <code>string</code> , <code>boolean</code> copied; objects and arrays shared - the same rule
<code>String</code> is an object, but immutable	<code>string</code> is a primitive (and immutable)
<code>a == b</code> on objects: the same object	<code>a === b</code> on objects: the same object
<code>a.equals(b)</code> , written by you	no <code>equals</code> ; tests use <code>assertEquals</code> for deep equality
<code>final List<String> list</code> - the list can still change	<code>const list: string[]</code> - the same
<code>Collections.unmodifiableList(list)</code>	<code>readonly string[]</code> (compile time only)
<code>new ArrayList<>(list)</code> - a shallow copy	<code>[...list]</code> - a shallow copy
no built-in deep copy	<code>structuredClone(data)</code> for plain data
composition: a field referring to another class's objects	the same, usually <code>private</code>
packages, <code>public</code> classes, <code>import model.Course;</code>	file modules, <code>export</code> , <code>import { Course } from './model/index.ts';</code>
<code>@BeforeEach</code> builds fresh test data	a function like <code>makeCourse()</code> , called at the start of each test

Summary

- primitives are copied; objects and arrays are shared - a variable holds a **reference**, and two references to one object are **aliases**
- `===` asks "the same object?" (`assertStrictEquals`); `assertEquals` asks "the same contents?"
- `const` fixes the variable, not the object; `readonly string[]` forbids changing methods, at compile time
- `[...a]` is a shallow copy (new array, same elements); `structuredClone(a)` is a deep copy of plain data
- a function that changes its argument surprises its caller; prefer returning something new, and test that the original is left alone
- a class must not let a reference to its private array escape: copy it coming in and going out, and type it `readonly`
- **composition**: the whole owns and creates its parts; **aggregation**: it refers to shared parts. Sharing is safe when the shared object cannot change, or guards its own rules
- every file is an ES module; use named exports, `import type` for types, and an `index.ts` as a folder's front door; in TypeScript the file, not the package, is the unit of privacy
- test data comes from a function that builds it fresh for each test

Challenges

Each challenge says which project to start from. Write the tests first.

1. Take it out again

Start from `ch09_project01_aliasing`. Write `withoutItem(basket: Item[], name: string): Item[]`, which gives back a basket without the item of that name, and leaves the original basket as it was. Test first: the item is gone from the result, the original still has it, and removing a name that is not there gives an equal basket. Add a **Remove eggs from B** button that uses it.

2. Move down

Start from `ch09_project02_playlist`. Add `moveDown(index)` to `Playlist`, with a ↓ button on each song. Test first: it swaps a song with the one after it, the last song stays where it is, and a position with no song throws, like `moveUp`.

3. Duplicate a playlist

Start from `ch09_project02_playlist`. Give `Playlist` a method `duplicate(name: string): Playlist` that makes a new playlist with the same songs. Test first: the duplicate has the same songs in the same order; removing a song from the duplicate leaves the original alone; and the songs in both are the very same `Song` objects. Write a sentence in a comment explaining why the songs do not need copying, but the array does.

4. Which modules is a student on?

Start from `ch09_project03_course_modules`. Add `modulesFor(id: string): Module[]` to `Course`, and show "Ann is on: OOP2, WEB2" on the page when a student in the list is clicked. Test first, using `makeCourse()`: a student on one module, a student on none, and an id that is not on the course.

Hint: `filter` the course's modules with `isEnrolled`. `isEnrolled` takes a `Student`; you have an `id - getStudent` turns one into the other.

5. A second folder

Start from `ch09_project03_course_modules`. Move `report.ts` into a new `src/report/` folder with its own `index.ts`, and add a new function `courseToCsv(course: Course): string` in `src/report/csv.ts` that gives one line per enrolment: `OOP2,C001,Aoife Byrne`, with a header line `module,id,name`. Test first. Every import of a model class in `src/report/` should be an `import type`. Check with `deno lint --rules-include=verbatim-module-syntax src/` that none was missed.

Hint: `index.ts` can re-export functions just like classes: `export { courseToCsv } from "../csv.ts";`. After moving `report.ts`, its import of the model needs one more `../`.

6. Save and restore

Start from `ch09_project03_course_modules`. Add `toData(): CourseData` to `Course`, which gives back the course as plain data in the same shape as `course.json`, and add **Save** and **Restore** buttons: Save keeps a snapshot, Restore rebuilds the course from it with `buildCourse`. Test first:

- `buildCourse(course.toData())` gives a course whose `toData()` is equal (`assertEquals`) to the original's
- changing the course after `toData()` does not change the saved data
- after restoring, the new course's modules are not the same objects as the old ones (`assertNotStrictEquals`)

Hint: `toData` must build new arrays and new objects (`map` is good at that) - if it handed out the course's own arrays, the snapshot would change along with the course. Why would `structuredClone(course)` not work as a snapshot? (Look back at the Note in Project 1.) `main.ts` will need `let course` instead of `const course`.

Next: [Chapter 10 - Interfaces and structural typing](#)

Chapter 10 - Interfaces and structural typing

In Java, an interface is a promise: "any class that says `implements Shape` will have these methods". TypeScript has interfaces too, written almost the same way - but it checks the promise differently. TypeScript does not care what a class says it is; it cares what it *has*. If an object has every member an interface asks for, it counts. This chapter shows how to write and implement interfaces, what this **structural typing** makes possible (fake objects in tests in one line, the browser's own canvas fitting an interface we wrote ourselves), and where it can surprise a Java programmer.

Shapes

Add circleAdd rectangleAdd triangleClear



6 shapes · total area 29593.3

Shape	Area	Perimeter
 Circle	5026.5	251.3
 Rectangle	5400.0	300.0
 Triangle	3600.0	273.6
 Circle	6939.8	295.3
 Triangle	2231.0	245.4
 Rectangle	6396.0	320.0

The Shapes project

What you will learn

- how to write an `interface`, and make a class keep it with `implements`
- letting the compiler drive test first: "incorrectly implements interface" as your first red
- **structural typing**: a type is about shape, not name - and how that differs from Java
- one class implementing two interfaces, and one object seen through both
- optional members (`strings?: number`) and `readonly` members
- `type` or `interface`: what each can do, and which to choose
- why interfaces vanish when the code runs, and what that means for `instanceof`
- object literals as quick fakes in tests

The projects

Project	What it shows
ch10_project01_shapes	a <code>Shape</code> interface, three classes that implement it, one <code>Shape[]</code> drawn on a <code><canvas></code> ; a <code>Pen</code> interface the canvas fits by structure; fake shapes and a fake pen in the tests
ch10_project02_instruments	<code>Instrument</code> and <code>Tunable</code> : a class implementing two interfaces, an optional member, <code>readonly</code>
ch10_project03_sort_anything	a <code>Comparable</code> -like <code>Sortable</code> interface: one function sorts songs, planets and mountains, developed test first

Why interfaces?

Think about drawing a picture made of circles, rectangles and triangles. The code that draws the picture wants to say "for every shape, draw it", and the code that totals the area wants to say "for every shape, add its area". Neither wants a chain of `if (it is a circle) ... else if (it is a rectangle) ...` - and neither should have to change when somebody adds a hexagon.

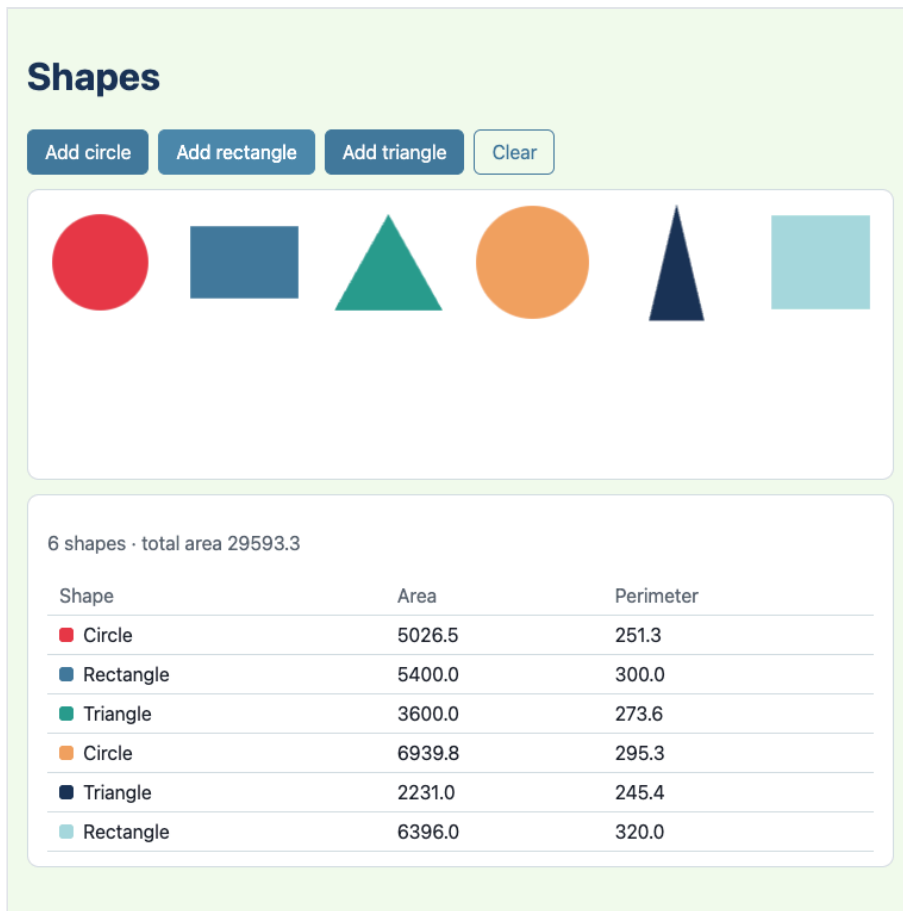
What they need is a **contract**: a list of what every shape can do, which the drawing and totalling code can rely on, and which every kind of shape must keep. In Java that contract is an interface:

```
interface Shape {
    String getName();
    double area();
    double perimeter();
}

class Circle implements Shape { ... }
```

The Java course's `SoundMaker` example made the same point: cats, cars and pianos are in different class hierarchies, but they can all promise a `getSound()` method. TypeScript keeps the idea, and almost the same syntax.

Project 1: Shapes



The Shapes project

Writing an interface

src/Shape.ts

```
// The contract every shape keeps. Anything with these members can be drawn and measured –
// the page, and the functions in shapes.ts, only ever see a Shape, never a Circle or a Triangle.

import type { Pen } from "./Pen.ts";

export interface Shape {
  /** What to call it in the table: "Circle", "Rectangle" ... */
  readonly name: string;
  /** A CSS colour, used to fill it. */
  readonly colour: string;
  area(): number;
  perimeter(): number;
  /** Draws the shape, centred on its position, with the pen it is given. */
  draw(pen: Pen): void;
}
```

An interface lists **members**, with their types and no bodies:

- `area(): number;` - a method that takes nothing and returns a number. Like Java, no body, just a semicolon
- `readonly name: string;` - a **property**. Java interfaces can only hold methods (and constants); a TypeScript interface can also say "every shape has a `name` you can read". `readonly` means code using a `Shape` may read it but not change it (more on that in Project 2)

- there is no `public`: everything in an interface is public, as in Java

`export interface` makes it importable, and `import type` (Chapter 9) brings in `Pen`, which we will meet shortly. One interface per file, named after it, like a class.

Implementing it

`src/Circle.ts`

```
const FULL_TURN = 2 * Math.PI; // a whole circle, in radians – what canvas arc() measures angles in

export class Circle implements Shape {
  public readonly name: string = "Circle";

  constructor(
    private readonly x: number,
    private readonly y: number,
    private readonly radius: number,
    public readonly colour: string,
  ) {}

  public area(): number {
    return Math.PI * this.radius * this.radius;
  }

  public perimeter(): number {
    return FULL_TURN * this.radius;
  }

  public draw(pen: Pen): void {
    pen.beginPath();
    pen.arc(this.x, this.y, this.radius, 0, FULL_TURN);
    pen.fill();
  }
}
```

`implements Shape` is the same as Java. The interface's `name` property is kept by a field (`public readonly name: string = "Circle"`) and `colour` by a parameter property (Chapter 5) - either is fine; the interface only cares that the member is there.

The interface is the type

Now the page can hold every kind of shape in one array, and treat them all alike:

`src/main.ts`

```
// The model: one array of Shape. Circles, rectangles and triangles all fit in it.
let shapes: Shape[] = [
  new Circle(60, 60, 40, COLOURS[0]),
  new Rectangle(180, 60, 90, 60, COLOURS[1]),
  new Triangle(300, 60, 90, 80, COLOURS[2]),
];
```

and the code that totals the areas never asks which kind of shape it has:

`src/shapes.ts`

```
/** The areas of all the shapes, added up. */
export const totalArea = (shapes: Shape[]): number => shapes.reduce((total, shape) => total +
shape.area(), 0);
```

`shape.area()` runs the circle's `area` for a circle and the triangle's for a triangle. (Running the right method for each object is **polymorphism**; Book 2 starts with it.) Add a `Hexagon` class that implements `Shape` and `totalArea` works with it, unchanged.

Rectangle, test first

`Circle` is done. Let's build `Rectangle` test first, and watch what the compiler does with `implements`. Start with one test, before there is any `Rectangle` at all:

tests/Rectangle.test.ts

```
Deno.test("a 3 by 4 rectangle has an area of 12", () => {
  const rectangle = new Rectangle(0, 0, 3, 4, "blue");
  assertEquals(rectangle.area(), 12);
});
```

Red, of course - there is nothing to import:

```
TS2307 [ERROR]: Cannot find module 'src/Rectangle.ts'.
  at tests/Rectangle.test.ts:2:27
```

The smallest step: a class with the constructor the test uses, which says it implements `Shape`:

src/Rectangle.ts

```
export class Rectangle implements Shape {
  constructor(
    private readonly x: number,
    private readonly y: number,
    private readonly width: number,
    private readonly height: number,
    public readonly colour: string,
  ) {}
}
```

Still red - and this time the compiler, not the test, is telling us what is missing:

```
TS2420 [ERROR]: Class 'Rectangle' incorrectly implements interface 'Shape'.
  Type 'Rectangle' is missing the following properties from type 'Shape': name, area, perimeter,
  draw
export class Rectangle implements Shape {
  ~~~~~
  at src/Rectangle.ts:6:14
```

That is the point of `implements`: it turns the interface into a checklist, and the compiler ticks it off. The test fails too, with `TypeError: rectangle.area is not a function` - type errors do not stop the tests running, so you see both.

Next step: the members the checklist asks for, doing as little as possible:

```

public readonly name: string = "Rectangle";

// ... constructor

public area(): number {
  return 0;
}

public perimeter(): number {
  return 0;
}

public draw(pen: Pen): void {
}

```

The compiler is happy now. The test is not - it is a real assertion failure, which is the red we wanted:

```

AssertionError: Values are not equal.

      [Diff] Actual / Expected
-    0
+   12

```

(The linter also says ``pen` is never used` - true for now, and it will be fixed when `draw` is written.) Make it green:

```

public area(): number {
  return this.width * this.height;
}

```

Then the same cycle for `perimeter` (a 3 by 4 rectangle: 14) and for `draw` - which raises a question. How do you test that something was *drawn*?

A Pen of our own

Drawing on a `<canvas>` uses its **2D context**, an object of the browser's type `CanvasRenderingContext2D`, with methods such as `beginPath`, `arc`, `rect` and `fill`. A shape could take one of those as its `draw` parameter. But the tests run in Deno, with no page and no canvas.

So the shapes do not ask for a canvas context. They ask for something smaller, which we wrote ourselves:

```
src/Pen.ts
```

```
// The drawing methods a shape needs. We wrote this interface ourselves – the browser's canvas
// context (CanvasRenderingContext2D) has never heard of it – yet the canvas context fits it,
// because it has every one of these methods. That is structural typing: the shape counts, not the
// name.
// It also means a test can hand a shape a small fake pen instead of a real canvas.

export interface Pen {
  beginPath(): void;
  arc(x: number, y: number, radius: number, startAngle: number, endAngle: number): void;
  rect(x: number, y: number, width: number, height: number): void;
  moveTo(x: number, y: number): void;
  lineTo(x: number, y: number): void;
  closePath(): void;
  fill(): void;
}
```

And `main.ts` hands the real canvas context to `draw`:

`src/main.ts`

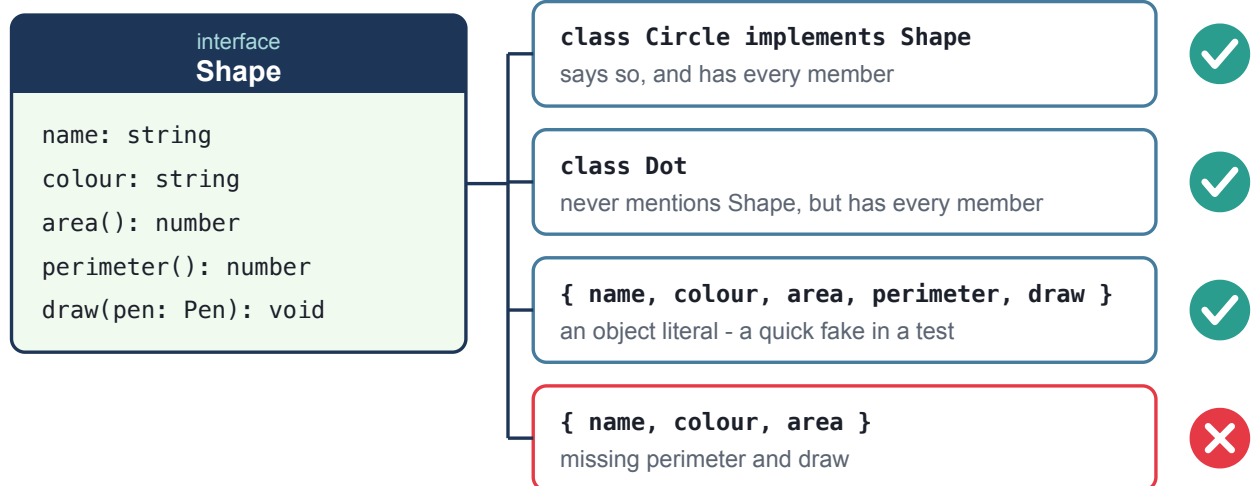
```
for (const shape of shapes) {
  context.fillStyle = shape.colour;
  // The real canvas context is passed where a Pen is wanted – it fits the Pen interface.
  shape.draw(context);
}
```

Stop and think about that from a Java point of view. `CanvasRenderingContext2D` was written by the people who wrote the browser, years ago. It certainly does not say `implements Pen`. In Java, passing it where a `Pen` is wanted would be an error, full stop. In TypeScript it compiles - because the canvas context has `beginPath()`, `arc(...)`, `rect(...)` and the rest, with matching types. That is enough.

Structural typing: shape, not name

Java's type system is **nominal** (by name): a class is a `Shape` only if it, or a superclass, says `implements Shape`. TypeScript's is **structural**: a value is a `Shape` if it has the structure a `Shape` needs - every member, with compatible types. The name of the class, and whether it mentions `Shape` at all, does not matter.

Is it a Shape? TypeScript checks the members, not the name



In Java only **Circle** would count: a Java class is a `Shape` only if it says `implements Shape`.

Is it a Shape?

So this class, which never mentions `Shape`, can go into a `Shape[]`:

```
// no "implements Shape"
export class Dot {
  public readonly name: string = "Dot";
  public readonly colour: string = "black";
  public area(): number { return 0; }
  public perimeter(): number { return 0; }
  public draw(pen: Pen): void { pen.fill(); }
}
const shapes: Shape[] = [new Dot()]; // fine
```

and so can a plain object literal, with no class at all:

tests/shapes.test.ts

```
/** A fake shape with a given area. It cannot draw, and does not need to. */
const fakeShape = (area: number): Shape => ({
  name: "fake",
  colour: "black",
  area: () => area,
  perimeter: () => 0,
  draw: () => {},
});
```

(An arrow function that returns an object literal wraps it in brackets, `=> ({ ... })`, so that the `{` is not read as the start of a block body.)

What does not fit is anything with a member missing, or a member of the wrong type:

```
export const blob: Shape = { name: "Blob", colour: "red", area: () => 1 };
```

```
TS2739 [ERROR]: Type '{ name: string; colour: string; area: () => number; }' is missing the following properties from type 'Shape': perimeter, draw
```

```
public area(): string { return "big"; } // in a class that implements Shape
```

```
TS2416 [ERROR]: Property 'area' in type 'Square' is not assignable to the same property in base type 'Shape'.
```

```
  Type '() => string' is not assignable to type '() => number'.
```

```
    Type 'string' is not assignable to type 'number'.
```

Why write `implements` at all?

If `Dot` fits without it, why does `Circle` say `implements Shape`? Because of *where* the error appears. Without `implements`, a mistake in `Dot` (say, `area` misspelled `aera`) is only reported where a `Dot` is first used as a `Shape` - perhaps in `main.ts`, far from the mistake. With `implements`, the error is on the class itself, as you saw with `Rectangle`, and the class tells the reader what it is for. So: **write `implements` on your own classes**; enjoy structural typing for fakes in tests and for objects you did not write, like the canvas context.

A fit that fails

Structural typing checks the types of the members, not just their names. The shapes need a fill colour, so it is tempting to put `fillStyle: string` in `Pen`. Try it, with a cut-down pen:

```
export interface StrictPen {  
  fillStyle: string;  
  fill(): void;  
}
```

```
TS2345 [ERROR]: Argument of type 'CanvasRenderingContext2D' is not assignable to parameter of type 'StrictPen'.
```

```
  Types of property 'fillStyle' are incompatible.
```

```
    Type 'string | CanvasGradient | CanvasPattern' is not assignable to type 'string'.
```

A canvas's `fillStyle` can hold a gradient or a pattern as well as a colour, so it does not fit `string`. That is why the colour is set in `main.ts` (`context.fillStyle = shape.colour`) and the `Pen` holds only methods.

Note - One extra check applies to object literals. If you write an object literal straight into a place that wants a `Shape`, and give it a member the interface does not have, TypeScript calls it a mistake: `{ ..., draw: () => {}, sides: 3 }` gives `Object literal may only specify known properties, and 'sides' does not exist in type 'Shape'`. Extra members are allowed on objects made elsewhere (the canvas context has dozens more than `Pen`), but in a fresh literal an extra member is usually a typo.

Fakes in tests

Structural typing makes test fakes cheap. `draw` is tested with a fake `Pen` that, instead of drawing, writes down every call it gets:

tests/recording_pen.ts

```
/** A pen that records its calls, and the list it records them in. */
export const recordingPen = (): { pen: Pen; calls: string[] } => {
  const calls: string[] = [];
  const pen: Pen = {
    beginPath: () => {
      calls.push("beginPath");
    },
    arc: (x, y, radius, startAngle, endAngle) => {
      calls.push(`arc ${x} ${y} ${radius} ${startAngle} ${endAngle.toFixed(2)}`);
    },
    rect: (x, y, width, height) => {
      calls.push(`rect ${x} ${y} ${width} ${height}`);
    },
    // ... moveTo, lineTo, closePath and fill, the same way
  };
  return { pen, calls };
};
```

No class, no `implements`, no mocking library: an object literal with the right methods *is* a `Pen`. (The arrow functions' parameters need no types - `Pen` says what they are, just as `Converter` did in Chapter 3.) Each arrow function remembers `calls` (a closure), so the test can read the list afterwards:

tests/Rectangle.test.ts

```
Deno.test("a rectangle is drawn from its top-left corner", () => {
  const recorder = recordingPen();
  new Rectangle(100, 50, 40, 20, "blue").draw(recorder.pen);
  assertEquals(recorder.calls, ["beginPath", "rect 80 40 40 20", "fill"]);
});
```

A rectangle centred on (100, 50) that is 40 wide and 20 high starts at (80, 40). The test proves `draw` works out the corner correctly - without a browser. Now `draw` can be written, and the last red goes green:

src/Rectangle.ts

```
public draw(pen: Pen): void {
  // canvas rect() wants the top-left corner, so step back half the size from the centre
  pen.beginPath();
  pen.rect(this.x - this.width / 2, this.y - this.height / 2, this.width, this.height);
  pen.fill();
}
```

`fakeShape` does the same job for `totalArea`: a test wants shapes with areas of exactly 10, 5 and 2.5, and gets them in one line each, without working out which rectangle has an area of 2.5.

```
Deno.test("the total area adds up every shape's area", () => {
  assertEquals(totalArea([fakeShape(10), fakeShape(5), fakeShape(2.5)]), 17.5);
});
```

Try it - Open `ch10_project01_shapes`, add a few shapes, and check the total area against the table. Then pass `"red"` (a string) to `shape.draw(...)` in `main.ts` and read the error: a string does not fit `Pen`.

Exercise 10.1 - The Triangle tests

`Triangle` has a base and a height. Before reading its code, write a test for its perimeter. Pick numbers that make the answer easy.

Here is one way. Half of a base of 6 is 3; with a height of 4 the sloping sides are exactly 5 (a 3-4-5 triangle), so the perimeter is $6 + 5 + 5$:

`tests/Triangle.test.ts`

```
Deno.test("a triangle with base 6 and height 4 has a perimeter of 16", () => {
  const triangle = new Triangle(0, 0, 6, 4, "green");
  assertEquals(triangle.perimeter(), 16);
});
```

Choosing friendly numbers means `assertEquals` works, with no `assertAlmostEquals` needed. (`Math.hypot(3, 4)` - the length of the long side of a right-angled triangle - is exactly 5.)

Project 2: Instruments

Instruments

4 instruments · 10 strings in all

Guitar · 6 strings

Play

Violin · 4 strings

Play

Drum · no strings

Play

Flute · no strings

Play

Play all

Tune all

Out of tune: Guitar, Violin

Guitar: Strum! (out of tune) Violin: Screech! (out of tune) Drum: Boom! Flute: Toot!

The Instruments project

A band has a guitar, a violin, a drum and a flute. Every instrument can be played. Some have strings and some do not. Some can drift out of tune and be tuned; others cannot. Interfaces describe all of that.

Optional members

src/Instrument.ts

```
export interface Instrument {
  readonly name: string;
  /** How many strings it has – left out by instruments without strings. */
  readonly strings?: number;
  /** The sound it makes, as text: "Strum!" */
  play(): string;
}
```

The `?` in `strings?` makes the member **optional**: an `Instrument` may have it or leave it out. The drum leaves it out:

src/Drum.ts

```
export class Drum implements Instrument {
  public readonly name: string = "Drum";

  public play(): string {
    return "Boom!";
  }
}
```

Java has no direct equivalent; you would return `0`, or `null`, or an `Optional`. In TypeScript, the type of `instrument.strings` is `number | undefined`, and the compiler makes you deal with the `undefined`:

```
export const withSpare = (instrument: Instrument): number => instrument.strings + 1;
```

```
TS18048 [ERROR]: 'instrument.strings' is possibly 'undefined'.
```

So code that reads it checks first, as you did with `find` in Chapter 3:

src/band.ts

```
/** "Guitar · 6 strings", or "Drum · no strings" when `strings` was left out. */
export const describeInstrument = (instrument: Instrument): string => {
  // strings is optional, so its type is number | undefined: check before using it
  if (instrument.strings === undefined) {
    return `${instrument.name} · no strings`;
  }
  return `${instrument.name} · ${instrument.strings} strings`;
};
```

After the `if`, TypeScript knows `instrument.strings` is a `number`. The tests use a fake harp (47 strings) and a fake kazoo (none):

tests/band.test.ts

```
const kazoo: Instrument = { name: "Kazoo", play: () => "Bzzz!" };
const harp: Instrument = { name: "Harp", strings: 47, play: () => "Twinkle!" };
```

One surprise is worth knowing about. Through the `Instrument` type you may ask for `strings`, and get `undefined`. Through the `Drum` type you may not ask at all: `new Drum().strings` is an error, Property `'strings'` does not exist on type `'Drum'`. - the `Drum` class has no such member. The test asks through the interface:

tests/instruments.test.ts

```
Deno.test("a drum, seen as an Instrument, has no strings", () => {
  // Drum itself has no `strings` at all; through the Instrument interface it is optional, so we
  may ask
  const drum: Instrument = new Drum();
  assertEquals(drum.strings, undefined);
  assertEquals(drum.play(), "Boom!");
});
```

Two interfaces, one class

Tuning is a separate idea from playing, so it gets its own interface:

src/Tunable.ts

```
export interface Tunable {
  readonly name: string;
  isInTune(): boolean;
  tune(): void;
}
```

A class can implement as many interfaces as it likes, separated by commas - exactly as in Java:

src/Guitar.ts

```
const GUITAR_STRINGS = 6;

export class Guitar implements Instrument, Tunable {
  public readonly name: string = "Guitar";
  public readonly strings: number = GUITAR_STRINGS;

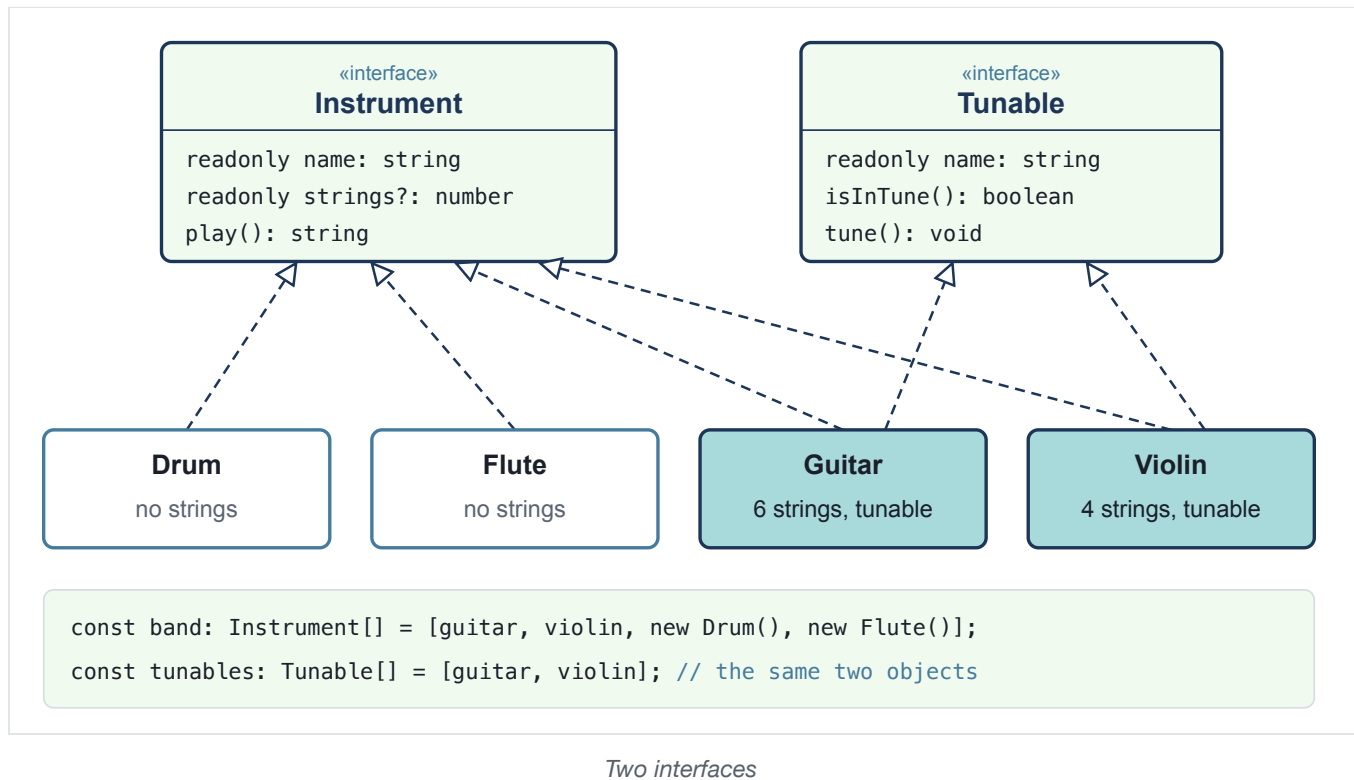
  // A new guitar arrives out of tune unless we say otherwise (a default parameter, Chapter 5).
  constructor(private inTune: boolean = false) {}

  public play(): string {
    return this.inTune ? "Strum!" : "Strum! (out of tune)";
  }

  public isInTune(): boolean {
    return this.inTune;
  }

  public tune(): void {
    this.inTune = true;
  }
}
```

Both interfaces ask for `name` ; one field keeps both promises.



The page puts the guitar and the violin into **two** arrays:

src/main.ts

```
const guitar = new Guitar();
const violin = new Violin();
const band: Instrument[] = [guitar, violin, new Drum(), new Flute()];
const tunables: Tunable[] = [guitar, violin];
```

As Chapter 9 showed, arrays hold references, so there is still only one guitar. `band` sees it as an `Instrument` (it can be played), `tunables` sees it as a `Tunable` (it can be tuned). "Tune all" works on `tunables` ; "Play all" on `band` ; and because they share the objects, playing after tuning gives a clean "Strum!". A test says exactly that:

tests/Guitar.test.ts

```
Deno.test("the same guitar can be used as an Instrument and as a Tunable", () => {
  const guitar = new Guitar();
  const instrument: Instrument = guitar;
  const tunable: Tunable = guitar;

  tunable.tune();

  // one object, two views of it: tuning through one is seen through the other
  assertEquals(instrument.play(), "Strum!");
});
```

Each interface is a **view** of the object: through `instrument` you can only `play` (and read `name` and `strings`); through `tunable` you can only `tune` and `isInTune` . Code that is given a `Tunable[]` , like

`tuneAll`, cannot play anything, even by accident. Small interfaces like these - one job each - are easier to implement, to fake, and to understand than one big one.

Exercise 10.2 - A fake Tunable

`outOfTune(tunables)` returns the names of everything out of tune. Write a test for it that uses no real instruments at all. Try it before reading on.

Here is one way: a small function that builds fake `Tunable`s, each remembering its own state in a closure:

`tests/band.test.ts`

```
/** A fake Tunable that starts in or out of tune. */
const fakeTunable = (name: string, inTune: boolean): Tunable => {
  let tuned = inTune;
  return {
    name,
    isInTune: () => tuned,
    tune: () => {
      tuned = true;
    },
  };
};

Deno.test("out of tune lists only the instruments that are out of tune", () => {
  const tunables = [fakeTunable("Cello", false), fakeTunable("Lute", true)];
  assertEquals(outOfTune(tunables), ["Cello"]);
});
```

`{ name, ... }` is short for `{ name: name, ... }`: when a property has the same name as the variable holding its value, you can write it once.

readonly

`readonly` on an interface member means: *through this interface*, you can read it but not assign it.

```
const shape: Shape = new Circle(0, 0, 1, "red");
shape.colour = "blue";
```

TS2540 [ERROR]: Cannot assign to 'colour' because it is a read-only property.

Note the words "through this interface". It is a promise about the *view*, not about the object. A class may keep `name` as a plain, changeable `public name: string`, and still implement `Instrument`: code holding the object as its own class can change it, while code holding it as an `Instrument` cannot. If you want a field that nobody can change, mark it `readonly` in the class too, as every class in this chapter does (Chapter 6 has the details).

type or interface?

Chapter 3 used `type` to name object shapes:

```
export type Student = { name: string; mark: number };
```

and this chapter uses `interface`. For describing an object, they are nearly interchangeable. This compiles, and a class can implement it just the same:

```
type InstrumentType = {
  readonly name: string;
  readonly strings?: number;
  play(): string;
};

export class Bell implements InstrumentType { ... }
```

The differences that matter at this stage:

	interface	type
an object shape	yes	yes
a class can <code>implements</code> it	yes	yes (if it is an object shape)
build on another	<code>interface Loud extends Instrument { ... }</code>	<code>type Loud = Instrument & { ... }</code>
a union, such as <code>"red" "amber" "green"</code> (Chapter 8)	no	yes
a function type, such as <code>(value: number) => number</code>	possible, but awkward	yes
a name for any other type: <code>type Id = string</code>	no	yes

An interface can `extend` one or more others (as in Java, interfaces may extend interfaces), and can extend an object `type` too.

This book's rule, which is the common one in TypeScript code:

- use **interface** for the shape of an object, and anything a class will implement
- use **type** for everything else: unions, function types, and short names for other types

Interfaces vanish at run time

Here is the start of the shapes project's `dist/app.js` - the JavaScript the browser actually runs:

```
// src/Circle.ts
var FULL_TURN = 2 * Math.PI;
var Circle = class {
  x;
  y;
  radius;
  colour;
  name;
  constructor(x, y, radius, colour) {
```

Look for `implements Shape` : it is gone. Look for `src/Shape.ts` or `src/Pen.ts` : there is nothing

- not even a comment. All the types (`: number`, `: Pen`, `private`, `readonly`) were removed when the TypeScript was turned into JavaScript. Interfaces are checked by the compiler, and then thrown away. That is why we import them with `import type` : there is nothing to import at run time.

In Java, an interface is a real thing at run time, so you can ask `if (thing instanceof Shape)`. In TypeScript you cannot, because there is no `Shape` left to ask about:

```
export const isShape = (value: object): boolean => value instanceof Shape;
```

```
TS2693 [ERROR]: 'Shape' only refers to a type, but is being used as a value here.
```

(`instanceof Circle` does work: classes are real at run time.) Most of the time you do not need to ask - keeping the guitar in a `Tunable[]` as well as in the band means the program always knows which objects can be tuned. When you really do need to find out at run time what something is, Book 2's chapter on **narrowing** shows the tools.

Project 3: Sort anything

Sort Anything

☒ Sorted

Songs shortest first

1. Song 2 - Blur (2:02)
2. Hey Ya! - OutKast (3:55)
3. Yellow - Coldplay (4:29)
4. Zombie - The Cranberries (5:06)
5. Bohemian Rhapsody - Queen (5:54)

Planets nearest the Sun first

1. Mercury (58 million km)
2. Venus (108 million km)
3. Earth (150 million km)
4. Mars (228 million km)
5. Jupiter (778 million km)

Mountains tallest first

1. Everest (8849 m)
2. Kilimanjaro (5895 m)
3. Mont Blanc (4806 m)
4. Ben Nevis (1345 m)
5. Carrauntoohil (1039 m)

The Sort Anything project

Java has a famous interface for putting things in order: `Comparable`, with its one method `compareTo`. A class that implements it decides its own "natural order", and `Collections.sort` can then sort a list of

them. This project builds a small TypeScript interface in the same spirit:

src/Sortable.ts

```
// Anything that can be put in order and shown in a list. Like Java's Comparable, each class
// decides its own order – here by giving a number to sort by (smaller comes first).

export interface Sortable {
  /** The text to show in a list. */
  label(): string;
  /** Where it goes in the order: smaller numbers come first. */
  sortKey(): number;
}
```

Three classes that have nothing else in common implement it. A `Song` sorts by its length; a `Planet` by its distance from the Sun; a `Mountain` tallest first:

src/Mountain.ts

```
export class Mountain implements Sortable {
  constructor(
    public readonly name: string,
    public readonly metres: number,
  ) {}

  /** "Everest (8849 m)" */
  public label(): string {
    return `${this.name} (${this.metres} m)`;
  }

  public sortKey(): number {
    return -this.metres;
  }
}
```

Minus the height gives the tallest mountain the smallest key, so it comes first. The class decides its own order; the sorting code does not need to know.

sortAll, test first

The function to write is `sortAll(items: Sortable[]): Sortable[]`. What should the tests use? Not songs or planets - the function must work for *any* `Sortable`, so the tests use the simplest `Sortable` possible: a label and a key, as an object literal.

tests/sorting.test.ts

```
/** A fake Sortable with a label and a key. */
const item = (label: string, key: number): Sortable => ({
  label: () => label,
  sortKey: () => key,
});

Deno.test("sortAll puts the smallest key first", () => {
  const items = [item("c", 3), item("a", 1), item("b", 2)];
  assertEquals(labels(sortAll(items)), ["a", "b", "c"]);
});
```

The labels make the expected order obvious at a glance. `labels` is a one-liner, `items.map((item) => item.label())`. To see the test fail for the right reason, `sortAll` starts by giving back what it was given:

```
export const sortAll = (items: Sortable[]): Sortable[] => items;
```

Red, and the diff shows the order is wrong:

```
AssertionError: Values are not equal.
```

```
[Diff] Actual / Expected
[
-   "c",
-   "a",
-   "b",
+   "c",
]
```

Green, with a comparator (Chapter 3) that compares keys:

src/sorting.ts

```
/** A sorted copy: smallest sort key first. The original array is left as it was. */
export const sortAll = (items: Sortable[]): Sortable[] => items.toSorted((a, b) => a.sortKey() - b.sortKey());
```

More tests pin down the edges: negative keys (mountains!), an empty array, and that the original array is not changed. Then the real classes get a test each, to check the order each one chose:

tests/classes.test.ts

```
Deno.test("mountains sort tallest first", () => {
  const mountains = [new Mountain("Ben Nevis", 1345), new Mountain("Everest", 8849)];
  assertEquals(labels(sortAll(mountains)), ["Everest (8849 m)", "Ben Nevis (1345 m)"]);
});
```

One function, three classes

The page loads its data from a JSON file, makes objects with `map` (Chapter 5), and shows each list with the same function:

src/main.ts

```
const songs: Song[] = data.songs.map((song) => new Song(song.title, song.artist, song.seconds));
const planets: Planet[] = data.planets.map((planet) => new Planet(planet.name, planet.millionKm));
const mountains: Mountain[] = data.mountains.map((mountain) => new Mountain(mountain.name, mountain.metres));
```



```
const render = (): void => {
  // A Song[] can be passed where a Sortable[] is wanted: every Song is a Sortable.
  showList("#songs", songs);
  showList("#planets", planets);
  showList("#mountains", mountains);
};
```

`showList` and `sortAll` were written once, and work for all three - and for any class you add, as long as it has a `label()` and a `sortKey()`.

Note - `sortAll` gives back `Sortable[]`, not `Song[]`: once a song has gone through it, the compiler only knows it is *something sortable*, so you could call `label()` on it but not read its `artist`. Java's `Comparable<T>` avoids this with a type parameter, the `<T>`. TypeScript has the same idea - **generics** - and Book 2 has a chapter on them. Here it does not matter: the page only needs the labels.

Exercise 10.3 - Where does it go?

Which way round do planets sort if `Planet.sortKey()` returns `-this.millionKm`? Which test fails?

Here is the answer: furthest first, Jupiter at the top. "planets sort nearest the Sun first" goes red, with Mars and Earth the wrong way round in the diff. `sortAll` is untouched, and its own tests still pass: they use fake items, so a change in `Planet` cannot break them. That is a good sign - each test checks one thing.

Java and TypeScript

Java	TypeScript
<code>interface Shape { double area(); }</code>	<code>interface Shape { area(): number; }</code>
interfaces hold methods and constants	interfaces can also hold properties: <code>readonly name: string;</code>
<code>class Circle implements Shape, Drawable</code>	<code>class Circle implements Shape, Drawable</code>
<code>interface B extends A</code>	<code>interface B extends A</code>
nominal: a class must say <code>implements Shape</code>	structural: anything with the right members is a <code>Shape</code>
a fake needs a class (or Mockito)	a fake can be an object literal: <code>{ area: () => 10, ... }</code>
no optional methods (only <code>default</code> ones)	optional members: <code>strings?: number</code>
<code>obj instanceof Shape</code> works	interfaces vanish at run time - <code>instanceof Shape</code> is an error
<code>Comparable<T></code> with <code>compareTo</code>	your own interface (<code>Sortable</code>), or a comparator function

Summary

- an `interface` lists the members an object must have: methods, and properties too, all public

- `implements` makes a class keep the contract; a missing member is a compile error on the class, which makes a good first red when working test first
- TypeScript's typing is **structural**: anything with the right members fits, whatever it is called and whether or not it says `implements` - even the browser's canvas context fits our `Pen`
- still write `implements` on your own classes, so mistakes are reported where they are made
- an object literal is the quickest fake in a test; a fresh literal may not have extra members
- a class can implement several interfaces; one object can be held through each of them
- `strings?: number` is optional, and its type is `number | undefined`; `readonly` stops assignment *through that type*
- use `interface` for object shapes and `type` for unions, function types and other names
- interfaces exist only for the compiler: they vanish from the JavaScript, so `instanceof Shape` cannot work

Challenges

Each challenge says which project to start from. Write the tests first.

1. Square

Start from `ch10_project01_shapes`. Add a `Square` class that implements `Shape`, with a centre and one side length, and an "Add square" button. Write the tests first: a square of side 5 has an area of 25 and a perimeter of 20, and a test with the recording pen checks where it is drawn. Watch the compiler's "incorrectly implements" message shrink as you add each member.

2. Cities

Start from `ch10_project03_sort_anything`. Add a `City` class that is `Sortable`, sorted by population, **biggest first**, and a fourth list on the page with five cities from the JSON file. Test first: the label (for example "Dublin (592,713)" - look up `toLocaleString`) and the order. Did `sortAll` need to change?

3. The biggest shape

Start from `ch10_project01_shapes`. Show "Largest: Circle (area 6939.8)" under the table. Write a function `largestShape(shapes: Shape[]): Shape | undefined` in `src/shapes.ts`, tested with fake shapes made from object literals only - no real `Circle`s. Remember the empty array.

4. Things that make a sound

Start from `ch10_project02_instruments`. The Java course's `SoundMaker` idea: a doorbell makes a sound, but it is not an instrument. Make a `SoundMaker` interface with `name` and `play()`, change `Instrument` so it `extends SoundMaker`, and add a `Doorbell` class that is a `SoundMaker` only. Add a "Sound check" button that plays the band *and* the doorbell, using a function `soundCheck(makers: SoundMaker[]): string[]` written test first.

Hint: once `Instrument` `extends SoundMaker`, it only needs to list `strings?`. Can you pass the `band` array (an `Instrument[]`) to a function that wants a `SoundMaker[]`? Why?

5. Ties

Start from `ch10_project03_sort_anything`. When two items have the same key, `sortAll` leaves them in whatever order they came. Change it so that ties are sorted alphabetically by label. Write the tests first, with fake items: `item("b", 1)` and `item("a", 1)` should come out "a", "b"; and items with different keys must still sort by key, whatever their labels.

Hint: in the comparator, if the keys are different, return their difference as now; if they are equal, return `a.label().localeCompare(b.label())`.

6. Click to choose

Start from `ch10_project01_shapes`. Click on a shape on the canvas to select it: show its details under the canvas ("Triangle: area 3600.0, perimeter 273.6"). Add a method `contains(x: number, y: number): boolean` to the `Shape` interface - and let the compiler list every class that now breaks. Test `contains` first for each class, inside and outside, including a point just outside each edge.

Hint: a circle contains a point if its distance from the centre (`Math.hypot`) is at most the radius. A rectangle checks both coordinates against its edges. For the triangle, checking its bounding rectangle is a fair first version; a better one checks the point is above both sloping sides. In `main.ts`, listen for `click` on the canvas; `event.offsetX` and `event.offsetY` give the position in the canvas - but the canvas is shown at a different size from its 720 by 240 pixels, so scale them by `canvas.width / canvas.clientWidth`. Use `find` to get the clicked shape, and search from the end of the array if you want the shape drawn on top to win.

Next: [Chapter 11 - Inheritance](#)

Chapter 11 - Inheritance

A cat is an animal. A student is a person. A savings account is a bank account. When one kind of thing is a special case of another, inheritance lets you write what they share once, in a **superclass**, and only what is different in each **subclass**. You know the idea from Java, and TypeScript's version will look very familiar: `extends`, `super`, `protected`, `abstract`. This chapter shows where TypeScript is the same, where it differs (`override` is a keyword, and there is no `final`), and how one set of tests can check every subclass at once.

Animals

Add a cat Add a dog Everybody speak

Tom says Meow · Rex says Woof · Felix says Meow · Bella says Woof

Tom, a cat

Speak

Rex, a dog

Speak

Felix, a cat

Speak

Bella, a dog

Speak

The Animals project

What you will learn

- how to make a subclass with `extends`, and what it inherits
- `abstract` classes and methods: a class you cannot make objects of, and methods every subclass must write
- `override`, and how the compiler (with `noImplicitOverride`) catches overriding mistakes
- `protected`: members a subclass may use but the outside world may not
- calling the superclass's constructor with `super(...)`, and its methods with `super.method()`
- overriding `toString()` so that each subclass describes itself
- what to do in TypeScript instead of Java's `final`
- writing **one test suite as a function**, and running it against every subclass

The projects

Project	What it shows
ch11_project01_animals	an abstract <code>Animal</code> , <code>Cat</code> and <code>Dog</code> that extend it, <code>override</code> , <code>protected</code> , and a mixed <code>Animal[]</code>
ch11_project02_staff	<code>Person</code> , <code>Student</code> and <code>Lecturer</code> from a JSON file: <code>super(...)</code> in constructors, <code>super.toString()</code> , a check inherited by every subclass
ch11_project03_bank_accounts	an abstract <code>BankAccount</code> holding the rules, <code>SavingsAccount</code> and <code>CurrentAccount</code> , an overdraft built test first, and one test suite run against both

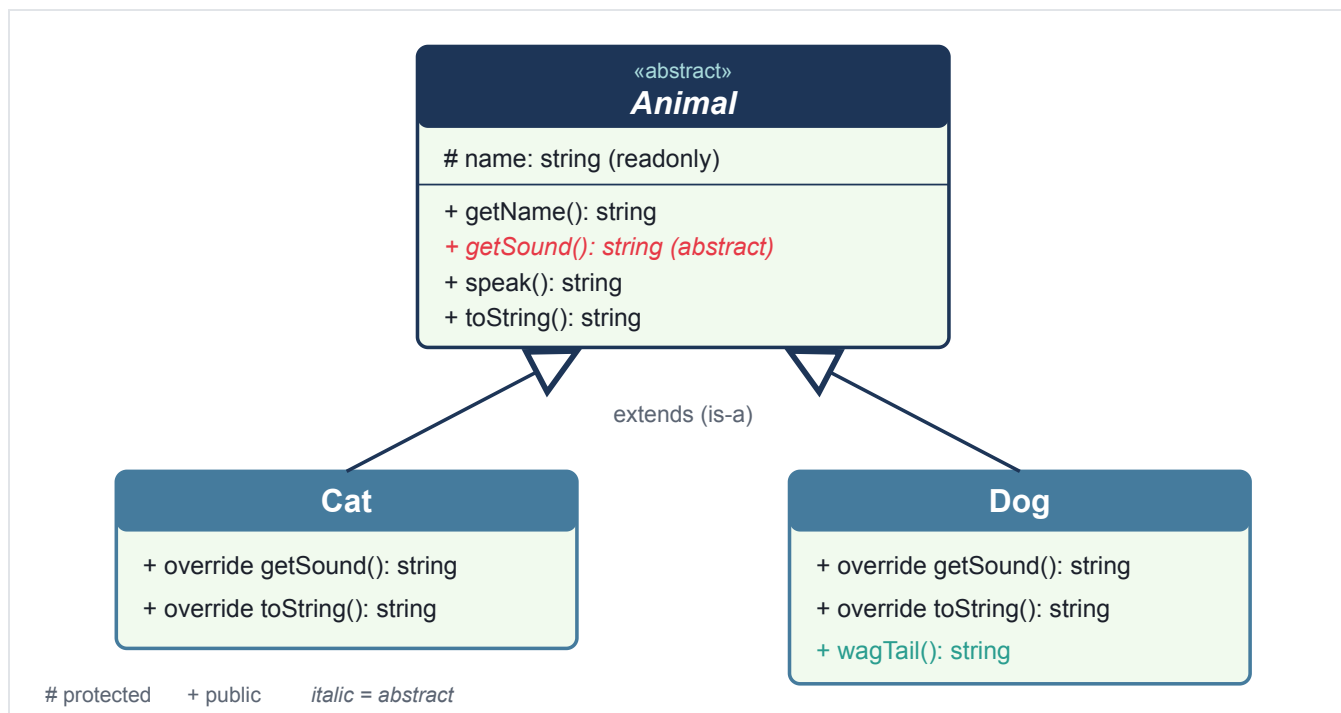
Is-a, and why inherit

In Java you met two ways for classes to be related. **Composition** - a playlist *has* songs, a course *has* modules - was Chapter 9's subject. **Inheritance** is for the other relationship, **is-a**: a cat *is an* animal, so anything that is true of every animal is true of a cat.

Without inheritance, `Cat` and `Dog` would each need their own `name` field, their own `getName()`, their own `speak()` - the same lines, twice, and any fix made in two places. Inheritance is one way to keep to the rule "don't repeat yourself": put what they share in `Animal`, and let `Cat` and `Dog` **inherit** it.

A quick test for whether inheritance fits: say "a ___ is a ___" out loud. "A dog is an animal" - yes. "A student is a person" - yes. "A car is an engine" - no: a car *has* an engine, so that is composition.

Project 1: Animals



Animal, Cat and Dog

The Java inheritance chapters built this up in eleven small projects, `cat1` to `cat11`. You know the ideas already, so here is the whole journey at once, with the TypeScript for each step:

Java project	The step	In TypeScript
<code>cat1</code>	one class, <code>Cat</code>	<code>class Cat { ... }</code>
<code>cat2</code>	a superclass <code>Animal</code> ; <code>Cat</code> extends <code>Animal</code>	<code>class Cat extends Animal { }</code> - the same
<code>cat3</code>	<code>Cat</code> overrides <code>getSound()</code> , with <code>@Override</code>	<code>public override getSound()</code> - a keyword, and required
<code>cat4</code> , <code>cat5</code>	an <code>Animal</code> variable can hold a <code>Cat</code>	<code>const pet: Animal = new Cat("Tom");</code> - the same
<code>cat6</code>	override <code>toString()</code>	<code>public override toString()</code> , used by <code>\${pet}</code>
<code>cat7</code>	abstract class <code>Animal</code>	<code>export abstract class Animal</code> - the same
<code>cat8</code> , <code>cat9</code>	<code>final</code> classes and methods	no <code>final</code> - see later in the chapter
<code>cat10</code>	abstract method <code>getSound()</code>	<code>public abstract getSound(): string;</code> - the same
<code>cat11</code>	cats and dogs together	<code>const animals: Animal[] = [new Cat("Tom"), new Dog("Rex")];</code>

Here is where the journey ends up.

An abstract superclass

`src/Animal.ts`

```

export abstract class Animal {
  // `protected`: this class and its subclasses can read `name`; code outside cannot.
  // `readonly`: an animal keeps its name.
  constructor(protected readonly name: string) {}

  public getName(): string {
    return this.name;
  }

  /** Every animal makes a sound, but each kind makes its own – so there is no body here. */
  public abstract getSound(): string;

  /** Written once, for every animal. It calls getSound(), and the subclass's version runs. */
  public speak(): string {
    return `${this.name} says ${this.getSound()}`;
  }

  /** A subclass can override this to say what kind of animal it is. */
  public toString(): string {
    return `${this.name}, an animal`;
  }
}

```

Three things to notice:

- `abstract class` means there is no such thing as "just an animal": you can make cats and dogs, but not `new Animal(...)`
- `getSound()` is an **abstract method**: it has a signature but no body, ending in `;`. Every (non-abstract) subclass must write it
- `speak()` is written once, here, and it *calls* `getSound()`. When a cat speaks, `this` is the cat, so the cat's `getSound()` runs. `Animal` does not know or care which subclass it is

The constructor uses a **parameter property** (Chapter 5): `protected readonly name: string` declares the field and sets it in one go.

If you try to make an `Animal`, the compiler stops you:

```
TS2511 [ERROR]: Cannot create an instance of an abstract class.
```

A subclass

`src/Cat.ts`

```
import { Animal } from "./Animal.ts";

export class Cat extends Animal {
  // `override`: this replaces a method of Animal. The compiler checks that Animal has one.
  public override getSound(): string {
    return "Meow";
  }

  public override toString(): string {
    // `name` is protected in Animal, so a subclass may use it.
    return `${this.name}, a cat`;
  }
}
```

`extends Animal` works exactly as in Java. `Cat` inherits `name`, `getName()`, `speak()` - everything `Animal` has. It has no constructor of its own, so it uses `Animal`'s: `new Cat("Tom")` runs `Animal`'s constructor with `"Tom"`. (In Java, constructors are never inherited; a Java `Cat` would need its own constructor calling `super(name)`. In TypeScript, a subclass with no constructor gets its superclass's, parameters and all.)

If `Cat` forgets `getSound()`, it is an error - the compiler will not let a class leave a promise unkept:

```
TS2515 [ERROR]: Non-abstract class 'Cow' does not implement inherited abstract member getSound from class 'Animal'.
```

(That message came from a `Cow` with no methods - Challenge 1 asks you to write a proper one.)

override

In Java, `@Override` is an annotation you may leave out. In TypeScript, `override` is a keyword, and these projects make it **compulsory**: every project's `deno.json` switches on the compiler option `noImplicitOverride`:

`deno.json`

```
"compilerOptions": {
  "lib": ["dom", "dom.iterable", "esnext", "deno.ns"],
  "noImplicitOverride": true
},
```

With it on, replacing a superclass's method *without* saying `override` is an error. Leave the word out of a `Cow`'s `toString()`, and:

```
TS4114 [ERROR]: This member must have an 'override' modifier because it overrides a member in the base class 'Animal'.
```

Why insist? Because it stops two kinds of accident. The first: you add a method to a subclass, not knowing the superclass already has one with that name, and silently replace it. The second is a typo. Write `override` `getSond()` and the compiler spots that there is nothing to override - and even guesses what you meant:

```
TS4117 [ERROR]: This member cannot have an 'override' modifier because it is not declared in the base class 'Animal'. Did you mean 'getSound'?
```


Java's `@Override` catches the typo too, but only if you remember to write it.

Note - Strictly, the compiler only *requires* `override` when a method replaces one that has a body. Writing an abstract method's body, like `getSound()` in `Cat`, is allowed without it. This book writes `override` on every one, so that a reader can see at a glance which methods come from the superclass.

Adding something new

A subclass can also add methods its superclass does not have:

src/Dog.ts

```
export class Dog extends Animal {
  public override getSound(): string {
    return "Woof";
  }

  public override toString(): string {
    return `${this.name}, a dog`;
  }

  /** Only dogs have this method. A variable of type Animal cannot call it. */
  public wagTail(): string {
    return `${this.name} wags their tail`;
  }
}
```

A cat is an animal

Because a cat is an animal, a cat can go anywhere an `Animal` is expected - in an `Animal` variable, an `Animal[]` array, or an `Animal` parameter. The page keeps all its animals in one array:

src/main.ts

```
// One array holds cats and dogs together, because both are Animals.
const animals: Animal[] = [new Cat("Tom"), new Dog("Rex"), new Cat("Felix")];
```

and `chorus` works with any animals at all:

src/farm.ts

```
/** Every animal speaks, in order. Each one uses its own getSound(). */
export const chorus = (animals: Animal[]): string[] => animals.map((animal) => animal.speak());
```

`chorus` only knows about `Animal`, yet each animal says its own sound. The method that runs is decided by the *object*, not by the type of the variable. Java works the same way, and Book 2 begins with this idea - it is called **polymorphism**.

There is a price. Through an `Animal` variable you can only use what *every* animal has:

```
const pet: Animal = new Dog("Rex");
pet.wagTail();
```

```
TS2339 [ERROR]: Property 'wagTail' does not exist on type 'Animal'.
```

The object really is a `Dog`, but the variable only promises an `Animal`, and not every animal has a tail to wag.

Classes, unlike Chapter 10's interfaces, still exist when the program runs, so `instanceof` works with them, as in Java:

tests/animals.test.ts

```
Deno.test("a cat is an Animal, and instanceof can tell", () => {
  const tom = new Cat("Tom");
  assert(tom instanceof Cat);
  assert(tom instanceof Animal);
  assertEquals(tom instanceof Dog, false);
});
```

(`assert` passes if what it is given is `true`.)

toString()

`Animal`, `Cat` and `Dog` each have a `toString()`. A template literal calls it for you: `${animal}` gives "Tom, a cat" or "Rex, a dog" - the subclass's version, even though the array is an `Animal[]`. That is how the page labels each card:

src/main.ts

```
// `${animal}` calls animal.toString() - the Cat or Dog version.
label.textContent = `${animal}`;
```

Without any `toString()` at all, an object turns into the unhelpful `[object Object]` - JavaScript's version of Java's `Animal@1b6d3586`.

protected

`Cat.toString()` uses `this.name`, which belongs to `Animal`. That works because `name` is **protected**: visible in the class and its subclasses, hidden from everyone else. From outside:

```
const tom = new Cat("Tom");
console.log(tom.name);
```

```
TS2445 [ERROR]: Property 'name' is protected and only accessible within class 'Animal' and its subclasses.
```

Modifier	The class itself	Its subclasses	Everyone else
private	yes	no	no
protected	yes	yes	no
public	yes	yes	yes

One difference from Java: Java's `protected` also opens a member to every class in the same package. TypeScript has no packages, so `protected` means exactly "this class and its subclasses". (And, as in Chapter 6, these checks are made by the compiler; `#private` fields are the ones that are private even at run time. A `#name` field cannot be seen by subclasses at all, so `protected` has no `#` version.)

Use `protected` sparingly. Every protected member is something subclasses may come to depend on, so it is harder to change later. Start `private`, and open a member up only when a subclass really needs it.

Exercise 11.1 - Which lines compile?

```
const pet: Animal = new Dog("Rex");
pet.speak();           // 1
pet.wagTail();         // 2
pet.name;              // 3
new Animal("Thing");   // 4
```

Decide for each line, then read on.

Only line 1 compiles: every `Animal` can speak, and the dog's own sound is used ("Rex says Woof"). Line 2 is TS2339 - `wagTail` is not on `Animal`. Line 3 is TS2445 - `name` is protected. Line 4 is TS2511 - `Animal` is abstract.

Project 2: Staff and students

Staff and Students

Everybody

Students

Lecturers

Showing 6 of 6 people

Student Aoife Byrne <aoife.byrne@student.college.ie>, student S1001, Computing

Lecturer Dr Ciara Kelly <ciara.kelly@college.ie>, lecturer in Computing, room A214

Lecturer Dr Mark Ryan <mark.ryan@college.ie>, lecturer in Mathematics, room B105

Student Jack Doyle <jack.doyle@student.college.ie>, student S1004, Data Science

Student Priya Nair <priya.nair@student.college.ie>, student S1003, Computing

Student Tomás Walsh <tomas.walsh@student.college.ie>, student S1002, Games Design

The Staff and Students project

A college directory lists students and lecturers. Both have a name and an email address; a student also has a student number and a course, and a lecturer a department and an office.

This time the superclass is **not** abstract. A visitor to the college is just a `Person`, so it makes sense to create one:

```
src/Person.ts
```

```

/** The roles the directory knows about (a string literal union, as in Chapter 8). */
export type Role = "Visitor" | "Student" | "Lecturer";

export class Person {
  // Parameter properties (Chapter 5). `protected`, so Student and Lecturer can use them too.
  constructor(protected readonly name: string, protected readonly email: string) {
    // Every Person checks this, so every Student and Lecturer gets the check for free.
    if (!email.includes("@")) {
      throw new Error(`Not an email address: ${email}`);
    }
  }

  public getName(): string {
    return this.name;
  }

  public getEmail(): string {
    return this.email;
  }

  public getRole(): Role {
    return "Visitor";
  }

  public toString(): string {
    return `${this.name} <${this.email}>`;
  }
}

```

super(...) - calling the superclass's constructor

`Student` needs more than a name and an email, so it needs its own constructor - and a subclass's constructor must call the superclass's constructor, with `super(...)`:

src/Student.ts

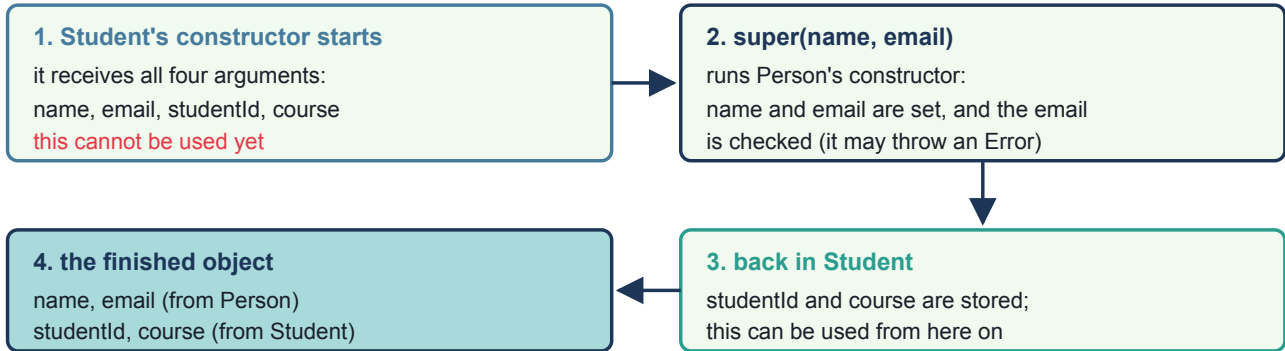
```

export class Student extends Person {
  // name and email are plain parameters, handed on to Person's constructor.
  // studentId and course are new parameter properties, belonging to Student.
  constructor(name: string, email: string, private readonly studentId: string, private readonly
course: string) {
    // A subclass constructor must call super(...) - Person's constructor - before it uses `this`.
    super(name, email);
  }
}

```

Look at the parameters carefully. `name` and `email` have no modifier: they are ordinary parameters, only passed on. `Person` already stores them. `studentId` and `course` are written as parameter properties, so `Student` stores those. (Writing `protected readonly name` again here would declare a second `name` - leave it to `Person`.)

`new Student("Ann", "ann@college.ie", "S1", "Computing")`



`toString()`: `super.toString()` gives "Ann <ann@college.ie>"
and Student adds " , student S1, Computing"

What happens in `new Student(...)`

The rules are Java's, and the compiler enforces them. Leave out `super(...)` :

```
TS2377 [ERROR]: Constructors for derived classes must contain a 'super' call.
```

Use `this` before calling it:

```
TS17009 [ERROR]: 'super' must be called before accessing 'this' in the constructor of a derived class.
```

Pass the wrong arguments, say `super(name)` :

```
TS2554 [ERROR]: Expected 2 arguments, but got 1.
```

("Derived class" and "base class", in these messages, are other names for subclass and superclass.)

The reason for the order: until `Person`'s constructor has run, the object is not a proper `Person` yet

- it has no name, and its email has not been checked. `super(...)` first means a subclass can always rely on the superclass's part being ready.

Inherited rules

`Person`'s constructor checks the email. Every `Student` and `Lecturer` goes through that constructor, so they all get the check, without a line of extra code. A test makes sure:

`tests/people.test.ts`

```
Denotest("Person's email check protects every subclass too", () => {
  assertThrows(() => new Person("Val", "nope"), Error, "Not an email address: nope");
  assertThrows(() => new Student("Ann", "ann.college.ie", "S1", "Computing"), Error, "Not an email address");
  assertThrows(() => new Lecturer("Bob", "bob", "Maths", "B105"), Error, "Not an email address");
});
```

`super.toString()` - building on the superclass's method

A student's `toString()` should say everything a person's does, and more. Rather than repeat `Person`'s code, it calls it with `super.toString()`:

`src/Student.ts`

```
public override toString(): string {
  // super.toString() runs Person's version: "Ann <ann@college.ie>". Student adds to it.
  return `${super.toString()}, student ${this.studentId}, ${this.course}`;
}
```

`super.method()` means "the superclass's version of this method" - the same as in Java. If `Person` later shows email addresses differently, students and lecturers change with it.

`Lecturer` is built the same way, and adds a method of its own that uses the protected `name`:

`src/Lecturer.ts`

```
/** How a lecturer signs an email. `name` is protected in Person, so Lecturer can read it. */
public signature(): string {
  return `${this.name}, Department of ${this.department}`;
}
```

From JSON to objects

`people.json` keeps the students and the lecturers in two lists. `peopleFrom` turns each list into objects with `map` (Chapter 5) and joins them with the spread operator (Chapter 9):

`src/directory.ts`

```
/** Everybody, as objects. The result is a Person[], holding Students and Lecturers together. */
export const peopleFrom = (data: DirectoryData): Person[] => {
  const students = data.students.map((s) => new Student(s.name, s.email, s.studentId, s.course));
  const lecturers = data.lecturers.map((l) => new Lecturer(l.name, l.email, l.department, l.office));
  return [...students, ...lecturers];
};

/** Only the people with this role. */
export const withRole = (people: Person[], role: Role): Person[] =>
  people.filter((person) => person.getRole() === role);
```

Each subclass overrides `getRole()` to return `"Student"` or `"Lecturer"`, so the filter buttons can ask any `Person` what it is. Because `getRole()` returns the union `Role`, a typo such as `withRole(people, "Lecturor")` is a type error.

Exercise 11.2 - Initials for everybody

Give every person a method `getInitials(): "Ann Lee" gives "AL"`. Write the test first. Where does the method go, and what do you test it on?

Here is one way. The method goes in `Person`, so `Student` and `Lecturer` inherit it:

```
public getInitials(): string {
  return this.name.split(" ").map((word) => word[0]).join("");
}
```

and the test uses a subclass, to show that the method really is inherited:

```
Deno.test("a student inherits getInitials from Person", () => {
  assertEquals(new Student("Ann Lee", "ann@college.ie", "S1", "Computing").getInitials(), "AL");
});
```

(`split(" ")` cuts a string into an array of words.)

Project 3: Bank accounts

Bank Accounts

Not enough money: €200.00 available

Savings account for Aoife: €200.00

Available: €200.00

Current account for Aoife: -€70.00

Available: €30.00

The Bank Accounts project

Every bank account takes deposits and allows withdrawals, and both must be more than zero. What differs is **how much you may take out**: a savings account only what is in it, a current account that and an overdraft as well. So the shared rules go in an abstract class, and each subclass answers one question.

`src/BankAccount.ts`


```

export abstract class BankAccount {
    // protected: subclasses need the balance (to work out what is available, or to add interest),
    // but code outside must go through deposit and withdraw, which check the rules.
    protected balance: number = 0;

    constructor(private readonly owner: string) {}

    public getOwner(): string {
        return this.owner;
    }

    public getBalance(): number {
        return this.balance;
    }

    public deposit(amount: number): void {
        this.requirePositive(amount);
        this.balance += amount;
    }

    /**
     * The same for every account: the amount must be positive, and no more than available().
     * Subclasses are not meant to override this – TypeScript has no `final` to stop them, so the
     * shared tests in tests/account_rules.ts check that every subclass keeps these rules.
     */
    public withdraw(amount: number): void {
        this.requirePositive(amount);
        if (amount > this.available()) {
            throw new Error(`Not enough money: ${formatEuro(this.available())} available`);
        }
        this.balance -= amount;
    }

    /** How much may be withdrawn now. Each kind of account has its own rule. */
    public abstract available(): number;

    /** "Savings account", "Current account" ... used by toString. */
    public abstract getKind(): string;
    // ... toString(), and a private requirePositive(amount)
}

```

`withdraw` is written once and calls the abstract `available()` - the same shape as `speak()` calling `getSound()`. (Book 3 gives this shape a name: the *Template Method* pattern.)

The fields are chosen with care. `balance` is `protected`, because subclasses need it. `owner` is `private`: no subclass needs to change it, and a `SavingsAccount` that tries to read `this.owner` is told

```
TS2341 [ERROR]: Property 'owner' is private and only accessible within class 'BankAccount'.
```

- it must use the public `getOwner()`, like everyone else.

A subclass with a default parameter

`src/SavingsAccount.ts`

```

const DEFAULT_INTEREST_RATE = 2; // per cent

export class SavingsAccount extends BankAccount {
  constructor(owner: string, private readonly interestRate: number = DEFAULT_INTEREST_RATE) {
    super(owner);
  }

  public override available(): number {
    return this.balance;
  }

  public override getKind(): string {
    return "Savings account";
  }
  // ... getInterestRate()

  /** Adds one year's interest. Uses the protected balance directly – no need for a deposit. */
  public addInterest(): void {
    this.balance = roundToCent(this.balance * (1 + this.interestRate / 100));
  }
}

```

`new SavingsAccount("Aoife")` earns 2%; `new SavingsAccount("Aoife", 5)` earns 5% - a default parameter (Chapter 5) in a subclass constructor, passing only `owner` up to `super`.

The overdraft, test first

`CurrentAccount` starts as the simplest thing that could work: copy the savings rule.

`src/CurrentAccount.ts` (first version)

```

public override available(): number {
  return this.balance;
}

```

Now the first test of what makes a current account different. With €50 in it and a €100 overdraft, taking out €120 should leave it at -€70:

`tests/current.test.ts`

```

Deno.test("a current account can withdraw into its overdraft", () => {
  const account = new CurrentAccount("Ann", 100);
  account.deposit(50);
  account.withdraw(120);
  assertEquals(account.getBalance(), -70);
});

```

Red. This time it is not a diff: `withdraw` throws, and the error ends the test:

```

not ok 10 - a current account can withdraw into its overdraft
---
message: |-
  Error: Not enough money: €50.00 available
    throw new Error(`Not enough money: ${formatEuro(this.available())} available`);
          ^
    at CurrentAccount.withdraw (src/BankAccount.ts:35:13)
    at tests/current.test.ts:12:11
at: tests/current.test.ts:9
...

```

The stack lines say where: the error was thrown in `BankAccount.ts` (by the `withdraw` that `CurrentAccount` inherited), called from line 12 of the test. The message says why: only €50 counts as available. So the fix is in `available()`, not in `withdraw`:

`src/CurrentAccount.ts`

```

/** The balance plus the overdraft: with €50 and a €100 overdraft, €150 may be taken out. */
public override available(): number {
  return this.balance + this.overdraftLimit;
}

```

Green. The next test - "a current account cannot go past its overdraft limit", withdrawing €151 - was green straight away: `withdraw`'s check, written once in `BankAccount`, already did the work. The two tests for `isOverdrawn()` were red until the one-line method was written.

One test suite for every subclass

Some rules are the same for every kind of account: a new account is empty, deposits add, a deposit of zero is rejected, you can never take out more than `available()` says. You could copy those tests into `savings.test.ts` and `current.test.ts` - and copy them again for each new kind of account, and fix every copy when a rule changes.

Instead, write them **once, as a function**:

`tests/account_rules.ts`

```

/** Something that makes a new, empty account for an owner. */
export type MakeAccount = (owner: string) => BankAccount;

export const testAccountRules = (kind: string, makeAccount: MakeAccount): void => {
  Deno.test(`${kind}: a new account is empty`, () => {
    assertEquals(makeAccount("Ann").getBalance(), 0);
  });

  Deno.test(`${kind}: no more than what is available can be withdrawn`, () => {
    const account = makeAccount("Ann");
    account.deposit(100);
    const before = account.getBalance();
    assertThrows(() => account.withdraw(account.available() + 1), Error, "Not enough money");
    assertEquals(account.getBalance(), before);
  });
  // ... seven more rules
};

```

`testAccountRules` does not run any tests itself. When it is *called*, it **registers** nine tests with `Deno.test`, each named after the kind of account. It is given a function, `makeAccount`, that makes a fresh account of whichever kind is being tested - so every test starts from a new, empty account (a helper function instead of a `beforeEach`, as in Chapter 4). The file's name does not end in `.test.ts`, so Deno does not run it on its own; the test files call it:

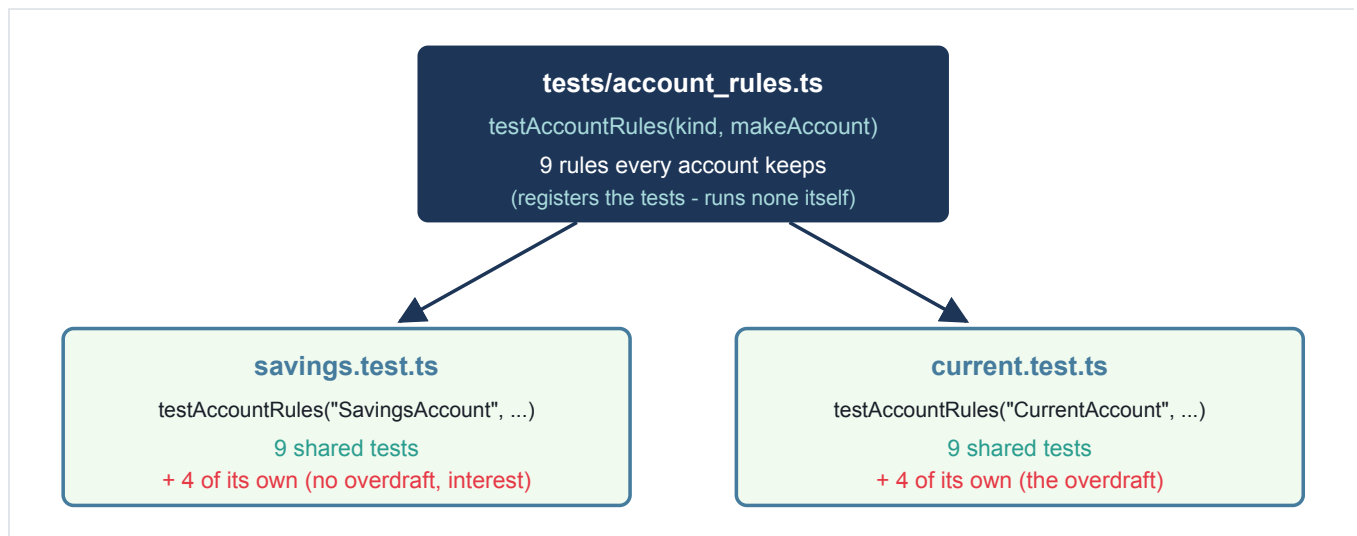
`tests/savings.test.ts`

```
testAccountRules("SavingsAccount", (owner) => new SavingsAccount(owner));
```

`tests/current.test.ts`

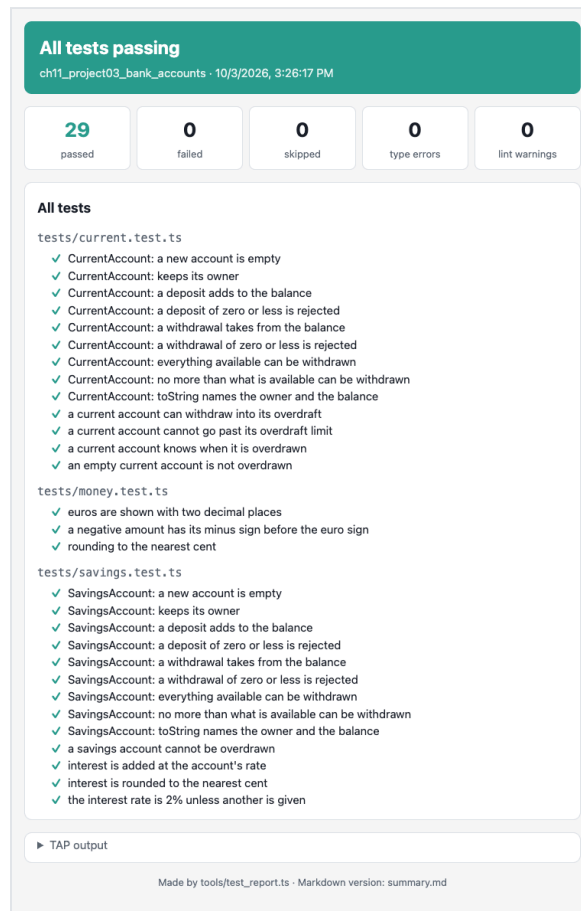
```
testAccountRules("CurrentAccount", (owner) => new CurrentAccount(owner));
```

One line each, and each kind of account gets all nine tests. The rules are written so that they hold for *any* account: "everything available can be withdrawn" asks the account what is available rather than assuming a number.



One rules function, run for each subclass

The report shows the shared tests twice, once for each subclass, followed by each subclass's own:



The test report

This is the test-first answer to an important rule of inheritance: **anywhere the program expects a `BankAccount`, any subclass must work**. (It has a name - the *Liskov substitution principle*, the "L" in SOLID.) A subclass that breaks a shared rule - say, a new account that lets a deposit of €0 through - fails its shared tests the moment `testAccountRules` is called for it.

Errors on the page

`withdraw` throws an `Error` when a rule is broken. The page must not crash; it should show the message. `main.ts` uses `try` and `catch`, which work as in Java:

src/main.ts

```
/** Runs an action on an account; if a rule is broken, shows the error's message instead. */
const attempt = (action: () => void, done: string): void => {
  try {
    action();
    message.textContent = done;
    message.className = "good";
  } catch (error) {
    message.textContent = error instanceof Error ? error.message : "Something went wrong";
    message.className = "bad";
  }
  render();
};
```

One difference: JavaScript can throw *anything*, not only errors, so `catch (error)` cannot say what type error is. `error instanceof Error` checks before `error.message` is used. Book 2's chapter on errors goes further.

`wireCard(prefix, account: BankAccount)` connects a card's Deposit and Withdraw buttons. Because it takes a `BankAccount`, it works for both cards. Only the savings card has an Add interest button, wired to the `savings` variable, whose type is `SavingsAccount`.

No `final` - and what to do instead

Java has `final` for three things, and TypeScript has a different answer for each.

A field that never changes - Java's `final` field - is TypeScript's `readonly` (Chapter 6), as in `protected readonly name`.

A class that nobody may extend - Java's `final class`. TypeScript has no keyword, but a class with a **private constructor** cannot be extended, because a subclass could not call `super(...)`. Objects are then made with a static method (Chapter 8):

```
export class Euro {
  private constructor(public readonly cents: number) {}
  public static of(cents: number): Euro { return new Euro(cents); }
}
export class Dollar extends Euro {}
```

```
TS2675 [ERROR]: Cannot extend a class '"file:///.../Euro.ts".Euro'. Class constructor is marked as private.
```

(The real message has the file's full path where the `...` is.)

Use this rarely. Most TypeScript code simply does not extend classes that were not designed for it.

A method no subclass may override - Java's `final` method. TypeScript has nothing that does this. `BankAccount.withdraw` holds the rules, and a subclass *could* override it and skip them. What you can do instead:

1. **Keep helpers private**. A subclass cannot call or replace a private method. Declare a `requirePositive` of its own and the compiler refuses: `TS2415 [ERROR]: Class 'Isa' incorrectly extends base class 'BankAccount'. Types have separate declarations of a private property 'requirePositive'.`
2. **Say so** in a comment, as `withdraw`'s does.
3. **Test it**: the shared suite checks that every subclass still keeps the rules, whoever wrote it.
4. **Keep hierarchies shallow**, and prefer composition when "is-a" is doubtful. Book 3 starts with a famous example of inheritance going wrong (ducks that fly when they should not) and the advice "favour composition over inheritance".

Abstract class or interface?

Chapter 10's interfaces also describe what a group of classes can do. Which should you use?

	Interface	Abstract class
Holds code (method bodies)	no	yes - <code>speak()</code> , <code>withdraw()</code>
Holds fields and a constructor	no	yes - <code>name</code> , <code>balance</code>
How many can a class have?	<code>implements A, B, C</code> - any number	<code>extends</code> exactly one
Exists at run time (<code>instanceof</code>)	no	yes
Objects need to come from the class?	no - any object of the right shape will do	for the shared code, yes

Use an **interface** when you only need to say what something can do - it is lighter, and an object literal can stand in for it in a test. Use an **abstract class** when the subclasses share real code or state, as the bank accounts share their rules. The two mix well: an abstract class can `implement` an interface and leave some of its methods abstract.

Java and TypeScript

Java	TypeScript
<code>class Cat extends Animal</code>	<code>class Cat extends Animal</code>
<code>@Override</code> (optional)	<code>override</code> keyword (required with <code>noImplicitOverride</code>)
<code>super(name)</code> ; first in the constructor	<code>super(name)</code> ; before any use of <code>this</code>
constructors are never inherited	a subclass with no constructor gets its superclass's
<code>protected</code> = subclasses and the same package	<code>protected</code> = the class and its subclasses only
<code>abstract class</code> , <code>abstract methods</code>	the same
<code>final</code> field	<code>readonly</code>
<code>final class</code>	no keyword; a <code>private constructor</code> prevents <code>extends</code>
<code>final</code> method	nothing - private helpers, comments and shared tests
every class extends <code>Object</code> ; default <code>toString()</code> gives <code>Animal@1b6d3586</code>	default gives <code>[object Object]</code>
<code>super.toString()</code>	<code>super.toString()</code>
<code>obj instanceof Animal</code>	the same (classes exist at run time; interfaces do not)

Summary

- inheritance is for **is-a**: a subclass `extends` a superclass and inherits all its members
- an `abstract` class cannot be instantiated; an `abstract` method has no body and every concrete subclass must write it
- with `noImplicitOverride`, replacing a method needs the `override` keyword, which catches accidental overrides and typos
- `protected` members can be used by subclasses but not from outside; start `private`
- a subclass constructor must call `super(...)` before using `this`; a subclass with no constructor uses its superclass's
- `super.method()` calls the superclass's version, so a subclass can build on it, as `toString()` does
- through a superclass variable you can only use superclass members, but each object runs its own version of them
- there is no `final`: use `readonly` for fields, a private constructor for classes, and private helpers plus tests for methods
- shared rules can be tested once, in a function that registers tests, called once per subclass

Challenges

Each challenge says which project to start from. Write the tests first.

1. A cow

Start from `ch11_project01_animals`. Add a `Cow` that says "Moo" and describes itself as "Daisy, a cow", and an "Add a cow" button. Write the tests first - copy the shape of the cat tests - and watch them fail before `Cow.ts` exists.

2. A technician

Start from `ch11_project02_staff`. The college also has technicians, each looking after a lab. Add a `Technician` subclass of `Person` with a `lab`, the role "Technician", and a `toString()` built on `super.toString()`: "Sam sam@college.ie, technician, lab C12". Add some technicians to `people.json` and a filter button. Test first, including the email check.

3. Legs

Start from `ch11_project01_animals`. Every animal should know how many legs it has. Give `Animal` a second constructor parameter `legs`, and a `getLegs()` method. Cats and dogs have four, so they now need constructors of their own that call `super(name, 4)`. Add a `Bird` with two. The tests come first: what should `new Cat("Tom").getLegs()` be? Do the old tests still pass without changes?

4. A student account

Start from `ch11_project03_bank_accounts`. A student account is a current account with a smaller overdraft (€50) that also refuses any single withdrawal over €200. Write `StudentAccount` extends

`CurrentAccount`. First call `testAccountRules` for it - before the class has any code of its own - then add tests for the €50 overdraft and the €200 limit.

Hint: the limit on one withdrawal is a rule about *withdrawing*, not about what is *available*. Can you put it in `available()`, or must you override `withdraw`? If you override it, call `super.withdraw(amount)` so the shared rules still run - and check that the shared tests agree.

5. Three levels deep

Start from `ch11_project02_staff`. A PhD student is a student who also teaches: `PhdStudent` extends `Student`, with a `module` they teach. Its `toString()` should build on `Student`'s: "Lee lee@college.ie, student S9, Computing, teaches Databases". Test first. Then write one test that checks a `PhdStudent` is a `Student` and a `Person`, and that the email check still applies.

Hint: `super(...)` in `PhdStudent` calls `Student`'s constructor, which calls `Person`'s. Which arguments does each need?

6. Month end

Start from `ch11_project03_bank_accounts`. At the end of each month, a savings account adds a twelfth of its yearly interest, and an overdrawn current account is charged a €5 fee (which may take it past its overdraft). Add an abstract method `monthEnd(): void` to `BankAccount`, write it in each subclass, and add an "End of month" button that runs it on every account in an array. Test first: add a shared rule to `testAccountRules` ("month end on a new, empty account leaves it at zero"), then tests for each subclass.

Hint: a fee is not a withdrawal - it must not be refused. The fee can change the protected `balance` directly. What does the compiler say about the subclasses the moment you add the abstract method, before you write them?

What next

That is the end of Book 1. You can now write and test small object-oriented TypeScript programs that run in a web page: classes and constructors, encapsulation, model and view, static members and unions, references and composition, modules, interfaces - and inheritance. You also have the habit that holds it together: test first, red, green, refactor.

Book 2, *Object-oriented TypeScript, further*, takes you on, one project per chapter and more slowly. It begins with **polymorphism** - the idea behind `chorus` and `wireCard`, properly - and goes on to **narrowing** (finding out what something is with `typeof`, `instanceof` and `in`), **discriminated unions** (a TypeScript alternative to class hierarchies), **generics** (`Stack<T>`), the **collections** `Map` and `Set`, **null and undefined** in depth, **errors** and `try / catch`, **test doubles** (fakes, stubs and spies), and finally **code quality**: measuring your code with static analysis and in the running page.

Appendix - Java to TypeScript cheat sheet

One page of everything Book 1 translates from Java. The chapter that explains each row is in the last column.

Basics

Java	TypeScript	Ch
<code>int n = 5; / double d = 2.5;</code>	<code>let n = 5; / let d: number = 2.5;</code> - one number type	1
<code>final String NAME = "Ada";</code>	<code>const NAME = "Ada";</code>	1
<code>String, boolean, char</code>	<code>string, boolean</code> (no <code>char</code> : a one-letter string)	1
<code>"Hi " + name + "!"</code>	<code>`Hi \${name}!`</code>	1
<code>a.equals(b)</code> (strings), <code>==</code> (numbers)	<code>a === b</code> for both; never <code>==</code>	1
<code>String[] xs = {"a", "b"}; / ArrayList<String></code>	<code>const xs: string[] = ["a", "b"];</code> (grows like a list)	1
<code>for (String x : xs)</code>	<code>for (const x of xs)</code> - of, not in	1
<code>xs.length / list.size()</code>	<code>xs.length</code>	1
<code>static int twice(int n) { ... }</code> in a class	<code>function twice(n: number): number { ... }</code> in any file	1
<code>public class, import pkg.Name;</code>	<code>export, import { Name } from "./Name.ts";</code>	1, 9
<code>null</code>	<code>null</code> and <code>undefined</code> ; only allowed if the type says so: <code>string null</code>	2

Functions

Java	TypeScript	Ch
<code>x -> x * 2</code>	<code>(x) => x * 2</code>	3
<code>Function<Integer, Integer></code>	<code>(x: number) => number</code>	3
<code>list.stream().map(f).collect(...)</code>	<code>xs.map(f)</code>	3
<code>.filter(p), .reduce(0, Integer::sum), .findFirst()</code>	<code>.filter(p), .reduce((s, x) => s + x, 0), .find(p)</code> (or <code>undefined</code>)	3
<code>list.sort(cmp)</code> (changes the list)	<code>xs.toSorted(cmp)</code> (a copy)	3
<code>obj::method</code>	<code>() => obj.method()</code> - never <code>obj.method</code> on its own	3

Java	TypeScript	Ch
<code>button.addActionListener(e -> ...)</code>	<code>button.addEventListener("click", () => { ... })</code>	2, 3

Classes

Java	TypeScript	Ch
<code>public class Student { ... }</code>	<code>export class Student { ... }</code> , one per file, <code>Student.ts</code>	2, 5
<code>count++</code> inside a method	<code>this.count++</code> - <code>this.</code> is compulsory	2
<code>private String name;</code> + constructor assigning it	<code>constructor(private name: string) {}</code> (parameter property)	5
several constructors / overloaded methods	one constructor/method with default (<code>x = 1</code>) or optional (<code>x?</code>) parameters	5
<code>toString()</code>	<code>toString(): string</code> - used by <code>\${obj}</code>	5
<code>private</code>	<code>private</code> (checked by the compiler) or <code>#field</code> (private at run time too)	6
<code>final</code> field	<code>readonly</code> field	6, 8
<code>getName()</code> , <code>isActive()</code> , <code>setName(...)</code>	methods, or <code>get name()</code> / <code>set name(v)</code> accessors	6
<code>throw new IllegalArgumentException("...")</code>	<code>throw new Error("...")</code>	4, 6
<code>static final int MAX = 3;</code>	<code>static readonly MAX = 3;</code> (or a module-level <code>const</code>)	8
<code>enum Colour { RED, GREEN }</code>	<code>type Colour = "red" "green";</code> (or <code>enum</code> , or as <code>const</code>)	8
<code>interface Shape { double area(); }</code>	<code>interface Shape { area(): number; }</code> - matched by shape, not name	10
<code>class Circle implements Shape</code>	<code>class Circle implements Shape</code> (optional - any object of the right shape fits)	10
<code>class Cat extends Animal, super(...)</code>	the same	11
<code>@Override</code>	<code>override</code> keyword - compulsory in this book's projects	11
<code>protected</code> , <code>abstract</code>	the same	11
<code>final class</code> / <code>final</code> method	no equivalent - prefer composition, or document "do not override"	11
<code>obj instanceof Shape</code> (interface)	not possible - interfaces vanish at run time	10

Tests

JUnit	Deno	Ch
a Javadoc example nobody runs	a doc comment <code>@example</code> - Deno runs and checks it	1
<code>@Test void name() { ... }</code>	<code>Deno.test("a sentence saying what should be true", () => { ... });</code>	1
<code>assertEquals(expected, actual)</code>	<code>assertEquals(actual, expected)</code> - the other way round	1
<code>assertEquals(e, a, delta)</code>	<code>assertAlmostEquals(actual, expected)</code>	3
<code>assertThrows(X.class, () -> ...)</code>	<code>assertThrows(() => ..., Error, "part of the message")</code>	4
<code>assertSame(e, a)</code>	<code>assertStrictEquals(actual, expected)</code>	4
<code>@BeforeEach</code>	a helper function each test calls	4
nested tests	<code>await t.step("...", () => { ... })</code> in an <code>async</code> test	4

Running things

Java	This book	Ch
<code>javac</code> then <code>java</code>	open the project in Celbridge: <code>deno task dev</code> builds, tests and watches; press refresh on the preview	1
the console	the TAP output in Celbridge's console, and <code>test_output/index.html</code>	1
a <code>.jar</code>	<code>dist/</code> - a plain web page, no server needed	1